

UNIX / Linux Programming



AMITY
UNIVERSITY

DIRECTORATE OF DISTANCE &
ONLINE EDUCATION

© Amity University Press

All Rights Reserved

No parts of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise without the prior permission of the publisher.

Advisory Committee

Chairman : Ms. Monica Agarwal

Members : Prof. Arun Bisaria

Dr. Priya Mary Mathew

Prof. Aindril De

Mr. Alok Awtans

Dr. Coral J Barboza

Dr. Monica Rose

Mr. Sachit Paliwal

SLM Review Committee

Mr. Gaurav Agarwal

Ms. Nitika Khanna

Ms. Rashmi Saxena

Ms. Renu Singh

Ms. Mona Chaudhary

Contents

Page No.

Module-I: Introduction

01

- 1.1 Unix Introduction
 - 1.1.1 Introduction to OS
 - 1.1.2 GNU
 - 1.1.3 Unix Family
 - 1.1.4 Unix Features
 - 1.1.5 Unix Layered Architecture
 - 1.1.6 "Types of Shell (Bourne, BASH, KORN,C),"
- 1.2 Unix File System
 - 1.2.1 Unix file system
 - 1.2.2 Absolute Path
 - 1.2.3 Relative Path
 - 1.2.4 Inode
- 1.3 Linux
 - 1.3.1 Two User Interfaces –Character User Interface(CUI) and Graphic User Interface (GUI)
 - 1.3.2 "LINUX booting Process"
 - 1.3.3 "LINUX Graphical environment, LINUX Desktop environment –GNOME and KDE"

Module-II: Processes

50

- 2.1 Process Basics
 - 2.1.1 Process state and transitions
 - 2.1.2 The context of a process
- 2.2 Running Processes
 - 2.2.1 Foreground Processes
 - 2.2.2 Background processes
 - 2.2.3 suspend and interrupt a process and kill A process
- 2.3 Process Commands
 - 2.3.1 The ps command
 - 2.3.2 The batch command
 - 2.3.3 The at command

Module-III: Unix/Linux Commands

84

- 3.1 Work on Unix Terminal
 - 3.1.1 Telnet connect through Login Account , password, logout ,
 - 3.1.2 Internal and External Commands
- 3.2 Unix File Handling

- 3.2.1 "current working directory, referring to home directories"
- 3.2.2 Commands to move around by path concept, creating new directories
- 3.2.3 creating new directories, creating files –touch , cat
- 3.2.4 Listing directory and File - ls, hidden files
- 3.2.5 "copying files; moving files, Deleting files and directories;"
- 3.2.6 View file - cat, more, pg, less
- 3.2.7 wildcards and Meta characters
- 3.3 Filters and Pipes
 - 3.3.1 Introduction to filters
 - 3.3.2 Head, Tail, Cut
 - 3.3.3 Sort, wc
 - 3.3.4 grep
 - 3.3.5 "cmp, comm., diff, uniq"
 - 3.3.6 expr , bc, exit
- 3.4 Standard Input/Output
 - 3.4.1 Introduction to redirection
 - 3.4.2 redirecting input
 - 3.4.3 redirecting output and error
- 3.5 Unix Permissions
 - 3.5.1 Understanding Permissions
 - 3.5.2 Symbolic and Numeric Permissions
 - 3.5.3 Changing Permissions
- 3.6 Unix Links
 - 3.6.1 File Links
 - 3.6.2 Types of Links
 - 3.6.3 The ln command

Module-IV: VI Editor and Shell Programming

181

- 4.1 Vi editor
 - 4.1.1 Command mode, insert mode and last line mode;
 - 4.1.2 "Change word, Change line, Delete current line, Delete n lines, Delete
 - 4.1.3 "command to delete character, insert line; deleting text, command for moving the cursor;
- 4.2 Shell Programming
 - 4.2.1 Shell Script Introduction
 - 4.2.2 Shell scripts and execution methods
 - 4.2.3 Setting positional parameters (set command), Shift ,
 - 4.2.4 shell variables, environment variables
 - 4.2.5 "Assignment Statements, Arithmetic, Logical and relational operators, read, echo"

- 4.2.6 Test Command through IF Statement: Numerical Test, File Test and String Test;
- 4.2.7 Test Command: Numerical Test, File Test and String Test;
- 4.2.8 for Loop
- 4.2.9 while and until loop
- 4.2.10 case and switch

Module-V: sed AND awk

237

- 5.1 sed
 - 5.1.1 Introduction and features sed
 - 5.1.2 Execution process of sed
 - 5.1.3 Syntax of sed and Addressing:
 - 5.1.4 "sed commands: Inserting / Deleting/Printing Text/Lines"
 - 5.1.5 sed commands: Substitution
 - 5.1.6 Multiple edits, reading from file and writing to file
 - 5.1.7 quit command, Modify a File with sed
- 5.2 awk
 - 5.2.1 Syntax for awk
 - 5.2.2 working of awk,
 - 5.2.3 Input from files, Input from commands
 - 5.2.4 Formatting output-print and printf function
 - 5.2.5 "concept of record and field in awk ."
- 5.3 System Administration
 - 5.3.1 Adding and Removing Users, Changing password
 - 5.3.2 Disk Management, Starting and shutting down the system
 - 5.3.3 File System Mounting and Unmounting
 - 5.3.4 Monitoring System Usage,
 - 5.3.5 Ensuring System Security

Module-I: Introduction

Notes

Structure:

- 1.1 Unix Introduction
 - 1.1.1 Introduction to OS
 - 1.1.2 GNU
 - 1.1.3 Unix Family
 - 1.1.4 Unix Features
 - 1.1.5 Unix Layered Architecture
 - 1.1.6 “Types of Shell (Bourne, BASH, KORN,C),”
- 1.2 Unix File System
 - 1.2.1 Unix file system
 - 1.2.2 Absolute Path
 - 1.2.3 Relative Path
 - 1.2.4 Inode
- 1.3 Linux
 - 1.3.1 Two User Interfaces –Character User Interface(CUI) and Graphic User Interface (GUI)
 - 1.3.2 “LINUX booting Process”
 - 1.3.3 “LINUX Graphical environment, LINUX Desktop environment –GNOME and KDE”

Notes

Unit-1.1: Unix Introduction

Objectives:

After studying this chapter, you will be able to:

- Learn Introduction of OS
- Understand GNU
- Learn About Unix Family
- Learn About Unix Features
- Understand Concept of Unix Layered Architecture
- Understand Types of Shell

Introduction

An operating system is used to execute other programmes, such as application software, on modern general-purpose computers, such as personal computers and mainframes. Microsoft Windows, Mac OS (including Darwin), UNIX, and Linux are examples of personal computer operating systems. The kernel is the most basic level of any operating system. When a system boots or starts up, this is the first layer of software that is loaded into memory. All other system and application programmes have access to the kernel's multiple common core services. Disk access, memory management, job scheduling, and access to other hardware devices are among the services provided.

In addition to the kernel, an operating system is frequently distributed with tools for programmes to display and manage a graphical user interface (although these tools are built into Windows and the Macintosh), as well as utility programmes for tasks such as managing files and configuring the operating system. They are also frequently delivered with application software that does not directly relate to the basic function of the operating system, but which the operating system distributor believes is advantageous to include with the operating system.

The distinction between the operating system and application software is not exact and is regularly disputed. From an economic or legal standpoint, the delineation might be influenced by the circumstances of the interests at stake. One of the significant issues in the United States v. Microsoft antitrust lawsuit, for example, was whether Microsoft's web browser was part of its operating system or a separate piece of application software.

The question of what exactly should constitute the "kernel," like the phrase "operating system," is controversial, with disagreements over whether features like file systems should be included in the kernel. Microkernels, monolithic kernels, and other approaches are advocated by various groups.

Most, but not all, computer systems employ operating systems. The simplest computers, including the smallest embedded systems, as well as many of the first computers, lacked operating systems. Instead, they relied on the application programmes to manage the minimum hardware on their own, sometimes with the

help of libraries designed for the purpose. Commercially available operating systems are present on almost all modern computers, ranging from personal computers to mainframes, as well as mobile computers such as PDAs and mobile phones.

1.1.1 Introduction to OS

An operating system is a significant part of almost every computer system. A computer system can be divided roughly into four components:

- ◇ Hardware
- ◇ Operating system
- ◇ Application programs
- ◇ Users

The basic computing resources are provided by the hardware, which includes the central processing unit (CPU), memory, and input/output devices. Application programmes, such as word processors, spreadsheets, compilers, and web browsers, define how these resources are used to solve users' computing problems. The operating system manages and coordinates the use of hardware by multiple application applications for different users. A computer system's components are its hardware, software, and data.

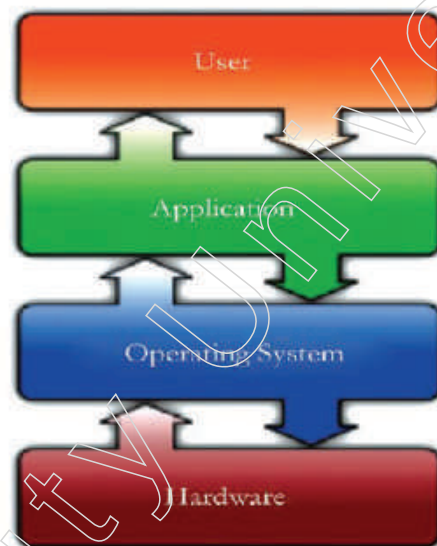


Fig: Operating System

The operating system provides the tools for these resources to be used properly in the computer system's operation. Operating systems can be investigated from two perspectives: the user's and the system's.

Did you know?

Early computers, such as calculators, were designed to accomplish a series of separate tasks. In the 1950s, basic operating system elements such as resident monitor functions that could automatically execute different programmes in succession to speed up processing were developed. Operating systems in its contemporary and more complex variants did not exist until the early 1960s.

Notes

Functions of Operating Systems

The operating system is the most important piece of software in a computer. It has numerous purposes and, at its most basic, serves as an interface between your computer and the outside world. A computer is described under the hardware section as having various component elements, including a monitor, keyboard, mouse, and other equipment. The operating system interfaces with these components via what are known as “drivers.” This is why, when you install a new printer or other piece of hardware, the system may prompt you to install additional software known as a driver.

A driver is a specially created programme that understands how the device it interacts with works, such as a printer, video card, sound card, or CD ROM drive. It converts commands from the operating system or the user into commands that the component computer part with which it communicates understands. It also converts responses from the component computer element into responses that the operating system, application programme, or user can understand. The diagram depicts the interfaces between the operating system and the computer component.

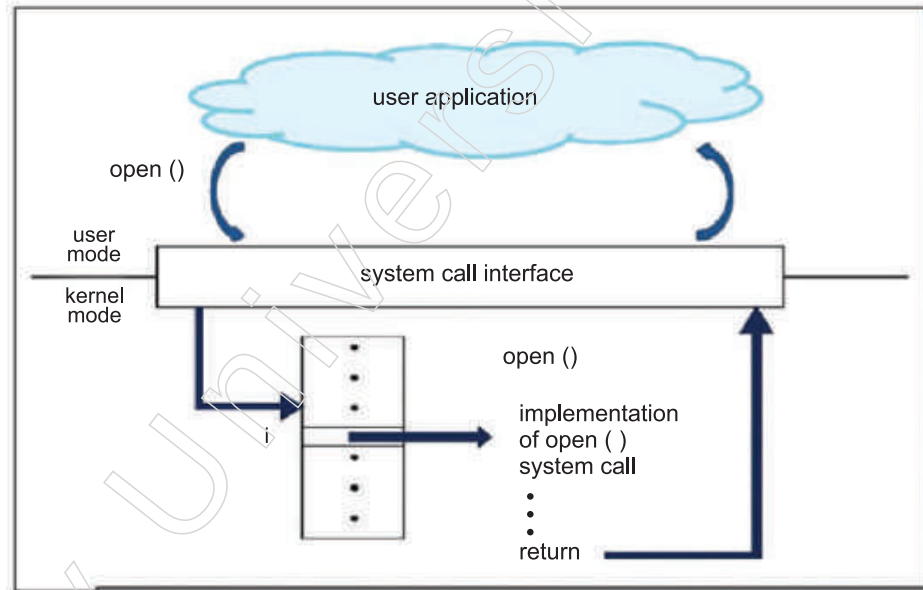


Fig: Operating System and Computer Component

The operating system provides for several other functions including:

- ◇ System tools (programmes) used to monitor computer performance, debug problems, or maintain elements of the system are provided by the operating system.
- ◇ A collection of libraries or functions that programmes can employ to execute specific tasks, particularly those involving connecting with computer system components.

The operating system ensures that these interfacing functions, along with its other functions, run smoothly and are usually transparent to the user.

1.1.2 GNU

Richard Stallman began development on the GNU operating system while working at the MIT Artificial Intelligence Laboratory. Stallman publicly introduced the GNU Project on the net.unix-wizards and net.usoft newsgroups on September 27, 1983. On

January 5, 1984, Stallman resigned from the Lab so that they could not claim ownership or interfere with the distribution of GNU components as free software. The idea was to create an entirely free software operating system. Stallman desired that computer users be allowed to study the source code of the software they use, share software with others, modify software behaviour, and publish their updated versions of the software. In March 1985, this ideology was released as the GNU Manifesto.

Following his experience with the Incompatible Timesharing System (ITS), an early operating system built in assembly language that became obsolete due to the demise of the PDP-10 computer architecture for which ITS was written, Richard Stallman decided that a portable system was required. As a result, it was determined that development would begin with C and Lisp as system programming languages, and that GNU would be UNIX compatible. UNIX was already a prominent proprietary operating system at the time. UNIX was designed to be modular, so it could be re-implemented piece by piece.

Much of the required software had to be created from scratch, but existing compatible third-party free software components, such as the TeX typesetting system, the X Window System, and the Mach microkernel that serves as the foundation for GNU Hurd's GNU Mach core, were also employed (the official kernel of GNU). With the exception of the aforementioned third-party components, the majority of GNU has been produced by volunteers, some in their own time and others paid by companies, educational institutions, and other non-profit organisations. Stallman founded the Free Software Foundation in October 1985. (FSF). The FSF employed software engineers in the late 1980s and early 1990s to write the software required for GNU.

As GNU grew in popularity, firms that were interested began contributing to its development or offering GNU software and technical help. Cygnus Solutions, which is now part of Red Hat, was the most visible and successful of these.

Did you know?

In the 1960s, Control Data Corporation created the SCOPE operating system for batch processing. During the 1970s, the Kronos and later the NOS operating systems were created in collaboration with the University of Minnesota, and they permitted simultaneous batch and timesharing use.

1.1.3 Unix Family

The UNIX family is composed of two components:

- ◇ Utilities
- ◇ The kernel

Utilities are applications that allow you to run or execute them. Utilities include the program person and date that you viewed. Almost every software you are familiar with is classified as a utility. Commands are not the same as utilities. The term utility refers to the name of a program, but the term command refers to the program as well as any inputs you pass to it to change its behaviour. For simple commands where only the program name to execute is specified, the term command may be used instead of utility. The kernel is the UNIX system's beating heart. It allows utilities to access the hardware of a machine. It also controls command scheduling and execution.

Notes

When a system is shut off, the kernel and utilities are both stored on the hard discs. However, when the machine boots, the kernel is loaded into memory from disc. The kernel is kept in memory until the machine is shut off. Utilities, on the other hand, are kept on disc and loaded into memory only when used.

For instance, when you run the command:

```
$ who
```

the kernel loads the who command from the hard disc, stores it in memory, and then executes it.

When the software completes its execution, it remains in the machine's memory for a brief time before being removed. This allows frequently used instructions to be executed more quickly. Consider what happens if you run the date command three times in a row:

```
$ date
```

```
Sun Dec 27 09:42:37 PST 1998
```

```
$ date
```

```
Sun Dec 27 09:42:38 PST 1998
```

```
$ date
```

```
Sun Dec 27 09:42:39 PST 1998
```

The date command can be loaded from the machine's hard disc the first time, but the second and third times it normally remains in the machine's memory, allowing it to execute faster.

The shell is a program that works similarly to the who command. The main distinction is that when you log in, the shell is loaded into memory.

Logging In

When you initially connect to a UNIX system, you should see something like this: login:

At this screen, you must enter your username. After you input your username, you will see the following prompt:

```
login:
```

```
Password:
```

At this point, you must enter your password.

An application named Getty generates these two prompts. These are its responsibilities:

- ◇ Show the login prompt.
- ◇ Wait for a user to enter their username.
- ◇ Display the password prompt when a username has been entered.
- ◇ Require the user to enter a password.
- ◇ Pass the user's username and password to the login command and exit.

When login receives your username and password, it searches the file /etc/passwd for an entry that matches the information you supplied.

If a match is found, login runs a shell and exits. As an example, on my system the matching entry for my username, in file /etc/passwd is:

name: x: 500 :100 : Full name:/home/name:/bin/bash

If no match is found, the login program issues an error message and exits. At this point the Getty program takes over and displays a new login prompt.

Shell Initialization

When the login software runs a shell, it is uninitialized. When a shell is uninitialized, critical parameters that the shell requires to function properly are not defined.

To configure these parameters, the shell goes through a setup phase. This is often a two-step operation in which the shell reads the following files:• /etc/profile

- ◇ profile

The process is as follows:

- ◇ The shell verifies the existence of the file /etc/profile.
- ◇ The shell reads it if it exists. If not, this file is skipped. There is no error message displayed.

The shell checks to see if the file. profile is present in your home directory. Your home directory is the first directory you see after logging in.

- ◇ If it exists, the shell reads it; if it does not exist, the shell skips it. There is no error message displayed.

When both of these files have been read, the shell prompts you:

This is the prompt where you can type commands to have them run.

1.1.4 Unix Features

UNIX operating systems have established themselves as industry standards, and this is no secret to anyone. These findings came as no surprise to programmers who have been working with the UNIX operating system for many years: the UNIX system provides an elegant and efficient environment in which to develop software. Because this is exactly what Dennis Ritchie and Ken Thompson aimed for when they worked on the UNIX operating system at Bell Laboratories in the late 1960s.

One of the most advantageous aspects of the UNIX operating system is the large number of programmes available. The standard operating system includes more than 200 basic commands that are easy to learn and use. From counting the number of lines in a file to sending electronic mail to displaying a calendar for any desired year, these commands (also known as tools) can perform a wide range of tasks on your computer. But it is not just this large collection of commands that makes the UNIX system so powerful; it is also how elegantly and simply these commands can be combined to perform far more complex functions. The shell was created in order to advance this goal as well as to provide a consistent buffer between the user and the heart of the UNIX system (the kernel), among other things. In its most basic form, the shell is a software that takes in the commands you input and translates them into a format that is more easily understood by the UNIX operating system. A number of essential programming

Notes

elements are also included, such as the ability to make decisions, loop, and store values in variables.

UNIX and Linux systems ship with the standard shell, which was originally developed by Stephen Bourne at Bell Labs and later adopted by AT&T as the standard shell. Since then, the IEEE has developed standards that are based on the Bourne shell as well as other more contemporary shell technologies. According to the most recent revision of this standard, the Shell and Utilities volume of IEEE Std 1003.1-2001 (commonly known as the POSIX standard) is the most up-to-date version. Many UNIX systems are still in use today, and many of them have Bourne shell variants and utilities that are not consistent with the POSIX specification. However, because there are so many different versions of UNIX from so many different manufacturers, it is just not possible to include every distinction between them all here. Alternatively, if you have an older UNIX system that does not include a POSIX-compliant shell, there is still hope for you. Programming programmes may be produced, changed, and debugged quickly and efficiently since the shell provides an interpreted programming language. As a first-choice programming language, we turn to the shell for assistance. After becoming proficient with shell programming, you may choose to start with it as a starting point. The popularity of the UNIX operating system has skyrocketed in recent years, thanks in large part to the introduction of the Linux operating system. This should come as no surprise to programmers and other UNIX users: UNIX was created to provide an environment that is both powerful and simple to use. One of the most significant advantages of UNIX is the enormous number of standard programmes that are included with it. These tools can be used to conduct a wide range of functions, ranging from listing your files to reading your incoming email. One of the most important characteristics of UNIX, as opposed to other operating systems, is that these programmes can be coupled to accomplish intricate tasks and solve your problems. The shell is one of the most powerful standard applications available in the UNIX operating system. It is a programme that provides a consistent and easy-to-use environment for executing applications under the UNIX operating system. If you have ever worked with a UNIX system, you have probably dealt with the shell in some capacity.

The shell's primary function is to read the commands you input and then request that the UNIX kernel carry out these commands on your behalf. The shell also has complex programming tools that allow you to make decisions, repeatedly execute commands, define functions and store values in variables. The Bourne shell, which is the basic UNIX shell, is the focus of this article. The shell that Dennis Ritchie and Ken Thompson used when they were developing the majority of UNIX in the early 1970s was extremely basic. Stephen Bourne's first genuine shell, written in the mid-1970s, was widely regarded as ground-breaking. The original Bourne shell has undergone minor modifications over the years; certain features have been added, while others have been removed, but its grammar and the power it provides have stayed unchanged. Create scripts is the most appealing aspect of the shell, as it allows you to do so quickly and easily. scripts are files that include a set of commands that you want to run when the script is executed. Given that every script is housed within a file, and that every file has a name, scripts allow you to mix existing programmes in order to develop wholly new programmes that handle your specific needs. Following a period of acclimation to screenplay writing, you will discover yourself solving an increasing number of problems with scripts.

1.1.5 Unix Layered Architecture

UNIX system tracks a layered approach.

It has 4 layers:

- ◇ The hardware layer is the deepest layer.
- ◇ The KERNEL is placed in the second layer.
- ◇ The utilities and other application program make up the third category.
- ◇ The fourth layer is the one with which the user interacts.

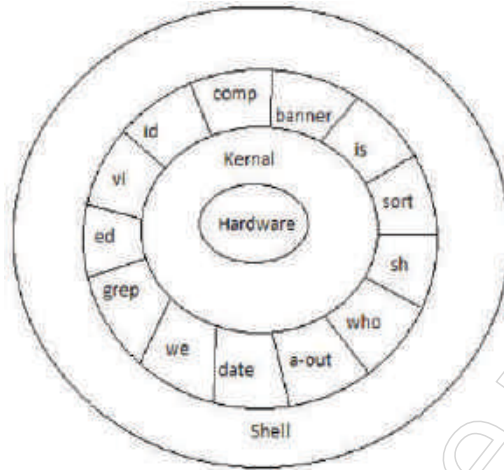


Fig: Unix System Architecture

Kernel

- ◇ Kernel is that fragment of OS which unswervingly makes interface with the hardware system.

Actions:

- ◇ Provides a mechanism for adding and removing processes.
- ◇ Manages CPU scheduling, memory and I/O.
- ◇ Enables inter-process communication.

The Shell

It is a utility program that comes with the UNIX system.

Features of shell are:

- ◇ Interactive processing
- ◇ Back ground processing
- ◇ I/ O Redirection
- ◇ Pipes
- ◇ Shell Scripts
- ◇ Shell Variables
- ◇ Programming constructs

Notes

Notes

Process Management

- ◇ A process is a program that is being carried out.
- ◇ A UNIX system can run several processes at the same time.
- ◇ The “fork ()” system call is commonly used to start a process.

The parent process is the one that makes the “fork ()” system call, and the newly generated process is known as the child process.

CPU Scheduling

Classical UNIX scheduling has been preemptive, time-sharing, and priority-based:

- ◇ Priority-based – where every process is assigned a dynamic and calculated priority and then the process with the highest priority runs
- ◇ Time-sharing – where multiple processes with same priority level can run in parallel and each process given a time slice (e.g. 10 ms). After the time is expired the next process with the same priority is executed.

Modern UNIX schedulers are modular and several scheduling classes are available according to needs of the applications. These support multi-cpu, multi-core systems and improves resource allocation using schedule threads.

Memory Management

- ◇ Virtual memory
- ◇ Swap area
- ◇ Demand paging

File Management

- ◇ UNIX employs a hierarchical file system with the letter ‘1’ as its root.
- ◇ Every non-leaf node in the tree is referred to as a directory file.
- ◇ Each leaf node can be a file or an empty directory.

Types of UNIX users

- ◇ Broad classification of users
 - ◆ root
 - ◆ non-root
- ◇ Group
 - ◆ Unix allows user ID's to be grouped
 - ◆ A single user ID can be member of multiple groups
- ◇ Differentiating users with respect to file access
 - ◆ OWNER
 - ◆ Group
 - ◆ others

Working with UNIX

- ◇ User logs in with a valid user ID.
- ◇ User logs out to terminate the login session.

Exec ()

Excel, Excel p, Excel e, Excel v, Excel vp – executes a file

Synopsis

```
# include <Unistd. h>
```

```
Extern char ** en viron;
```

```
Int excel (const char * path , const char * arg,.....);
```

```
Int excel p (const char * file , const char * arg,.....);
```

```
Int excel e (const char * path , const char * avg,..., char * const envp [] );
```

Return Value

If any of the exec () functions returns, an error will have occurred. The return value is -1, and the global variable err no will be set to indicate the error.

Files

/bin/sh

System ()

System - executes a shell command

Synopsis

```
# include <stdlib.h>
```

```
Int system (const char * command;)
```

Description

System () calls /bin/sh – c command to run the command supplied in command and returns after the command is finished. SIGCHLD will be blocked and SIGINT and SIGQUIT will be ignored if the command is executed.

Return Value

‘ - 1 ’ on error

If the value of command is NULL, sys n () returns non – Zero if the shell is available and if not system () does not affect wait status of at other children.

PS:

\$ PS:

PiD	TTY	TIME	CMD
2561	Pts/0	00:00:00	Bash
2752	Pts/0	00:00:00	PS

\$ bash // opened bash for your own

Notes

\$ Ps - l:

F	S	UID	PID	PPID	0	PRI	NT	ADDR	S7	WCHAN	TTY	TIME	CMD
0	S	6524	2561	2559	0	75	0	-	1177	wait	Pts/0	00:00:00	Bash
0	S	6524	2767	2561	0	76	0	-	1176	Wait	Pts/0	00:00:00	Bash
0	S	6524	2785	2767	0	76	0	-	1084	-	Pts/0	00:00:00	PS

1.1.6 Types of Shell (Bourne, BASH, KORN,C)

The kernel launches a new shell instance whenever a user registers into the system or starts a console window. The kernel is the central processing unit (CPU) of any operating system.

It is in charge of controlling, managing, and executing processes, as well as ensuring optimum system resource use.

What is a Shell?

The shell serves as an interface to the UNIX system. It collects your input and performs programs based on it. When a program completes its execution, the output of that program is displayed.

A shell is a program that allows us to run commands, applications, and shell scripts. There are various varieties of shells, just as there are various flavours of operating systems. Each shell flavour has its own collection of known commands and functions.

Shells are a powerful tool because they can communicate with the kernel. A user cannot utilise the utilities provided by their machine's operating system if they are unable to interact with the kernel.

Shell Prompt

The shell generates the prompt, which is known as the command prompt. You can type a command while the prompt is shown.

After you press Enter, the shell reads your input. It looks at the first word of your input to decide which command you wish to perform. A word is a continuous string of characters. Words are separated by spaces and tabs.

Types of Shells:

There are two types of shells in UNIX:

1. The shell of Bourne. The \$ character is the default prompt in a Bourne-type shell.
2. The shell in C. The percent character is the default prompt when using a C-type shell. There are again various subcategories for Bourne Shell which are listed as follows:

- ◆ Bourne shell (sh)
- ◆ Korn shell (ksh)
- ◆ Bourne Again shell (bash)
- ◆ POSIX shell (sh)

The different C-type shells follow:

- ◆ C shell (csh)
- ◆ TENEX/TOPS C shell (tcsh)

Notes

The Original Bourne Shell:

- ◇ Stephen R. Bourne created the first UNIX shell in the mid-1970s while working at AT&T Bell Labs in New Jersey. Because the Bourne shell was the first to appear on UNIX systems, it is commonly referred to as “the shell.” On most UNIX systems, the Bourne shell is installed as /bin/sh. As a result, it is the shell of choice for writing scripts for usage on various UNIX versions. The Bourne shell is a strong language with programming syntax similar to that of the ALGOL language, in addition to being a command interpreter. It has the following characteristics:
 - ◇ Process control
 - ◇ Variables
 - ◇ Regular expressions
 - ◇ Flow control
 - ◇ Powerful input and output controls
 - ◇ Functions

These characteristics are supported by all Bourne-type shells.

One of the primary disadvantages of the original Bourne shell is that it is difficult to use interactively. The others are:

- ◇ No file name completion is one of the three key downsides.
- ◇ There is no command history or command editing.
- ◇ Difficulty carrying out several background processes or jobs.

The C Shell

Bill Joy created the C shell in the early 1980s while studying at the University of California, Berkeley. It was created to make interactive use of the shell more convenient for users. Another design aim was to switch the shell's syntax from the older ALGOL language of the Bourne shell to the current C style.

The C shell's C language style was meant to be an upgrade because the C language was familiar to Berkeley programmers working on UNIX. The concept was that a shell using C language syntax would be easier to develop programmes in than a shell using ALGOL syntax.

As it turned out, the C shell could only be used for the most basic scripts. Some of the key disadvantages are as follows:

- ◇ Inadequate input and output controls
- ◇ A lack of functions
- ◇ Confusing syntax as a result of a “lazy” command interpreter

Although the C shell was not popular for scripting, it has grown in popularity for interactive use. Some of the main advancements that have contributed to its popularity are as follows:

- ◇ The Command History

Notes

You can re-execute commands that you previously ran. You can also modify the command before re-executing it.

◇ Aliases

Commands can be given brief mnemonic names. Aliases are a condensed version of the Bourne shell functions.

◇ File Name Completion

By inputting a few characters of the file's name, you can have the C shell automatically complete it.

◇ Job Controls

The job command in the C shell allows you to execute many processes and control them.

Most systems have the C shell installed as `/bin/csh`.

The TENEX/TOPS C shell, `tcsh`, is a more recent version of the C shell that allows you to scroll over the command history by using the up and down arrow keys. It also lets you edit commands by using the right and left arrow keys.

Korn Shell

For a long time, the only shells available were the Bourne shell and the C shell. This required most users to be familiar with two shells: the Bourne shell for programming and the C shell for interactive use.

To address this issue, AT&T Bell Labs' David Korn created the Korn shell, `ksh`, which mixes all of the C shell's interactive features into the Bourne shell's syntax. As a result, the Korn shell has become a popular favourite.

In recent years, most vendors have begun to include the Korn shell with their UNIX versions. It is typically installed as `/bin/ksh` or `/usr/bin/ksh`.

In general, `ksh` is fully compatible with `sh`, however some differences will prohibit scripts from working properly.

◇ The following are some of the additional features that the Korn shell adds to the Bourne shell: Command history and history substitution

◇ Command aliases and functions

◇ File name completion

◇ Array

◇ Built-in integer arithmetic

There are three major versions of `ksh` available:

◇ The Official version (`ksh`)

◇ The Public Domain version (`pdksh`)

◇ The Desktop version (`dtksh`)

There is no difference between the Official and Public Domain versions of `ksh` for shell programmers; all scripts that run in one version will run in the other. The Official version has a few handy features for shell users, such as command line completion with the Tab key rather than the Esc key. The Desktop version includes all main CDE

implementations. This version supports the creation and display of Graphical User Interfaces (GUIs) using the ksh syntax. Scripts developed for the previous two versions of ksh will work properly with this version.

The POSIX shell is a variation on the Korn shell. Hewlett-Packard is the only major vendor currently selling the POSIX shell. The POSIX shell is installed as /bin/sh in HP-UX 11.0. The Bourne shell is found in /usr/old/bin/sh.

The Bourne Again Shell

The Bourne Again shell, bash, was created as part of the GNU project and has mostly superseded the Bourne shell, sh, on GNU-based systems such as Linux. Bash is the default shell replacement in all major Linux distributions, including Red Hat, Slackware, and Caldera. Despite having C shell (csh and tcsh) and Korn shell (ksh) features, bash maintains syntax compatibility with the Bourne shell, allowing it to run practically all Bourne shell scripts.

Check your Understanding

A-Fill in the Blanks

1. A _____ is described under the hardware section as having various component elements, including a monitor, keyboard, mouse, and other equipment.
2. _____ operating systems have established themselves as industry standards, and this is no secret to anyone.
3. A user cannot utilize the utilities provided by their machine's operating system if they are unable to interact with the _____.
4. The Official version has a few handy features for _____ users, such as command line completion with the Tab key rather than the Esc key.
5. A _____ is a specially created programmed that understands how the device it interacts with works, such as a printer, video card, sound card, or CD ROM drive

B-State True and False

1. The distinction between the operating system and application software is exact and is regularly disputed.
2. The operating system manages and coordinates the use of hardware by multiple application applications for different users
3. The operating system provides the tools for these resources to be used properly in the computer system's operation.
4. When you install a new printer or other piece of hardware, the system may prompt you to uninstall additional software known as a driver.
5. Stallman publicly introduced the GNU Project on the net.unix-wizards and net.usoft newsgroups on September 27, 1983.

Summary

- The basic computing resources are provided by the hardware, which includes the central processing unit (CPU), memory, and input/output devices. Application

Notes

programmers, such as word processors, spreadsheets, compilers, and web browsers, define how these resources are used to solve users' computing problems.

- The operating system provides the tools for these resources to be used properly in the computer system's operation. Operating systems can be investigated from two perspectives: the user's and the system's.
- Early computers, such as calculators, were designed to accomplish a series of separate tasks. In the 1950s, basic operating system elements such as resident monitor functions that could automatically execute different programmes in succession to speed up processing were developed. Operating systems in its contemporary and more complex variants did not exist until the early 1960s.
- The operating system is the most important piece of software in a computer. It has numerous purposes and, at its most basic, serves as an interface between your computer and the outside world. A computer is described under the hardware section as having various component elements, including a monitor, keyboard, mouse, and other equipment.
- A driver is a specially created programme that understands how the device it interacts with works, such as a printer, video card, sound card, or CD ROM drive. It converts commands from the operating system or the user into commands that the component computer part with which it communicates understands.
- Richard Stallman began development on the GNU operating system while working at the MIT Artificial Intelligence Laboratory. Stallman publicly introduced the GNU Project on the net.unix-wizards and net.usoft newsgroups on September 27, 1983. On January 5, 1984, Stallman resigned from the Lab so that they could not claim ownership or interfere with the distribution of GNU components as free software.
- In the 1960s, Control Data Corporation created the SCOPE operating system for batch processing. During the 1970s, the Kronos and later the NOS operating systems were created in collaboration with the University of Minnesota, and they permitted simultaneous batch and timesharing use.
- UNIX operating systems have established themselves as industry standards, and this is no secret to anyone. These findings came as no surprise to programmers who have been working with the UNIX operating system for many years: the UNIX system provides an elegant and efficient environment in which to develop software. Because this is exactly what Dennis Ritchie and Ken Thompson aimed for when they worked on the UNIX operating system at Bell Laboratories in the late 1960s.
- UNIX and Linux systems ship with the standard shell, which was originally developed by Stephen Bourne at Bell Labs and later adopted by AT&T as the standard shell. Since then, the IEEE has developed standards that are based on the Bourne shell as well as other more contemporary shell technologies.
- The shell's primary function is to read the commands you input and then request that the UNIX kernel carry out these commands on your behalf. The shell also has complex programming tools that allow you to make decisions, repeatedly execute commands, define functions and store values in variables. The Bourne shell, which is the basic UNIX shell, is the focus of this article.

- The kernel launches a new shell instance whenever a user registers into the system or starts a console window. The kernel is the central processing unit (CPU) of any operating system.
- The shell generates the prompt, which is known as the command prompt. You can type a command while the prompt is shown. After you press Enter, the shell reads your input. It looks at the first word of your input to decide which command you wish to perform. A word is a continuous string of characters. Words are separated by spaces and tabs.
- Bill Joy created the C shell in the early 1980s while studying at the University of California, Berkeley. It was created to make interactive use of the shell more convenient for users. Another design aim was to switch the shell's syntax from the older ALGOL language of the Bourne shell to the current C style.
- The Bourne Again shell, bash, was created as part of the GNU project and has mostly superseded the Bourne shell, sh, on GNU-based systems such as Linux. Bash is the default shell replacement in all major Linux distributions, including Red Hat, Slackware, and Caldera. Despite having C shell (csh and tcsh) and Korn shell (ksh) features, bash maintains syntax compatibility with the Bourne shell, allowing it to run practically all Bourne shell scripts.

Activity

1. Prepare a presentation on Introduction of UNIX in MS PowerPoint.

Questions & Exercise

1. Define a single-user system.
2. Explain Unix Architecture.
3. Describe a link in UNIX.
4. What is the 'nohup' in UNIX?
5. Explain a path in UNIX and different types of pathnames.
6. Differentiate between swapping and paging?

Glossary

- **Child Process** - A process that has been created from a parent process. This is the typical way a Unix command is started and executed, i.e., a parent process initializes and starts execution of a child process (see fork, exec).
- **Home Directory** - The directory location where a user resides upon login, as specified in the /etc/passwd file (in the last field). Also, the default location when the cd command is used without arguments.
- **Exit Status** - An integer value returned from a command, program, or function indicating the success of its completion. An exit status of zero indicates success while a non-zero status indicates non-success.
- **She-Bang** - The name of the characters #!. When located in a shell program on the first line in the first character position, followed by the file specification of an interpreter program, the she-bang instructs the shell to load that specific

Notes

interpreter to execute the shell program. For example, to execute a program using the bash interpreter

- **Job** - A name for a running program, typically a running program of some duration.
- **Shell** - A program which typically runs at each logged-in terminal which serves as the interpreter for user commands. The shell interprets a user's command, checks the commands validity and interfaces with the kernel in executing the command.
- **Process** - A program that has been loaded into memory and is in a state of execution. A process is an instance of a running program.
- **MULTICS** - A project which was the predecessor of what is now Unix. The name was an acronym from MULTiplexed Information and Computing Service.

Further Readings

1. Sumitabha Das: UNIX: Concept and Applications, McGraw Hill Education, 4th Edition.
2. M.G. Venkatesh Murthy: Introduction to Unix and Shell Programming, Pearson Education, 2005 Edition.

Check Your Understanding- Answer

Fill in the Blanks

1. Computer
2. UNIX
3. Kernel
4. Shell
5. Driver

State True and False

1. False
2. True
3. True
4. False
5. True

Unit-1.2: Unix File System

Notes

Objectives:

After studying this chapter, you will be able to:

- Understand UNIX File System.
- Learn UNIX Paths.
- Know about Ordinary Files, Binary File, Dictionary File, Device and Hidden File.
- Understand Types of Users.
- Learn about Symbolic Connection.

Introduction

A file is a container for data storage. It is a series of bits, bytes, or lines recorded on a storage device such as a disc. Everything in Unix is kept in the form of files. A programme, executable code, a file, a set of instructions, a database, a directory, or a subdirectory are all possibilities. A Unix file is a tool or application that defines the structure and format of a file.

- ◇ Unix filenames are case sensitive! So, there is a file called my data. Txt is distinct from mydata.txt, which is distinct from mydata.txt.
- ◇ Unix filenames can only contain letters, digits, and the (underscore)&. (dot) character. Other characters should be avoided at all costs. The /(slash) characters are particularly crucial since they are used to indicate subdirectories.
- ◇ Additional dots in the filename are also permissible.
- ◇ The component of the name that comes after the dot is frequently used to indicate the kind of file.
 - ◆ Text files have a file extension of.TXT.
 - ◆ Files with the extension.C contain source code written in the 'C' programming language.
 - ◆ Files with the extension.HTML are intended for use on the web.
- ◇ However, this is merely a convention, not a law imposed by the operating system. This is a solid and logical practice that you should adhere to.
- ◇ Most UNIX systems limit the length of a filename to 14 characters. It is, however, dependent on the UNIX variant utilised.

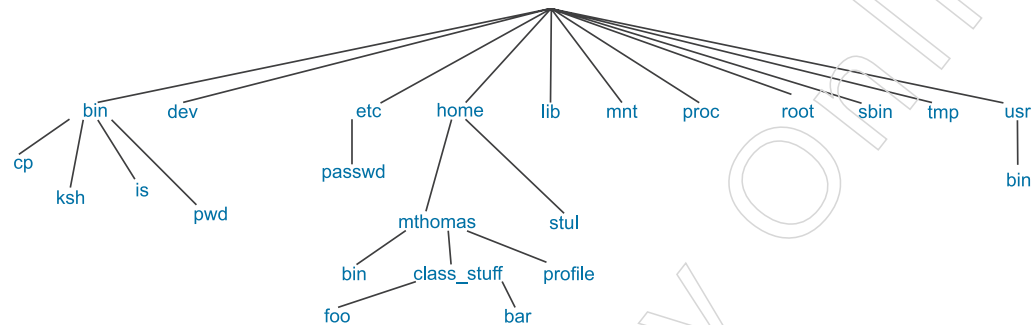
1.2.1 Unix File System

A filesystem is where all of the information on a Unix computer is held. When you start a shell login session, the shell thinks you to be anywhere in a filesystem. Although being "placed" someplace in a computer's filesystem may appear bizarre, the concept is not dissimilar to real life. You cannot just be; you have to be somewhere. The location in the filesystem tree where you are is referred to as the current working directory.

The Unix filesystem is structured in a hierarchical manner (resembling a tree structure). The root, denoted by a slash "/", is where the tree is anchored. A file or a

Notes

directory is the only two types of items in the Unix filesystem tree. A directory functions similarly to a container. A directory can house files as well as other directories. A directory that is included within another is referred to as the child of the other. A directory in the filesystem tree can have multiple children, but only one parent. A file can store data but cannot contain other files or directories.



Directories are organised in a hierarchical framework that spreads out like an upside-down tree.

The topmost directory is known as root, and it is denoted by the symbol (/).

The roots contain subdirectories, and each of them may have more subdirectories, and so on.

The Unix operating system is based on the concept of a file system, which is used to store all information. This includes the kernel of the operating system itself. There are seven different sorts of files:

1. Ordinary or Regular files
2. Directory files
3. Device files or special files
4. Hidden files
5. Pipes
6. Sockets
7. Symbolic Link

Ordinary files

These files can include text, data, or program information, but they cannot contain other files or directories. Use the underscore ('_') sign instead. It is a standard file.

There are two sorts of ordinary files.

- ◇ Text file
- ◇ Binary file

Text File: A text file contains only printable characters. It contains character lines, each of which is finished by a newline character. Text files, for example, contain C and Java program, as well as shell and Perl scripts.

Binary File: a binary file contains both printable and unprintable character (0 to 255 ASCII code). Binary files, for example, contain Unix commands, c program object code, images, sound, and video files.

Directory files

Directories are containers or folders that hold files, and other directories. Subdirectories are directories that lead to other directories.

In general, folders include files and other subdirectories, as well as link information. A directory typically has two bits of information.

- i. The file names
- ii. A unique identification numbers.

When a user adds or removes a file, the kernel updates the relevant directory by adding or removing the file's inode number and filename.

Device Files

To enable apps to easily access hardware devices. They can be used in the same way that regular files are in UNIX. Device files are typically used to access your computer's display screen, RAM, disc, input ports, and other similar devices.

In Unix, there are two kinds of device files.

- ◇ Block-oriented: These are devices that send data in blocks. For example, a hard disc.
- ◇ Character-oriented devices: Character-oriented devices that transport data byte by byte (e.g. modems, printers and networks).

Hidden files

The filename can begin with a dot character is called hidden file.

Types of users:

- a. The owner of file(user)
- b. The users who belong to the same group as the file(group)
- c. Everyone else(other)

Pipes

UNIX allows you to connect commands with a pipe. The pipe functions as a temporary file, storing data from one command until it is read by another. A Unix pipe allows data to travel in just one direction. The first command sequence's output or result is used as the input to the second command sequence. To create a pipe, place a vertical bar (|) between two commands on the command line. For example: `who | wc -l`

In long-format output of `ls -l`, named pipes are marked by the "p" symbol

Sockets

A Unix socket (also known as an inter-process communication socket) is a specific file that enables advanced inter-process communication. In a client-server application framework, a Unix Socket is employed. It is essentially a data stream, comparable to network streams (and network sockets), but all transactions are confined to the filesystem.

Unix sockets are denoted by the "s" symbol in the long-format output of `ls -l`.

Notes

Symbolic Connection

A symbolic link is used to refer to another file on the file system. Soft link is another name for symbolic link. It comprises a text representation of the path to the file it refers to. To an end user, a symbolic link appears to have its own name, but when you try to read or write data to it, it will instead refer to the file it points to. Even if we removed the soft link, the data file would remain. If we delete or relocate the source file, the symbolic file will no longer function properly.

Symbolic links are denoted by the "l" symbol in the long-format output of ls -l. (that's a lower-case L).

The Unix file system is organized as a hierarchical tree structure. The structure of a simple Unix file system divided into 4 parts.

1. The Boot blocks
2. The Super block
3. The Inode block
4. The Data blocks

1) The Boot Block:

- ◇ The boot block is positioned at the start of the file system.
 - ◇ It can be accessed using minimal code contained in the computer's ROM bios.
 - ◇ It contains the first bootstrap program that is used to load the operating system.
- It is typically a part of the disk label, a special set of blocks containing data on the disk layout.

2) The Super Block:

- ◇ It includes statistics data to maintain track of the entire file system. The super block is accessed whenever disc modification is required.

The following information is contained in the Super Block:

1. **Size of the file system:** This is the storage size of the device.
2. **List of storage blocks:** The storage space is divided up into a series of standard size blocks.
3. Number of free blocks on the file system.
4. A list of free blocks with their location.
5. Index to next free block on the list.
6. A list of free inodes.

3) The Inode Block:

- ◇ An inode is a specific kernel structure that stores information about each file in the file system.
- ◇ It contains a pointer to the disc blocks that hold the data in the file, as well as information such as file type, permission bits, the owner and group, file size, file modification, and so on.

- ◇ The name of each file is listed in the directory with which it is related. A directory is a specific type of file that contains a list of filenames and their associated inodes.
- ◇ When a user searches for a file by name, the name is looked up in the directory. Wherever the relevant inode is discovered

An inode for a file contains the following information:

- ◆ **File owner id:** It is the numeric id used in the password file to uniquely identify a user on the system.
- ◆ **Group id (GID):** This identifies a group that can be granted special access by the owner.
- ◆ **File type:** It indicates whether inode represents a file, a directory, a FIFO, character device.

File access permissions:

- ◇ **User access:** Access by the person who possesses the file.
- ◇ **Group access:** Access by the members of a stated group.
- ◇ **Other access:** The rest of the world, who are not the owner.

Types of Access:

- ◇ **Read access:** To be able to read the data stored in file.
- ◇ **Write access:** To be able to modify the data stored in file.
- ◇ **Execute access:** To be able to request that the system attempt to execute the file.

4) Data Block:

- ◇ It holds the actual data included in the files;
- ◇ These blocks adhere to the inode table and take up the majority of the storage device capacity.
- ◇ If it is assigned to one file, it cannot be assigned to another unless the two files are linked.

Advantages:

- ◇ Data in tiny files can be obtained directly from the inode.
- ◇ With one read operation highlighting the inode and another highlighting the first data block.
- ◇ Larger files can be accessed quickly.
- ◇ The disc can be completely filled with very little wasted space.

Disadvantages:

- ◇ Because inode information is maintained distinct from data, accessing data often necessitates a long search when files are first accessed.
- ◇ The original file system only employs 512-byte blocks, which is an insufficient transfer size.
- ◇ The free list quickly becomes jumbled, increasing the overhead of locating free blocks.

Notes

1.2.2 Unix Paths

What is a Path?

A path is a unique location to a file or folder in an operating system's file system. A file path is made up of / and alpha-numeric characters.

Absolute Path:

An absolute path is one that specifies the location of a file or directory relative to the root directory (/).

To write an absolute path-name:

- ◇ Begin at the root directory (/) and work your way down.
- ◇ Use a slash (/) after each directory name (last one is optional)

For example:

```
$cat abc.sql
```

will only work if the file "abc.sql" is present in your current directory. However, if this file is not in your working directory and is located somewhere else, such as /home/kt, then this command will only work if you use it as described below:

```
cat /home/kt/abc.sql
```

If the initial character of a pathname is /, the file's location must be established with regard to root in the preceding example. When there is more than one / in a pathname, you must drop one level in the file system for each such /, as in the above kt, which is one level below home and so two levels below root.

An absolute path is one that specifies the location of a file or directory relative to the root directory (/). In other words, an absolute route is a complete path beginning with the / directory and ending with the end of the physical filesystem.

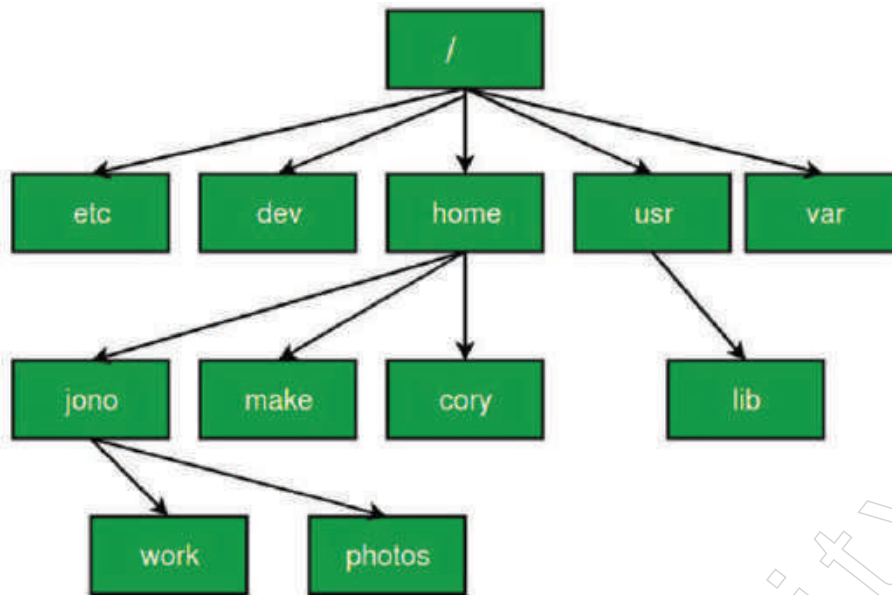
Examples of absolute path:

```
/var/ftp/pub  
/etc/samba.smb.conf  
/boot/grub/grub.conf
```

Relative Path:

The relative path is defined as the path that is directly related to the present (pwd). It always begins in your current directory and never begins with a /.

To be more detailed, consider the image below, in which if we are looking for photographs, the absolute path will be provided as /home/jono/photos, but assuming that we are already in the Jono directory, the relative path can be written as simple photos.



Using . and ... in Relative Path-names:

UNIX provides a shortcut in the form of the relative pathname, which utilises the current or parent directory as a reference and gives the path relative to it. A relative path-name contains one of the following cryptic symbols: The current directory is represented by. (a single dot). The parent directory is represented by..(two dots).

This means that if we are currently in the directory /home/kt/abc, we may use.. as a parameter to cd to move to the parent directory /home/kt as follows:

```

$pwd
/home/kt/abc
$cd ..      ***moves one level up***
$pwd
/home/kt
    
```

NOTE: Now / when used with .. has a different meaning; instead of moving down a level, it moves one level up:

```

$pwd
/home/kt/abc      ***moves two level up***
$cd ../../
$pwd
/home
    
```

Example of Absolute and Relative Path:

Changing directory with relative path concept:

```

$pwd
/home/kt
$cd abc
$pwd
/home/kt/abc
    
```


Notes

Changing directory with absolute path concept:

```
$pwd
/home/kt
$cd /home/kt/abc
$pwd
/home/kt/abc
```

Difference Between Absolute and Relative Path:

Parameter of Comparison	Absolute Path	Relative Path
By definition	specifies the location from the root directory	related to the location from current directory
Function of delimiting character	Begins with a delimiting character	Never begins with a delimiting character
Navigates to	Content from other domains	Content from the same domain
URL used	Uses absolute URL	Used relative URL
Other names	Full-path or File path	Non-absolute path

- ◇ The primary distinction between an absolute and a relative path is that an absolute route indicates the location from the root directory, whereas a relative path refers to the current directory.
- ◇ Another distinguishing feature of the two paths is that an absolute road begins with a delimiting character such as “/,” whereas a relative pathway begins with characters such as “.” but never “/” or “”.
- ◇ When a link navigates to information on another website, an absolute pathway is utilised, whereas a relative pathway is used when linking to anything on the same website.
- ◇ An absolute path is also referred to as a full pathway, whereas a relative path is referred to as a non-absolute path or a partial pathway.
- ◇ An absolute pathway employs an absolute URL to travel to pages outside of a single domain, whereas a relative pathway only navigates to pages within the same domain.

1.2.3 Inode

In UNIX operating systems, an inode is a data structure that stores vital information about files in a file system. When a file system is formed under UNIX, a certain number of inodes are also created. The inode table typically takes up roughly 1% of the entire file system disc space.

An inode is, by definition, an index node. It is used to identify a specific piece of metadata on a given filesystem. Each piece of metadata defines what we call a file. That is accurate, inodes work independently on each filesystem. When you learn that each inode is stored in a shared table, things become more complicated. In a nutshell, each filesystem mounted on your computer has its own set of inodes. An inode number can be used multiple times by the same filesystem, but never by the same filesystem

twice. The filesystem id is combined with the inode number to provide a one-of-a-kind identification label.

proc inodes

Every Linux file or directory (technically, there is not much of a distinction) has an inode, which stores all of the file's metadata (i.e., all the administrative data needed to read a file is stored in its inode).

For example, the inode holds a list of all the blocks in which a file is stored, as well as the file's owner information, permissions, and any other attributes that have been defined for the file.

Inode limits are set per filesystem and are determined at the moment the filesystem is created. The maximum directory size is determined by the filesystem, and so the precise limit varies.

Make your directories smaller for better speed by arranging files into subcategories rather than having one giant directory.

What is an inode number?

The inode number is often referred to as the index number. An inode is a one-of-a-kind number issued to files and directories when they are created. The inode number will be distinct across the entire filesystem.

On a classic Unix-style file system, such as ext3 or ext4, an inode is a data structure that stores the attributes of files and directories.

The inode table is an array of these inodes that is maintained by Linux extended filesystems such as ext3 or ext4. This table gives a list of all files in the specified filesystem. The inode number is a unique number (unique to that filesystem) assigned to each individual inode in the inode table. The following information is stored in inode:

- ◇ File type: regular file, directory, pipe etc.
- ◇ Permissions to that file: read, write, execute
- ◇ Link count: The number of hard link relative to an inode
- ◇ User ID: owner of file
- ◇ Group ID: group owner
- ◇ Size of file: or major/minor number in case of some special files
- ◇ Time stamp: access time, modification time and (inode) change time
- ◇ Attributes: immutable' for example
- ◇ Access control list: permissions for special users/groups
- ◇ Link to location of file
- ◇ Other metadata about the file

Note: The inode does not store the name of the file but its content only.

How to check inode in Linux:

If you want to have a look at inodes, on any ext. file system you can use below commands to check the properties of the file system and files that are used in it.

Notes

a. Display file data information:

You can display the inode data on a file or directory by using stat command.

You need to indicate the name of the file as following:

```
# stat hello

File: 'hello'

Size: 66 Blocks: 8 IO Block: 4096 regular file

Device: fd01h/64769d Inode: 530461 Links: 2

Access: (0774/-rwxrwxr--) Uid: ( 0/ root) Gid: ( 0/ root)

Access: 2017-05-15 20:12:32.540352591 +0000

Modify: 2017-05-15 20:12:16.901527357 +0000

Change: 2017-05-19 17:41:37.394470321 +0000

Birth: -
```

The stat output tells you the various time-stamps of the file, its ownership and permissions, and where it's stored. The file's data is kept in the disk block, which is shown in the inode's stat command output.

You can choose to list only the inode number of a file with the --format option as below:

```
# stat --format=%i hello

530461
```

b. Print index number of files:

The ls command is used to list file and directories information. The -i option with ls displays the inode number of each file. We can combine it with -l option to list information in detail

```
# ls -li

total 140984
520170 dr-xrw-rw- 2 linoadmin linoadmin  4096 Mar  9  2013 asciiquarium_1.1
263206 -rwxr-xr-x 1 linoadmin linoadmin  15436 Mar  9  2013 asciiquarium.tar.gz
519187 drwxr----- 2 root    root      4096 Apr 13 01:35 baba
258717 -rwSr--r-- 1 root    root      5747 Apr 25 01:45 bootstrap
655799 drwxr-xr-x 2 root    root      4096 May 16 17:46 course
528927 drwxr-xr-x 3 root    root      4096 Apr 29 00:29 environments
```

The first column gives the inode number. You can display a particular file's inode as below:

```
# ls -i continue.sh
519450 continue.sh
```

c. Display filesystem inode space information:

By default, df command summarizes available and used disk space. You can instead receive a report on available and used inodes by passing the -i or --inodes option.

```
# df -i
```

Filesystem	Inodes	IUsed	IFree	IUse%	Mounted on
/dev/vda1	1292800	126091	1166709	10%	/
devtmpfs	60205	319	59886	1%	/dev
tmpfs	62556	1	62555	1%	/dev/shm
tmpfs	62556	367	62189	1%	/run
/dev/vda2	128	13	115	11%	/mnt/vda2
tmpfs	62556	1	62555	1%	/run/user/0

This information can be helpful if a partition has very many small files, which can deplete available inodes sooner than they deplete available disk space.

d. List the contents of the filesystem superblock:

You can use tune2fs -l command to displays all information related to inode.

```
# tune2fs -l /dev/sda6 | grep inode
```

Filesystem features: has_journal ext_attr resize_inode dir_index filetype needs_recovery extent flex_bg sparse_super large_file huge_file uninit_bg dir_nlink extra_isize

```
Free inodes:      2224350
First inode:      11
Journal inode:    8
First orphan inode: 1575905
Journal backup:   inode blocks
```

e. Manipulate the filesystem meta data:

The debugfs command displays the contents of an inode as it exists on an Ext4 file system. To view the contents of the inode, use the stat command provided in the file system debugger. When you are finished, use exit to exit the debugfs environment.

While working in debugfs, ensure that no files on the file system may be accessed. You should think about remounting the file system with mount -o remount / yourfilesystem.

Notes

```
# debugfs /dev/sda1
debugfs 1.42.13 (17-May-2015)
debugfs: stat <19>
Inode: 19 Type: regular
Mode: 0644 Flags: 0x0
Generation: 2632480000
User: 0 Group: 0 Size: 8211957
File ACL: 0 Directory ACL: 0
Links: 1 Blockcount: 16106
Fragment: Address: 0 Number: 0 Size: 0
ctime: 0x48176267 -- Tue Apr 29 14:01:11 2008
atime: 0x485ea3e9 -- Sun Jun 22 15:11:37 2008
mtime: 0x48176267 -- Tue Apr 29 14:01:11 2008
BLOCKS:
(0-11):22749-22760, (IND):22761, (12-267):22762-23017, (DIND):23018,
(IND):23019,
```

You can use debugfs to undelete a file by using its inode and indicating a file

proc_inode

The kernel provides a data structure called `proc_inode` to support an inode-oriented view of the `proc` filesystem entries. This structure is defined as follows:

```
<proc_fs.h> union proc_op {
    int (*proc_get_link)(struct inode *, struct dentry **, struct vfsmount **); int (*proc_read)(struct
    task_struct *task, char *page);
};

struct proc_inode { struct pid *pid;
    int fd;
    union proc_op op;
    struct proc_dir_entry *pde; struct inode vfs_inode;
};
```

The structure's function is to connect the `proc`-specific data to the VFS layer's inode data. Each entry's `pde` contains a pointer to the `proc` dir entry instance; at the end of the structure is an instance of `inode`. This is the same data that the VFS layer uses for inode management. In other words, there is additional data in memory just before each instance of an `inode` structure linked with the `proc` filesystem that may be taken from a specific instance of `proc` inode via the container technique. Because the kernel needs to access this information frequently, it defines the following auxiliary procedure:

```
<proc_fs.h>

static inline struct proc_inode *PROC_I(const struct inode *inode)
{
    return container_of(inode, struct proc_inode, vfs_inode);
}
```

This returns the inode-specific data associated with a VFS inode.

Figure illustrates the situation in memory.



The remaining structural elements are only used if the inode represents a process-specific entry (and is thus located in the proc/ pid directory). Their respective meanings are as follows:

- ◇ pid is a pointer to a process's pid instance. Because a substantial quantity of process-specific information may be accessed in this manner, it is obvious that a process-specific inode should be directly associated with this data.
- ◇ proc_get_link and proc read (collected in a union because only one at a time makes sense) are used in the Virtual Filesystem to get process-specific information or to generate links to process-specific data.
- ◇ fd contains the filedescriptor for which a file in /proc/pid>/fd/ contains information. All files in this directory can use the same file operations with the help of fd.

Linux operations with files and its relation with inodes:

With the exception of the rm or mv commands, which remove (or transfer) the soft link itself, most operations (such as copy) done on soft links will affect the actual linked file.

Here are certain file operations where inodes come in handy: **a. copy files:**

When we copy a file, a new file with a new inode is created.

```
# cp myfile ..
# ls -li myfile ../myfile
2501 -rw----- 1 shyam shyam 49 Jun 20 10:10 myfile
3746 -rw----- 1 shyam shyam 49 Jun 20 10:10 ../myfile
```

b. move files:

When migrating across filesystems, the mv command works similarly to the cp command above, except that the original file is deleted from its position. However, while migrating inside a filesystem, the inode does not change; just the inode's directory mapping changes; the actual data on the hard disc (file contents) does not change.

Notes

```
# ls -li samplefile.txt
2497 -rw----- 1 shyam shyam 49 Jun 20 10:10 samplefile.txt
```

Now let's move the file and check the result:

```
# mv samplefile.txt ..
# ls -li ../samplefile.txt
2497 -rw----- 1 shyam shyam 49 Jun 20 10:12 ../samplefile.txt
```

c. remove files:

When rm command is issued, first it checks the link count of the file. If the link count is greater than 1, then it removes that directory entry and decreases the link count. Still, data is present, nor is the inode affected. And when link count is 1, the inode is deleted from the inode table, inode number becomes free, and the data blocks that this file was occupying are added to the free data block list.

```
# ls -li myfile myfile.hardlink

2501 -rw----- 2 shyam shyam 49 Jun 20 10:12 myfile
2501 -rw----- 2 shyam shyam 49 Jun 20 10:12 myfile.hardlink
```

Let's delete the file and check the result.

```
# rm myfile.hardlink
#ls -li myfile
2501 -rw----- 1 shyam shyam 49 Jun 20 10:12 myfile
```

You can see that the number of inode is decreased.

Check your Understanding

A-Fill in the Blanks

1. _____ files can include text, data, or program information, but they cannot contain other files or directories. Use the underscore ('_') sign instead. It is a standard file.
2. The _____ functions as a temporary file, storing data from one command until it is read by another.
3. A _____ file contains only printable characters. It contains character lines, each of which is finished by a newline character.
4. UNIX provides a shortcut in the form of the relative pathname, which utilizes the current or _____ directory as a reference and gives the path relative to it.
5. The inode table is an array of these inodes that is maintained by _____ extended filesystems such as ext3 or ext4.

B-State True and False

1. Directories are containers or folders that hold files, and other directories. Subdirectories are directories that lead to individual directories.
2. A Unix pipe allows data to travel in just one direction. The command sequence's output.
3. An absolute path is one that specifies the location of a file or directory relative to the root directory.
4. In UNIX operating systems, an inode is a data structure that stores vital information about files in a file system.
5. On a classic Unix-style file system, such as ext3 or ext4, an inode is a data structure that stores the attributes of files and directories.

Summary

- The Unix filesystem is structured in a hierarchical manner (resembling a tree structure). The root, denoted by a slash "/", is where the tree is anchored. A file or a directory is the only two types of items in the Unix filesystem tree. A directory functions similarly to a container. A directory can house files as well as other directories.
- These files can include text, data, or program information, but they cannot contain other files or directories. Use the underscore ('_') sign instead. It is a standard file.
- Text File: A text file contains only printable characters. It contains character lines, each of which is finished by a newline character. Text files, for example, contain C and Java program, as well as shell and Perl scripts.
- Directories are containers or folders that hold files, and other directories. Subdirectories are directories that lead to other directories.
- They can be used in the same way that regular files are in UNIX. Device files are typically used to access your computer's display screen, RAM, disc, input ports, and other similar devices.
- UNIX allows you to connect commands with a pipe. The pipe functions as a temporary file, storing data from one command until it is read by another. A Unix pipe allows data to travel in just one direction. The first command sequence's output or result is used as the input to the second command sequence.
- A Unix socket (also known as an inter-process communication socket) is a specific file that enables advanced inter-process communication. In a client-server application framework, a Unix Socket is employed. It is essentially a data stream, comparable to network streams (and network sockets), but all transactions are confined to the filesystem.
- A symbolic link is used to refer to another file on the file system. Soft link is another name for symbolic link. It comprises a text representation of the path to the file it refers to. To an end user, a symbolic link appears to have its own name, but when you try to read or write data to it, it will instead refer to the file it points to.
- It contains a pointer to the disc blocks that hold the data in the file, as well as information such as file type, permission bits, the owner and group, file size, file modification, and so on.

Notes

- If the initial character of a pathname is /, the file's location must be established with regard to root in the preceding example. When there is more than one / in a pathname, you must drop one level in the file system for each such /, as in the above kt, which is one level below home and so two levels below root.
- UNIX provides a shortcut in the form of the relative pathname, which utilizes the current or parent directory as a reference and gives the path relative to it. A relative path-name contains one of the following cryptic symbols: The current directory is represented by. (a single dot). The parent directory is represented by... (two dots).
- In UNIX operating systems, an inode is a data structure that stores vital information about files in a file system. When a file system is formed under UNIX, a certain number of inodes are also created. The inode table typically takes up roughly 1% of the entire file system disc space.
- The inode number is often referred to as the index number. An inode is a one-of-a-kind number issued to files and directories when they are created. The inode number will be distinct across the entire filesystem.

Activity

1. Prepare a handmade worksheet on UNIX Paths

Questions & Exercise

1. What are links and symbolic links in UNIX file system?
2. How do you create special files like named pipes and device files?
3. Explain the mount and unmount system calls.
4. Define 'inode'.
5. Explain about the directory representation in UNIX.
6. What is a FIFO?

Glossary

- **Interrupt** - Sending a signal to a process to terminate the process with varying degrees of severity. Some signals terminate processes more severely than others.
- **Exec** - A system function that overlays the current process image with a new process image, assigning the new process the PID of the original process.
- **Standard Error** - The default destination from where a command delivers its errors, typically the monitor (as does stdout). Also referred to as stderr.
- **Environment Variable** - A named variable used within the shell to store a value. Similar to variables in other programming languages. Environment variables can be system or user defined.
- **Foreground** - Executing a program such that its parent (usually the shell) waits for its completion before resuming control. Typically, only one program can run in the foreground in any one terminal window.
- **Preemptive**- Describes an operating system where the task-switching operation can occur at any point. The kernel can preempt the operation of any process in

order to switch tasks. The process need not relinquish control or signify in any way that it is ready to be switched.

- **Virtual Memory**- Logically-assigned memory that may or may not exist physically. Through the use of paging and the swap area, more memory can be referenced and allocated than actually exists on the system.
- **Core Dump**- An image of a terminated process saved for debugging. A core file is created under the name core in the current directory of the process when an abnormal event occurs that results in the process's termination.

Further Reading

1. Rosen and Kenneth: Unix: The Complete Reference, McGraw Hill, 2007 Edition.
2. Yashavant P. Kanetkar: Unix Shell Programming, BPB Publications, 2003 Edition.

Check Your Understanding- Answer

Fill in the Blanks

1. Ordinary Files
2. Pipe
3. Text
4. Parent
5. Linux

State True and False

1. False
2. False
3. True
4. True
5. True

Notes

Unit-1.3: Linux

Objectives:

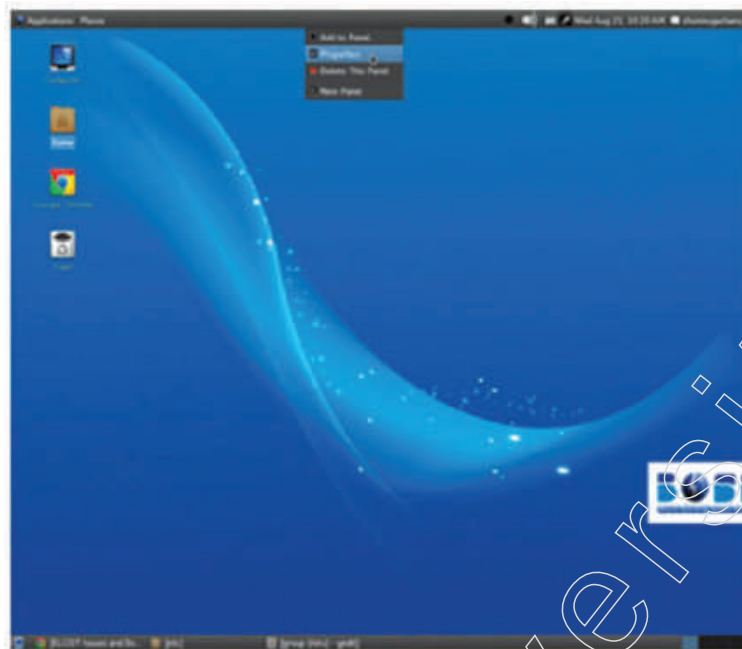
At the end of this unit, you will be able to:

- Define Linux
- Explain User Interface
- Explain Character User Interface
- Explain Graphic User Interface
- Learn Difference between GUI and CUI
- Explain LINUX Booting Process

Introduction

Linux is available from a variety of businesses and in a variety of versions. A corporation will frequently create its own version with unique upgrades or adjustments. These are then commercially released and referred to as distributions. Although Linux officially only consists of the kernel, it is widely seen as including all associated programs and utilities. A commercial distribution is made up of the kernel, utilities, and sometimes some applications. Linus Torvalds designed the kernel for Linux as a student at the University of Helsinki. In general, most people understand what Linux is. To imply that Linux is an operating system implies that it is intended to be used as a replacement for existing operating systems such as Windows, Mac OS, MS-DOS, Solaris, and others. Linux is not a single software, such as a word processor, or a collection of programs, such as an office suite. Linux is a software interface that connects computer/server hardware with the programs that run on it. Linus Torvalds used a version of the UNIX operating system named 'Minx' while studying at the University of Helsinki. Linus and other users sent Minx's developer, Andrew Tanenbaum, with recommendations for changes and enhancements, but he felt they were unnecessary. That is when Linus decided to design his own operating system, one that would incorporate user feedback and suggestions for enhancements. In its most basic form, Linux is an operating system. It is the software on a computer that allows applications and the computer user to access the computer's devices and perform specific functions. The operating system (OS) delivers instructions from an application to the computer's CPU, for example. The processor executes the work at hand and then returns the results to the application via the operating system. In these regards, Linux is extremely comparable to other operating systems like Windows and Mac OS X. Linux is created collaboratively as an open operating system, which means that no single business is fully responsible for its development or continuous support. Companies in the Linux economy pool their research and development costs with their partners and competitors. This distribution of development costs across individuals and businesses has resulted in a big and efficient ecosystem, as well as unforeseen software innovation. Linux is already popular on a wide range of devices, but there are many technological areas in which Linux is progressing, even while desktop and server development continues to outpace that of any other operating system today. Linux is being placed in the system BIOS of laptop and notebook computers,

allowing customers to turn on their devices in seconds and boot into a simplified Linux environment. This environment will include Internet connectivity capabilities such as a web browser and an e-mail client, allowing users to operate on the Internet without having to boot their device all the way into its principal operating system—even if that operating system is Windows.



Simultaneously, Linux is becoming available on mobile Internet devices (MIDs). This includes embedded devices like cellphones and PDAs, as well as netbooks, which are small laptop-type machines that offer the same essential functionality as their bigger counterparts but in a smaller, more energy-efficient design. Cloud computing's rise is a natural fit for Linux, which already powers many of the Internet's web servers. Linux enables cloud services like Amazon's A3 to perform more efficiently in delivering online apps and information to customers.

The well-known success of Linux on supercomputers, both in the high-performance computing (HPC) and high-availability (HA) areas, where academic research in physics and bioengineering, as well as firms in the financial and energy industries, require reliable and scalable computing power to achieve their goals, is related to Linux's growth in cloud computing. Many popular Web 2.0 services, such as Twitter, Linked In, YouTube, and Google, all use Linux as their operating system. As new web services become available in the future, Linux will become the platform that powers these new technologies.

1.3.1 Two User Interfaces –Character User Interface (CUI) and Graphic User Interface (GUI)

The software that allows a person to communicate with a computer is referred to as a user interface, or UI. The user interface (UI) defines how the system looks and feels, as well as how simply and efficiently it may be used.

The phrase “Windows” refers to a certain proprietary operating system and includes all of its characteristics, including the API and the user interface. When most people think of Windows, they envision the Start menu, the Control Panel, and so on.

Notes

Notes

Similarly, the term “Macintosh” refers to a specific product and conjures up thoughts of the “Dock” and a menu bar at the top of the screen rather than attached to a window.

The name “Unix,” on the other hand, indicates an API but not a specific user interface. There are numerous graphical user interfaces (GUIs) available for Unix systems. In fact, a Unix-based machine can have multiple UIs installed, and each user can select the one they prefer when they log in.

User Interfaces are classified into two types: GUI and CUI. GUI stands for Graphical User Interface.

CUI is for Character User Interface while UUI stands for User Interface.

Graphical User Interfaces (GUIs):

A graphical user interface (GUI) is a sort of menu-driven interface in which the menu items are graphical icons. Some menu systems are nothing more than text.

While modern Unix systems include graphical user interfaces (GUIs), much work is still done via the command line, or shell.

Menu-driven systems are considerably easier to use while you are new to the system or use it seldom, but they can become difficult when used on a regular basis. An ATM (automated teller machine) with a command-driven interface, for example, would almost certainly be disliked among banking customers.

When a user requires access to dozens or hundreds of features, they cannot all be presented as icons on the screen at the same time. As a result, you will have to browse through numerous levels of menus or screens to obtain the functionality you require. Even if they could be presented all at once, finding the one you want would be a headache in such a cluttered environment.

Because of these constraints, most graphical user interfaces (GUIs) provide hot keys, which are special key combinations that can be used to access certain functionality without having to navigate menus. Hot keys are frequently displayed in menus with the capabilities that they enable. On Mac OS X, for example, Command+q can be used to end most graphical applications.

In a menu-driven system, tasks are also difficult to automate. Some systems have this capability, although the majority do not, and the manner of automation varies each system.

In general, open source developers lack the time and technical abilities required to create and maintain a sophisticated graphical user interface.

Character User Interface (CUI):

CUI means that you must use a keyboard to type commands in order to communicate with the computer. To provide orders to the computer, you can only write text, just like in MS-DOS or command prompt. There are no visuals or graphics on the screen, and the interface is basic. Computers had to be operated through this interface at first, and users who have seen it say it was a black screen with white lettering only. There was no need for a mouse back then because CUI did not support the use of pointer devices. CUIs have gradually become obsolete, with more complex GUIs supplanting them. Even the most contemporary computers, however, include a modified version of CUI known as CLI (Command Line Interface).

Difference between GUI and CUI:

- ◇ CUI and GUI are computer user interfaces.
- ◇ CUI is the antecedent to GUI and stands for character user interface, where the user must type on the keyboard to advance. GUI, on the other hand, stands for Graphical User Interface and allows you to use a mouse instead of a keyboard.
- ◇ GUI is considerably easier to browse than CUI.
- ◇ In the case of CUI, there is only text, whereas in the case of GUI, there are graphics and other visual cues.
- ◇ The majority of current computers employ graphical user interfaces (GUIs) rather than command-line interfaces (CLI).
- ◇ DOS is an example of a command-line interface (CUI), whereas Windows is an example of a graphical user interface (GUI).
- ◇ We can use both the keyboard and the mouse (or any other pointing device) on the GUI, but only the keyboard can be used with the CUI.
- ◇ While GUI requires a lot of memory to function properly, CUI can run with less memory than GUI.
- ◇ Because GUI is very configurable, you can modify your appearance experience; nevertheless, the main difference between GUI and CUI is that CUI Appearance cannot be changed.
- ◇ In terms of performance, CUI outperforms GUI.

Disadvantages of CUI:

- ◇ Interaction: CUI interaction is only possible with the help of instructions (only text).
- ◇ Ease of Operation: Handling the task on CUI-based systems was far more complicated and required knowledge.
- ◇ Flexibility: Because it is a command-based user interface, the flexibility was on the low side.
- ◇ Customizing Appearance: as it is not a graphics grounds user interface the Appearance cannot be altered.

Disadvantages of GUI:

- ◇ The graphical user interface is not resource optimised. The goal is to create a user-friendly interface, so it consumes more computer memory.
- ◇ Applications based on the graphical user interface (GUI) require more RAM to run than those based on other interface types.
- ◇ To find secret commands, users must consult the Help file.
- ◇ When compared to other types of interfaces, GUI consumes greater processing power.

1.3.2 LINUX Booting Process

Understanding the Linux boot and startup processes is essential for configuring Linux and troubleshooting startup issues. This section provides a summary of the

Notes

bootup sequence with the GRUB2 bootloader and the startup sequence with the systemd initialization system.

In reality, there are two sequences of actions that must occur in order for a Linux computer to boot and become usable: boot and startup. When you switch on your computer, the boot sequence begins and ends when the kernel is initialised and systemd is activated. The startup procedure then takes over and completes the task of bringing the Linux computer up to speed.

Overall, the Linux boot and starting process is straightforward. It consists of the following steps, which will be explained in further detail in the following sections.

BIOS	Basic input / output system executes MBR
MBR	Master Boot Record executes GRUB
GRUB	Grand Unified Bootloader executes Kernel
Kernel	Kernel executes the /sbin/init program
Init	Init executes Runlevel programs
Runlevel	Runlevel programs are executed from /etc/rc.d/rc*.d/

Stage 1: BIOS:

BIOS is an abbreviation for Basic Input/Output System. BIOS is responsible for searching for and executing boot loader (GRUB, LILO) programmes.

BIOS boot loader search into a floppy, CD-rom, or hard disc During BIOS starting, you can alter the boot sequence from BIOS setup.

BIOS grants control to the boot loader programme after it has been discovered and loaded into memory. As a result, the BIOS loads and executes the MBR boot loader.

Stage 2: MBR:

MBR is an abbreviation for Master Boot Record.

MBR is found in the first sector of the bootable drive. In most cases, /dev/had or /dev/sda.

MBR is 512 bytes in size and consists of three components.

- ◇ The primary boot loader information is stored in the first 446 bytes.
- ◇ Partition table data is stored in the next 64 bytes.
- ◇ The MBR validation check is stored in the last two bytes.

MBR contains information about any boot loader; in our example, it contains information about the GRUB boot loader.

In short MBR loads and executes the GRUB boot loader.

Stage 3: GRUB:

GRUB is an abbreviation for Grand Unified Bootloader.

It saves all information about the operating system image that will be loaded and executed.

If you have more than one operating system, all of them will be listed in this GRUB file, and you can choose which one to use as the default.

On system boot, GRUB displays a splash screen and waits a few seconds for user input to select an operating system; if you do not type anything, it loads the default kernel image defined in the grub configuration file.

Grub is familiar with your operating system's filesystem.

The GRUB configuration file can be found in "/boot/grub/grub.conf," a sample of which is shown below:

```
#boot=/dev/sda
default=0
timeout=5
splashimage=(hd0,0)/boot/grub/splash.xpm.gz
hiddenmenu
title CentOS (7.2.18-324.el5PAE)
    root (hd0,0)
    kernel /boot/vmlinuz-7.2.18-324.el5PAE ro root=LABEL=/
    initrd /boot/initrd-7.2.18-324.el5PAE.img
```

As you can see in the file, it has information of kernel and initrd image. In simple term GRUB loads and executes Kernel and initrd images.

Stage 4: Kernel:

When kernel loads, it mounts the root file system and execute /sbin/init program. As init was the first program run by the Linux kernel, it has one(1) as process id (PID), which you can check by using below command:

```
# ps -ef | grep init
```

The initrd stands for Initial RAM Disk.

Stage 5: Init:

It checks file "/etc/inittab" to decide the Linux run level.

There are seven (7) run levels available with the Linux operating system:

- ◇ 0 - halt
- ◇ 1 - single-user mode
- ◇ 2 - Multiuser, without NFS
- ◇ 3 - Full multiuser mode

Notes

- ◇ 4 - unused
- ◇ 5 - X11
- ◇ 6 - reboot

Init will check and identifies the default initlevel from file “/etc/inittab” to load all appropriate program required for the run level.

You can check your system's configured default run level by executing below command:

```
$ grep initdefault /etc/inittab
```

Stage 6: Runlevel:

One default run level identified; it will execute all required program for that run level.

The system will check and execute run level programs from the following directories:

- ◇ Run level 0 – /etc/rc.d/rc0.d/
- ◇ Run level 1 – /etc/rc.d/rc1.d/
- ◇ Run level 2 – /etc/rc.d/rc2.d/
- ◇ Run level 3 – /etc/rc.d/rc3.d/
- ◇ Run level 4 – /etc/rc.d/rc4.d/
- ◇ Run level 5 – /etc/rc.d/rc5.d/
- ◇ Run level 6 – /etc/rc.d/rc6.d/

There is symbolic linked directory also available in “/etc” directory, /etc/rc0.d is linked to /etc/rc.d/rc0.d and so on.

You can check all program under the directory /etc/rc.d/rc*.d/ which is starting with S and K.

The program which starts with S used during startup. S stands for the startup.

The program which starts with K used during shutdown. K stands for Kill.

You can see a number in the name after S or K which are the sequence number in which programs should be started or killed.

1.3.3 LINUX Graphical Environment, LINUX Desktop Environment – GNOME and KDE

The average user may not be concerned with login settings, but Linux provides a plethora of flashy window and desktop managers for use with X, the graphical environment. Window managers and desktops are simple to use and configure, and may even resemble the normal MS Windows, Apple, or UNIX CDE environment, however many Linux users prefer flashier desktops and fancier window managers.

- ◇ On both monochrome and colour displays, X enables overlapping hierarchical sub-windows as well as text and graphics activities. The X server is used by a huge number of X client programs. Among the programs included in the core X Consortium release are:
- ◇ **xterm**: a terminal emulator

- ◇ **twm:** a minimalistic window manager
- ◇ **xdm:** a display manager
- ◇ **xconsole:** a console redirect program
- ◇ **bitmap:** a bitmap editor
- ◇ **xauth, xhost and iceauth:** access control programs
- ◇ **xset, xmodmap and many others:** user preference setting programs
- ◇ **xclock:** a clock
- ◇ **xlsfonts and others:** a font displayer, utilities for listing information about fonts, windows and displays
- ◇ **xfs:** a font server

Display names:

From the user's perspective, every X server has a display name in the form of:

hostname: displaynumber. screennumber

The application uses this information to determine how to connect to the X server and which screen to utilise by default (on displays with multiple monitors):

- ◇ **hostname:** The name of the client machine to which the display is physically attached is specified by the hostname. If no host name is specified, the most efficient method of interacting with a server on the same machine is utilised.
- ◇ **displaynumber:** The term "display" normally refers to a group of monitors that share a similar keyboard and pointer (mouse, tablet, etc.). Most workstations have only one keyboard and, as a result, only one display. Larger, multi-user systems, on the other hand, usually include many displays so that more than one person may work on graphics at the same time. To avoid confusion, when the X server for that display is started, each display on the workstation is issued a display number (starting at 0). A display name must always include the display number.
- ◇ **screen number:** Some displays share a single keyboard and pointer among two or more monitors. Because each monitor has its own set of windows, when the X server for that display is started, each screen is issued a screen number (starting at 0). If no screen number is specified, screen 0 is utilised.

The default display name on POSIX systems is kept in the DISPLAY environment variable. The xterm terminal emulator automatically sets this variable. However, if you log into another system on a network, you may need to manually configure DISPLAY to point to your display.

Window and desktop managers:

Window managers are programs that regulate the layout of windows on the screen. Although many window managers will honour supplied geometry parameters, some may choose to ignore them (for example, requiring the user to explicitly draw the window's region on the screen with the pointer).

Because window managers are typical (though complicated) client applications, a wide range of user interfaces can be created. The X Consortium distribution includes a

Notes

window manager called twm, although most users prefer something more sophisticated when system resources allow. Sawfish and Enlightenment are two prominent examples that allow each user to customise their desktop based on their mood and style.

A desktop manager makes use of one or more window managers to arrange your graphical desktop in a user-friendly manner, including menu bars, drop-down menus, informational messages, a clock, a program manager, a file manager, and so on. Gnome and KDE are two of the most popular desktop managers, and they both operate on practically every Linux distribution and many other UNIX systems.

LINUX Desktop Environment –GNOME and KDE:

The desktop environment is the appearance and feel of an operating system's graphical user interface (GUI). It has no counterpart in the worlds of OS X and Windows, as those operating systems each have only one desktop environment. Distinct versions of OS X and Windows may have different styles and interfaces, but there is no way to get the Windows 7 interface on Windows 10.

As previously stated, Linux is a kernel. It manages the operating system, but the desktop environment is how your machine looks and feels. It governs elements like how windows are presented, as well as minor details like where the date and time are displayed on the interface.

GNOME Desktop Environment:

GNOME is a popular desktop environment that is used by many distributions as the default. GNOME 3 is the current version, which differs from GNOME 2, which appeared and performed similarly to Windows XP.

GNOME, like most Linux desktop environments and OS X, supports multiple desktops, allowing users to have various apps open in different desktops, almost as if they were working on multiple PCs, but all on a single monitor. This type of functionality is well suited to desktop work, as users often switch between graphical apps. Some people use Alt-Tab to switch between windows and applications, however multiple desktops allow a word processor to be open in one desktop and a web browser and PDF reader to be open in another, with the user flipping between desktops rather than programs. This approach enables some apps to run full-screen in one workspace while others share the same window in another.

Many computer users rely heavily on their desktop to store files and icons. Although GNOME provides a desktop file path, the desktop does not display files by default. Some users are perplexed by this. Others like the clean appearance it gives their machine. Files saved on the desktop are always accessible via the file manager, but users who are accustomed to browsing to their desktop to open files and folders may need to modify this default by running GNOME Tweak Tool, a tool that allows you to easily adjust specific GNOME settings from a GUI.

While GNOME is primarily a desktop environment, the project's developers are also working on products to complement the desktop. Among these projects are gedit, a text editor, Epiphany, a web browser, and Notes, a note-taking application. These programs are designed to fit with the GNOME look and feel and operate across desktop environments.

GNOME feels different than a typical desktop, but if you get used to the idea behind it—that you are not so much looking through menus as you are calling programs and files—you might find it difficult to go back to a more traditional desktop.

The other prominent Linux desktop environment is KDE, which stands for the K Desktop Environment. The desktop is technically KDE Plasma, with KDE being the bigger project that generates apps in addition to Plasma. However, most people simply refer to KDE Plasma as KDE.

KDE has traditional menus as well as a launcher. Most people should be able to navigate the UI with ease. What distinguishes and popularises KDE is that almost every aspect of it can be customised to look exactly as you want it to.

Power users will most likely be interested in KDE's Activities capability, which allows you to configure separate desktops for different types of jobs. So, for example, you could have a writing activity that contains desktop shortcuts to a word editor and a citation manager. A research activity with shortcuts to a web browser and a PDF reader may also be included. A data worker may have a data activity with an open data folder and a shortcut to the R statistical tool.

Activities are similar to virtual desktops, except they include software and folders that have been pre-selected based on how the desktop is used. The concept appears to be overkill for most users, particularly less skilled ones who likely spend the most of their computer time in a web browser and possibly an email client. However, activities have a lot of potential for anyone wishing to completely customise their desktop experience, especially if their desktop work is sophisticated.

Linux programs are compatible with all desktop environments. However, KDE has its own set of programmes, all of which begin with the letter K. So, Kate is the default text editor in KDE, while KRunner is the default programme launcher in KDE. Konqueror, KDE's web browser, is even available. This is not to suggest that any of those programmes will not work with GNOME. It simply means that KDE has its own software ecosystem.

Some users, especially new ones, test KDE and are immediately enamoured with all of the configuration choices and are willing to invest time and work perfecting their desktop. Others find the procedure vexing, wishing that so many decisions were not left to the user. It is the polar opposite of GNOME in that it is customisable, but not in an evident sense. KDE may appear to some to be extremely configurable. However, there may be no better desktop environment for those who desire a more personal, concentrated desktop experience.

Check your Understanding

A-Fill in the Blanks

1. The _____ executes the work at hand and then returns the results to the application via the operating system.
2. _____ refers to a certain proprietary operating system and includes all of its characteristics, including the API and the user interface.
3. CUI means that you must use a keyboard to type _____ in order to communicate with the computer.

Notes

4. When you switch on your _____, the boot sequence begins and ends when the kernel is initialized and system is activated.
5. A _____ makes use of one or more window managers to arrange your graphical desktop in a user-friendly manner, including menu bars, drop-down menus, so on.

B-State True and False

1. When a user requires access to dozens or hundreds of features, they can all be presented as icons on the screen at the same time.
2. GNOME is a popular desktop environment that is used by many distributions as the default.
3. The desktop is technically KDE Plasma, with KDE being the smaller project that generates apps in addition to Plasma.
4. There may be a better desktop environment for those who desire a more personal, concentrated desktop experience.
5. The software that allows a person to communicate with a computer is referred to as a user interface, or UI.

Summary

- Linux is available from a variety of businesses and in a variety of versions. A corporation will frequently create its own version with unique upgrades or adjustments. These are then commercially released and referred to as distributions. Although Linux officially only consists of the kernel, it is widely seen as including all associated programs and utilities.
- Companies in the Linux economy pool their research and development costs with their partners and competitors. This distribution of development costs across individuals and businesses has resulted in a big and efficient ecosystem, as well as unforeseen software innovation. Linux is already popular on a wide range of devices, but there are many technological areas in which Linux is progressing, even while desktop and server development continues to outpace that of any other operating system today.
- The well-known success of Linux on supercomputers, both in the high-performance computing (HPC) and high-availability (HA) areas, where academic research in physics and bioengineering, as well as firms in the financial and energy industries, require reliable and scalable computing power to achieve their goals, is related to Linux's growth in cloud computing.
- The software that allows a person to communicate with a computer is referred to as a user interface, or UI. The user interface (UI) defines how the system looks and feels, as well as how simply and efficiently it may be used.
- The phrase "Windows" refers to a certain proprietary operating system and includes all of its characteristics, including the API and the user interface. When most people think of Windows, they envision the Start menu, the Control Panel, and so on.

- A graphical user interface (GUI) is a sort of menu-driven interface in which the menu items are graphical icons. Some menu systems are nothing more than text. While modern Unix systems include graphical user interfaces (GUIs), much work is still done via the command line, or shell.
- CUI means that you must use a keyboard to type commands in order to communicate with the computer. To provide orders to the computer, you can only write text, just like in MS-DOS or command prompt. There are no visuals or graphics on the screen, and the interface is basic. Computers had to be operated through this interface at first, and users who have seen it say it was a black screen with white lettering only.
- In reality, there are two sequences of actions that must occur in order for a Linux computer to boot and become usable: boot and startup. When you switch on your computer, the boot sequence begins and ends when the kernel is initialised and systemd is activated. The startup procedure then takes over and completes the task of bringing the Linux computer up to speed.
- The average user may not be concerned with login settings, but Linux provides a plethora of flashy window and desktop managers for use with X, the graphical environment. Window managers and desktops are simple to use and configure, and may even resemble the normal MS Windows, Apple, or UNIX CDE environment.
- Window managers are programs that regulate the layout of windows on the screen. Although many window managers will honour supplied geometry parameters, some may choose to ignore them (for example, requiring the user to explicitly draw the window's region on the screen with the pointer).
- The desktop environment is the appearance and feel of an operating system's graphical user interface (GUI). It has no counterpart in the worlds of OS X and Windows, as those operating systems each have only one desktop environment. Distinct versions of OS X and Windows may have different styles and interfaces, but there is no way to get the Windows 7 interface on Windows 10.
- GNOME, like most Linux desktop environments and OS X, supports multiple desktops, allowing users to have various apps open in different desktops, almost as if they were working on multiple PCs, but all on a single monitor.
- While GNOME is primarily a desktop environment, the project's developers are also working on products to complement the desktop. Among these projects are gedit, a text editor, Epiphany, a web browser, and Notes, a note-taking application. These programs are designed to fit with the GNOME look and feel and operate across desktop environments.
- Power users will most likely be interested in KDE's Activities capability, which allows you to configure separate desktops for different types of jobs. So, for example, you could have a writing activity that contains desktop shortcuts to a word editor and a citation manager. A research activity with shortcuts to a web browser and a PDF reader may also be included. A data worker may have a data activity with an open data folder and a shortcut to the R statistical tool.

Activity

1. Built a Clustering Server with Linux.

Notes

2. Create a Smart Magic Mirror.

Questions & Exercise

1. Define Linux?
2. What is Linux Kernel?
3. Which are the Shells used in Linux?
4. Explain file permission in Linux.
5. What is LINUX Booting Process?
6. What is the difference between CUI and GUI?

Glossary

- **AIX-** IBM's proprietary form of Unix that runs on the company's proprietary hardware, as well as commodity hardware based around AMD and Intel processors. Nowadays, IBM is slowly deprecating AIX in favor of Linux.
- **Closed Source-** The reverse of Open Source in which the source code is not available for others to see, share, or modify.
- **Dependency-** A way of referring to system files that a program requires in order to run. If the dependencies are not present during program installation, a program might refuse to install.
- **GNOME-** GNU Network Object Model Environment; a GUI-based desktop environment used by Ubuntu, as well as several other distributions. It uses the GTK+ libraries. See also KDE.
- **Hacker-** Term used within the community to describe anyone who enjoys computers and possesses some skill therein, either in a professional capacity or as a hobby. This term is distinct from connotations of maliciously breaking into computers propagated by the media.
- **Open Source-** Method and philosophy of developing software whereby the source code—the original listing created by the programmer—is available for all to see. Note that open source is not the same as Free Software; describing software as open source doesn't imply that the code can be shared or used by others (although this is often the case).
- **Apache-** Popular Open-Source web server software that runs on Unix, Linux, and other operating system platforms. Considered responsible in part for the rise in popularity of Linux in the late 1990s.
- **Command-** Input typed at the shell that performs a specific task, usually related to administration of the system and/or the manipulation of files.

Further Reading

1. Christopher Negus: Linux Bible, Wiley, 2020 Edition.
2. Daniel P. Bovet, Marco Cesati: Understanding the Linux Kernel, O'Reilly Media, 2005 Edition.

Check Your Understanding- Answer

Fill in the Blanks

1. Processor
2. Windows
3. Commands
4. Computer
5. Desktop Manager

State True and False

1. False
2. True
3. True
4. False
5. True

Notes

Notes

Module-II: Processes

Structure:

- 2.1 Process Basics
 - 2.1.1 Process state and transitions
 - 2.1.2 The context of a process
- 2.2 Running Processes
 - 2.2.1 Foreground Processes
 - 2.2.2 Background processes
 - 2.2.3 suspend and interrupt a process and kill A process
- 2.3 Process Commands
 - 2.3.1 The ps command
 - 2.3.2 The batch command
 - 2.3.3 The at command

Unit-2.1: Process Basics

Notes

Objectives:

At the end of this Unit, the reader will be able to:

- Understand what is process
- Discuss process state and transitions
- Talk about the process Life Cycle
- Comprehend the context of a process

Introduction

Processes are responsible for carrying out tasks within the operating system. A programme is a collection of machine code instructions and data that is saved in an executable image on disc and is thus a passive entity; a process is a computer programme in operation. Processes are a critical component in the UNIX universe. Essentially, practically every programme you run is running in a process.

Although you may not need to engage with them all of the time, you must rely on them to get anything done in a UNIX system.

What is a process?

Although this is not the official definition of a process, I like to think of it as a container. This container has a running programme (for example, vim), a collection of metadata attributes that characterise the programme (who is running it, what its id is, and so on), and it can receive and send messages to other containers.

A more formal definition is “an instance of a computer programme that is being executed,” as defined by Wikipedia.

2.1.1 Process State and Transitions

A process is a running instance of a programme. A programme is made up of a collection of procedures that work together to form a whole.

In Unix, processes are classified into two types:

- ◇ User processes: They are performed in user mode.
- ◇ Kernel processes: They run in kernel mode.

Process States

Process states are the states that a Process enters when operating from start to finish. These are as follows:

- 1) **New State:** When a user requests a Service from the System, the System will first begin the process, also known as an initial Process. As a result, the New Born Process refers to any new operation that is requested by the system.
- 2) **Running State:** When a process is running under the CPU, or when a programme is being executed by the CPU, this is referred to as the Running

Notes

state, and when a process is running, it will also display certain outputs on the screen.

- 3) **Waiting:** When a process is waiting for certain input or output operations, it is said to be in the Waiting State. And in this case, the process is not being executed; instead, it is being stored in memory, and when the user provides input, it will return to the ready state.
- 4) **Ready State:** This is the state in which the process is ready to execute but is waiting for the CPU to execute. After the inputs and outputs have been completed, the process will be in the Ready state, which means it will wait for the Processor to execute.
- 5) **Terminated State:** The CPU will automatically terminate the process after it has completed. As a result, this is also known as the Process's Terminated State. The Processor will deallocate the Memory that was allocated to the Process after it has completed the entire process. As a result, this is referred to as the Terminated Process.

This is not accurate, as we know there are several processes running at the same time. A processor can only run one process at a time. There are numerous Process States that determine which Process will be executed. The Processor will execute all processes based on their states; processes in the Waiting State will not be performed, and the CPU will split his time for execution if there are many processes ready to execute.

When a process transitions from one state to another, this is referred to as a process state transition. In this case, a running process may enter the Wait State, whereas a ready process may enter the Wait State, and the Wait State may enter the Running State.

A process is not always ready to be started. It must occasionally wait for events from other sources beyond its control, such as keyboard input in a text editor. The procedure cannot run until the event occurs.

Process Life Cycle

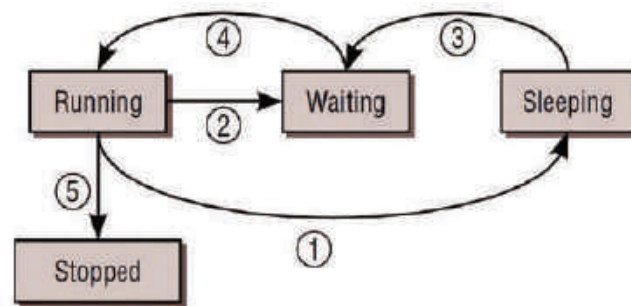
When switching between tasks, the scheduler must be aware of the status of every process in the system; it makes no sense to devote CPU time to processes that are idle. Transitions between individual process states are equally important. For example, if a process is waiting for data from a peripheral device, it is the scheduler's responsibility to change the process's state from waiting to runnable once the data arrives.

A process can be in one of three states:

- ◇ **Running:** The procedure is now running.
- ◇ **Waiting:** The process is capable of running but is unable to do so because the CPU is being used by another process. If the scheduler so desires, it can select the process at the next task transition.
- ◇ **Sleeping:** The process is sleeping and cannot run because it is waiting for something to happen outside of it. The process cannot be selected by the scheduler at the next task switch.

The system saves all processes, whether they are running, sleeping, or waiting, in a process table. Sleeping processes, on the other hand, are specially marked so that

the scheduler knows they are not ready to start. There are also a number of queues that organise sleeping processes so that they can be roused at the appropriate time, such as when an external event that the process has been waiting for occurs.



Let us begin our investigation of the various transitions with a queued runnable process; the process is ready to run but is unable to do so since the CPU is assigned to another process (its state is thus “waiting”). It will continue in this condition until the scheduler allocates CPU time to it. When this occurs, its status changes to “running”. (path 4)

When the scheduler decides to withdraw CPU resources from the process, we investigate the reasons why the process state quickly switches from “running” to “waiting” (path 2), and the cycle begins again. In actuality, there are two “sleeping” states that differ depending on whether or not they may be disrupted by signals. This difference is not significant right now, but it will be when we look more closely at implementation.

If the process is forced to wait for an event, its state (path 1) switches from “running” to “sleeping.” It cannot, however, go immediately from “sleeping” to “running”; once the event it was waiting for occurs, the process returns to the “waiting” state (route 3) and then resumes the normal cycle.

When a program’s execution ends (for example, when a user stops an application), the process state changes from “running” to “stopped” (path 5). The “zombie” state is a unique process state that is not specified above. As the name implies, such processes are no longer in use but are nonetheless alive and well. In reality, they are dead because their resources (RAM, peripheral connections, etc.) have already been depleted, and they cannot and will never run again. They are still alive, however, because there are entries for them in the process table.

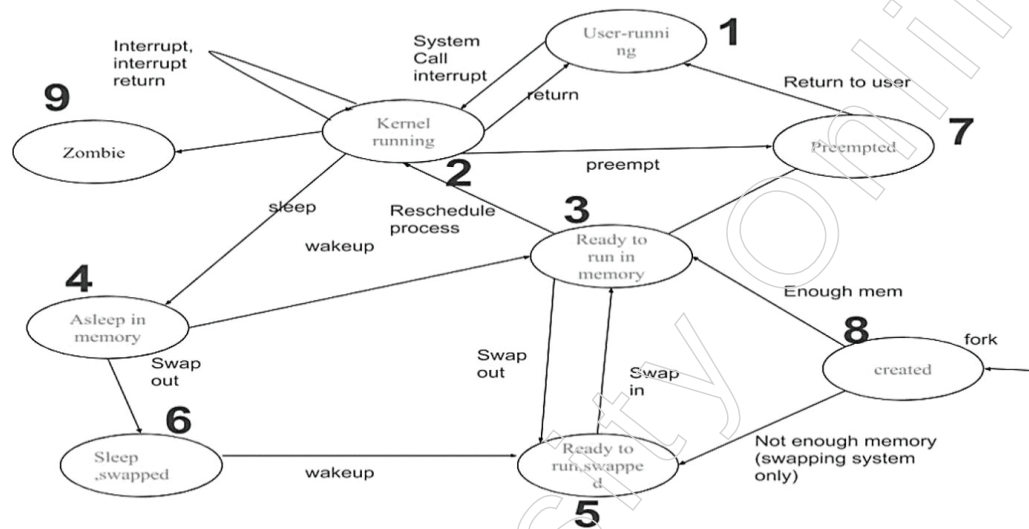
What causes zombies to exist? The cause is found in the UNIX process creation and destruction structure. A program terminates when two events occur: first, the program is killed by another process or by a user (this is usually done by sending a SIGTERM or SIGKILL signal, which is equivalent to terminating the process regularly); second, when the child process terminates, the parent process must invoke or has already invoked the wait4 (read: wait for) system call.

This tells the kernel that the parent process has acknowledged the child’s death. The system call allows the kernel to liberate resources that the child process has reserved. A zombie happens when just the first condition (the program has been terminated) but not the second applies (wait4). Between termination and the removal of its data from the process table, a process always momentarily enters the zombie state. A zombie can become deeply embedded in the process table and remain there until the next reset. Reading the output of process tools such as ps or top reveals this. This is not a big deal because residual data takes up very little space in the kernel.

Notes

Process Transitions

A process's lifetime can be divided into a number of states. The list below describes the entire set of process states.



The following steps describe how the Process works:

1. **User-running:** The process is running as a user.
2. **Kernel-running:** The process has been allocated to the kernel and is hence in kernel mode.
3. **Ready to run in memory:** Further, after accessing in main memory process is resubmitted to the Kernel i.e., The process is not now running but will be as soon as the kernel schedules it.
4. **Asleep in memory:** The process is sleeping but is still in main memory. It is awaiting the start of the task.
5. **Ready to run, swapped:** The process is ready to run and has been swapped into main memory by the CPU, allowing the kernel to schedule it for execution.
6. **Sleep, Swapped:** The process is in a sleep state in secondary memory, freeing up space in main memory for the execution of other processes. It may resume once the assignment has been completed.
7. **Pre-empted:** The kernel preempts an ongoing process in order to allocate another process while the first is transitioning from kernel to user mode.
8. **Created:** A new process has been created but is not yet running. All processes begin at this condition.
9. **Zombie:** The process has been thoroughly conducted, and the exit call has been enabled.

As a result, the procedure no longer exists. It does, however, keep a statistical record of the procedure. This is the end result of all processes.

Multitasking in Advance

The structure of Linux process management necessitates the employment of two additional process states: user mode and kernel mode. These represent the fact that

all modern CPUs have (at least) two execution modes, one of which has unlimited rights and the other is subject to certain restrictions, such as access to specific memory locations. This distinction is critical for building locked “cages,” which contain current processes and prevent them from interacting with other areas of the system.

Normally, the kernel is in user mode, which allows it to access only its own data and so cannot interfere with other applications on the system; it may not even be aware that there are other programs than itself.

If a process needs to access system data or functions (the latter handle resources shared by all processes, such as file system space), it must enter kernel mode. Obviously, this is only conceivable if everything is under control; otherwise, all created protective systems would be redundant, and all paths would be clearly known.

A second method of moving from user mode to kernel mode is through interrupts, which are then activated automatically. Interrupts occur more or less arbitrarily, as opposed to system calls, which are actively triggered by user applications. In general, the actions required to handle interrupts have little to do with the process that was running at the time the interrupt occurred. When an external block device transfers data to RAM, for example, an interrupt is raised, even if the data could be meant for any process operating on the system. In the same way, arriving network packages are communicated via an interrupt. Again, the inbound package is unlikely to be meant for the currently executing process.

As a result, Linux executes these activities in such a way that the running process is completely oblivious of them. The kernel's preemptive scheduling paradigm creates a hierarchy that defines which process states can be interrupted by which other ones.

Even other processes can cause normal processes to be disrupted. When an important process, like as an editor, becomes runnable, the scheduler can decide whether to launch the process immediately, even if the current process is still happily running. This type of preemption contributes significantly to optimal interactive behaviour and minimal system latency.

If the system is in kernel mode and is processing a system call, no other process in the system can cause CPU time to be withdrawn. The scheduler is required to wait until the system call's execution has completed before selecting another process. An interrupt, on the other hand, can halt the system call.

Interrupts have the ability to suspend processes in both user and kernel mode. They are given top priority since it is critical to deal with them as quickly as possible once they are issued.

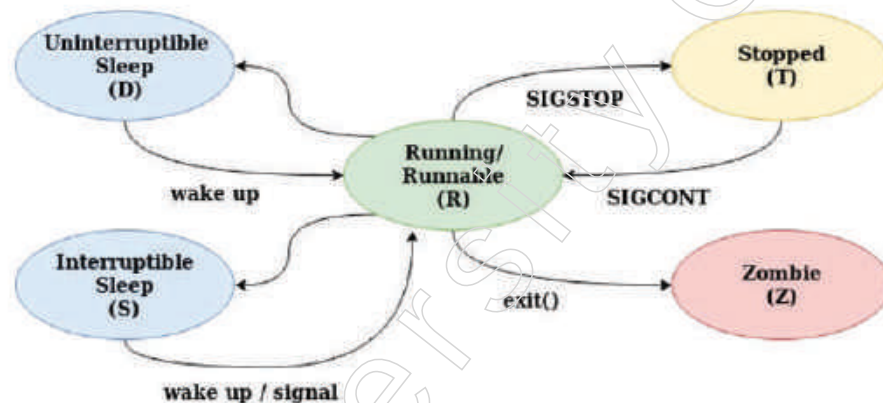
During the kernel's development, one option known as kernel preemption was added. This option allows you to switch to another process, if necessary, even while system calls are being executed in kernel mode (but not during interrupts). Although the kernel makes every effort to perform system calls as rapidly as feasible, the time required may be insufficient for some applications that rely on continual data streams. Kernel preemption can cut down on such wait times, ensuring “smoother” application execution. However, this comes at the expense of increased kernel complexity, as numerous data structures must now be safeguarded against concurrent access, even on single-processor platforms.

Notes

Process States in Unix/Linux

In Unix/Linux operating systems, the states that a Process enters in working from start till end are known as Process states. These are listed below as:

1. RUNNING and RUNNABLE
2. INTERRUPTABLE_SLEEP
3. Uninterruptable_Sleep
4. Stop
5. Zombie



RUNNING and RUNNABLE

The CPU is in a RUNNING state when it executes a process. The process is in the RUNNABLE state when it is not waiting for any resources and is ready to be executed by the CPU.

Sleeping

The Sleeping state shows that the process is currently waiting on resources (such as I/O, locks, application code causing the process to sleep, and so on). Sleeping processes are classified into two types:

- ◇ INTERRUPTABLE_SLEEP: When a process is in INTERRUPTABLE_SLEEP, it will wake up from the middle of sleep and process new signals sent to it.
- ◇ UNINTERRUPTABLE_SLEEP: When a process is in UNINTERRUPTABLE_SLEEP, it will not wake up from the middle of sleep even though new signals are sent to it.

How to Kill the Sleeping Process?

If the process is in the INTERRUPTABLE_SLEEP state, sending a SIGKILL signal to it (i.e., 'kill -9') will terminate it immediately. If the process is in the UNINTERRUPTABLE_SLEEP state, however, giving a SIGKILL signal will not immediately terminate it. The process will only terminate after it completes its sleep/waiting procedure. Thus, if you want to kill a process that has been in the UNINTERRUPTABLE_SLEEP state for an extended period of time, you must reboot the system; there is no other method.

Stopped

The STOPPED condition implies that the procedure has been halted and will not continue. When you use the 'Ctrl + Z' command in Linux, it sends a SIGSTOP signal to the process. When the process receives this signal, it is suspended/stopped from continuing to execute. When a process is STOPPED, it only handles SIGKILL and SIGCONT signals. SIGKILL signals will end the process, whereas SIGCONT signals will restart it in the RUNNING/RUNNABLE state.

Zombie

A process will terminate when it invokes the 'system exit' API or when it is killed by another process. When a process exits, it releases all of the data structures and resources it is holding. It will, however, not release its space in the 'process' table. The process will instead emit a SIGCHLD signal to its parent process. The parent process must now surrender the child process slot in the 'process' table. From the time the child process delivers the SIGCHLD signal until the parent process releases the slot in the 'process' table, the process is in a ZOMBIE state.

How to Kill the ZOMBIE Process?

Because the ZOMBIE process does not exist, issuing 'kill -9' on it has no effect. However, the ZOMBIE Process can be destroyed by issuing a SIGCHLD signal to the parent process and using the kill command listed below:

```
kill -s SIGCHLD parent-pid
```

This signal instructs the parent process to call wait () and clear up its ZOMBIE process.

NOTE: If the parent process is not properly configured to handle SIGCHLD signals, the above command may fail. You must terminate the ZOMBIE process's parent process.

How to Determine the Status of a Process

The process state can be obtained from the following source:

The process state will be reported in the column's' by the Unix/Linux command-line utility, 'top.' The state of the process is communicated using a single character:

- ◇ R — RUNNING/RUNNABLE
- ◇ S — INTERRUPTABLE_SLEEP
- ◇ D — UNINTERRUPTABLE_SLEEP
- ◇ T — STOPPED
- ◇ Z — ZOMBIE

2.1.2 The Context of a Process

In the operating system, there are times when you need to restore a running process to a different state, such as ready or wait/block. If a running process needs to conduct an I/O action, you must first remove it from the running state and then place it in the I/O queue. The process may use a round-robin scheduling technique in which the process must return to the ready state from the running state after every fixed time quantum. As a result, Context Switching is used to do these process switchings. In this

Notes

blog, we will learn about the notion of Context Switching in the Operating System, as well as the benefits and drawbacks of Context Switching. So, let us get this party started.

What is Context Switching?

A context switch is a process in which the CPU switches from one process or task to another. The kernel suspends the execution of the process that is in the running state, and the CPU executes another process that is in the ready state.

It is a necessary element of the multitasking operating system. The processes are shifted so quickly that the user gets the impression that they are all being run at the same time.

However, the context switching procedure entails a number of processes that must be completed. A process cannot be switched from the running to the ready state. You must keep the context of that process. If you do not save the context of any process P, then when the process P returns to the CPU for execution after a while, it will begin executing from the beginning. However, it should proceed from where it left the CPU in its prior execution. As a result, before starting any other process, the context of the current process should be stored.

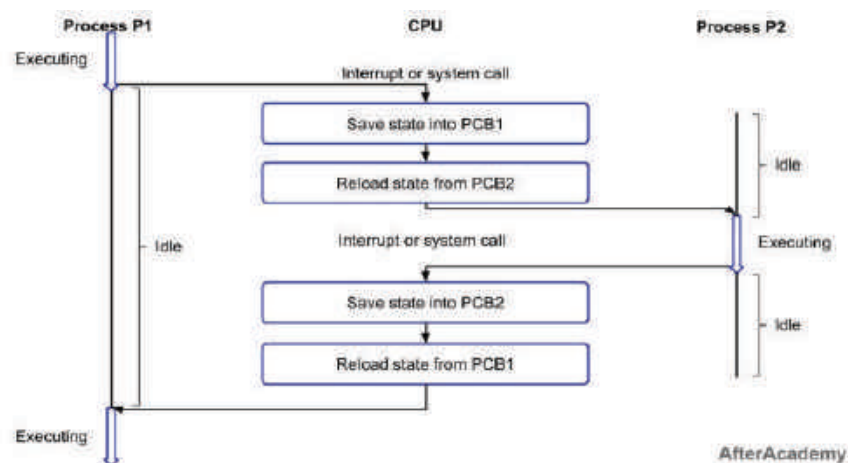
Triggers for Context Switching:

Context flipping is triggered by three key factors. These are listed below:

- ◇ **Multitasking:** In a multitasking environment, one process is switched off the CPU to allow another to execute. The old process's state is stored, and the new process's state is loaded. Processes on a pre-emptive system may be switched out by the scheduler.
- ◇ **Interrupt Handling:** When an interrupt occurs, the hardware shifts a portion of the context. This happens on its own. To reduce the time necessary to handle the interrupt, only a portion of the context is updated.
- ◇ **User and Kernel Mode Switching:** A context switch may occur in the operating system when a shift between user mode and kernel mode is necessary.

Context Switching involves the following steps:

A number of phases are involved in the context switching process. The graphic below demonstrates the context switching process between the two processes P1 and P2.



In the diagram above, you can see that process P1 is originally in the running state, whereas process P2 is in the ready state. When an interruption occurs, you must transition the process P1 from running to ready after saving the context, and the process P2 from ready to running. The following steps will be taken:

1. First, the context of process P1, i.e., the process in its operating state, will be saved in the Process Control Block of process P1, i.e., PCB1.
2. Now you must shift PCB1 to the appropriate queue, such as the ready queue, I/O queue, waiting queue, and so on.
3. Select the new process to be executed from the ready state, i.e., process P2.
4. Now, set the process status to running and update the Process Control Block of process P2, i.e., PCB2. If the CPU previously executed the process P2, you can obtain the place of the last executed instruction and resume the execution of P2.
5. Similarly, if you wish to repeat the process P1, you must follow the same steps as before (from step 1 to 4).

Cost of Context Switching

Context switching incurs extra costs due to TLB flushes, sharing the cache across several jobs, executing the task scheduler, and so on. Because threads share the same virtual memory mappings, context switching between two threads of the same process is faster than moving between two separate processes. As a result, TLB flushing is not required. The Context Switching Time is the amount of time it takes to switch from one process to another.

Advantage of Context Switching

Context switching is used to accomplish multitasking, often known as multiprogramming with time-sharing (learn more about multitasking from here). Multitasking provides users the impression that more than one process is running at the same time. In actuality, a CPU is only performing one task at a given moment. The context switching is so quick in this case that the user believes the CPU is doing many tasks at the same time.

Disadvantage of Context Switching

The downside of context switching is that it takes time to switch between contexts, i.e., the context switching time. It takes time to save the context of one process that is currently operating and then get the context of another process that is about to enter the running state. From the user's perspective, the CPU is doing nothing valuable at that period. So, in this case, context switching is just a waste of time.

Check Your Understanding

Fill Ups

1. A process is a _____ in operation.
2. In Unix, processes are classified into _____ and _____.
3. When a process transitions from one state to another, this is referred to as a _____.

Notes

4. The structure of Linux process management necessitates the employment of _____ and _____.
5. A _____ is a process in which the CPU switches from one process or task to another.

True/False

1. Processes are a critical component in the UNIX universe.
2. Under Process's Terminated State, the CPU requires directions to terminate the process after it has completed.
3. A processor can run multiple process at a time.
4. The system saves all processes, whether they are running, sleeping, or waiting, in a process table.
5. the ZOMBIE Process can be destroyed by issuing a SIGCHLD signal to the parent process.

Summary

- Processes are responsible for carrying out tasks within the operating system.
- A programme is a collection of machine code instructions and data that is saved in an executable image on disc and is thus a passive entity; a process is a computer programme in operation.
- Processes are a critical component in the UNIX universe. Essentially, practically every programme you run is running in a process.
- Although you may not need to engage with them all of the time, you must rely on them to get anything done in a UNIX system.
- Process states are the states that a Process enters when operating from start to finish.
- When switching between tasks, the scheduler must be aware of the status of every process in the system; it makes no sense to devote CPU time to processes that are idle.
- A process's lifetime can be divided into a number of states.
- The CPU is in a RUNNING state when it executes a process. The process is in the RUNNABLE state when it is not waiting for any resources and is ready to be executed by the CPU.
- If a running process needs to conduct an I/O action, you must first remove it from the running state and then place it in the I/O queue.
- The process may use a round-robin scheduling technique in which the process must return to the ready state from the running state after every fixed time quantum.

Activity

1. Divide the class into the groups of 5 and give presentation on the topic "Preventing the Context Switching from sabotaging one's productivity".

Glossary

- **Context Switch-** A process in which the CPU switches from one process or task to another.
- **Programme-** A collection of machine code instructions and data that is saved in an executable image on disc and is thus a passive entity; a process is a computer programme in operation.
- **Process-** An instance of a computer programme that is being executed.
- **Process states-** The states that a Process enters when operating from start to finish.
- **New State-** Any new operation that is requested by the system.
- **Running State-** A process running under the CPU, or a programme being executed by the CPU.
- **Waiting:** A process waiting for certain input or output operations.
- **Ready State:** The state in which the process is ready to execute but is waiting for the CPU to execute.

Question & Exercises

1. What is process? Discuss in brief the process state and transition.
2. Write in brief about process life cycle.
3. Elucidate Context Switching. Determine its advantages and disadvantages.
4. How to Determine the Status of a Process?
5. What are steps involved in context switching?

Further Readings

1. The Linux Programming Interface: A Linux and UNIX System Programming
2. How Linux Works: What Every Superuser Should Know
3. The Linux Command Line: A Complete Introduction
4. Advanced Programming in the UNIX Environment
5. Advanced UNIX Programming — Rockkind
6. UNIX and Linux System Administration Handbook, 4th Edition
7. Unix Network Programming, Volume 1: The Sockets Networking API
8. UNIX Network Programming, Volume 2: Interprocess Communications
9. The Art of UNIX Programming
10. The Unix Programming Environment — Kernighan, Pike

Answers to Check Your Understanding

Fill-Ups

1. Computer Programme
2. User Processes, Kernel Processes

Notes

Notes

3. Process State Transition
4. User Mode, Kernel Mode
5. Context Switch

True/False

1. True
2. False
3. False
4. True
5. True

Unit-2.2: Running Processes

Notes

Objectives:

At the end of this Unit, the reader will be able to:

- Understand what is Foreground Processes
- Discuss Background processes
- Talk about how to suspend and interrupt a process and kill A process
- Comprehend Listing Running Processes

Introduction

Process The operating system keeps track of processes using a five-digit ID number called the pid or process ID. Every process in the system has its own pid.

Pids eventually repeat since all potential numbers are exhausted and the next pid rolls or begins again. At any given time, no two processes with the same pid exist in the system since the pid is the identifier used by Unix to track each process.

2.2.1 Foreground Processes

Starting a Process

When you start a process (run a command), there are two ways you can run it –

- ◇ Foreground Processes
- ◇ Background Processes

Foreground Processes

Every process that you launch runs in the forefront by default. It receives input from the keyboard and outputs to the screen.

This is demonstrated via the ls command. If you wish to list all the files in your current directory, you can use the following command –

If you want to see a list of all the files in your current directory, use the command –

```
$ls ch*.doc
```

This would show all of the files whose names begin with ch and conclude with .doc –

```
ch01-1.doc ch010.doc ch02.doc ch03-2.doc
ch04-1.doc ch040.doc ch05.doc ch06-2.doc
ch01-2.doc ch02-1.doc
```

The process runs in the forefront, the output is routed to my screen, and the ls command waits for input from the keyboard (which it does not).

Notes

While a time-consuming application is running in the foreground, no other commands can be executed (no other processes can be started) since the prompt will not be available until the program finishes processing and exits.

Listing Running Processes

It is simple to see your own processes by issuing the `ps` (process status) command as shown below.

```
$ps
PID    TTY    TIME    CMD
18358  tty3   00:00:00 sh
18361  tty3   00:01:31 abiword
18789  tty3   00:00:00 ps
```

One of the most commonly used flags for `ps` is the `-f` (f for full) option, which provides more information as shown in the following example –

One of the most commonly used flags for `ps` is the `-f` (f for full) option, which provides more information as shown in the following example:

```
$ps -f
UID    PID  PPID  C  STIME  TTY  TIME  CMD
amrood 6738 3662 0 10:23:03 pts/6 0:00 first_one
amrood 6739 3662 0 10:22:54 pts/6 0:00 second_one
amrood 3662 3657 0 08:10:53 pts/6 0:00 -ksh
amrood 6892 3662 4 10:51:50 pts/6 0:00 ps -f
```

Here is the description of all the fields displayed by `ps -f` command:

Sr.No.	Column & Description
1	UID User ID that this process belongs to (the person running it)
2	PID Process ID
3	PPID Parent process ID (the ID of the process that started it)
4	C CPU utilization of process
5	STIME Process start time
6	TTY Terminal type associated with the process
7	TIME CPU time taken by the process

Notes

8	CMD The command that started this process
---	---

There are other options which can be used along with ps command –

Sr.No.	Option & Description
1	-a Shows information about all users
2	-x Shows information about processes without terminals
3	-u Shows additional information like -f option
4	-e Displays extended information

Stopping Processes

A process can be terminated in a variety of ways. Typically, sending a CTRL + C keystroke (the normal interrupt character) from a console-based command will exit the command. When the process is running in the forefront, this works.

If a process is running in the background, use the ps command to obtain its Job ID. After that, use the kill command to terminate the process, as seen below.

```
$ps -f
UID    PID  PPID  C  STIME   TTY   TIME CMD
amrood  6738 3662  0  10:23:03 pts/6  0:00 first_one
amrood  6739 3662  0  10:22:54 pts/6  0:00 second_one
amrood  3662 3657  0  08:10:53 pts/6  0:00 -ksh
amrood  6892 3662  4  10:51:50 pts/6  0:00 ps -f
$kill 6738
```

Terminated

Here, the kill command terminates the first_one process. If a process ignores a regular kill command, you can use kill -9 followed by the process ID as follows:

```
$kill -9 6738
```

Terminated

Parent and Child Processes

Each Unix process is allocated two ID numbers: the Process ID (pid) and the Parent process ID (ppid). Each system user process has a parent process.

The shell is the parent of the majority of the commands you perform. Consider the ps -f command, which displays both the process ID and the parent process ID.

Notes

Zombie and Orphan Processes

When a child process is destroyed, the parent process is normally updated through a SIGCHLD signal. The parent can then do something else or start a new child if necessary. However, the parent process is sometimes killed before its child. In this situation, the init process, the “parent of all processes,” becomes the new PPID (parent process ID). These processes are sometimes referred to as orphan processes.

When a process is terminated, a ps listing may still show it with a Z state. This is a dormant or dead process. The process has died and is no longer in use. These processes are not the same as orphan processes. They have finished execution, but there is still an item in the process table.

Daemon Processes

Daemons are system-related background processes that frequently operate with root privileges and respond to service calls from other processes.

A daemon has no command-and-control terminal. It is unable to open /dev/tty. If you run “ps -ef” and look at the tty field, you will notice that all daemons have a? for the tty.

A daemon is a process that runs in the background, usually waiting for anything that it can work with to happen. A printer daemon, for example, might be waiting for print commands.

If you have an application that requires extensive processing, you should make it a daemon and execute it in the background.

2.2.2 Background Processes

A background process operates in the background while you are not connected to your keyboard. If the background process requires keyboard input, it will wait for it.

The benefit of running a process in the background is that you can run other commands while it is running; you do not have to wait for it to finish before starting another!

To begin a background process, simply append an ampersand (&) to the end of the command.

```
$ls ch*.doc &
```

This displays all files with names that begin with ch and conclude with .doc.

```
ch01-1.doc ch010.doc ch02.doc ch03-2.doc
ch04-1.doc ch040.doc ch05.doc ch06-2.doc
ch01-2.doc ch02-1.doc
```

If the ls command requires input (which it does not), it enters a halt state until we move it to the foreground and provide it with data from the keyboard.

The first line comprises background process information, including the job number and process ID. You must know the task number in order to move it between the background and the foreground.

When you press the Enter key, you will see the following:

```
[1] + Done      ls ch*.doc &
$
```

The first line indicates that the ls command background process has completed successfully. The second is a command prompt for another.

The Highest Command

The top command is a highly handy tool for quickly displaying processes that have been sorted according to various criteria.

It is an interactive diagnostic tool that displays information on physical and virtual memory, CPU usage, load averages, and your busy processes on a regular basis.

Here is a simple syntax for running top and viewing information on CPU consumption by different processes.

```
$top
```

Job ID Versus Process ID

Typically, job numbers are used to modify background and suspended processes (job ID). This is a separate number than the process ID and is used since it is shorter.

Furthermore, a job can be made up of many processes that execute sequentially or concurrently. Tracking separate processes is more difficult when using the job ID.

Foreground and Background Jobs

Foreground processes are those that require a user to initiate or engage with them. Background processes are processes that run independently of the user. By default, programs and commands run as foreground processes. To run a process in the background, add an ampersand (&) to the end of the command name used to initiate the process.

Changing the focus of a process between the foreground and the background

Note

A foreground process consumes your shell (terminal window), which means that any new instructions you type will have no impact until the previous command is completed. This is to be expected, although it can be perplexing when running long-running programs such as the afni or suma GUI (graphical user interface).

Note

Because afni now operates in the background by default, it is no longer a relevant example (it creates a child process and returns the user to an active command prompt). As a result, suma will now be used in this demonstration.

Notes

Notes

Commands and descriptions:

suma : the main AFNI GUI for viewing surface data
ctrl-z : keystroke: suspend the foreground process
bg : put the recently suspended process in the background
fg : put the recently suspended process in the foreground
& : run a program in the background to begin with
jobs : list child processes under terminal shell

start *suma*

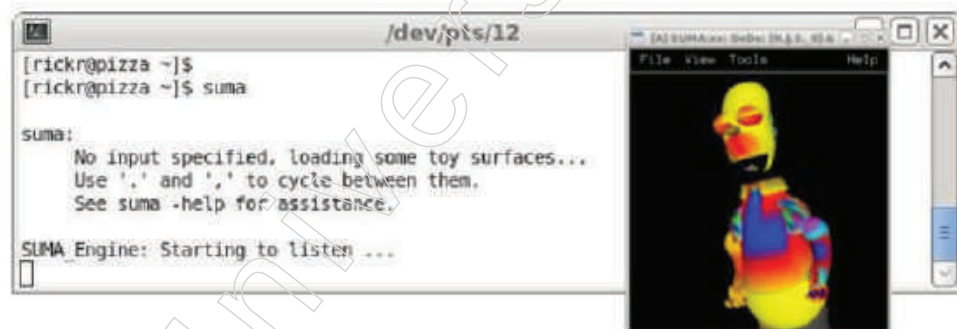
Start *suma* (from any directory). It will display a random dataset (which can be changed with '.').

commands (type these in the terminal window):

suma

When we open the *suma* GUI, it shows us anything (a random internal dataset of its choosing), which varies. But since this is about Unix, we do not mind what it reveals.

To see if *suma* is active, use the left mouse button (press and hold) to “grab” and rotate the surface data.



fail to do anything

Try to type commands in the original terminal window.

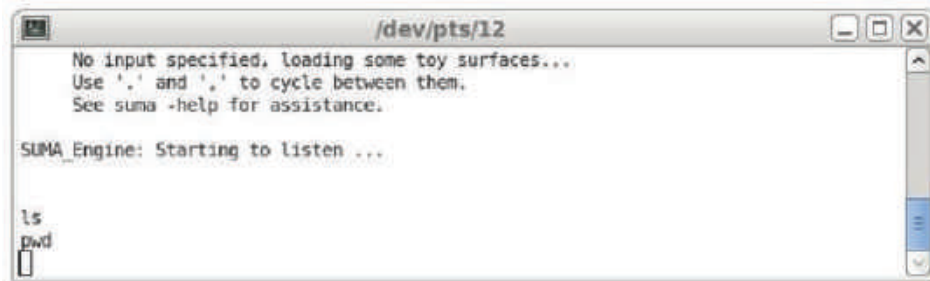
Locate the terminal window from which *suma* was launched. Try to enter some commands into it.

commands (type these in the terminal window):

<Enter>
 <Enter>
 ls
 pwd
 <Enter>

We do not receive a prompt, and the commands have no impact.

This is because the *suma* software is running in the terminal shell's foreground. *Suma* has taken up residence in the shell.



```

/dev/pts/12
No input specified, loading some toy surfaces...
Use '.' and ',' to cycle between them.
See suma -help for assistance.

SUMA_Engine: Starting to listen ...

ls
pwd

```

suspend suma

Press ctrl-z (hold the ctrl key and press 'z') in the terminal window.

command (enter in the terminal window):

ctrl-z

jobs

The terminal window displays the word 'Suspended.' That refers to the suma program, which means that suma is no longer executing yet the terminal is accessible.

Suma is also suspended, according to the jobs command.



```

/dev/pts/12
SUMA_Engine: Starting to listen ...

ls
pwd
^Z
Suspended
[rick@pizza ~]$ jobs
[1]  + Suspended                  suma
[rick@pizza ~]$

```

try, try, again

Repeat of B: try to type commands in the terminal window.

commands (type these in the terminal window):

<Enter>

<Enter>

ls

pwd

<Enter>

This time the commands work. The suma program is no long occupying the terminal shell.



```

/dev/pts/12
[rick@pizza ~]$
[rick@pizza ~]$ jobs
[1]  + Suspended                  suma
[rick@pizza ~]$
[rick@pizza ~]$ ls
APNI_data6 Desktop abin afni data rbin subject_results
[rick@pizza ~]$ pwd
/home/rickr
[rick@pizza ~]$
[rick@pizza ~]$

```


Notes

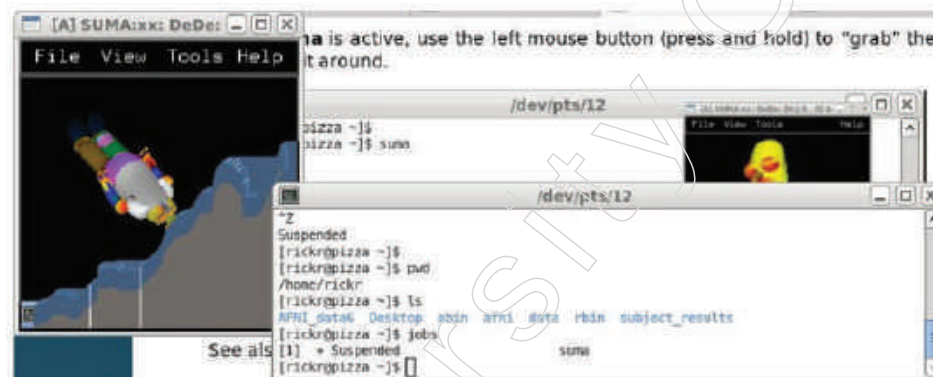
is suma broken?

Try to use the suma GUI.

Try again to grip and spin the surface using the left mouse button.

Next, take another window (perhaps the terminal or a web browser), move it to partially obscure the suma window, and then move it away.

The windows are essentially erased, possibly leaving duplicates of the obscuring image in the suma window. The window manager is no longer redrawing the suma image. This demonstrates that 'suma' is no longer active (since it was suspended).



wake up, suma!

Put suma in the background before attempting to use it again.

Return to the same terminal window and type the bg command (for background). However, keep an eye on the suma GUI when you press Enter>.

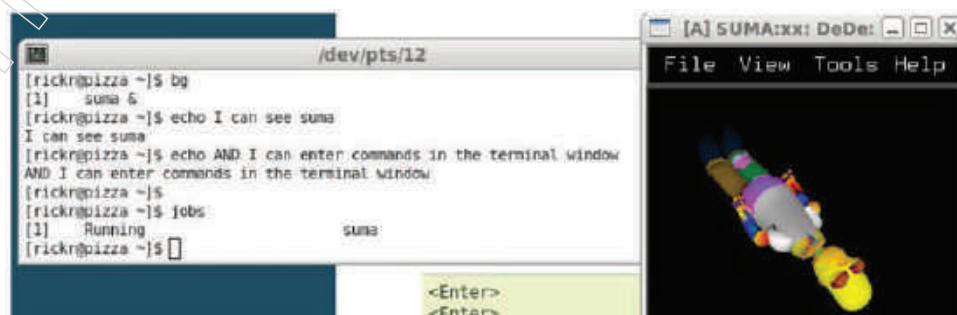
commands (enter them into the terminal window):

bg

jobs

As soon as the bg command was input, suma began running again. It is also worth noting that we can still type commands into the terminal window. Not only is afni up and operating, but we also have access to the shell.

Also, jobs indicates that suma is operating.



start over

Be effective. To begin, run 'suma' in the background. Then ctrl-z and bg would be unnecessary.

Begin by exiting the current suma GUI. To close the suma image window, use the 'x' in the upper right corner of the window, or press esc>, and then click Yes (or simply press Enter>).

Restart suma by inserting the & letter after the instruction.

commands (type these in the terminal window):

suma &

To begin with, suma is in the backdrop. Suma can be used, and commands can be entered into the terminal window.

Note:

When a program runs in the background, any text produced from the background program appears in the same terminal window, which can be confusing.

2.2.3 Suspend and Interrupt a Process and Kill a Process

Although Linux is regarded as a stable operating system with few application bugs, programmes can become unresponsive at times. When this happens, they might eat a large amount of system resources or bring the entire system down. These applications are typically unable to be restarted automatically.

If the process or application is still operating and will not be entirely shut down, you must use a command to terminate it in Linux. When this occurs, it is required to either restart the entire system or terminate the specific programme process. Because restarting the entire system takes time and might be inconvenient for clients, killing a process under Linux is significantly easier.

The Linux operating system includes the ability to terminate a process. The command allows the server to continue running without having to reboot after a large change/update. This is where Linux's immense power comes into play, and it is one of the reasons why Linux runs on 96.4 percent of the world's servers.

In Linux, the KILL command (found in /bin/kill) is a built-in command that is used to manually terminate processes.

Kill command sends a signal, a specified signal to a currently running process. The kill command can be executed in a number of ways, directly or from a shell script.

The common syntax for kill command is:

kill [signal or option] PID(s)

For a kill command a Signal Name could be:

Signal Name	Signal Value	Behaviour
SIGHUP	1	Hangup
SIGKILL	9	Kill Signal
SIGTERM	15	Terminate

Notes

Clearly, based on the behaviour described above, SIGTERM is the default and safest approach to terminate a process. SIGHUP is less secure than SIGTERM for terminating a process. SIGKILL is the most dangerous method of killing a process since it kills the process without preserving it.

To stop a process, we must first determine its Process ID. A Process is a program instance. When a program starts, a unique PID is automatically generated for that process.

In Linux, every process has a pid. The - init process is the first to run when a Linux system boots, hence it is usually assigned a value of '1'.

Init is the master process and cannot be killed in this manner, ensuring that the master process is not killed by accident. Init determines and enables itself to be killed, where kill is just a shutdown request.

Signals

Signals are a type of communication between processes, whether they be from other processes, the kernel, or the specific process itself. Each process has a present behaviour or disposition, which can be classified into five types:

- ◇ Term
- ◇ Ign
- ◇ Core
- ◇ Stop
- ◇ Cont

When a signal is provided to the process, these qualities govern how it will behave. The default action of the Term is to terminate the process, which is commonly utilised by the kill command.

With the kill command, a total of 64 signals are employed. The one we send will be determined by the desired consequence, as many signals have varied impacts and outcomes. The first 31 of these 64 signals are standard signals. The remaining signals are in real time.

The primary distinction between the two types of communications is that standard signals cannot be queued whereas real-time signals can. This indicates that a process receives the information associated with the first instance of a standard signal. Real-time signals will queue the information connected with them, allowing numerous signals to be received.

We send an appropriate signal to a process in order to terminate it. The Linux operating system relies heavily on signal processing. Signals allow us to instruct processes on what to do. The most commonly used signals are:

- ◇ 1 (HUP) - Reload a process.
- ◇ 9 (KILL) - Kill a process.
- ◇ 15 (TERM) - Gracefully stop a process.

kill -l:

To get a list of all available signals, invoke the command with the -l option:

\$ kill -l

```

root@AJ7:/home/demon# kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS      8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH   29) SIGIO       30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
root@AJ7:/home/demon#

```

Signals can be specified in three different ways:

- ◇ Using number (e.g., -1 or -s 1).
- ◇ Using the “SIG” prefix (e.g., -SIGHUP or -s SIGHUP).
- ◇ Without the “SIG” prefix (e.g., -HUP or -s HUP).

The following commands are equivalent to one another:

- ◇ \$kill -1 PID_NUMBER
- ◇ \$kill -SIGHUP PID_NUMBER
- ◇ \$kill -HUP PID_NUMBER

PID

To send a signal to a process with kill, we require the process’s identifier, abbreviated as PID. The PID of a process can be acquired using commands such as ps or top, but this is beyond the scope of this tutorial.

Let us pretend we have a hanging process with PID 123 for the rest of this section. While we will use a single PID in our examples, we can always send the same signal to several processes by providing a space-separated string of PIDs.

- ◇ Negative PID numbers represent the process group ID. If you pass a process group ID, the signal will be sent to all processes in that group.
- ◇ A PID of -1 denotes all processes except kill and init, which is the parent process of all processes on the system.
- ◇ The command ps will show you a list of running processes together with their PID numbers if you want to see a list of running processes. The PID is required to designate which process should receive the kill signal.

Notes

Notes

Syntax:

\$ps

kill pid: To show how to use a PID with the kill command.

Syntax:

\$kill pid

kill -s: To show how to send signal to processes.

Syntax:

\$kill {-signal | -s signal} pid

kill -L: This command is used to list available signals in a table format.

Syntax:

\$kill {-l | --list[=signal] | -L | --table

Check Your Understanding

Fill-Ups

1. The operating system keeps track of processes using a five-digit ID number called the _____.
2. _____ receives input from the keyboard and outputs to the screen.
3. One of the most commonly used flags for ps is the _____.
4. If a process ignores a regular kill command, _____ can be used.
5. Each Unix process is allocated _____ ID numbers.

True/False

1. Every process in the system has its own pid.
2. While a time-consuming application is running in the foreground, other commands can be executed.
3. A process can be terminated in a variety of ways.
4. A daemon has a command-and-control terminal.
5. The Linux operating system includes the ability to terminate a process.

Summary

- **Process** The operating system keeps track of processes using a five-digit ID number called the pid or process ID.
- Every process in the system has its own pid.
- Pids eventually repeat since all potential numbers are exhausted and the next pid rolls or begins again.
- At any given time, no two processes with the same pid exist in the system since the pid is the identifier used by Unix to track each process.
- Although Linux is regarded as a stable operating system with few application bugs, programmes can become unresponsive at times. When this happens, they might eat a large amount of system resources or bring the entire system down.
- If the process or application is still operating and will not be entirely shut down, you must use a command to terminate it in Linux.
- When this occurs, it is required to either restart the entire system or terminate the specific programme process. Because restarting the entire system takes time and might be inconvenient for clients, killing a process under Linux is significantly easier.
- The Linux operating system includes the ability to terminate a process. The command allows the server to continue running without having to reboot after a large change/update.
- This is where Linux's immense power comes into play, and it is one of the reasons why Linux runs on 96.4 percent of the world's servers.

Activity

1. Get divided into the group of five and make a presentation on "1 (HUP) - Reload a process or 9 (KILL) - Kill a process or 15 (TERM) - Gracefully stop a process"

Glossary

- **Daemon-** Background program that provides vital functions for the day-to-day running of Linux.
- **Device-** Linux shorthand describing something on your system that provides a function for the user or that the system requires in order to run. This usually refers to hardware, but it can also describe a virtual device that is created to provide access to a particular Linux function.
- **GNU/Linux-** Another name for the operating system referred to as Linux.
- **Linux-** A kernel program created by Torvalds.
- **PID-** Process ID; the numbering system used to refer to a process.

Question & Exercises

1. How a process is started. Discuss foreground and background processes.
2. Write in brief about the listing of running processes.
3. Elucidate interruption and killing of a process.

Notes

4. What is a command?
5. Write a short note on PID.

Further Readings

1. The Linux Programming Interface: A Linux and UNIX System Programming
2. How Linux Works: What Every Superuser Should Know
3. The Linux Command Line: A Complete Introduction
4. Advanced Programming in the UNIX Environment
5. Advanced UNIX Programming — Rochkind
6. UNIX and Linux System Administration Handbook, 4th Edition
7. Unix Network Programming, Volume 1: The Sockets Networking API
8. UNIX Network Programming, Volume 2: Interprocess Communications
9. The Art of UNIX Programming
10. The Unix Programming Environment — Kernighan, Pike

Answers to Check Your Understanding

Fill-Ups

1. Process ID or Pid
2. Foreground Processes
3. -f
4. kill -9
5. Two

True/False

1. True
2. False
3. True
4. False
5. True

Unit-2.3: Process Commands

Notes

Objectives:

At the end of this Unit, the reader will be able to:

- Understand what is ps Command
- Discuss Batch Command
- Talk about how to Schedule tasks using batch
- Comprehend at command

Introduction

A new process is normally created when an existing process makes an exact copy of itself in memory. The child process will have the same environment as its parent, but only the process ID number is different.

There are two conventional ways used for creating a new process in Linux:

- ◇ Using The System() Function – this method is relatively simple, however, it's inefficient and has significantly certain security risks.
- ◇ Using fork() and exec() Function – this technique is a little advanced but offers.

2.3.1 The ps Command

It displays information about a selection of the active processes on the system as shown below:

```
# ps
# ps -e | head
```

One of the most commonly used flags for ps is the -f (f for full) option, which provides more information as shown in the following example –

```
$ps -f
UID    PID  PPID  C  STIME  TTY  TIME CMD
amrood  6738 3662  0 10:23:03 pts/6 0:00 first_one
amrood  6739 3662  0 10:22:54 pts/6 0:00 second_one
amrood  3662 3657  0 08:10:53 pts/6 0:00 -ksh
amrood  6892 3662  4 10:51:50 pts/6 0:00 ps -f
```

Here is the description of all the fields displayed by ps -f command –

Sr.No.	Column & Description
1	UID User ID that this process belongs to (the person running it)
2	PID Process ID

Notes

3	PPID Parent process ID (the ID of the process that started it)
4	C CPU utilization of process
5	STIME Process start time
6	TTY Terminal type associated with the process
7	TIME CPU time taken by the process
8	CMD The command that started this process

There are other options which can be used along with ps command -

Sr.No.	Option & Description
1	-a Shows information about all users
2	-x Shows information about processes without terminals
3	-u Shows additional information like -f option
4	-e Displays extended information

2.3.2 The Batch Command

The cron daemon keeps a list of jobs that it executes at certain times. At the times specified, these processes and programs execute in the background. This gives you a lot of leeway when it comes to scheduling jobs that must be repeated. You can set up cron to run a task once every hour, at a certain time each day, or once a month or year.

However, this is ineffective if you wish to schedule a job to run only once. You can set it up in cron, but then you have to remember to remove the crontab line after the action has completed, which is cumbersome.

If you are having trouble with Linux, it is almost certain that someone else has, too. Fortunately, because Unix-like operating systems have been around for so long, there is a good possibility someone has already solved your problem.

Using sh, at and batch read commands from standard input or a specified file that will be run later.

Scheduling tasks using batch:

The label statement in Batch Script is used to define a function. When a function is first defined, it may accept one or more values as input 'parameters,' process the functions in the main body, and return the results to the functions as output 'return types.'

The batch command is used to take commands from standard input or a given file and execute them when system load levels allow it, that is, when the load average falls below 1.5.

Syntax:

Batch

It should be noted that batch does not take any parameters.

Use the batch command to do a one-time task when the load average is less than 0.8.

The `at>` prompt appears after you type the batch command. Enter the command to be executed, then hit Enter and Ctrl-D. Typing each command followed by the Enter key will allow you to specify multiple commands. After you have entered all of the commands, hit Enter to move to a blank line and type Ctrl-D. Alternatively, a shell script can be entered at the prompt, followed by Enter after each line of the script and exited by entering Ctrl-D on a blank line. If a script is executed, the shell set in the user's SHELL environment, the user's login shell, or `/bin/sh` is utilised (whichever is found first). The collection of commands or script is executed as soon as the load average falls below 0.8.

If the set of instructions or script attempts to send data to standard out, the result is emailed to the user.

It should be noted that batch does not take any parameters.

Use the batch command to do a one-time task when the load average is less than 0.8.

The `at>` prompt appears after you type the batch command. Enter the command to be executed, then hit Enter and Ctrl-D. Typing each command followed by the Enter key will allow you to specify multiple commands. After you have entered all of the commands, hit Enter to move to a blank line and type Ctrl-D. Alternatively, a shell script can be entered at the prompt, followed by Enter after each line of the script and exited by entering Ctrl-D on a blank line. If a script is executed, the shell set in the user's SHELL environment, the user's login shell, or `/bin/sh` is utilised (whichever is found first). The collection of commands or script is executed as soon as the load average falls below 0.8.

If the set of instructions or script attempts to send data to standard out, the result is emailed to the user.

2.3.3 The at Command

The `at` command is a command-line utility that allows you to schedule a command to run at a specific time in the future. Jobs created with the `at` command are only executed once. The `at` command can be used to run any application or send any email at any point in the future. It performs commands at predefined times. It accepts times of the pattern `HH:MM` to run a job at a predefined time of day. To plan a job, use expressions such as `noon`, `midnight`, `teatime`, `tomorrow`, `next week`, `next Monday`, and so on with the `at` command.

Syntax:

Notes

at [OPTION...] runtime

Normally, the at command expects time as an input. TIME can be indicated using keywords like;

noon, midnight or teatime (4:00 PM) ,

next week, tomorrow, teatime tomorrow, next tuesday, next tuesday, next tuesday, next tuesday, next tuesday, next tuesday, next tuesday, next tuesday, next tuesday

When no instructions or scripts are supplied to it, at commands operates interactively, expecting you to enter the commands to execute from the prompt.

Enter the command to run at the prompt. As an example,

When you are finished typing commands, hit Ctrl+D to end input. You will see the command line if you hit Ctrl+D.

You may also send commands to at command from the command line by specifying the file with commands with the -f option. Consider the file mycommands.sh, which contains two shell scripts.

To have these commands executed next week at midnight on the same day;

You may also use the usual input redirection sign;

You can also stream tasks/commands to at command. To make a notification, for example, use the notify-send command.

This will send you an alert at 5 p.m. to replace the gas.

Check Your Understanding

Fill-Ups

1. _____ displays information about a selection of the active processes on the system.
2. PPID stands for _____.
3. The _____ keeps a list of jobs that it executes at certain times.
4. The _____ in Batch Script is used to define a function.
5. _____ performs commands at preferred times.

True/False

1. A new process is normally created when an existing process makes an exact copy of itself in memory.
2. -a Shows information about processes without terminals.
3. -x Shows information about all users.
4. The batch command is effective if you wish to schedule a job to run only once.
5. The at command can be used to run any application or send any email at any point in the future

Notes

Summary

- A new process is normally created when an existing process makes an exact copy of itself in memory.
- The child process will have the same environment as its parent, but only the process ID number is different.
- There are two conventional ways for creating a new process in Linux.
- Using The System () Function – this method is relatively simple, however, it's inefficient and has significantly certain security risks.
- Using fork () and exec () Function – this technique is a little advanced but offers.
- The ps Command displays information about a selection of the active processes on the system.
- The cron daemon keeps a list of jobs that it executes at certain times. At the times specified, these processes and programs execute in the background.
- This gives you a lot of leeway when it comes to scheduling jobs that must be repeated. You can set up cron to run a task once every hour, at a certain time each day, or once a month or year.
- The label statement in Batch Script is used to define a function.
- When a function is first defined, it may accept one or more values as input 'parameters,' process the functions in the main body, and return the results to the functions as output 'return types.'
- The at command is a command-line utility that allows you to schedule a command to run at a specific time in the future. Jobs created with the at command are only executed once.
- The at command can be used to run any application or send any email at any point in the future. It performs commands at predefined times.
- It accepts times of the pattern HH:MM to run a job at a predefined time of day.

Activity

1. Get divided into the group of five and make a presentation on any of the following
 1. PS Command
 2. Batch Command
 3. AT Command

Glossary

- **Daemon-** Background program that provides vital functions for the day-to-day running of Linux.
- **Device-** Linux shorthand describing something on your system that provides a function for the user or that the system requires in order to run. This usually refers to hardware, but it can also describe a virtual device that is created to provide access to a particular Linux function.
- **GNU/Linux-** Another name for the operating system referred to as Linux.

Notes

- **Linux**- A kernel program created by Torvalds.
- **PID**- Process ID; the numbering system used to refer to a process.
- **PS Command**- It displays information about a selection of the active processes on the system
- **C- CPU** utilization of process
- **STIME**- Process start time
- **TTY**- Terminal type associated with the process
- **TIME**- CPU time taken by the process
- **CMD**- The command that started this process
- **-a**- Shows information about all users
- **-x**- Shows information about processes without terminals
- **-u**- Shows additional information like -f option
- **-e**- Displays extended information

Question & Exercise

Questions

1. What are Process commands?
2. Write in brief about PS Command.
3. Elucidate the Batch command and how a task can be scheduled through it.
4. Discuss about AT Command and its benefits in scheduling a command.

Further Readings

1. The Linux Programming Interface: A Linux and UNIX System Programming
2. How Linux Works: What Every Superuser Should Know
3. The Linux Command Line: A Complete Introduction
4. Advanced Programming in the UNIX Environment
5. Advanced UNIX Programming — Rochkind
6. UNIX and Linux System Administration Handbook, 4th Edition
7. Unix Network Programming, Volume 1: The Sockets Networking API
8. UNIX Network Programming, Volume 2: Interprocess Communications
9. The Art of UNIX Programming
10. The Unix Programming Environment — Kernighan, Pike

Answers to Check Your Understanding

Fill-Ups

1. PS Command
2. Parent process ID

3. Cron Daemon
4. label statement
5. AT Command

True/False

1. True
2. False
3. False
4. False
5. True

Notes

Module-III: Unix/Linux Commands

Structure:

- 3.1 Work on Unix Terminal
 - 3.1.1 Telnet connect through Login Account , password, logout ,
 - 3.1.2 Internal and External Commands
- 3.2 Unix File Handling
 - 3.2.1 "current working directory, referring to home directories"
 - 3.2.2 Commands to move around by path concept, creating new directories
 - 3.2.3 creating new directories, creating files –touch , cat
 - 3.2.4 Listing directory and File - ls, hidden files
 - 3.2.5 "copying files; moving files, Deleting files and directories;"
 - 3.2.6 View file - cat, more, pg, less
 - 3.2.7 wildcards and Meta characters
- 3.3 Filters and Pipes
 - 3.3.1 Introduction to filters
 - 3.3.2 Head, Tail, Cut
 - 3.3.3 Sort, wc
 - 3.3.4 grep
 - 3.3.5 "cmp, comm., diff, uniq"
 - 3.3.6 expr , bc, exit
- 3.4 Standard Input/Output
 - 3.4.1 Introduction to redirection
 - 3.4.2 redirecting input
 - 3.4.3 redirecting output and error
- 3.5 Unix Permissions
 - 3.5.1 Understanding Permissions
 - 3.5.2 Symbolic and Numeric Permissions
 - 3.5.3 Changing Permissions
- 3.6 Unix Links
 - 3.6.1 File Links
 - 3.6.2 Types of Links
 - 3.6.3 The ln command

Unit-3.1: Work on Unix Terminal

Notes

Objectives:

At the end of this Unit, the reader will be able to:

- Understand Telnet connect through Login Account, password, logout,
- Learn Internal and External Commands

Introduction

A terminal is a type of device file in Unix that implements a number of additional commands (ioctls) in addition to read and write. The kernel can offer some terminals on behalf of a hardware device, such as those with input from the keyboard and output to a text mode screen, or those with input and output broadcast over a serial line. Terminal emulators provide other terminals (via a thin kernel layer) that are frequently referred to as pseudo-terminals or pseudo-ttys. The following are examples of terminal emulators:

- ◇ Xterm, Gnome Terminal, Konsole, Terminator, and other graphical apps run in the X Window System.
- ◇ Screen and tmux are programmes that provide a barrier between a programme and another terminal.
- ◇ Ssh is a programme that allows you to connect a terminal on one system to programmes on another.
- ◇ Expect it when it comes to programming terminal interactions.

The term terminal can also refer to a device that allows you to interface with a computer, often through a keyboard and monitor. An X terminal, for example, is a thin client, a special-purpose computer whose sole job is to control a keyboard, display, mouse, and occasionally additional human interface peripherals, while the main applications are run on a separate, more powerful computer.

3.1.1 Telnet Connect Through Login Account, Password, Logout

TELNET is an abbreviation for TERminal NETwork. It is a form of protocol that allows one computer to communicate with another. It is a standard TCP/IP protocol for virtual terminal service provided by ISO. The computer that initiates the connection is referred to as the local computer. The computer to which the connection is made, i.e., the computer that accepts the connection, is referred to as the remote computer. When the connection is established between local and remote computer. During telnet operation whatever that is being performed on the distant computer will be presented by local computer. Telnet works on a client/server model. Local computer uses telnet client programme while the remote computers uses telnet server programme.

TELNET Commands:

Commands of the telnet are identified by a prefix character, Interpret As Command (IAC) which is having code 255. After IAC, there are command and option codes. The command's basic format is illustrated in the picture below:

Notes



Figure: Telnet command format

Following are some of the important TELNET commands:

Character	Decimal	Binary	Meaning
WILL	251	11111011	1. Offering to enable. 2. Accepting a request to enable.
WON'T	252	11111100	1. Rejecting a request to enable. 2. Offering to disable. 3. Accepting a request to disable.
DO	253	11111101	1. Approving a request to enable. 2. Requesting to enable.
DON'T	254	11111110	1. Disapproving a request to enable. 2. Approving an offer to disable. 3. Requesting to disable.

Following are some common options used with the TELNET:

Code	Option	Meaning
0	Binary	It interpret as 8-bit binary transmission.
1	Echo	It will echo the data that received on one side to the other side.
3	Suppress go ahead	It will suppress go ahead signal after data.
5	Status	It will request for the status of TELNET.
6	Timing mark	It define the timing marks.
8	Line width	It specifies the line width.
9	Page size	It specifies the number of lines in a page.
24	Terminal type	It set the terminal type.
32	Terminal speed	It set the terminal speed.
34	Line mode	It will change to the line mode.

Modes of Operation:

Most telnet implementation operates in one of the following three modes:

- ◇ Default mode
- ◇ Character mode
- ◇ Line mode

Default Mode:

- ◇ When no other modes are triggered, this mode is used.
- ◇ The client performs echoing in this mode.

In this mode, the user inputs a character, and the client echoes it on the screen but does not send it until the entire line is done.

Character Mode:

- ◇ Each character written in this mode is delivered to the server by the client.
- ◇ In this mode, the server generally repeats the character back to the client's screen.

Line Mode:

- ◇ Line editing, such as echoing and character deleting, is performed on the client side.
- ◇ The client sends the entire line to the server.

Logging in to the CLI with a telnet connection

When secure management access for the Management server is disabled, you can connect to it via the network using a telnet session.

About this task

Note: By default, secure management access is enabled for the Management server; therefore, you cannot connect to the Management server by using a telnet connection. For more information about enabling and disabling secure management access, see `mgmt. secure`.

Procedure

1. In Windows, select Start > Run.
2. Enter `telnet management server-host-name`.
The login prompt is exhibited at first access.
3. Log in to the Management server with a user name and password, which have administrative privileges. The default user name and password is:
 - ◆ Login: admin
 - ◆ Password: `ln0r1t5@C`

Note: Please keep in mind that this console has a US keyboard layout, which means that the `@` symbol may appear in a different location than on other keyboard layouts. Please double-check that you typed the password correctly if you are not using a US keyboard.

Results

When you log in, the prompt changes to the Management server's host name.

If you do not specify a hostname, the prompt defaults to the Management server's IP address.

Password Command

In Linux, the `passwd` command is used to update user account passwords. The root user has the ability to change the password for any user on the system, but a normal user can only change the password for his or her own account.

Notes

Syntax:

`passwd [options] [username]`

Example:

Command: `passwd`

Command [root]: `passwd user1`

```
root@DESKTOP-: /home/hp# passwd user1
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@DESKTOP-: /home/hp#
```

Note: Normal users can use `sudo` to invoke root privileges and change the password for root. This is especially useful when a user is a member of the admin group (has a position in the sudoers list (/etc/sudoers) and may use commands with `sudo`) and the root password is not set, as is the situation with many mainstream Linux distributions.

Command: `sudo passwd root`

```
hp@DESKTOP-:~$ sudo passwd root
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
hp@DESKTOP-:~$
```

Processing in passwd command:

1. Verify current user password: When the user enters the `passwd` command, it requests for the current user password, which is compared to the password saved in the user's /etc/shadow file. The root user can skip this step and update the password directly, allowing forgotten passwords to be recovered.
2. Check password ageing information: In Linux, a user password can be set to expire after a certain amount of time. A user may also be barred from changing his or her password for an extended length of time. This password ageing information (together with the password) is saved in the file /etc/shadow.
3. Change the password: Following authentication, the user is invited to enter the new password and confirm it by retyping it.

/etc/shadow file: The shadow file is a list of colon separated values with 9 fields, as shown below:

```
user1:$6$x8wAJRp$EWC97sXW5tqac10Q2TQyXkR.1I1jdK4VLK1pkZKmA2mbA
6UnSGyo94Pis074viWBA3sVbkCptSZzuP2K.yan/:17887:0:99999:7:::
```



- passwd options:**

- ```
hp@DESKTOP-... ~ % sudo passwd -e user1
[sudo] password for hp:
passwd: password expiry information changed.
hp@DESKTOP-... ~ % su user1
Password:
You are required to change your password immediately (root enforced)
Changing password for user1.
(current UNIX password:
Enter new UNIX password:
Retype new UNIX password:
$
```

- Amity Directorate of Distance & Online Education

## Notes

example: `passwd -i 3 user1`

- ◇ **-k, --keep-tokens:** This option is used when you only want to change the password if it is expired. It keeps the authentication tokens for the verification if the password is not yet expired, even if you requested to change it. Note that if the expiry period for a user is set to 99999, then this selection will not keep tokens and the password will be changed.
- ◇ **-l, --lock:** Lock the password of user. This appends the encrypted password of the user with a character '!', and thus making it unable to match with any of input password combinations. This does not disable the account but prevents the user from logging in using a password. Though other authentication methods like ssh keys can be used to login to the account.
- ◇ **-n, --mindays MIN\_DAYS:** Change the minimum number of days between password changes to MIN\_DAYS so that the user can't change the password for MIN\_DAYS.
- ◇ **-q, --quiet:** This option is used for quiet mode. While using this option to change a password, the message "Changing password for \$user.", which usually gets printed before changing a password, does not get echoed.

```
hp@DESKTOP- :/mnt/c/Users/HP$ passwd
Changing password for hp.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
hp@DESKTOP- :/mnt/c/Users/HP$ passwd -q
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
hp@DESKTOP- :/mnt/c/Users/HP$
```

- ◇ **-r, --repository REPO:** This option is used to change password for repository named "REPO".
- ◇ **-R, --root CHROOT\_DIR:** Apply changes in the CHROOT\_DIR directory and use the configuration files from the CHROOT\_DIR directory. This basically changes the root directory for the passwd process for once, and since CHROOT\_DIR is a sub-directory of the root, it cannot access the configuration files outside the CHROOT\_DIR.
- ◇ **-S, --status:** Shows the password status (7 fields) of user in the following format:  
user1 P 12/22/2018 0 99999 7 3

The first field is the user's login name. The second field indicates if the user account has a locked password (L), has no Password (NP), or has a usable password (P). The third field gives the date of the last password change. The next four fields are the minimum age, maximum age, warning period, and inactivity period for the password. These ages are expressed in days.

**-S [, --status] -a [, --all]:** This combination of options shows password status for all users. Note that -a or --all cannot be used without -S option.

- ◇ **-u, --unlock:** Unlock the password of an account.



- ◇ **-w, --warndays WARN\_DAYS:** This option is used to change the number of days before the password is to expire, to display the warning for expiring password.
- ◇ **-x, --maxdays MAX\_DAYS** Set the maximum number of days for which the password remains valid. After MAX\_DAYS, the password will expire and the user will be forced to change password.

## Logout Command

### NAME

**logout** - session-logout.

### SYNOPSIS

```
session-logout[options]
```

### DESCRIPTION

**logout** command allows you to programmatically logout from your session. causes the session manager to take the requested action immediately.

### EXAMPLES

Example-1:

- ◇ To logout from current user session:
- ◇ \$ logout
- ◇ output:
- ◇ no output on screen, current user session will be logged out.

## 3.1.2 Internal and External Commands

### Internal and External Commands in Linux

The UNIX system is command-based, which means that things happen as a result of the commands you type in. UNIX commands are rarely more than four characters.

They are classified into two groups:

- ◇ **Internal Commands:** These are commands that are built into the shell. The execution of all shell built-in commands is rapid because the shell does not have to search the supplied path for them in the PATH variable, and no process is needed to execute it.  
Examples include source, cd, fg, and so on.
- ◇ **External Commands:** These are commands that are not incorporated into the shell. When an external command is to be executed, the shell looks for its path in the PATH variable, spawns a new process, and executes the command. They are typically found at /bin or /usr/bin. For example, when you run the "cat" command, which is generally found in /usr/bin, the executable /usr/bin/cat is run.  
Examples include ls, cat, and others.



## Notes

If you are familiar with UNIX commands, you have probably heard of the `ls` command. Because `ls` is a programme or file with its own existence in the `/bin` directory (or `/usr/bin`), it is labelled as an external command. This indicates that the `ls` command is not built into the shell and is existing in a separate file. In other words, when you type the `ls` command, it will be discovered in `/bin` and executed. Most commands are external in nature, but there are a few that are not truly located anywhere and are not generally performed even if they are in one of the directories indicated by `PATH`. Consider the `echo` command:

### \$type echo

`echo` is a shell builtin.

`echo` is not an external command in the sense that when you write `echo`, the shell will not check in its `PATH` for it (even if it is in `/bin`). It will instead execute it from its own collection of built-in commands, which are not saved in separate files. These built-in commands, of which `echo` is one, are referred to as internal commands.

You may have observed that the shell is the one who actually does all of these things. This software begins to execute as soon as the user logs in and terminates when the user logs out. The shell is an external command with a twist in that it has its own set of internal commands. So, if a command exists both as an internal shell command and as an external command (in `/bin` or `/usr/bin`), the shell will prioritise its own internal command of the same name.

This is also true for `echo`, which is likewise found in `/bin` but is rarely used since the shell ensures that the internal `echo` command takes precedence over the external. Now, let us speak about internal and external commands in greater detail.

### Getting the list of Internal Commands

If you are using `bash` shell you can get the list of shell built-in commands with `help` command:

#### \$help

```
//this will list all
the shell built-in commands//
```

### How to find out whether a command is internal or external?

In addition to this you can also find out about a specific command i.e., whether it is internal or external with the help of `type` command:

#### \$type cat

```
cat is/bin/cat
```

```
//specifying that cat is
external type//
```

#### \$type cd

cd is a shell builtin

//specifying that cd is

internal type//

### Internal vs External

The question of when to use which command, internal or external, is meaningless because the user employs a command based on the needs of the problem he wants to solve. The only difference between internal and external commands is that internal commands execute significantly faster than external commands because the shell must look for the path when using external commands.

In some circumstances, you can avoid using external by substituting internal. For example, if you need to add two numbers, you can do so as://use of internal command let

for addition//

**\$let c=a+b**

instead of using:

//use of external command expr

for addition//

**\$c=`expr \$a+\$b`**

In such a case, the use of let will be a better option as it is a shell built-in command so it will work faster than the expr which is an external command.

### Check Your Understanding

#### Fill Ups

1. Commands of the telnet are identified by a ..... character
2. ....are commands that are built into the shell
3. Most commands are ..... in nature.
4. Normal users can use ..... to invoke root privileges and change the password for root.

#### True/False

1. When no other modes are triggered, default mode is used.
2. Line editing, such as echoing and character deleting, is performed on the client side
3. UNIX commands are rarely more than two characters.
4. External commands execute significantly faster than internal commands

#### Summary

- When the connection is established between local and remote computer. During telnet operation whatever that is being performed on the distant computer will be presented by local computer. Telnet works on a client/server model. Local

Notes

## Notes

computer uses telnet client programme while the remote computers uses telnet server programme.

- When secure management access for the Management server is disabled, you can connect to it via the network using a telnet session.
- Logout command allows you to programmatically logout from your session. Causes the session manager to take the requested action immediately.
- The UNIX system is command-based, which means that things happen as a result of the commands you type in. UNIX commands are rarely more than four characters.

### Activity

1. Prepare a PowerPoint presentation on internal v/s external commands

### Glossary

- **TELNET-** TELNET is an abbreviation for TErminaL NETwork. It is a form of protocol that allows one computer to communicate with another. It is a standard TCP/IP protocol for virtual terminal service provided by ISO.
- **Password Command-** In Linux, the passwd command is used to update user account passwords. The root user has the ability to change the password for any user on the system, but a normal user can only change the password for his or her own account.

### Question and Exercises

1. Describe the different modes of operation?
2. How to process the passwd command?
3. What is the difference between Internal and External Commands
4. What is echo?

### Further Readings

1. The Linux Programming Interface: A Linux and UNIX System Programming by Michael Kerrisk
2. How Linux Works: What Every Superuser Should Know

### Answers to check your Understanding

#### Fill Ups

- |             |                      |
|-------------|----------------------|
| 1. Prefix   | 2. Internal Commands |
| 3. External | 4. sudo              |

#### True/False

- |          |          |
|----------|----------|
| 1. True  | 2. True  |
| 3. False | 4. False |

## Unit-3.2: Unix File Handling

### Notes

### Objectives:

At the end of this unit you will be able to:

- Understand current working directory, referring to home directories
- Learn the Commands to move around by path concept, creating new directories
- Learn creating new directories, creating files –touch, cat
- Listing directory and File - ls, hidden files
- copying files; moving files, Deleting files and directories
- View file - cat, more, pg, less
- wildcards and Meta characters

### Introduction

The Unix file system is a logical technique of organising and storing huge volumes of data in an easy-to-manage fashion. A file is the smallest unit of storage for information. The Unix file system includes a number of useful features. In Unix, all data is arranged into files. All of the files are separated into folders. The file system is a tree-like structure that organises these folders.

A directory tree is a multi-level hierarchy structure that organises files in a Unix system. A directory called “root” is represented by a “/” at the very top of the file system. All other files are root’s “descendants.”

### 3.2.1 Current Working Directory, Referring to Home Directories

#### Current working directory

The directory in which the user is now working is referred to as the current working directory. You are operating within a directory every time you interact with your command prompt.

When you connect onto your Linux system, the current working directory is set to your home directory by default. To change the working directory use the **cd** command.

For example, to change the current working directory to **/tmp** you would type:

**cd /tmp**

If you have a customized **bash** or **zsh** prompt, the path to the current working directory may be shown in the shell prompt.

**user@host:/tmp#**

#### pwd Command

The **pwd** command stands for print working directory. It is one of the most basic and frequently used commands in Linux. When invoked the command prints the complete path of the current working directory.

## Notes

**pwd** is a shell builtin in most modern shells such as bash and zsh. Its behavior is slightly different than the standalone **/bin/pwd** executable. You can use the **type** command to display all locations containing **pwd**:

**type -a pwd**

**pwd is a shell builtin**

**pwd is /bin/pwd**

As you can see from the output below, the shell builtin has priority over the standalone executable and it is used whenever you type **pwd**. If you want to use the standalone **pwd** binary type the full path to the file **/bin/pwd**

### How to Find your Current Working Directory

To find out what directory you are currently in, type **pwd** your terminal:

**pwd**

The output will look something like this:

**/home/linuxize**

What the **pwd** does is printing the **PWD environment variable**.

You will get the same output if you type:

**echo \$PWD**

**/home/linuxize**

The **pwd** command can accept only two arguments:

- ◇ **-L (--logical)** - Do not resolve symlinks.
- ◇ **-P (--physical)** - Display the physical directory, without any symbolic links.

By default, if no option is used, **pwd** behaves as if **-L** option is specified.

To better illustrate how the **-P** option works, let's **create a directory** and **symlink** pointing to the directory:

**mkdir /tmp/directoryln -s /tmp/directory /tmp/symlink**

Now if you navigate to the **/tmp/symlink** directory and you type **pwd** in your terminal:

**pwd**

The output will show that your current working directory is **/tmp/symlink**:

**/tmp/symlink**

If you run the same command using the **-P** option:

**pwd -P**

The command will print the directory to which the symlink points to:

**/tmp/directory**

The current working directory is the directory from which you invoke commands in your terminal.

The **pwd** command is used to display the current working directory.

### Referring to home directories

The Linux home directory is a directory that contains individual files for a certain user of the system. It is also known as the login directory. After logging into a Linux machine, this is the first location that appears. It is built automatically as “/home” for each user in the directory’. It is a normal root directory subdirectory. The root directory holds all of the system’s directories, subdirectories, and files. It is represented by a forward slash (/).

Except for root, the home directory can be thought of as a personal working space for all users. Each user has his or her own directory. For instance, two users named ‘jtp1’ and ‘jtp2’ will have folders named “/home/jtp1” and “/home/jtp2.” These users will have full access to their directory files.

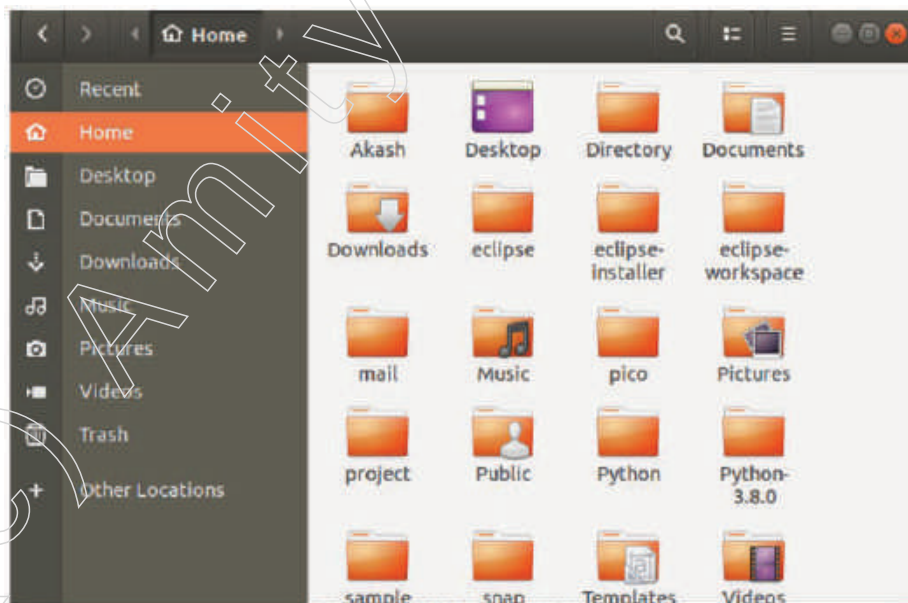
The root (administrative) user is the only one whose home directory is by default in a different place. The root user’s path is ‘/root/’, where it has control over all directories and files.

### How to find the Home Directory?

There are multiple ways to access and return to the home directory. Some commands are very helpful for the directories such as cd

- , pwd
- , mkdir
- , pwd
- , ls
- , and rmdir

To access the home directory graphically, open the files application, and click on the Home option from the left side menu. Consider the below image:



## Notes

Here, we can explore our home directory.

Generally, our terminal opens with the user's particular directory. To change directory to home directory, execute the cd command as follows:

```
cd /home
```

The above command will change the directory to home.

```
[root@localhost ~]# cd /home
[root@localhost home]#
```

To list the home directory, execute the ls command as follows:

```
ls
```

Consider the below output:

```
[root@localhost home]# ls
Books Docs
[root@localhost home]#
```

We can get back to our home directory by executing the cd command without any argument. It will back to our home directory from any directory we are working on. Execute it as follows:

```
cd
```

Consider the below output:

```
[root@localhost ~]# cd /home/Docs/report
[root@localhost report]# cd
[root@localhost ~]#
```

From the above output, we were in /home/Docs/report and by executing only cd command, we reached to root directory. We can use the "cd ~ or cd \$HOME" commands as well, to do the same. Consider the below commands:

```
cd ~
```

```
[root@localhost ~]# cd /home/Docs/report
[root@localhost report]# cd ~
[root@localhost ~]#
```

```
cd $Home
```

```
[root@localhost ~]# cd /home/Docs/report
[root@localhost report]# cd $Home
[root@localhost ~]#
```

To display the current working directory, execute the pwd command as follows:

```
pwd
```

Consider the below output:



```
[root@localhost ~]# cd /home/Docs/report
[root@localhost report]# pwd
/home/Docs/report
[root@localhost report]#
```

To create a directory under a directory, execute the mkdir command as follows:

mkdir <directory name>

Consider the below output:

```
[root@localhost report]# mkdir new_dir
[root@localhost report]# ls
new_dir
[root@localhost report]#
```

In the above output, we have created a directory as 'new\_dir', and displayed it by executing the ls command.

We can remove a directory as well. To remove a directory, execute the rmdir command as follows:

rmdir <directory name>

Consider the below output:

```
[root@localhost report]# rmdir new_dir
[root@localhost report]# ls
[root@localhost report]#
```

In the above output, we have removed the 'new\_dir' directory.

### 3.2.2 Commands to Move Around by Path Concept, Creating New Directories

MV stands for move. mv is used to move one or more files or directories from one place to another in a file system like UNIX. It has two distinct functions:

- (i) It renames a file or folder.
- (ii) It moves a group of files to a dissimilar directory.

No additional space is expended on a disk during renaming. This command normally works silently means no prompt for validation.

Syntax:

mv [Option] source destination

Let us consider 4 files having names a.txt, b.txt, and so on till d.txt.

To rename the file a.txt to sample.txt (not exist):

```
[root@localhost report]# ls
a.txt b.txt c.txt d.txt
[root@localhost report]# mv a.txt sample.txt
[root@localhost report]# ls
b.txt c.txt d.txt sample.txt
[root@localhost report]#
```

## Notes

The destination file will be created if it does not already exist. `mv` simply replaces the source filename in the directory with the destination filename in the preceding command (new name). If the destination file already exists, it will be overwritten, and the source file will be removed. `mv` does not prompt for overwriting existing files by default, so be cautious!! Let's try to understand with an example, moving `sample.txt` to `b.txt` (exist):

```
[root@localhost report]# ls
b.txt c.txt d.txt sample.txt
[root@localhost report]# cat sample.txt
sample text a
[root@localhost report]# cat b.txt
sample text b
[root@localhost report]# mv sample.txt b.txt
[root@localhost report]# ls
b.txt c.txt d.txt
[root@localhost report]# cat b.txt
sample text a
[root@localhost report]#
```

### Options:

1. **-i (Interactive):** Like in `cp`, the `-i` option makes the command ask the user for confirmation before moving a file that would overwrite an existing file, you have to press `y` for confirm moving, any other key leaves the file as it is. This option doesn't work if the file doesn't exist, it simply rename it or move it to new location.
2. **-f (Force):** `mv` prompts for confirmation overwriting the destination file if a file is write-protected. The `-f` option overrides this minor protection and overwrites the destination file forcefully and deletes the source file.

```
[root@localhost report]# ls
a.txt b.txt c.txt d.txt new_file.txt ro_file.txt
[root@localhost report]# cat b.txt
sample text b
[root@localhost report]# ls -l b.txt
-r--r--r-- 1 root root 14 Jan 31 17:31 b.txt
[root@localhost report]# mv -i new_file.txt b.txt
mv: overwrite 'b.txt'? n
[root@localhost report]# ls
a.txt b.txt c.txt d.txt new_file.txt ro_file.txt
[root@localhost report]# mv -f new_file.txt b.txt
[root@localhost report]# ls
a.txt b.txt c.txt d.txt ro_file.txt
[root@localhost report]# cat b.txt
sample text a
[root@localhost report]#
```

3. **-n (no-clobber):** With `-n` option, `mv` prevent an existing file from being overwritten.

In the following example the effect is for nothing to happen as a file would be overwritten.

```
[root@localhost report]# ls
a.txt b.txt c.txt d.txt ro_file.txt
[root@localhost report]# cat b.txt
sample text b
[root@localhost report]# mv -n a.txt b.txt
[root@localhost report]# ls
a.txt b.txt c.txt d.txt ro_file.txt
[root@localhost report]# cat b.txt
sample text b
[root@localhost report]#
```

4. **-b(backup)**: With this choice, it is easier to take a backup of an existing file that will be overwritten as a result of the mv command. This will create a backup file with the tilde character (~) attached to it.

```
[root@localhost report]# ls
a.txt b.txt c.txt d.txt ro_file.txt
[root@localhost report]# mv -b a.txt b.txt
[root@localhost report]# ls
b.txt b.txt~ c.txt d.txt ro_file.txt
[root@localhost report]#
```

5. **--version**: This option is used to display the version of mv which is currently running on your system.

```
[root@localhost report]# mv --version
mv (GNU coreutils) 8.32
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Written by Mike Parker, David MacKenzie, and Jim Meyering.
[root@localhost report]#
```

Linux **which** command is used to classify the location of a given executable that is executed when you type the executable name (command) in the terminal prompt. The command searches for the executable specified as an argument in the directories listed in the PATH environment variable.

```
[root@localhost report]# which ls
ls='ls --color=auto
/bin/ls'
[root@localhost report]#
```

### What is PATH

In Linux, **PATH** is an **environmental variable** that tells the shell and other programs which directories to search for executable files. It consists of a list of colon-separated absolute paths to directories containing the executables.

To view the contents of your PATH variable, use the **echo command** with **\$PATH** as an argument:

**echo \$PATH**

The output will look something like below:

## Notes

```
[root@localhost report]# echo $PATH
/usr/local/sbin:/bin:/sbin:/usr/bin:/usr/sbin:/usr/local/bin
[root@localhost report]#
```

### How to Use the which Command

The syntax for the **which** command is as follows:

**which [OPTIONS] FILE\_NAME...**

For example, to find the full path of the **ping** command, you would type the following:

**which ping**

The output will be something like this:

```
[root@localhost report]# which ping
/bin/ping
[root@localhost report]#
```

You can also provide more than one arguments to the **which** command:

**which ls uptime**

The output will include full paths to both **ls** and **uptime** executables:

```
[root@localhost report]# which ls uptime
ls='ls --color=auto
/bin/ls
/bin/uptime
[root@localhost report]#
```

The search is done from left to right, and if more than one matches are found in the directories listed in the **PATH** path variable, **which** will print only the first one. To print all matches, use the **-a** option:

**which -a touch**

The output will show two full paths to the **touch** command:

```
[root@localhost report]# which -a touch
/bin/touch
/usr/bin/touch
[root@localhost report]#
```

Usually one of the executables is only a **symlink** to the other one, but in some cases, you may have two versions of the same command installed in different locations or totally different commands using the same name.

### Creating new directories

The **mkdir** stands for 'make directory'. With the help of **mkdir** command, you can create a new directory wherever you want in your system. Just type "**mkdir <dir name>**", in place of **<dir name>** type the name of new directory, you want to create and then press enter.

Syntax:



**mkdir <dirname>**

```
[root@localhost exercise]# pwd
/home/Docs/exercise
[root@localhost exercise]# ls
existing_dir
[root@localhost exercise]# mkdir new_dir
[root@localhost exercise]# pwd
/home/Docs/exercise
[root@localhost exercise]# ls
existing_dir new_dir
[root@localhost exercise]# mkdir new_dir
mkdir: cannot create directory 'new_dir': File exists
[root@localhost exercise]#
```

In above example, I am in /home/Docs/exercise directory. I have made a directory 'new\_dir' by passing command "mkdir new\_dir".

Now, if I try to create a new directory with the name 'new\_dir' that technically already exists, I receive an error notice.

Note: If you do not provide a location, your file will be produced in the current directory only. If you want to construct your directory somewhere else, enter the path to your desired location, and your file will be generated there.

### To make multiple directories

Syntax:

1. **mkdir <dirname1> <dirname2> <dirname3> ...**

```
[root@localhost exercise]# ls
existing_dir new_dir
[root@localhost exercise]# mkdir dir1 dir2 dir3
[root@localhost exercise]# ls
dir1 dir2 dir3 existing_dir new_dir
[root@localhost exercise]#
```

You can also create multiple directories simultaneously. In the example above, we have created multiple directories 'dir1 dir2 dir3'.

Mkdir Options

| Options             | Description                                 |
|---------------------|---------------------------------------------|
| mkdir -p, -parents  | Add directory including its sub directory.  |
| mkdir -v, -verbose  | Print a message for each created directory. |
| mkdir -m -mode=MODE | Set access privilege.                       |

### 3.2.3 Creating New Directories, Creating Files –touch, Cat

#### Creating files

##### Linux touch command

touch command is a way to create empty files (there are some other methods also). You can update the modification and access time of each file with the help of touch command.

Notes

## Notes

Syntax:

`touch <filename>`

Example:

`touch myfile1`

```
[root@localhost exercise]# touch myfile1
[root@localhost exercise]# ls
dir1 dir2 dir3 existing_dir myfile1 new_dir
[root@localhost exercise]# touch myfile2
[root@localhost exercise]# ls
dir1 dir2 dir3 existing_dir myfile1 myfile2 new_dir
[root@localhost exercise]# touch myfile3 myfile4
[root@localhost exercise]# ls
dir1 dir2 dir3 existing_dir myfile1 myfile2 myfile3 myfile4 new_dir
[root@localhost exercise]#
```

Look at the screenshot above; we used the touch command to create two files, 'myfile1' and 'myfile2'. To make numerous files, use a single touch command followed by the enter key to enter all of the file names. For example, if you would like to create 'myfile3' and 'myfile4' simultaneously, then your command will be:

`touch myfile3 myfile4`

touch Options

| Option                | Function                                                     |
|-----------------------|--------------------------------------------------------------|
| <code>touch -a</code> | To change file access and modification time.                 |
| <code>touch -m</code> | It is used to only modify time of a file.                    |
| <code>touch -r</code> | To update time of one file with reference to the other file. |
| <code>touch -t</code> | To create a file by specifying the time.                     |
| <code>touch -c</code> | It doesn't create an empty file.                             |

### Linux touch -a command

touch command with option 'a' is used to change the access time of a file. By default, it will take the current time of your system.

Syntax:

`touch -a <filename>`

Example:

`touch -a c.txt`

To see the access and change time of your file, you need to use stat command.

```
[root@localhost report]# ls
b.txt b.txt~ c.txt d.txt no_file.txt
[root@localhost report]# stat c.txt
 File: c.txt
 Size: 14 Blocks: 8 IO Block: 4096 regular file
Device: ch/12d Inode: 229154 Links: 1
Access: (0644/-rw-r--r--) Uid: (0/ root) Gid: (0/ root)
Access: 2022-01-31 17:04:44.087000000 +0530
Modify: 2022-01-31 17:04:44.087000000 +0530
Change: 2022-01-31 17:04:44.080000000 +0530
 Birth: -
[root@localhost report]# touch -a c.txt
[root@localhost report]# stat c.txt
 File: c.txt
 Size: 14 Blocks: 8 IO Block: 4096 regular file
Device: ch/12d Inode: 229154 Links: 1
Access: (0644/-rw-r--r--) Uid: (0/ root) Gid: (0/ root)
Access: 2022-01-31 17:04:44.087000000 +0530
Modify: 2022-01-31 17:04:44.087000000 +0530
Change: 2022-01-31 18:02:27.586000000 +0530
 Birth: -
[root@localhost report]#
```

In above snapshot we have used 'stat' command (which we'll learn in later tutorial) just to check the status of our file (c.txt). So don't get confused with that. Now you can match the access time of file c.txt before and after passing the command 'touch -a c.txt'. It has taken the default access time of our system.

Linux touch -m command

The touch '-m' option will help you to change only the modification time of a file.

Syntax:

touch -m <filename>

Example:

touch -m c.txt

```
[root@localhost report]# stat c.txt
 File: c.txt
 Size: 14 Blocks: 8 IO Block: 4096 regular file
Device: ch/12d Inode: 229154 Links: 1
Access: (0644/-rw-r--r--) Uid: (0/ root) Gid: (0/ root)
Access: 2022-01-31 17:04:44.087000000 +0530
Modify: 2022-01-31 17:04:44.087000000 +0530
Change: 2022-01-31 18:02:27.586000000 +0530
 Birth: -
[root@localhost report]# touch c.txt
[root@localhost report]# touch -m c.txt
[root@localhost report]# stat c.txt
 File: c.txt
 Size: 14 Blocks: 8 IO Block: 4096 regular file
Device: ch/12d Inode: 229154 Links: 1
Access: (0644/-rw-r--r--) Uid: (0/ root) Gid: (0/ root)
Access: 2022-01-31 18:05:06.403000000 +0530
Modify: 2022-01-31 18:05:06.403000000 +0530
Change: 2022-01-31 18:05:06.403000000 +0530
 Birth: -
[root@localhost report]#
```



## Notes

Notice carefully in the above snapshot, only modification time has been changed.

Linux touch -r option

This command will update time with reference to the other mentioned command. There are two ways to use this command. Both works the same.

In below example, we want to change time-stamp of d.txt' with reference to 'c.txt'. First you can write it as,

```
touch -r c.txt d.txt
```

or

```
touch d.txt -r c.txt
```

First we'll see the status of the file before using touch command.

```
[root@localhost report]# ls
b.txt b.txt~ c.txt d.txt ro_file.txt
[root@localhost report]# stat d.txt
 File: d.txt
 Size: 14 Blocks: 8 IO Block: 4096 regular file
Device: ch/12d Inode: 229155 Links: 1
Access: (0644/-rw-r--r--) Uid: (0/ root) Gid: (0/ root)
Access: 2022-01-31 17:04:57.894000000 +0530
Modify: 2022-01-31 17:04:57.894000000 +0530
Change: 2022-01-31 17:04:57.894000000 +0530
 Birth: -
[root@localhost report]# touch -r c.txt d.txt
[root@localhost report]# stat d.txt
 File: d.txt
 Size: 14 Blocks: 8 IO Block: 4096 regular file
Device: ch/12d Inode: 229155 Links: 1
Access: (0644/-rw-r--r--) Uid: (0/ root) Gid: (0/ root)
Access: 2022-01-31 18:05:06.403000000 +0530
Modify: 2022-01-31 18:05:06.403000000 +0530
Change: 2022-01-31 18:07:12.085000000 +0530
 Birth: -
[root@localhost report]#
```

Now after using touch -r d.txt c.txt command, time of c.txt has been changed with reference to time of d.txt

```
[root@localhost report]# stat c.txt
 File: c.txt
 Size: 14 Blocks: 8 IO Block: 4096 regular file
Device: ch/12d Inode: 229154 Links: 1
Access: (0644/-rw-r--r--) Uid: (0/ root) Gid: (0/ root)
Access: 2022-01-31 18:05:06.403000000 +0530
Modify: 2022-01-31 18:05:06.403000000 +0530
Change: 2022-01-31 18:05:06.403000000 +0530
 Birth: -
[root@localhost report]# touch -r d.txt c.txt
[root@localhost report]# stat c.txt
 File: c.txt
 Size: 14 Blocks: 8 IO Block: 4096 regular file
Device: ch/12d Inode: 229154 Links: 1
Access: (0644/-rw-r--r--) Uid: (0/ root) Gid: (0/ root)
Access: 2022-01-31 18:05:06.403000000 +0530
Modify: 2022-01-31 18:05:06.403000000 +0530
Change: 2022-01-31 18:09:45.402000000 +0530
 Birth: -
[root@localhost report]#
```

### Linux touch -t command

with this command, you can change the access time of a file by determining a specified time to it.

It will modify the time by specified time instead of default time.

Format of time will be:

`touch -t YYYYMMDDhhmm.ss`

Below screenshot shows status of file b.txt before and after the touch command,

```
[root@localhost report]# stat b.txt
File: b.txt
Size: 14 Blocks: 8 IO Block: 4096 regular file
Device: ch/12d Inode: 229168 Links: 1
Access: (0644/-rw-r--r--) Uid: (0/ root) Gid: (0/ root)
Access: 2022-01-31 18:00:42.322000000 +0530
Modify: 2022-01-31 18:00:42.322000000 +0530
Change: 2022-01-31 18:00:42.322000000 +0530
Birth: -
[root@localhost report]# touch -t 202201261830 b.txt
[root@localhost report]# stat b.txt
File: b.txt
Size: 14 Blocks: 8 IO Block: 4096 regular file
Device: ch/12d Inode: 229168 Links: 1
Access: (0644/-rw-r--r--) Uid: (0/ root) Gid: (0/ root)
Access: 2022-01-26 18:30:00.000000000 +0530
Modify: 2022-01-26 18:30:00.000000000 +0530
Change: 2022-01-31 18:12:19.271000000 +0530
Birth: -
[root@localhost report]#
```

### Linux touch -c command

Using 'c' option with touch command will not create an empty file, if that file doesn't exist.

Syntax:

`touch -c <filename>`

Example:

`touch -c z.txt`

```
[root@localhost report]# ls
b.txt b.txt~ c.txt d.txt ro_file.txt
[root@localhost report]# touch -c z.txt
[root@localhost report]# ls
b.txt b.txt~ c.txt d.txt ro_file.txt
[root@localhost report]#
```

In above figure, we wanted to create file 'z.txt' but with 'c' option no file has been created.

### Cat

Cat (concatenate) command is very frequently used in Linux. It reads data from the file and gives their content as output. It helps us to create, view, concatenate files. So let us see some frequently used cat commands.

Notes

## Notes

### 1) To view a single file:

Command:

`$cat filename`

Output is like this:

```
[root@localhost report]# cat b.txt
sample text b
[root@localhost report]#
```

It will show content of given filename.

### 2) To view multiple files:

Command:

`$cat file1 file2`

Output is below:

```
[root@localhost report]# cat b.txt c.txt
sample text b
sample text c
[root@localhost report]#
```

This will show the content of file1 and file2.

### 3) To view contents of a file preceding with line numbers:

Command:

`$cat -n filename`

Output will show content with line number:

```
[root@localhost report]# cat d.txt
sample text d.
second line in this file.
[root@localhost report]# cat -n d.txt
 1 sample text d.
 2 second line in this file.
[root@localhost report]#
```

### 4) Create a file:

Command:

`$ cat > e.txt`

Output shows the file named e.txt:

```
[root@localhost report]# cat > e.txt
file e has one line.
^C
[root@localhost report]# ls
b.txt b.txt~ c.txt d.txt e.txt ro_file.txt
[root@localhost report]# cat e.txt
file e has one line.
[root@localhost report]#
```

**5) Copy the contents of one file to another file:**

Command:

`$cat [filename-whose-contents-is-to-be-copied] > [destination-filename]`

Output

```
[root@localhost report]# ls
b.txt b.txt~ c.txt d.txt e.txt no_file.txt
[root@localhost report]# cat d.txt
sample text d.
second line in this file.
[root@localhost report]# cat e.txt
file e has one line.
[root@localhost report]# cat d.txt > e.txt
[root@localhost report]# cat e.txt
sample text d.
second line in this file.
[root@localhost report]#
```

The content will be copied in destination file

**6) Cat command can suppress repeated empty lines in output:**

Command:

`$cat -s d.txt`

Output

```
[root@localhost report]# cls
sh: cls: command not found
[root@localhost report]# cat d.txt
sample text d.

second line in this file.

another line.
[root@localhost report]# cat -s d.txt
sample text d.

second line in this file.

another line.
[root@localhost report]#
```

It has suppressed repeated empty lines in output.

**7) Cat command can append the contents of one file to the end of another file:**

Command:

`$cat file1 >> file2`

Output



## Notes

```
[root@localhost report]# cat c.txt
sample text c
[root@localhost report]# cat d.txt
sample text d.

second line in this file.

another line.
[root@localhost report]# cat c.txt >> d.txt
[root@localhost report]# cat d.txt
sample text d.

second line in this file.

another line.
sample text c
[root@localhost report]#
```

It has appended the contents of c.txt file to the end of d.txt file.

### 8) Cat command can display content in reverse order using tac command:

Command:

\$tac filename

Output

```
[root@localhost exercise]# ls
a.txt b.txt c.txt d.txt
[root@localhost exercise]# cat d.txt
Sample Text d.
This file originally had two lines.

[root@localhost exercise]# tac d.txt
This file originally had two lines.
Sample Text d.
```

It displays content in reverse order.

### 9) Cat command can highlight the end of line:

Command:

\$cat -E "filename"

Output

```
[root@localhost exercise]# cat -E d.txt
Sample Text d.$
This file originally had two lines.$
$
$
[root@localhost exercise]#
```

Text Lines are highlighted with the end of line marker \$.

- 10) If you want to use the -v, -E and -T option together, then instead of writing:

vET in the command, you can just use the -A command line option.

Command

\$cat -A "filename"

```
[root@localhost exercise]# cat -A d.txt
Sample Text d.$
This file originally had two lines.$
$
$
```

- 11) Cat command to open dashed files:

Command:

\$cat -- "-dashfile"

Output

```
[root@localhost exercise]# ls
a.txt b.txt c.txt d.txt -new
[root@localhost exercise]# cat -- -new
The quick brown fox jumps right over the lazy dog.
[root@localhost exercise]#
```

It displays the content of -new file.

- 12) Cat command if the file has a lot of content and can't fit in the terminal:

Command:

\$cat "filename" | more

Output

```
[root@localhost exercise]# ls
a.txt b.txt c.txt d.txt long.txt
[root@localhost exercise]# cat long.txt | more
The quick brown fox jumps right over the lazy dog. 0010
The quick brown fox jumps right over the lazy dog. 0020
The quick brown fox jumps right over the lazy dog. 0030
The quick brown fox jumps right over the lazy dog. 0040
The quick brown fox jumps right over the lazy dog. 0050
The quick brown fox jumps right over the lazy dog. 0060
The quick brown fox jumps right over the lazy dog. 0070
The quick brown fox jumps right over the lazy dog. 0080
The quick brown fox jumps right over the lazy dog. 0090
The quick brown fox jumps right over the lazy dog. 0100
The quick brown fox jumps right over the lazy dog. 0110
The quick brown fox jumps right over the lazy dog. 0120
The quick brown fox jumps right over the lazy dog. 0130
The quick brown fox jumps right over the lazy dog. 0140
The quick brown fox jumps right over the lazy dog. 0150
The quick brown fox jumps right over the lazy dog. 0160
The quick brown fox jumps right over the lazy dog. 0170
The quick brown fox jumps right over the lazy dog. 0180
The quick brown fox jumps right over the lazy dog. 0190
The quick brown fox jumps right over the lazy dog. 0200
The quick brown fox jumps right over the lazy dog. 0210
The quick brown fox jumps right over the lazy dog. 0220
The quick brown fox jumps right over the lazy dog. 0230
The quick brown fox jumps right over the lazy dog. 0240
The quick brown fox jumps right over the lazy dog. 0250
The quick brown fox jumps right over the lazy dog. 0260
The quick brown fox jumps right over the lazy dog. 0270
The quick brown fox jumps right over the lazy dog. 0280
The quick brown fox jumps right over the lazy dog. 0290
--More--
```

## Notes

Will show that much content, which could fit in terminal and will ask to show more.

### 13) Cat command to merge the contents of multiple files:

Command:

```
$cat "filename1" "filename2" "filename3" > "merged_filename"
```

It will merge the contents of file in respective order and will insert that content in "merged\_filename".

```
[root@localhost exercise]# ls
a.txt b.txt c.txt d.txt long.txt
[root@localhost exercise]# cat a.txt
sample text a
[root@localhost exercise]# cat b.txt
Sample Text b
[root@localhost exercise]# cat c.txt
Sample text c
[root@localhost exercise]# cat a.txt b.txt c.txt > abc.txt
[root@localhost exercise]# cat abc.txt
sample text a
Sample Text b
Sample text c
[root@localhost exercise]#
```

In this example, contents of a.txt, b.txt and c.txt are merged into abc.txt file retaining all source formatting.

### 14) Cat command to display the content of all text files in the folder:

Command:

```
$cat *.txt
```

Output will show the content of all text files present in the folder:

```
[root@localhost exercise]# ls
abc.txt a.txt b.txt c.txt d.txt dump
[root@localhost exercise]# cat *.txt
sample text a
Sample Text b
Sample text c
sample text a
Sample Text b
Sample text c
Sample Text d.
This file originally had two lines.

[root@localhost exercise]#
```

### 15) Cat command to write in an already existing file:

Command :

```
$cat >> d.txt
```

This line was appended using cat from console.

Output



```
[root@localhost exercise]# cat d.txt
Sample Text d.
This file originally had two lines.

[root@localhost exercise]# cat >> d.txt
This line was appended using cat from console.
^C
[root@localhost exercise]# cat d.txt
Sample Text d.
This file originally had two lines.

This line was appended using cat from console.
[root@localhost exercise]#
```

Will append the new text to the end of the file.

### 3.2.4 Listing Directory and File - ls, hidden files

#### Hidden files

The easiest way to show hidden files on Linux is to use the ls command with the “-a” option for “all”.

\$ ls -a <path>

```
LS(1) User Commands LS(1)
NAME
 ls - list directory contents
SYNOPSIS
 ls [OPTION]... [FILE]...
DESCRIPTION
 List information about the FILES (the current directory by default).
 Sort entries alphabetically if none of -cftuvSUX nor --sort is speci-
 fied.

 Mandatory arguments to long options are mandatory for short options
 too.

 -a, --all
 do not ignore entries starting with .
 -A, --almost-all
 do not list implied . and ..
```

For example, in order to show hidden files in a user home directory, this is the command that you would run.

Alternatively, you can use the “-A” flag in order to show hidden files on Linux.

Using “A”, implied files will not be shown (for example the previous folder also named “.”).

\$ ls -al ~

```
total 356
drwxr-xr-x 14 schkn schkn 4096 Oct 26 06:08 . <-- Not shown with -A
drwxr-xr-x 3 root root 4096 Jan 5 2019 .. <-- Not shown with -A
-rw-r----- 1 schkn schkn 43436 Oct 26 06:08 .bash_history
```

## Notes

```
-rw-r--r-- 1 schkn schkn 220 Apr 4 2018 .bash_logout
-rw-r--r-- 1 schkn schkn 3771 Apr 4 2018 .bashrc
drwx----- 2 schkn schkn 4096 Jan 5 2019 .cache
```

```
$ ls -A ~
```

```
total 348
```

```
-rw----- 1 schkn schkn 43436 Oct 26 06:08 .bash_history
-rw-r--r-- 1 schkn schkn 220 Apr 4 2018 .bash_logout
-rw-r--r-- 1 schkn schkn 3771 Apr 4 2018 .bashrc
drwx----- 2 schkn schkn 4096 Jan 5 2019 .cache
```

In this case, the hidden files are the `.bash_history`, the `.bash_logout`, the `.bashrc` and the `.cache` files.

### Show Exclusively Hidden Files using ls

In some cases, you are not interested in other files that the ones that are hidden.

To show exclusively hidden files on Linux, use the `ls` command with a special regex.

```
$ ls -dl .[^.]* <path>
```

For example, given the example we have described before, we would get the following output.

```
$ ls -dl .[^.]* ~
```

```
-rw----- 1 schkn schkn 43436 Oct 26 06:08 .bash_history
-rw-r--r-- 1 schkn schkn 220 Apr 4 2018 .bash_logout
-rw-r--r-- 1 schkn schkn 3771 Apr 4 2018 .bashrc
drwx----- 2 schkn schkn 4096 Jan 5 2019 .cache
drwx----- 5 schkn schkn 4096 Jan 5 2019 .config
```

Note that the previous command also show hidden directories in the path specified.

### Show Hidden Files on Linux using find

Another effective method for locating hidden files across your entire system is to utilise the search command with a globbing character. To show all the hidden files on your system, run “find” with the name option.

```
$ find / -name ".*" 2> /dev/null
```

It is worth noting that the command’s output is diverted to `/dev/null` to avoid being presented with directories that you cannot access.

Run “find” with the `maxdepth` option to display hidden files in the current working directory. `$ find . -name ".*" -maxdepth 1 2> /dev/null`

```
schkn@schkn-ubuntu:~$ find ~ -name ".*" -maxdepth 1 -type d 2> /dev/null
/home/schkn/.ssh
/home/schkn/.config
/home/schkn/.keras
/home/schkn/.cache
/home/schkn/.pm2
/home/schkn/.npm
/home/schkn/.gnupg
schkn@schkn-ubuntu:~$
```

### Show Hidden Directories using find

To show hidden directories in the current working directory, without recursive search, use the “find” command and specify the “d” type.

```
$ find . -name ".*" -maxdepth 1 -type d 2> /dev/null
```

### Show Hidden Files on Linux using dir

The dir command is a command close to the ls command on Linux : it displays directory contents on your system.

Similarly, to the ls command, it can be used in order to show hidden files in a directory.

To show hidden files, run the “dir” command with the “-a” or the “-A” option (in order to exclude implied files and folders).

```
$ dir -a <path>
```

```
$ dir -A <path>
```

For example, to show hidden files in your home directory, you would run

```
$ dir -A ~
```

```
.bash_history .dbshell .mongorc.js .viminfo scripts
```

Note that the dir command also shows hidden directories that may be located in the path you are search for.

Similarly, to the ls command, you can choose to show hidden files exclusively in a folder, not to be bothered with all the other files.

```
$ dir -dl .[^.]* <path>
```

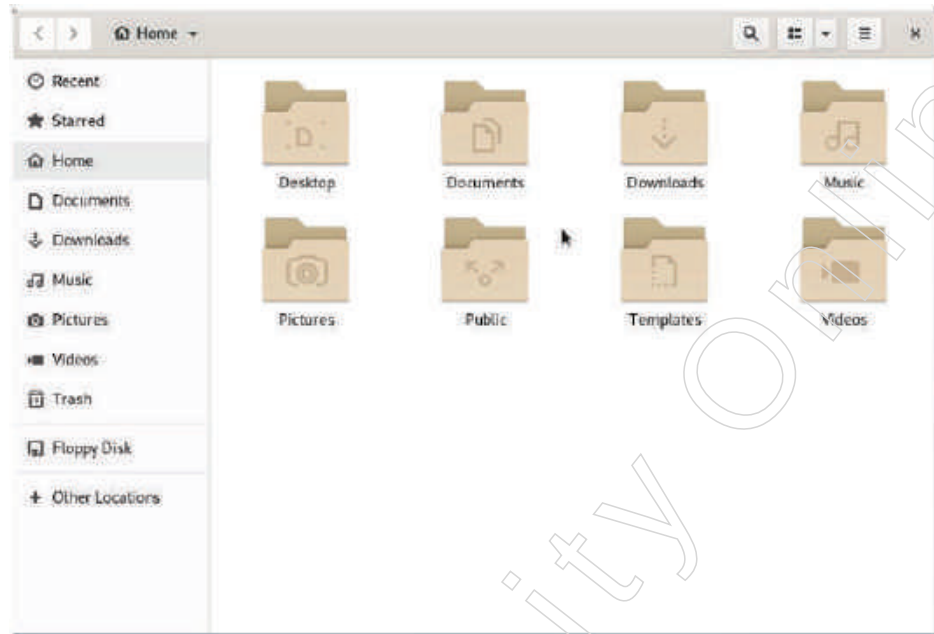
For example, in the home directory, that would give

```
$ dir -dl .[^.]* ~
-rw----- 1 schkn schkn 43436 Oct 26 06:08 .bash_history
-rw-r--r-- 1 schkn schkn 220 Apr 4 2018 .bash_logout
-rw-r--r-- 1 schkn schkn 3771 Apr 4 2018 .bashrc
drwx----- 2 schkn schkn 4096 Jan 5 2019 .cache
```

### Display Hidden Files in GNOME Desktop Environment

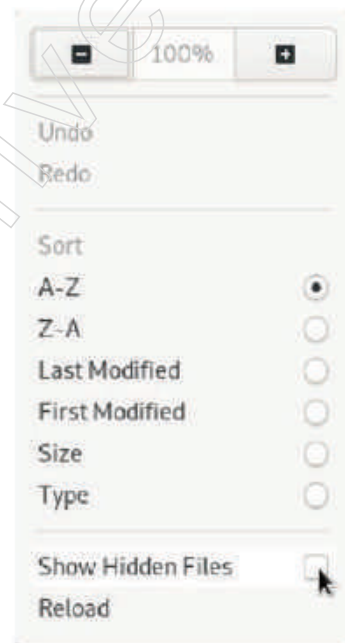
Finally, if you are using the GNOME desktop environment, you can display hidden files when browsing your system with a file explorer.

## Notes

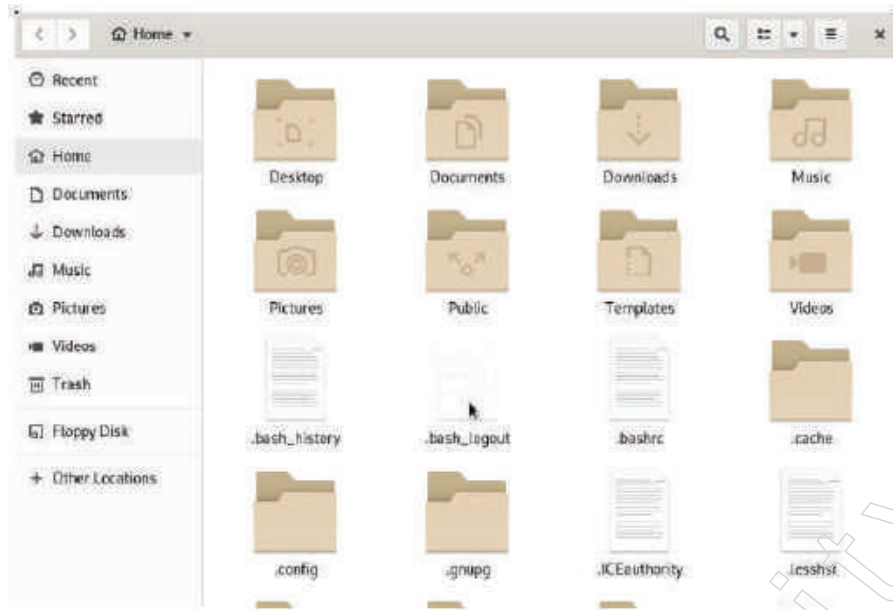


To view hidden files in the GNOME interface, click the small bottom arrow in the upper-right corner of the screen.

Check the “Show Hidden Files” checkbox when a little menu displays.



As a consequence, hidden files and folders will be revealed in the File Explorer.



### 3.2.5 Copying Files; Moving Files, Deleting Files and Directories

#### Linux Copy Command – cp: Copying Files

When it comes to conducting a Linux copy command, the cp command is available. You can quickly copy files and directories with this command. The cp follows this basic syntax:

```
1 cp [OPTION] SOURCE DEST
```

So, in essence, the linux copy file command above copies the SOURCE to the DEST.

Let us look at a basic linux copy command utilising the “cp command.” (Note: in Linux, copying a file creates a duplicate of it.)

```
1 cp sample_data.txt data/
```

- ◇ As seen in the above example, we use the cp Linux copy command to copy the “sample data.txt” file to the “data” folder. (Note: If the target folder does not exist at the time of the copy, Linux will create it automatically.)
- ◇ Expert VPS Hosting Provider
- ◇ Root Access to Run Any Software Stack
- ◇ Professional Linux Administrators on Hand
- ◇ 24/7/365 Technical Support

#### Linux Copy File / Directory Options

Furthermore, when invoking the copy command in Linux, arguments can be sent to it. For example, while using the Linux copy file command, we can specify the following options. :-i: the “i” stands for “interactive” and with this option you can specify if existing file (i.e., sample\_data.txt) in the destination directory (i.e. data) should be overwritten during the copy process.

-r: the “r” stands for “recursive” and it allows you to copy recursively meaning all sub-directories and files thus preserving the existing structure.

## Notes

-v: the “v” stands for “verbose” which tells the shell do display copy process as it’s being executed on the screen.

As you can see, the Linux copy file command is simple to use while also being highly powerful. There is a lot more you can do with it that is outside the scope of this post.

Let us go on to the next section, which is about moving files and folders. **Linux Move Command – mv : Moving (and Renaming) Files**

Following that, we will look at how to move a file in Linux. When migrating a directory, this can also be used.

In Linux, the mv command is useful for moving and renaming files. This command, also known as the Linux move command, is used to move and rename files. For example:

```
1 mv sample_data.txt data/
```

The above command simply moves sample\_data.txt to the data directory. In the event the data directory does not exist, the system will create it and move sample\_data.txt there.

And if sample\_data.txt is already existing in data, it will be overwritten.

### Linux mv Command Options

Just as with the Linux copy command, the Linux mv command allows you to specify options:

-i: the “i” stands for “interactive” and with this option you can specify if existing file (i.e. sample\_data.txt) in the destination directory (i.e. data) should be overwritten during the copy process.

-f: the “f” stands for “force” and it disables all interactivity thus running the Linux mv command without prompts. This can be risky so make sure you know what you’re doing when using the -f option.

-v: the “v” stands for “verbose” which tells the shell do display copy process as its being executed on the screen.

### Linux Delete File – rm : Deleting Files

You have probably caught the gist of it by now. Linux has the rm command for deleting files, just like the cp and mv commands.

The syntax for the Linux delete file command is as follows:

```
1 rm sample_data.txt
```

Where sample\_data.txt is the file being deleted by this Linux delete file command.

### Linux Delete File Command Options

As with the rest of the other commands, the Linux delete file command provides additional options such as -i interactive, -f force, -v verbose, and -r recursive.

Furthermore, you can apply it move than one file, for example:



```
1 rm sample_data.txt no_data.txt my_data.txt, your_data.txt, test.txt
```

As well as use wildcard character \* when working with it, for example:

```
1 rm *_data.txt
```

The above will remove sample\_data.txt no\_data.txt my\_data.txt, your\_data.txt BUT not test.txt.

The rm Linux delete file command can also be applied to directories.

For example, this command:

```
1 rm -r testfolder
```

will delete the testfolder and everything in it because the -r recursive option is applied.

Now that you've gotten basic understanding on how to copy, move and delete files in Linux, there are few things you should remember:

-i (Interactive) is Your Friend

Using the -i with your commands is a wise approach as it allows you to confirm your selection and avoid unnecessary accidents.

-r (Recursive) / -f (Force) – Use with Caution

In addition, it is prudent to use the -r and -f options with caution.

Equally most important when you're using wildcard \*. For example, can you imagine accidentally running the following command:

```
1 rm -r *
```

Let that sink in a moment.

You've just deleted everything!

Therefore, it's wise to be prudent.

### 3.2.6 View file - cat, more, pg, less

#### The pg and more Commands

A long output can normally be zipped by you on the screen, but if you run text through more or use the pg command as a filter; the display stops once the screen is full of text.

Let's assume that you have a long directory listing. To make it easier to read the sorted listing, pipe the output through more as follows:

## Notes

```
$ls -l | grep "Aug" | sort +4n | more
```

```
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
```

```
-rw-rw-r-- 1 john doc 2488 Aug 15 10:51 intro
```

```
-rw-rw-rw- 1 john doc 8515 Aug 6 15:30 ch07
```

```
-rw-rw-r-- 1 john doc 14827 Aug 9 12:40 ch03
```

```
.
```

```
.
```

```
.
```

```
-rw-rw-rw- 1 john doc 16867 Aug 6 15:56 ch05
```

```
--More--(74%)
```

Once the screen is full of text consisting of lines arranged by file size, the screen will fill up. The more prompt is at the bottom of the screen, where you can write a command to move through the sorted content.

When you are finished viewing this screen, you can use any of the commands provided in the more program's description.

### 3.2.7 Wildcards and Meta Characters

#### Wildcards

In Linux, a wildcard is a symbol or collection of symbols that stands in for other characters. It can stand in for any other character or characters in a string. You can, for example, use a wildcard to retrieve a list of all files in a directory that start with the letter O.

- ◇ In Linux, three types of wildcards are commonly used: **?** – matches a single character. For example, **O??d** matches anything that begins with **O**, ends with **d** and has two characters in between (like **Oind**, **Okhd**, **Oerd**, but not **Oereed**, **Oad**, **Oerererd**.)
- ◇ **\*** – matches any character or set of characters, including no character. For example, **O\*d** matches anything that begins with **O** and ends with **d** (like **Oind**, **Okhd**, **Oerd**, **Oereed**, **Oad**, **Oerererd**, **Od**, **Oarmeerrd**). The number of characters in between **O** and **d** is not important.
- ◇ **Bracketed values** – match characters enclosed in square brackets. For example, **O[ac]d** matches only **Oad** and **Ocd**. You can also specify a range of values: **O[a-e]d** matches **Oad**, **Obd**, **Ocd**, **Odd** and **Oed**.

Let's look at a couple of examples. We've created files mentioned above:

If we want to list all files that begin with O, end with d and have two characters in between, we can use the following syntax:

```
bob@ubuntu:~/reg_expr$ ls -l 0??d
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0erd
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0ind
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0khd
bob@ubuntu:~/reg_expr$
```

To list all files that begin with **0** and end with **d**, no matter the number of character in between, we can use the following syntax:

```
bob@ubuntu:~/reg_expr$ ls -l 0*d
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0ad
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0armeerrd
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0bd
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0cd
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0d
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0dd
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0ed
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0erd
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0ereed
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0erererd
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0ind
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0khd
bob@ubuntu:~/reg_expr$
```

To list all files that begin with **0** and end with **d** and have **a** or **c** in between, we need to use the following syntax:

```
bob@ubuntu:~/reg_expr$ ls -l 0[ac]d
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0ad
-rw-rw-r-- 1 bob bob 0 May 9 05:27 0cd
bob@ubuntu:~/reg_expr$
```

### Metacharacters:

In a certain context, a character that has a distinctive significance to the shell or another application. Metacharacters are used in the shell's recognition of ambiguous file references and in the regular expressions recognised by various utilities. If you want to utilise a metacharacter without evoking its particular meaning, you must quote it.

For your convenience, the shell metacharacters are given below:

| Symbol  | Meaning                                                    |
|---------|------------------------------------------------------------|
| >       | Output redirection                                         |
| >>      | Output redirection (append)                                |
| <       | Input redirection                                          |
| *       | File substitution wildcard; zero or more characters        |
| ?       | File substitution wildcard; one character                  |
| [ ]     | File substitution wildcard; any character between brackets |
| `cmd`   | Command Substitution                                       |
| \$(cmd) | Command Substitution                                       |
|         | The Pipe ( )                                               |
| ;       | Command sequence, Sequences of Commands                    |
| [ ]     | File substitution wildcard; any character between brackets |
|         | OR conditional execution                                   |
| &&      | AND conditional execution                                  |
| ( )     | Group commands, Sequences of Commands                      |

## Notes

|    |                                                        |
|----|--------------------------------------------------------|
| &  | Run command in the background, Background Processes    |
| #  | Comment                                                |
| \$ | Expand the value of a variable                         |
| \  | Prevent or escape interpretation of the next character |
| << | Input redirection                                      |

Like traditional regular expressions, SED also supports metacharacters. These are Perl style regular expressions. Note that metacharacter support is GNU SED specific and may not work with other variants of SED. Let us discuss metacharacters in detail.

### Word Boundary (\b):

In regular expression terminology, “\b” matches the word boundary. For example, “\bthe\b” matches “the” but not “these”, “there”, “they”, “then”, and so on. The following example illustrates this.

```
[jerry]$ echo -e "these\nthe\nthey\nthen" | sed -n '\bthe\b/ p'
```

On executing the above code, you get the following result:

```
the
```

### Non-Word Boundary (\B):

In regular expression terminology, “\B” matches non-word boundary. For example, “the\B” matches “these” and “they” but not “the”. The following example illustrates this.

```
[jerry]$ echo -e "these\nthe\nthey" | sed -n '/the\B/ p'
```

On executing the above code, you get the following result:

```
these
they
```

### Single Whitespace (\s):

In SED, “\s” implies single whitespace character. The following example matches “Line\t1” but does not match “Line1”.

```
[jerry]$ echo -e "Line\t1\nLine2" | sed -n '/Line\s/ p'
```

On executing the above code, you get the following result:

```
Line 1
```

### Single Non-Whitespace (\S):

In SED, “\S” implies single non-whitespace character. The following example matches “Line2” but does not match “Line\t1”.

```
[jerry]$ echo -e "Line\t1\nLine2" | sed -n '/Line\S/ p'
```

On executing the above code, you get the following result:

Line2

### Single Word Character (\w):

In SED, “\w” implies single word character, i.e., alphabetical characters, digits, and underscore (\_). The following example illustrates this.

```
[jerry]$ echo -e "One\n123\n1_2\n&#" | sed -n '/\w/ p'
```

On executing the above code, you get the following result:

```
One
123
1_2
```

### Single Non-Word Character (\W):

In SED, “\W” implies single non-word character which is exactly opposite to “\w”. The following example illustrates this.

```
[jerry]$ echo -e "One\n123\n1_2\n&#" | sed -n '/\W/ p'
```

On executing the above code, you get the following result:

```
&#
```

### Beginning of Pattern Space (\^):

In SED, “\^” implies the beginning of the pattern space. The following example matches only the word “One”.

```
[jerry]$ echo -e "One\nTwo One" | sed -n '/^\^One/ p'
```

On executing the above code, you get the following result:

```
One
```

### Check Your Understanding

#### Fill Ups

1. MV stands for .....
2. To view the contents of your PATH variable, use the ..... command with \$PATH as an argument
3. touch command is a way to create .....
4. The ..... command is a command close to the ls command on Linux : it displays directory contents on your system.
5. The ..... command arranges lines of text alphabetically

#### True/False

1. The mkdir command stands for ‘make directory’

## Notes

- Using 'c' option with touch command will create an empty file, if that file doesn't exist.
- The easiest way to show hidden files on Linux is to use the ls command with the "-a" option for "all".

You can quickly copy files and directories with cp command

### Summary

- The Linux home directory is a directory that contains individual files for a certain user of the system. It is also known as the login directory. After logging into a Linux machine, this is the first location that appears.
- Linux which command is used to classify the location of a given executable that is executed when you type the executable name (command) in the terminal prompt. The command searches for the executable specified as an argument in the directories listed in the PATH environment variable.
- With the help of mkdir command, you can create a new directory wherever you want in your system.
- Linux touch -r option command will update time with reference to the other mentioned command. There are two ways to use this command. Both works the same.
- Another effective method for locating hidden files across your entire system is to utilise the search command with a globbing character. To show all the hidden files on your system, run "find" with the name option.
- when invoking the copy command in Linux, arguments can be sent to it. For example, while using the Linux copy file command, we can specify the following options. -i: the "i" stands for "interactive" and with this option you can specify if existing file (i.e., sample\_data.txt) in the destination directory (i.e. data) should be overwritten during the copy process.
- A long output can normally be zipped by you on the screen, but if you run text through more or use the pg command as a filter; the display stops once the screen is full of text.
- In a certain context, a character that has a distinctive significance to the shell or another application. Metacharacters are used in the shell's recognition of ambiguous file references and in the regular expressions recognised by various utilities. If you want to utilise a metacharacter without evoking its particular meaning, you must quote it.

### Activity

- List down all the Linux commands and prepare a table for the same.

### Glossary

- Pwd-** The pwd command stands for print working directory. It is one of the most basic and frequently used commands in Linux. When invoked the command prints the complete path of the current working directory.
- PATH-** In Linux, PATH is an environmental variable that tells the shell and other programs which directories to search for executable files. It consists of a list of colon-separated absolute paths to directories containing the executables.



- **Mv**- the mv command is useful for moving and renaming files

## Notes

### Questions and Exercises

1. State two distinct functions of MV?
2. Explain Cat command?
3. Explain Linux Delete File Command?
4. State the different ways in which hidden files can be discovered?

### Further Readings

1. The Linux Command Line: A Complete Introduction
2. Advanced Programming in the UNIX Environment

### Answers to check your Understanding

#### Fill Ups

1. move
2. echo
3. empty files
4. dir
5. sort

#### True/False

1. True
2. False
3. True
4. False

## Notes

### Unit-3.3: Filters and Pipes

#### Objectives:

At the end of this unit, the reader will know about:

- Filters
- Head, tail, Cut
- Sort, WC
- Grep
- Cmp, comm., diff, uniq

#### Introduction

M.D McIlroy introduced the novel idea of Pipes in June 1972– version 2, 10 UNIX installations. Piping is a method of passing the output of one command (written on the left side) as input to another command (written on RHS). The vertical bar “|” symbol is used to connect commands.

Filters are the set of commands in UNIX/Linux that take input from the standard input stream, stdin, perform some operations, and write output to the standard output stream, stdout. Using redirection and pipes, you can manage stdin and stdout to your liking. grep, more, and sort are examples of common filter commands.

The ‘pipe’ command is supported by both the UNIX and Linux operating systems. Pipes are input/output ideas in a command that help integrate two or more commands. We use more than one pipe in command in the Linux operating system so that the output of one command before a pipe functions as input for the second command after the pipe. We have gone over a lot of examples in this post to help you grasp the fundamental operation of pipes.

#### Prerequisites

To use pipe commands on Linux, you must have a Linux environment installed on your system. This can be accomplished by obtaining a virtual box and configuring it with an Ubuntu file. Users must be granted access to the appropriate applications.

#### Syntax

```
Command 1 | command 2 | command 3 |
```

#### Sort the list using pipes:

The pipe has much functionality used to filter, sort, and display the text in the list. One of the common examples is described here. Suppose we have a file named file1.txt having the names of the students. We have used the cat command to fetch the record of that file.

```
$ Cat file1.txt
```

```
aqsayasin@virtualbox:~$ cat File1.txt
Aqsa
Hanna
Zainab
Ayesha
Hafsa
Laraib
Fatima
Haleema
Sundus
```

The data present in this file is unordered. So, to sort the data, we need to follow a piece of code here.

```
$ Cat file1.txt | sort
```

```
aqsayasin@virtualbox:~$ cat File1.txt | sort
Aqsa
Ayesha
Fatima
Hafsa
Haleema
Hanna
Laraib
Sundus
Zainab
```

Through the respective output, you can see that students' names are arranged alphabetically in a sequence from a to z.

Beside this. Suppose we want to get an output in sorted form plus removing redundancy. We will use the same command and a "uniq" keyword in addition to the default command. Let's consider a file named file2.txt having the names of subjects in it. The same command is used for fetching data.

```
$ Cat file2.txt
```

```
aqsayasin@virtualbox:~$ cat file2.txt
Biology
physics
chemistry
mathematics
chemistry
English
Science
Social Studies
Physics
English
```

Now we will use the command to remove all the words that are duplicated in the file.

```
$ Cat file2.txt | sort | uniq
```

```
aqsayasin@virtualbox:~$ cat file2.txt | sort | uniq
Biology
chemistry
English
mathematics
physics
Science
Social Studies
```

The output shows that the elements are organized and arranged alphabetically. At the same time, all the words that were duplicated are removed. The above command will only display the output, but we will use the below-cited command to save them.

```
$ cat file2.txt | sort | uniq > list4.txt
```

## Notes

The output will be saved in another file with the same extension.

### Display file data of a corresponding range:

It is very annoying when you want to get some data only from the start, but the command gives you all the matching items in your system. You can use the 'head' keyword. It helps to limit your output with concerning some range. i.e., in this example, we have declared the range up to 4. So the data will be from the first 4 lines of the file. Consider the same file file2.txt as we have taken an example above.

```
$ Cat file2.txt | head -4
```

```
aqsayasin@virtualbox:~$ cat file2.txt | head -4
Biology
physics
chemistry
mathematics
```

Similar to head, we can also use the tail option. This will limit the output to the last lines according to the range given.

### Pipe and more command:

By using more command, all the output is displayed at a time on the screen. The pipe act as a container and displays all the output data as an input of ls-l. Because the output is a long list of files.

```
$ ls -al | more
```

```
aqsayasin@virtualbox:~$ ls -al | more
total 80
drwxr-xr-x 15 aqsayasin aqsayasin 4096 17:49 15 .
drwxr-xr-x 3 root root 4096 12:04 15 ..
-rw-r--r-- 1 aqsayasin aqsayasin 220 11:41 15 .bash_logout
-rw-r--r-- 1 aqsayasin aqsayasin 3771 11:41 15 .bashrc
drwxr-xr-x 11 aqsayasin aqsayasin 4096 12:03 15 .cache
drwxr-xr-x 12 aqsayasin aqsayasin 4096 12:14 15 .config
drwxr-xr-x 2 aqsayasin aqsayasin 4096 12:00 15 Desktop
drwxr-xr-x 2 aqsayasin aqsayasin 4096 12:00 15 Documents
drwxr-xr-x 2 aqsayasin aqsayasin 4096 12:00 15 Downloads
-rw-r--r-- 1 aqsayasin aqsayasin 61 12:19 15 File1.txt
-rw-r--r-- 1 aqsayasin aqsayasin 95 17:49 15 file2.txt
drwxr-xr-x 3 aqsayasin aqsayasin 4096 12:03 15 .gnupg
drwxr-xr-x 3 aqsayasin aqsayasin 4096 12:00 15 .local
drwxr-xr-x 2 aqsayasin aqsayasin 4096 12:00 15 Music
drwxr-xr-x 2 aqsayasin aqsayasin 4096 12:00 15 Pictures
-rw-r--r-- 1 aqsayasin aqsayasin 807 11:41 15 .profile
drwxr-xr-x 2 aqsayasin aqsayasin 4096 12:00 15 Public
drwxr-xr-x 2 aqsayasin aqsayasin 4096 12:03 15 .ssh
-rw-r--r-- 1 aqsayasin aqsayasin 0 17:27 15 .sudo_as_admin_successfu
l
drwxr-xr-x 2 aqsayasin aqsayasin 4096 12:00 15 Templates
drwxr-xr-x 2 aqsayasin aqsayasin 4096 12:00 15 Videos
```

Ls is used to display all possible data of the respective command. It firstly displays the total number of data related to the corresponding query

### Count the number of files:

It is a common need to know the number of files currently present. And it is not necessary to use the grep or cat command to fetch data of all the types. We can use pipe in this case either. The command used is written as:

```
$ ls | wc -l
```

### Process identification:

Many complicated tasks are also performed by using the pipe in our commands. The command we are discussing now is used to display the process ids of the systemd processes.

```
$ ps -ef | grep systemd | awk '{print $2}'
```

### Get sub-directories using pipe:

One of the pipe queries in the pipeline commands we used to get all the present subdirectories in the current directory is one of the pipeline commands we used. In this case, we used the grep command. Grep only displays data beginning with the letter 'd.' The pipe will aid in retrieving the data from each directory. The letter 'd' is used here. \$ ls -al | grep '^d'

### Get files using pipe:

To get the files from the system of respective extensions, we can get this by using the pipe in our commands. The first example is finding the java files in the system. 'locate' and 'grep' help get the files of respective extensions.

```
$ locate "*.java" | grep java
```

'\*' is used to fetch all the files in the system.

### Use multiple pipes in a single command:

In this example, unlike the earlier ones, we have used more than one pipe in a single command to elaborate its functionality. Consider a file named file3.txt.

```
aqsayasin@virtualbox:~$ cat file3.txt
Arham 20
sajid 40
mazhar 60
Wahab 20
Yasin 10
Jawad 60
sajid 40
Yasin 10
```

Now we want to get the record of the word that matched with the name we have provided in the command. Here cat command is used to fetch the data from a particular file. Grep is used to select that specific word from the file. 'tee' is used to save the result in another file. And wc is to count the resultant data. So, the result is shown below.

```
$ Cat file3.txt | grep "yasin" | tee file4.txt | wc -l
```

```
aqsayasin@virtualbox:~$ cat file3.txt | grep "Yasin" | tee file4.txt | wc -l
2
```

The word is matched with the 2 contents. We can display the data from the new sample file to display the whole result, where the result is being stored.

## Notes

### Fetch particular data with pipes

In this example, we want to get the data from the file having 'h' in its content.

```
$ Cat file3.txt | grep h
```

```
aqsayasin@virtualbox:~$ cat file3.txt | grep h
Arham 20
mazhar 60
Walab 20
```

The result shows that the fetched data is according to the search by the 'h' command. Moving towards the following example. Here we want to fetch the items of the file having 's' in it, but we have applied a condition of case sensitivity. Both upper and lower case alphabets will be fetched.

```
$ Cat file2.txt | grep -i s
```

```
aqsayasin@virtualbox:~$ cat file2.txt | grep -i s
physics
chemistry
mathematics
chemistry
English
Science
Social Studies
physics
English
```

The result is shown in the image. Next, we will display the students' names having alphabets 'a' and 't' combined in the word. The result is in the below-cited image.

```
$ cat file1.txt | grep "a+t"
```

```
aqsayasin@virtualbox:~$ cat File1.txt | grep "a+t"
Saksh
Katrina
Fatima
Batik
tatina
```

### 3.3.1 Introduction to Filters

#### Filters in UNIX

In UNIX/Linux, filters are the set of commands that take input from standard input stream i.e. stdin, perform some operations and write output to standard output stream i.e. stdout. The stdin and stdout can be managed as per preferences using redirection and pipes. Common filter commands are: grep, more, sort.

1. **grep Command:** It is a pattern or expression matching command. It searches for a pattern or regular expression that matches in files or directories and then prints found matches.

Filters are programs that take plain text(either stored in a file or produced by another program) as standard input, transforms it into a meaningful format, and then returns it as standard output. Linux has a number of filters.



In UNIX/Linux, filters are the set of commands that take input from standard input stream i.e. stdin, perform some operations and write output to standard output stream i.e. stdout. The stdin and stdout can be managed as per preferences using redirection and pipes. Common filter commands are: grep, more, sort.

1. **grep Command:** It is a pattern or expression matching command. It searches for a pattern or regular expression that matches in files or directories and then prints found matches.

### The grep Command

The grep command searches a file or files for lines that have a certain pattern. The syntax is –

```
$grep pattern file(s)
```

The name “grep” comes from the ed (a Unix line editor) command g/re/p which means “globally search for a regular expression and print all lines containing it”.

A regular expression is either some plain text (a word, for example) and/or special characters used for pattern matching.

The simplest use of grep is to look for a pattern consisting of a single word. It can be used in a pipe so that only those lines of the input files containing a given string are sent to the standard output. If you don't give grep a filename to read, it reads its standard input; that's the way all filter programs work –

```
$ls -l | grep "Aug"
-rw-rw-rw- 1 john doc 11008 Aug 6 14:10 ch02
-rw-rw-rw- 1 john doc 8515 Aug 6 15:30 ch07
-rw-rw-r-- 1 john doc 2488 Aug 15 10:51 intro
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
$
```

There are various options which you can use along with the grep command –

| Sr.No. | Option & Description                                                  |
|--------|-----------------------------------------------------------------------|
| 1      | -v<br>Prints all lines that do not match pattern.                     |
| 2      | -n<br>Prints the matched line and its line number.                    |
| 3      | -l<br>Prints only the names of files with matching lines (letter “l”) |
| 4      | -c<br>Prints only the count of matching lines.                        |
| 5      | -i<br>Matches either upper or lowercase.                              |

Let us now use a regular expression that tells grep to find lines with “carol”, followed by zero or other characters abbreviated in a regular expression as “.\*”), then followed by “Aug”.–

## Notes

Here, we are using the -i option to have case insensitive search –

```
$ls -l | grep -i "carol.*aug"
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
$
```

### The sort Command

The sort command arranges lines of text alphabetically or numerically. The following example sorts the lines in the food file –

```
$sort food
Afghani Cuisine
Bangkok Wok
Big Apple Deli
Isle of Java
Mandalay
Sushi and Sashimi
Sweet Tooth
Tio Pepe's Peppers
$
```

The **sort** command arranges lines of text alphabetically by default. There are many options that control the sorting –

| Sr.No. | Description                                                                       |
|--------|-----------------------------------------------------------------------------------|
| 1      | -n<br>Sorts numerically (example: 10 will sort after 2), ignores blanks and tabs. |
| 2      | -r<br>Reverses the order of sort.                                                 |
| 3      | -f<br>Sorts upper and lowercase together.                                         |
| 4      | +x<br>Ignores first x fields when sorting.                                        |

More than two commands may be linked up into a pipe. Taking a previous pipe example using grep, we can further sort the files modified in August by the order of size.

The following pipe consists of the commands ls, grep, and sort –

```
$ls -l | grep "Aug" | sort +4n
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
-rw-rw-r-- 1 john doc 2488 Aug 15 10:51 intro
-rw-rw-rw- 1 john doc 8515 Aug 6 15:30 ch07
-rw-rw-rw- 1 john doc 11008 Aug 6 14:10 ch02
$
```

This pipe sorts all files in your directory modified in August by the order of size, and prints them on the terminal screen. The sort option +4n skips four fields (fields are separated by blanks) then sorts the lines in numeric order.

### The pg and more Commands

A long output can normally be zipped by you on the screen, but if you run text through more or use the **pg** command as a filter; the display stops once the screen is full of text.

Let's assume that you have a long directory listing. To make it easier to read the sorted listing, pipe the output through **more** as follows –

```
$ls -l | grep "Aug" | sort +4n | more
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
-rw-rw-r-- 1 john doc 2488 Aug 15 10:51 intro
-rw-rw-rw- 1 john doc 8515 Aug 6 15:30 ch07
-rw-rw-r-- 1 john doc 14827 Aug 9 12:40 ch03
.
.
.
-rw-rw-rw- 1 john doc 16867 Aug 6 15:56 ch05
--More--(74%)
```

The screen will fill up once the screen is full of text consisting of lines sorted by the order of the file size. At the bottom of the screen is the more prompt, where you can type a command to move through the sorted text.

Once you're done with this screen, you can use any of the commands listed in the discussion of the more program.

### 3.3.2 Head, Tail, Cut

#### HEAD command

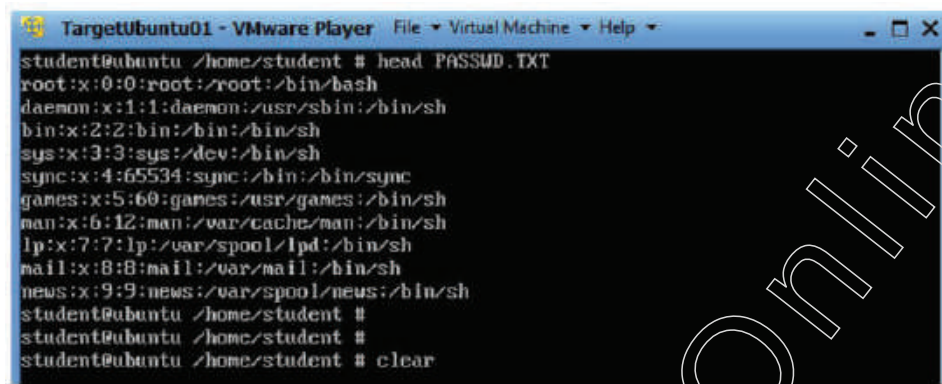
Output the first part of the file first.

head by default shows 10 lines at a time.

clear

head PASSWD.TXT

## Notes



```

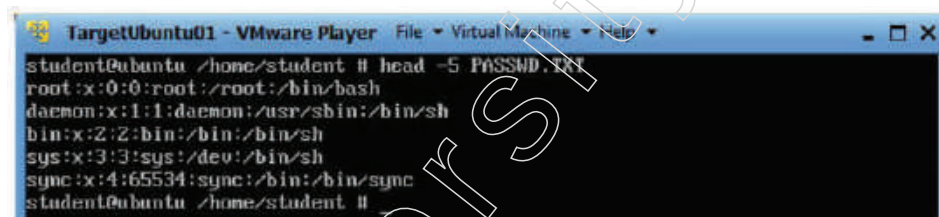
TargetUbuntu01 - VMware Player File Virtual Machine Help
student@ubuntu /home/student # head PASSWD.TXT
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
student@ubuntu /home/student #
student@ubuntu /home/student #
student@ubuntu /home/student # clear

```

clear

head -5 PASSWD.TXT

Show me the first 5 lines of the PASSWD.TXT file.



```

TargetUbuntu01 - VMware Player File Virtual Machine Help
student@ubuntu /home/student # head -5 PASSWD.TXT
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
student@ubuntu /home/student #

```

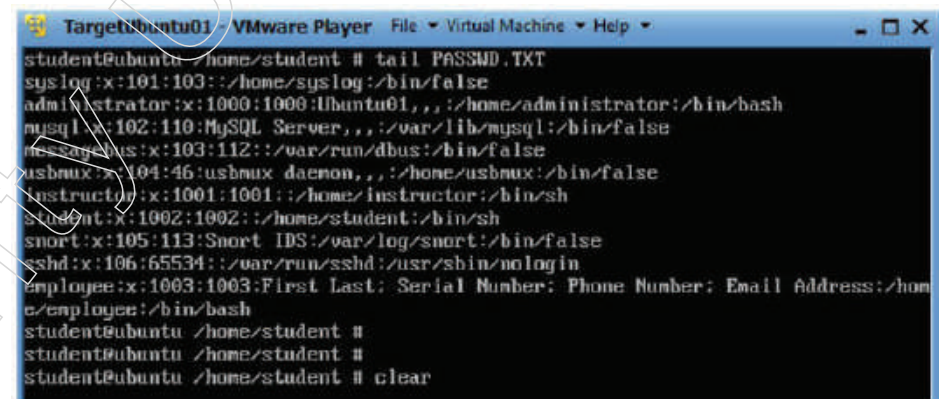
### TAIL Command

Output the last part of the file first.

last by default shows 10 lines at a time.

clear

tail PASSWD.TXT



```

TargetUbuntu01 - VMware Player File Virtual Machine Help
student@ubuntu /home/student # tail PASSWD.TXT
syslog:x:101:103:/home/syslog:/bin/false
administrator:x:1000:1000:Ubuntu01,,,:/home/administrator:/bin/bash
mysql:x:102:110:MySQL Server,,,:/var/lib/mysql:/bin/false
messagebus:x:103:112:/var/run/dbus:/bin/false
usbmux:x:104:46:usbmux daemon,,,:/home/usbmux:/bin/false
instructor:x:1001:1001:/home/instructor:/bin/sh
student:x:1002:1002:/home/student:/bin/sh
snort:x:105:113:Snort IDS:/var/log/snort:/bin/false
sshd:x:106:65534:/var/run/sshd:/usr/sbin/nologin
employee:x:1003:1003:First Last: Serial Number: Phone Number: Email Address:/home/employee:/bin/bash
student@ubuntu /home/student #
student@ubuntu /home/student #
student@ubuntu /home/student # clear

```

clear

tail -5 PASSWD.TXT

Show me the last 5 lines of the PASSWD.TXT file.

```

student@ubuntu /home/student # tail -5 PASSWD.TXT
instructor:x:1001:1001::/home/instructor:/bin/sh
student:x:1002:1002::/home/student:/bin/sh
snort:x:105:113:Snort IDS:/var/log/snort:/bin/false
sshd:x:106:65534::/var/run/sshd:/usr/sbin/nologin
employee:x:1003:1003:First Last: Serial Number: Phone Number: Email Address:/home/employee:/bin/bash
student@ubuntu /home/student #

```

### 3.3.3 Sort, wc

#### The sort Command

The sort command arranges lines of text alphabetically or numerically. The following example sorts the lines in the food file –

```

$sort food
Afghani Cuisine
Bangkok Wok
Big Apple Deli
Isle of Java
Mandalay
Sushi and Sashimi
Sweet Tooth
Tio Pepe's Peppers
$

```

The sort command arranges lines of text alphabetically by default. There are many options that control the sorting –

| Sr.No. | Description                                                                       |
|--------|-----------------------------------------------------------------------------------|
| 1      | -n<br>Sorts numerically (example: 10 will sort after 2), ignores blanks and tabs. |
| 2      | -r<br>Reverses the order of sort.                                                 |
| 3      | -f<br>Sorts upper and lowercase together.                                         |
| 4      | +x<br>Ignores first x fields when sorting.                                        |

More than two commands may be linked up into a pipe. Taking a previous pipe example using grep, we can further sort the files modified in August by the order of size.

The following pipe consists of the commands ls, grep, and sort –

Notes

## Notes

```
$ls -l | grep "Aug" | sort +4n
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
-rw-rw-r-- 1 john doc 2488 Aug 15 10:51 intro
-rw-rw-rw- 1 john doc 8515 Aug 6 15:30 ch07
-rw-rw-rw- 1 john doc 11008 Aug 6 14:10 ch02
$
```

This pipe sorts all files in your directory modified in August by the order of size, and prints them on the terminal screen. The sort option `+4n` skips four fields (fields are separated by blanks) then sorts the lines in numeric order.

### The wc command

The commands that normally take their input from the standard input can have their input redirected from a file in this manner. For example, to count the number of lines in the file `users` generated above, you can execute the command as follows –

```
$ wc -l users
2 users
$
```

Upon execution, you will receive the following output. You can count the number of lines in the file by redirecting the standard input of the `wc` command from the file `users` –

```
$ wc -l < users
2
$
```

Note that there is a difference in the output produced by the two forms of the `wc` command. In the first case, the name of the file `users` is listed with the line count; in the second case, it is not.

In the first case, `wc` knows that it is reading its input from the file `users`. In the second case, it only knows that it is reading its input from standard input so it does not display file name.

### 3.3.4 grep

The `grep` command searches a file or files for lines that have a certain pattern. The syntax is –

```
$grep pattern file(s)
```

The name “`grep`” comes from the `ed` (a Unix line editor) command `g/re/p` which means “globally search for a regular expression and print all lines containing it”.

A regular expression is either some plain text (a word, for example) and/or special characters used for pattern matching.



The simplest use of grep is to look for a pattern consisting of a single word. It can be used in a pipe so that only those lines of the input files containing a given string are sent to the standard output. If you don't give grep a filename to read, it reads its standard input; that's the way all filter programs work –

```
$ls -l | grep "Aug"
-rw-rw-rw- 1 john doc 11008 Aug 6 14:10 ch02
-rw-rw-rw- 1 john doc 8515 Aug 6 15:30 ch07
-rw-rw-r-- 1 john doc 2488 Aug 15 10:51 intro
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
$
```

There are various options which you can use along with the grep command –

| Sr.No. | Option & Description                                                  |
|--------|-----------------------------------------------------------------------|
| 1      | -v<br>Prints all lines that do not match pattern.                     |
| 2      | -n<br>Prints the matched line and its line number.                    |
| 3      | -l<br>Prints only the names of files with matching lines (letter "l") |
| 4      | -c<br>Prints only the count of matching lines.                        |
| 5      | -I<br>Matches either upper or lowercase.                              |

Let us now use a regular expression that tells grep to find lines with "carol", followed by zero or other characters abbreviated in a regular expression as ".\*"), then followed by "Aug".–

Here, we are using the -i option to have case insensitive search –

```
$ls -l | grep -i "carol.*aug"
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
$
```

### 3.3.5 cmp, comm., diff, uniq

#### cmp Command

Compare two files, and if they differ, tells the first byte and line number where they differ.

#### SYNOPSIS

cmp options... FromFile [ToFile]

#### EXAMPLES

## Notes

Compare two files

```
$ cat sample.txt
```

This is a sample text file

```
$ cat sample1.txt
```

This is another sample file

```
$ cmp sample.txt sample1.txt
```

sample.txt sample1.txt differ: byte 10, line 1

### **comm. Command**

compare two sorted files line by line and write to standard output: the lines that are common, plus the lines that are unique.

#### SYNOPSIS

```
comm [options]... File1 File2
```

#### EXAMPLES

Show the lines unique to words.txt

```
$ cat words.txt
```

Apple

Banana

Orange

India

US

Canada

```
$ cat countries.txt
```

India

US

Canada

```
$ comm -23 <(sort words.txt | uniq) <(sort countries.txt | uniq)
```

Apple

Banana

Orange

Show the lines common to words.txt and countries.txt.

```
$ comm -12 <(sort words.txt | uniq) <(sort countries.txt | uniq)
```

India

US

Canada

## Notes

### Diff Command

For example, we have two files as file.txt and file1.txt. The data has been inserted into file.txt as shown below –

I need to buy apples.

I need to run the laundry.

I need to wash the dog.

I need to get the car detailed.

file1.txt contains the data as shown below

I need to buy apples.

I need to do the laundry.

I need to wash the car.

I need to get the dog detailed.

Use the diff command to compare both the files as shown below –

```
linux@linux:~$ diff /home/linux/Desktop/file.txt /home/linux/Desktop/file1.txt
```

The above command should give the result as shown below –

```
linux@linux:~$ diff /home/linux/Desktop/file.txt /home/linux/Desktop/file1.txt
```

```
2,4c2,4
```

```
< I need to run the laundry.
```

```
< I need to wash the dog.
```

```
< I need to get the car detailed. --- > I need to do the laundry.
```

```
> I need to wash the car.
```

```
> I need to get the dog detailed.
```

The options of the result should be like this –

a - Added the text to file

c - Changes are made in the file

d - Deletion operation is performed

< Lines from the first file

> Lines from the second file

From the output, 2,4c2,4 means “Lines 2 through 4 in the first file needs to be changed in order to match lines 2 through 4 in the second file”

Let's look at another example, two text files should be like this-

## Notes

### file.txt

I need to go to the store.  
I need to buy some apples.  
When I get home, I'll wash the dog.

### file1.txt

I need to go to the store.  
I need to buy some apples.  
Oh yeah, I also need to buy grated cheese.  
When I get home, I'll wash the dog.

Use the diff command to compare both files. The command should be like this-

```
$ diff /home/linux/Desktop/file.txt /home/linux/Desktop/file1.txt
```

The above command should give the result as shown below –

```
2a3
```

```
> Oh yeah, I also need to buy grated cheese.
```

From the output, 2a3 means “After line 2 in the first file, a line needs to be added: line 3 from the second file”.

### Uniq Command

The uniq command in Linux is a command line utility that reports or filters out the repeated lines in a file.

In simple words, uniq is the tool that helps to detect the adjacent duplicate lines and also deletes the duplicate lines. uniq filters out the adjacent matching lines from the input file(that is required as an argument) and writes the filtered data to the output file .

### Syntax of uniq Command :

```
//...syntax of uniq...//
```

```
$uniq [OPTION] [INPUT[OUTPUT]]
```

The syntax of this is quite easy to understand. Here, INPUT refers to the input file in which repeated lines need to be filtered out and if INPUT isn't specified then uniq reads from the standard input. OUTPUT refers to the output file in which you can store the filtered output generated by uniq command and as in case of INPUT if OUTPUT isn't specified then uniq writes to the standard output.

Now, let's understand the use of this with the help of an example. Suppose you have a text file named kt.txt which contains repeated lines that needs to be omitted. This can simply be done with uniq.

```
//displaying contents of kt.txt//
```

### \$cat kt.txt

```
I love music.
I love music.
I love music.
I love music of Kartik.
I love music of Kartik.
Thanks.
```

Now, as we can see that the above file contains multiple duplicate lines. Now, let's use uniq command to remove them:

```
//...using uniq command.../
```

### \$uniq kt.txt

```
I love music.
I love music of Kartik.
Thanks.
/* with the use of uniq all
the repeated lines are removed*/
```

As you can see that we just used the name of input file in the above uniq example and as we didn't use any output file to store the produced output, the uniq command displayed the filtered output on the standard output with all the duplicate lines removed.

Note: uniq isn't able to detect the duplicate lines unless they are adjacent to each other. The content in the file must be therefore sorted before using uniq or you can simply use sort -u instead of uniq command.

### Options For uniq Command:

1. **-c – -count:** It tells how many times a line was repeated by displaying a number as a prefix with the line.
2. **-d – -repeated:** It only prints the repeated lines and not the lines which aren't repeated.
3. **-D – -all-repeated[=METHOD]:** It prints all duplicate lines and METHOD can be any of the following:
  - ◆ **none:** Do not delimit duplicate lines at all. This is the default.
  - ◆ **prepend:** Insert a blank line before each set of duplicated lines.
  - ◆ **separate:** Insert a blank line between each set of duplicated lines.
4. **-f N – -skip-fields(N):** It allows you to skip N fields(a field is a group of characters, delimited by whitespace) of a line before determining uniqueness of a line.
5. **-i – -ignore case:** By default, comparisons done are case sensitive but with this option case insensitive comparisons can be made.

## Notes

6. **-s N – -skip-chars(N)**: It doesn't compare the first N characters of each line while determining uniqueness. This is like the -f option, but it skips individual characters rather than fields.
7. **-u – -unique**: It allows you to print only unique lines.
8. **-z – -zero-terminated**: It will make a line end with 0 byte(NULL), instead of a newline.
9. **-w N – -check-chars(N)**: It only compares N characters in a line.
10. **-- help**: It displays a help message and exit.
11. **-- version**: It displays version information and exit.

### Examples of uniq with Options

1. **Using -c option**: It tells the number of times a line was repeated.

//using uniq with -c//

**\$uniq -c kt.txt**

3 I love music.

1

2 I love music of Kartik.

1

1 Thanks.

/\*at the starting of each  
line its repeated number is  
displayed\*/

2. **Using -d option**: It only prints the repeated lines.

//using uniq with -d//

**\$uniq -d kt.txt**

I love music.

I love music of Kartik.

/\*it only displayed one  
duplicate line per group\*/

3. **Using -D option**: It also prints only duplicate lines but not one per group.

//using -D option//

**\$uniq -D kt.txt**

I love music.

I love music.

I love music.

I love music of Kartik.

I love music of Kartik.



```
/* all the duplicate lines
are displayed*/
```

4. **Using -u option:** It prints only the unique lines.

```
//using -u option//
```

```
$uniq -u kt.txt
```

Thanks.

```
/*only unique lines are
displayed*/
```

5. **Using -f N option:** As told above, this allows the N fields to be skipped while comparing uniqueness of the lines. This option is helpful when the lines are numbered as shown in the example below:

```
//displaying contents of f1.txt//
```

```
$cat f1.txt
```

1. I love music.
2. I love music.
3. I love music of Kartik.
4. I love music of Kartik.

```
//now using uniq with -f N option//
```

```
$uniq -f 2 f1.txt
```

1. I love music.
3. I love music of Kartik.

```
/*2 is used cause we needed to
compare the lines after the
numbering 1,2.. and after dots*/
```

6. **Using -s N option:** This is similar to -f N option but it skips N characters but not N fields.

```
//displaying content of f2.txt//
```

```
$cat f2.txt
```

```
#%@I love music.
^&(I love music.
*!@thanks.
#%@!thanks.
```

```
//now using -s N option//
```

```
$uniq -s 3 f2.txt
```

```
#%@I love music.
*!@thanks.
#%@!thanks.
```

```
/*lines same after skipping
3 characters are filtered*/
```

## Notes

7. **Using -w option:** Similar to the way of skipping characters, we can also ask uniq to limit the comparison to a set number of characters. For this, -w command line option is used.

//displaying content of f3.txt//

**\$cat f3.txt**

How it is possible?

How it can be done?

How to use it?

//now using -w option//

**\$uniq -w 3 f3.txt**

How

/\*as the first 3 characters  
of all the 3 lines are same  
that's why uniq treated all these  
as duplicates and gave output  
accordingly\*/

8. **Using -i option:** It is used to make the comparison case-insensitive.

//displaying contents of f4.txt//

**\$cat f4.txt**

I LOVE MUSIC

i love music

THANKS

//using uniq command//

**\$uniq f4.txt**

I LOVE MUSIC

i love music

THANKS

/\*the lines aren't treated  
as duplicates with simple  
use of uniq\*/

//now using -i option//

**\$uniq -i f4.txt**

I LOVE MUSIC

THANKS

/\*now second line is removed  
when -i option is used\*/

9. **Using -z option:** By default, the output uniq produces is newline terminated. However, if you want, you want to have a NULL terminated output instead (useful while dealing with uniq in scripts). This can be made possible using the -z command line option.

Syntax:

//syntax of using uniq  
with -z option//

**\$uniq -z file-name**

## Notes

### 3.3.6 expr , bc, exit

#### Expr and bc Command

bc command is used for command line calculator. It is similar to basic calculator by using which we can do basic mathematical calculations.

Arithmetic operations are the most basic in any kind of programming language. Linux or Unix operating system provides the bc command and expr command for doing arithmetic calculations. You can use these commands in bash or shell script also for evaluating arithmetic expressions.

Syntax:

bc [ -hlwsqv ] [long-options] [ file ... ]

Options:

- h, {- -help } : Print the usage and exit
- i, {- -interactive } : Force interactive mode
- l, {- -mathlib } : Define the standard math library
- w, {- -warn } : Give warnings for extensions to POSIX bc
- s, {- -standard } : Process exactly the POSIX bc language
- q, {- -quiet } : Do not print the normal GNU bc welcome
- v, {- -version } : Print the version number and copyright and quit

The bc command supports the following features:

- ◇ Arithmetic operators
- ◇ Increment or Decrement operators
- ◇ Assignment operators
- ◇ Comparison or Relational operators
- ◇ Logical or Boolean operators
- ◇ Math functions
- ◇ Conditional statements
- ◇ Iterative statements

#### 1. Arithmetic Operators

Examples:

Input : \$ echo "12+5" | bc

Output : 17

Input : \$ echo "10^2" | bc

## Notes

Output : 100

How to store the result of complete operation in variable?

Example:

Input:

```
$ x=`echo "12+5" | bc`
```

```
$ echo $x
```

Output:17

Explanation: Stores the result of first line of input in variable x and then display variable x as \$x.

## 2. Assignment Operators

The list of assignments operators supported are:

- ◆ `var = value` : Assign the value to the variable
- ◆ `var += value` : similar to `var = var + value`
- ◆ `var -= value` : similar to `var = var - value`
- ◆ `var *= value` : similar to `var = var * value`
- ◆ `var /= value` : similar to `var = var / value`
- ◆ `var ^= value` : similar to `var = var ^ value`
- ◆ `var %= value` : similar to `var = var % value`

Examples:

Input: `$ echo "var=10;var" | bc`

Output: 10

Explanation: Assign 10 to the variable and print the value on the terminal.

Input: `$ echo "var=10;var^=2;var" | bc`

Output: 100

Explanation: Squares the value of the variable and print the value on the terminal.

How to store the result of complete operation in variable?

Example:

Input:

```
$ x=`echo "var=500;var%=7;var" | bc`
```

```
$ echo $x
```

Output:3

Explanation: Stores the result of 500 modulo 7 i.e. remainder of 500/7 in variable x and then display variable x as \$x.

### 3. Increment Operators

There are 2 kinds of increment operators:

- ◆ ++var : Pre increment operator, variable is increased first and then result of variable is stored.
- ◆ var++ : Post increment operator, result of the variable is used first and then variable is incremented.

Examples:

Input: `$ echo "var=10;++var" | bc`

Output: 11

Explanation: Variable is increased first and then result of variable is stored.

Input: `$ echo "var=10;var++" | bc`

Output: 10

Explanation: Result of the variable is used first and then variable is incremented.

### 4. Decrement Operators

There are 2 kinds of decrement operators:

- ◆ -- var : Pre decrement operator, variable is decreased first and then result of variable is stored.
- ◆ var -- : Post decrement operator, result of the variable is used first and then variable is decremented.

Examples:

Input: `$ echo "var=10;--var" | bc`

Output: 9

Explanation: Variable is decreased first and then result of variable is stored.

Input: `$ echo "var=10;var--" | bc`

Output: 10

Explanation: Result of the variable is used first and then variable is decremented.

### 5. Comparison or Relational Operators

Relational operators are used to compare 2 numbers. If the comparison is true, then result is 1. Otherwise(false), returns 0. These operators are generally used in conditional statements like if.

The list of relational operators supported in bc command are shown below:

- ◆ `expr1<expr2` : Result is 1 if expr1 is strictly less than expr2.
- ◆ `expr1<=expr2` : Result is 1 if expr1 is less than or equal to expr2.
- ◆ `expr1>expr2` : Result is 1 if expr1 is strictly greater than expr2.
- ◆ `expr1>=expr2` : Result is 1 if expr1 is greater than or equal to expr2.
- ◆ `expr1==expr2` : Result is 1 if expr1 is equal to expr2.

## Notes

- ◆ `expr1!=expr2` : Result is 1 if `expr1` is not equal to `expr2`.

Examples:

Input: `$ echo "10>5" | bc`

Output: 1

Input: `$ echo "1==2" | bc`

Output: 0

### 6. Logical or Boolean Operators

Logical operators are mostly used in conditional statements. The result of the logical operators is either 1(TRUE) or 0(FALSE).

- ◆ `expr1 && expr2` : Result is 1 if both expressions are non-zero.
- ◆ `expr1 || expr2` : Result is 1 if either expression is non-zero.
- ◆ `! expr` : Result is 1 if `expr` is 0.

Examples:

Input: `$ echo "10 && 5" | bc`

Output: 1

Input: `$ echo "0 || 0" | bc`

Output: 0

Input: `$ echo "! 0" | bc`

Output: 1

### 7. Mathematical Functions

The built-in math functions supported are :

- ◆ `s (x)`: The sine of `x`, `x` is in radians.
- ◆ `c (x)` : The cosine of `x`, `x` is in radians.
- ◆ `a (x)` : The arctangent of `x`, arctangent returns radians.
- ◆ `l (x)` : The natural logarithm of `x`.
- ◆ `e (x)` : The exponential function of raising `e` to the value `x`.
- ◆ `j (n,x)` : The bessel function of integer order `n` of `x`.
- ◆ `sqrt(x)` : Square root of the number `x`. If the expression is negative, a run time error is generated.

In addition to the math functions, the following functions are also supported :

- ◆ `length(x)` : returns the number of digits in `x`.
- ◆ `read()` : Reads the number from the standard input.
- ◆ `scale(expression)` : The value of the scale function is the number of digits after the decimal point in the expression.



- ◆ `ibase` and `obase` define the conversion base for input and output numbers. The default for both input and output is base 10.

- ◆ `last` (an extension) is a variable that has the value of the last printed number.

Examples:

Input:

```
$ pi=`echo "h=10;4*a(1)" | bc -l`
```

```
$ echo $pi
```

```
Output: 3.14159265358979323844
```

Explanation: Assign the value of "pi" to the shell variable pi. Here, a refers to the arctangent function, which is part of the math library loaded with the -l option.

```
Input: $ echo "scale($pi)" | bc -l
```

```
Output: 20
```

Explanation: Gives the number of digits after decimal point in value of "pi" calculated in previous example.

```
Input: $ echo "s($pi/3)" | bc -l
```

```
Output: .86602540378443864675
```

Explanation: Gives sine values at "pi/3" angle. Angle must be in radians. Here, s refers to the sine function

```
Input: $ echo "c($pi/3)" | bc -l
```

```
Output: .50000000000000000001
```

Explanation: Gives cosine values at "pi/3" angle. Angle must be in radians. Here, c refers to the cosine function.

```
Input: $ echo "e(3)" | bc -l
```

```
Output: 20.08553692318766774092
```

Explanation: Gives exponential^value as output.

```
Input: $ echo "l(e(1))" | bc -l
```

```
Output: .99999999999999999999
```

Explanation: Gives natural logarithm of the value i.e. w.r.t. base 'e'.

```
Input: $ echo "obase=2;15" | bc -l
```

```
Output: 1111
```

Explanation: Convert Decimal to Binary.

```
Input: $ echo "obase=8;9" | bc -l
```

```
Output: 11
```

Explanation: Convert Decimal to Octal.

```
Input: $ echo "ibase=2;1111" | bc -l
```

```
Output: 15
```

## Notes

Explanation: Convert Binary to Decimal.

Input: `$ echo "ibase=2;obase=8;10" | bc -l`

Output: 2

Explanation: Convert Binary to Octal.

### 8. Conditional Statements

Conditional Statements are used to take decisions and execute statements based on these decisions. bc command supports the if condition.

Syntax:

```
if(condition) {statements} else {statements}
```

Example:

Input: `$ echo 'n=8;m=10;if(n>m) print "n is greater" else print "m is greater" ' | bc -l`

Output: m is greater

### 9. Iterative statements

bc command supports the for loop and while loop for doing iterations.

Syntax:

```
for(assignment; condition; updation)
```

```
{
 statements.....

}
```

OR

```
while(condition)
```

```
{
 statements.....

}
```

Examples:

Input: `$ echo "for(i=1; i<=10; i++) {i;}" | bc`

Output:

1

2

3  
4  
5  
6  
7  
8  
9  
10

Input: `$ echo "i=1;while(i<=10) {i; i+=1}" | bc`

Output:

1  
2  
3  
4  
5  
6  
7  
8  
9  
10

Explanation: Both examples prints numbers from 1 to 10 using the respective looping syntax.

#### Some Pseudo Statements:

- ◆ **break:** This statement causes a forced exit of the most recent enclosing while statement or for statement.
- ◆ **continue:** The continue statement (an extension) causes the most recent enclosing for statement to start the next iteration.
- ◆ **halt:** The halt statement (an extension) is an executed statement that causes the bc processor to quit only when it is executed. For example, "if (0 == 1) halt" will not cause bc to terminate because the halt is not executed.
- ◆ **return:** Return the value 0 from a function. (See the section on functions).
- ◆ **return(expression):** Return the value of the expression from a function. (See the section on functions). As an extension, the parenthesis are not required.
- ◆ **limits:** Print the local limits enforced by the local version of bc. (This is an extension).

## Notes

- ◆ **quit:** When the quit statement is read, the bc processor is terminated, regardless of where the quit statement is found. For example, “if (0 == 1) quit” will cause bc to terminate.
- ◆ **warranty:** Print a warranty notice. (This is an extension).

### 10. Functions :

Functions allow you to define a computation that will be run later. Functions in bc always compute and return a value to the caller. Function definitions are “dynamic” in the sense that a function is undefined until it encounters a definition in its input. The definition function is then used until another definition function with the same name is encountered. The new definition then supersedes the previous definition. Syntax:

```
define name (parameters)
{
 statements.....

 return statement
}
```

### 11. We can write our arithmetic expressions in a file and then execute those statements by providing the filename to the bc command.

Example:

Input :

```
$ cat >> example.txt
```

```
2+5;
```

```
var = 10*3
```

```
var
```

```
print var
```

```
quit
```

Press ctrl+D

```
$ bc example.txt
```

Output :

```
bc 1.06.95
```

Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006 Free Software Foundation, Inc.

This is free software with ABSOLUTELY NO WARRANTY.

For details type `warranty'.

7

30

TO AVOID SYSTEM GENERATED MESSAGE ON OUTPUT SCREEN, USE:

\$ bc -q example.txt

Output :

7

30

### Important Points:

1. The semicolon (;) or newline is used as the statement separator by the bc command.
2. Use curly brackets to group statements. Use in conjunction with functions, if statements, for and while loops.
3. If simply an expression is supplied as a statement, the bc command evaluates it and sends the result to standard output.
4. If an assignment operator is discovered. The bc command assigns the value to the variable but does not print it to the terminal.
5. Before calling a function, it should be defined. The function definition should always come before the calling statements.
6. If a standalone variable is found as a statement, the bc programme prints the variable's value. The print statement can also be used to display a list of values on the terminal.

### Exit Command

exit-The exit command terminates a script, just as in a C program. It can also return a value, which is available to the script's parent process.

### Synopsis

exit

### Description

exit-Issuing the exit command at the shell prompt will cause the shell to exit.

In some cases, if you have jobs running in the background, the shell will remind you that they are running and simply return you to the command prompt. In this case, issuing exit again will terminate those jobs and exit the shell. Common aliases for exit include "bye", "logout", and "lo". Every command returns an exit status (sometimes referred to as a return status or exit code). A successful command returns a 0, while an unsuccessful one returns a non-zero value that usually can be interpreted as an error code. Well-behaved UNIX commands, programs, and utilities return a 0 exit code upon successful completion, though there are some exceptions.

Likewise, functions within a script and the script itself return an exit status. The last command executed in the function or script determines the exit status. Within a script, an exit nnn command may be used to deliver an nnn exit status to the shell (nnn must be an integer in the 0 - 255 range).

## Notes

### EXAMPLE-1:

```
To exit from shell:
$ exit

output:
su ubuntu

ubuntu@ubuntu:~$ exit
exit
root@ubuntu:/home/ubuntu#
```

### EXAMPLE-2:

exit command is used to return the success/failure of functionality in script.  
# cat test.sh

```
#!/bin/bash
echo "This is a test."
Terminate our shell script with success message
exit 0
root@ubuntu:/home/ubuntu# ./test.sh
This is a test.root@ubuntu:/home/ubuntu# echo $?
0
```

```
root@ubuntu:/home/ubuntu# cat test.sh
#!/bin/bash
echo "This is a test."
Terminate our shell script with failure message
exit 1
```

```
root@ubuntu:/home/ubuntu# ./test.sh
This is a test.
root@ubuntu:/home/ubuntu# echo $?
1
```

### Check Your Understanding

#### Fill-Ups

- \_\_\_\_\_ are the set of commands that take input from standard input stream.
- \_\_\_\_\_ are input/output ideas in a command that help integrate two or more commands.
- By using \_\_\_\_\_ command, all the output is displayed at a time on the screen.
- The \_\_\_\_\_ & \_\_\_\_\_ can be managed as per preferences using redirection and pipes.
- \_\_\_\_\_ is a pattern or expression matching command.

#### True/False

- The 'pipe' command is only supported by the UNIX.
- User can use more than one pipe in command in the Linux.
- Ls is used to display all possible data of the respective command.
- The sort command arranges lines of text randomly by default.
- bc command is used for command line calculator.

## Summary

- The 'pipe' command is supported by both the UNIX and Linux operating systems.
- Pipes are input/output ideas in a command that help integrate two or more commands.
- To use pipe commands on Linux, it must have a Linux environment installed on the system.
- This can be accomplished by obtaining a virtual box and configuring it with an Ubuntu file. Users must be granted access to the appropriate applications.
- The pipe has much functionality used to filter, sort, and display the text in the list.
- In UNIX/Linux, filters are the set of commands that take input from standard input stream i.e. stdin, perform some operations and write output to standard output stream i.e. stdout.
- The stdin and stdout can be managed as per preferences using redirection and pipes. Common filter commands are: grep, more, sort.
- grep Command: It is a pattern or expression matching command. It searches for a pattern or regular expression that matches in files or directories and then prints found matches.
- Filters are programs that take plain text (either stored in a file or produced by another program) as standard input, transforms it into a meaningful format, and then returns it as standard output.
- Linux has a number of filters.

## Activity

1. Suppose you have a file named file8.txt having the names of the students. You have to use the cat command to fetch the record of that file.

## Glossary

- **Pipes-** Input/output ideas in a command that help integrate two or more commands.
- **Filters-** The set of commands that take input from standard input stream.
- **stdin-** the device from which the system receives input.
- **stdout-** The standard output device
- **Grep command-** Command that searches a file or files for lines that have a certain pattern.
- -v- Prints all lines that do not match pattern.
- -n- Prints the matched line and its line number.
- -l- Prints only the names of files with matching lines (letter "l")
- -c- Prints only the count of matching lines.
- -I- Matches either upper or lowercase.

## Questions and Exercises

1. Differentiate between pipe and filters.

## Notes



## Notes

2. Write in short about grep command.
3. Discuss the usages of bc command.
4. What do you understand from head and tail command?
5. Write in brief about sort command.

### Further Readings

1. Advanced UNIX Programming — Rochkind
2. UNIX and Linux System Administration Handbook, 4th Edition

### Answers to Check Your Understanding

#### Fill-Ups

1. Filters
2. Pipes
3. More
4. Stdin, stdout
5. Grep command

#### True/False

1. False
2. True
3. True
4. False
5. True

## Unit-3.4: Standard Input/Output

### Notes

#### Objectives:

At the end of this unit, the reader will be able to:

- Understand the concept of redirection
- Comprehend the redirection input
- Talk about the redirection output and Errors

#### Introduction

The standard input device (stdin) is the tool wherein the system takes input. The keyboard is typically used for this, but you can clarify that input arrived from some other source, such as a serial port or a disc file. The standard output device, also known as stdout, is where system output is sent. This is typically a display, but it could also be a serial port or a file.

The operator redirects input on operating-system commands and MQ control commands. The input is read from a file when this operator is followed by a file name. Similarly, the > operator redirects output; if it is followed by a filename, output is directed to that file.

The standard input device, often known as stdin, is the device from which the system receives input. Typically, this is the keyboard, but you can specify that input come from another source, such as a serial port or a disc file. The standard output device, often known as stdout, is the device to which system output is sent. This is usually a display, although it can also be a serial port or a file.

The operator redirects input on operating-system instructions and MQ control commands. When this operator is followed by a file name, the input is read from the file. Similarly, the > operator redirects output; if accompanied by a file name, output is directed to that file.

```
$ cat
This is a line of text.
This is a line of text.
Cat keeps copying lines of text
Cat keeps copying lines of text
Until you press CONTROL-D at the beginning
Until you press CONTROL-D at the beginning
of a line.
of a line.
CONTROL-D
$
```

The cat utility copies standard input to standard output

#### 3.4.1 Introduction to Redirection

The standard input device, often known as stdin, is the device from which the system receives input. Typically, this is the keyboard, but you can specify that input

## Notes

come from another source, such as a serial port or a disc file. The standard output device, often known as stdout, is the device to which system output is sent. This is usually a display, although it can also be a serial port or a file.

The operator redirects input on operating-system instructions and MQ control commands. When this operator is followed by a file name, the input is read from the file. Similarly, the > operator redirects output; if accompanied by a file name, output is directed to that file.

### 3.4.2 Redirecting Input

You can redirect standard input in the same way that you can redirect standard output. The redirect input symbol (<) tells the shell to redirect a command's input to come from the given file rather than the keyboard. A command line that redirects input has the following syntax:

```
command [arguments] < filename
```

where **command** is any executable program (such as an application program or a utility), **arguments** are optional arguments, and **filename** is the name of the ordinary file the shell redirects the input from.

In this method, commands that ordinarily take their input from standard input can have their input redirected from a file. For example, to count the number of lines in the file users generated above, run the following command:

```
$ wc -l users
```

```
2 users
```

```
$
```

Following execution, you will receive the output shown below. By redirecting the standard input of the wc command from the file users, you may count the number of lines in the file.

```
$ wc -l < users
```

```
2
```

```
$
```

It is worth noting that the output of the two variants of the wc command differs. In the first scenario, the file users' names are printed alongside the line count; in the second case, they are not. In the first scenario, wc is aware that it is receiving input from file users. In the second example, it simply knows that it is reading from standard input, thus no file name is displayed.

#### Here Document:

A here document is used to redirect input into a shell script or application that is interactive. We can execute an interactive programme within a shell script without requiring user interaction by providing the necessary input for the interactive programme or interactive shell script.

A here document takes the following general form:

```
command << delimiter
document
delimiter
```

In this case, the shell understands the operator as an instruction to read input until it finds a line that contains the provided delimiter. All of the input lines up to the line containing the delimiter are subsequently fed into the command's standard input.

The delimiter informs the shell that the here document is finished. Without it, the shell will read the input indefinitely. The delimiter must be a single word with no spaces or tabs.

```
The following is the output of the command wc -l, which counts the total
number of lines.$wc -l << EOF
 This is a simple lookup program
 for good (and bad) restaurants
 in Cape Town.
EOF
3
$
```

You can use the here document to print multiple lines using your script as follows –

```
#!/bin/sh
cat << EOF
This is a simple lookup program
for good (and bad) restaurants
in Cape Town.
EOF
```

Upon execution, you will receive the following result –

```
This is a simple lookup program
for good (and bad) restaurants
in Cape Town.
```

The following script runs a session with the vi text editor and saves the input in the file test.txt.

```
#!/bin/sh
filename=test.txt
vi $filename <<EndOfCommands
i
This file was created automatically from
a shell script
^[
ZZ
EndOfCommands
```

If you run this script with vim acting as vi, then you will likely see output like the following –

## Notes

```
$ sh test.sh
Vim: Warning: Input is not from a terminal
$
```

After running the script, you should see the following added to the file test.txt –

```
$ cat test.txt
This file was created automatically from
a shell script
$
```

### 3.4.3 Redirecting Output and Error

The *redirect output symbol* (>) instructs the shell to redirect the output of a command to the specified file instead of to the screen. The syntax of a command line that redirects output is

*command [arguments] > filename*

where **command** is any executable program (e.g., an application program or a utility), **arguments** are optional arguments, and **filename** is the name of the ordinary file the shell redirects the output to.

There is no output shown at the terminal. This is because the output has been redirected from the terminal's default standard output device into the chosen file. You can find the whole material in the users file. \$ cat users

```
oko tty01 Sep 12 07:30
ai tty15 Sep 12 13:32
ruth tty21 Sep 12 10:10
pat tty24 Sep 12 13:07
steve tty25 Sep 12 13:03
$
```

If a command has its output redirected to a file and the file already contains some data, that data will be lost. Consider the following example –

```
$ echo line 1 > users
$ cat users
line 1
$
```

You can use >> operator to append the output in an existing file as follows –

```
$ echo line 2 >> users
$ cat users
line 1
line 2
$
```

### Discard the output:

You may need to run a command but do not want the output visible on the screen. In such circumstances, the output can be discarded by diverting it to the file /dev/null. \$ command > /dev/null

You may need to run a command but do not want the output visible on the screen. In such circumstances, the output can be discarded by diverting it to the file /dev/null. \$ command > /dev/null 2>&1

Here 2 represents STDERR and 1 represents STDOUT. You can display a message on to STDERR by redirecting STDOUT into STDERR as follows –

```
$ echo message 1>&2
```

### Redirection Commands:

Following is a complete list of commands which you can use for redirection –

| Sr.No. | Command & Description                                                                       |
|--------|---------------------------------------------------------------------------------------------|
| 1      | <b>pgm &gt; file</b><br>Output of pgm is redirected to file                                 |
| 2      | <b>pgm &lt; file</b><br>Program pgm reads its input from file                               |
| 3      | <b>pgm &gt;&gt; file</b><br>Output of pgm is appended to file                               |
| 4      | <b>n &gt; file</b><br>Output from stream with descriptor n redirected to file               |
| 5      | <b>n &gt;&gt; file</b><br>Output from stream with descriptor n appended to file             |
| 6      | <b>n &gt;&amp; m</b><br>Merges output from stream n with stream m                           |
| 7      | <b>n &lt;&amp; m</b><br>Merges input from stream n with stream m                            |
| 8      | <b>&lt;&lt; tag</b><br>Standard input comes from here through next tag at the start of line |
| 9      | <b> </b><br>Takes output from one program, or process, and sends it to another              |

Note: That the file descriptor 0 is normally standard input (STDIN), 1 is standard output (STDOUT), and 2 is standard error output (STDERR).

### Check Your Understanding

#### Fill-Ups

- \_\_\_\_\_ is the device from which the system receives input.
- \_\_\_\_\_ is the device to which system output is sent.
- The operator redirects input on \_\_\_\_\_ and \_\_\_\_\_ control commands.
- The \_\_\_\_\_ informs the shell that the here document is finished.

Notes

## Notes

5. The \_\_\_\_\_ instructs the shell to redirect the output of a command to the specified file instead of to the screen.

### True/False

1. Standard input could be redirected in the same way as the standard output could be redirected.
2. The output of the two variants of the wc command does not differ.
3. A here document is used to redirect input into a shell script or application that is interactive.
4. A file could be created automatically from a shell script.
5. The file descriptor 1 is normally standard input (STDIN).

### Summary

- The standard input device, often known as stdin, is the device from which the system receives input.
- This is the keyboard, but you can specify that input come from another source, such as a serial port or a disc file.
- The standard output device, often known as stdout, is the device to which system output is sent. This is usually a display, although it can also be a serial port or a file.
- The operator redirects input on operating-system instructions and MQ control commands.
- When this operator is followed by a file name, the input is read from the file. Similarly, the > operator redirects output; if accompanied by a file name, output is directed to that file.
- You can redirect standard input in the same way that you can redirect standard output.
- The redirect input symbol ( ) tells the shell to redirect a command's input to come from the given file rather than the keyboard.
- where command is any executable program (such as an application program or a utility), arguments are optional arguments, and filename is the name of the ordinary file the shell redirects the input from.
- A here document is used to redirect input into a shell script or application that is interactive.
- The redirect output symbol (>) instructs the shell to redirect the output of a command to the specified file instead of to the screen.
- where command is any executable program (e.g., an application program or a utility), arguments are optional arguments, and filename is the name of the ordinary file the shell redirects the output to.

### Activity

1. Get divided into the groups of five and prepare a presentation on Commands used in Here Document.



## Glossary

- **stdin**- the device from which the system receives input.
- **stdout**- The standard output device
- **Here Document**- Document used to redirect input into a shell script or application that is interactive.
- **pgm > file**- Output of pgm is redirected to file
- **pgm < file**- Program pgm reads its input from file
- **pgm >> file**- Output of pgm is appended to file
- **n > file**- Output from stream with descriptor n redirected to file
- **n >> file**- Output from stream with descriptor n appended to file
- **n >& m**- Merges output from stream n with stream m
- **n <& m**- Merges input from stream n with stream m
- **<< tag**- Standard input comes from here through next tag at the start of line
- **|**- Takes output from one program, or process, and sends it to another

## Question & Exercises

1. Give a brief introduction to redirection.
2. What is a Here Document?
3. What do you mean by redirecting input?
4. Discuss in brief about redirection output and error.

## Further Readings

1. Unix Network Programming, Volume 1: The Sockets Networking API
2. UNIX Network Programming, Volume 2: Interprocess Communications

## Answers to Check Your Understanding

### Fill-Ups

1. Stdin
2. Stdout
3. Operating-System Instructions, MQ
4. Delimiter
5. Redirect output symbol (>)

### True/False

- |          |          |
|----------|----------|
| 1. True  | 2. False |
| 3. True  | 4. True  |
| 5. False |          |

## Notes

## Notes

## Unit-3.5: Unix Permissions

**Objectives:**

At the end of this unit, the reader will be able to:

- Understand the concept of permissions
- Comprehend the symbolic and numeric permissions
- Talk about changing permissions

**Introduction**

When you run a “ls” command, you get no information about the security of the files because “ls” only lists the names of files by default. More information can be obtained by combining a “option” with the “ls” command. All options begin with a ‘-’ sign. For example, to run “ls” with the “long listing” option, type `ls -l`.

When you do this, each file will be listed in long format on a separate line.

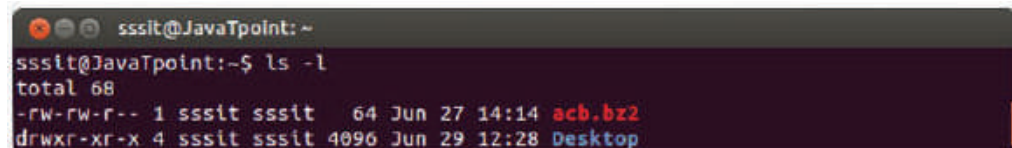
**3.5.1 Understanding Permissions**

In the Linux system, three sorts of permissions are defined for each of the three owners (user owner, group, and others). The three sorts of permissions are represented by nine characters.

1. **Read (r):** The read permission allows you to open and read a file’s content. However, you are unable to edit or modify the file.
2. **Write (w):** The write permission allows you to change, delete, or rename files. For example, if a file exists in a directory and write permission is set on the file but not on the directory, you can update the file’s content but not remove or rename it.
3. **Execute (x):** You cannot run or execute a programme in a Unix-like system unless the execute permission is set.

However, such a permission is not available in Windows. Permissions are listed below:

| permission  | on a file                | on a directory                   |
|-------------|--------------------------|----------------------------------|
| r (read)    | read file content (cat)  | read directory content (ls)      |
| w (write)   | change file content (vi) | create file in directory (touch) |
| x (execute) | execute the file         | enter the directory (cd)         |

**Permission Set**


```

sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ ls -l
total 68
-rw-rw-r-- 1 sssit sssit 64 Jun 27 14:14 acb.bz2
drwxr-xr-x 4 sssit sssit 4096 Jun 29 12:28 Desktop

```

Source: <https://www.javatpoint.com/>

Look at the above snapshot, there are ten characters (-rw-rw-r--) before the user owner. We'll describe these ten characters here.

#### File permissions for (-rw-rw-r--)

| position | characters | ownership            |
|----------|------------|----------------------|
| 1        | -          | denotes file type    |
| 2-4      | rw-        | permission for user  |
| 5-7      | rw-        | permission for group |
| 8-10     | r--        | permission for other |

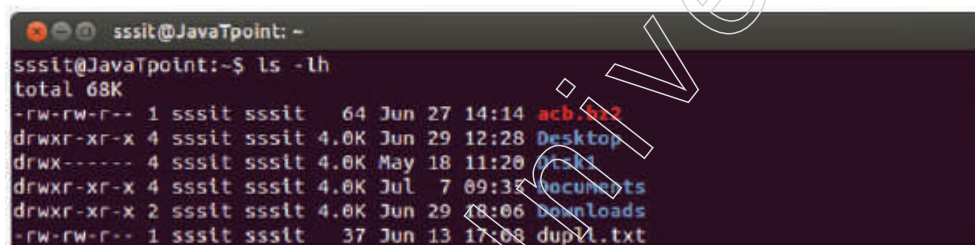
When you are the User owner, you have access to the User owner privilege. Other permissions do not apply to you.

When you are the Group, you have access to the group permissions. Other permissions do not apply to you.

When you are the Other, you have the other permission. You are uninterested in user and group permissions.

#### Permission Example

Now we'll show some examples how permissions can be seen for a file or directory.



```

sssit@JavaTpoint: ~$ ls -lh
total 68K
-rw-rw-r-- 1 sssit sssit 64 Jun 27 14:14 acb.txt
drwxr-xr-x 4 sssit sssit 4.0K Jun 29 12:28 Desktop
drwx----- 4 sssit sssit 4.0K May 18 11:20 Desktop
drwxr-xr-x 4 sssit sssit 4.0K Jul 7 09:35 Documents
drwxr-xr-x 2 sssit sssit 4.0K Jun 29 18:06 Downloads
-rw-rw-r-- 1 sssit sssit 37 Jun 13 17:08 dup1.txt

```

Source: <https://www.javatpoint.com/>

Take a look at the screenshot above; different directories and files have different permissions.

The files and directories are represented by the first letter (-) or d, respectively.

The first triplet of the remaining nine characters now signifies the permission for the user owner. The permission for group owner is represented by the second triplet. The permission for others is represented by the third triplet.

### 3.5.2 Symbolic and Numeric Permissions

There are two ways to represent the MODE:

1. Using symbolic modes (letters to indicate the categories and permission)
2. Using numeric modes (An octal (base 8) number that represents the mode)

Using the "numeric modes" way of setting these permissions is shorter than the symbolic method, but not as flexible because you can't build on top of existing permissions which is possible when using "symbolic modes".

## Notes

The `chmod` command allows you to change the permissions on a file using either a symbolic or numeric mode or a reference file. We will explain the modes in more detail later in this article. The command can accept one or more files and/or directories separated by space as arguments.

Only root, the file owner or user with `sudo` privileges can change the permissions of a file. Be extra careful when using `chmod`, especially when recursively changing the permissions.

### Symbolic (Text) Method

The syntax of the `chmod` command when using the symbolic mode has the following format:

```
chmod [OPTIONS] [ugoa...][-=]perms...[...] FILE...
```

Copy

The first set of flags (`[ugoa...]`), users flags, defines which users classes the permissions to the file are changed.

- ◇ u - The file owner.
- ◇ g - The users who are members of the group.
- ◇ o - All other users.
- ◇ a - All users, identical to `ugo`.

If the users flag is omitted, the default one is `a` and the permissions that are set by `umask` are not affected.

The second set of flags (`[-=]`), the operation flags, defines whether the permissions are to be removed, added, or set:

- ◇ - Removes the specified permissions.
- ◇ + Adds specified permissions.
- ◇ = Changes the current permissions to the specified permissions. If no permissions are specified after the `=` symbol, all permissions from the specified user class are removed.

The permissions (`perms...`) can be explicitly set using either zero or one or more of the following letters: `r`, `w`, `x`, `X`, `s`, and `t`. Use a single letter from the set `u`, `g`, and `o` when copying permissions from one to another users class.

When setting permissions for more than one user classes (`[,...]`), use commas (without spaces) to separate the symbolic modes.

Below are some examples of how to use the `chmod` command in symbolic mode:

- ◇ Give the members of the group permission to read the file, but not to write and execute it:

```
chmod g=r filenameCopy
```

- ◇ Remove the execute permission for all users:

```
chmod a-x filenameCopy
```

- ◇ Recursively remove the write permission for other users:

```
chmod -R o-w dirnameCopy
```

- ◇ Remove the read, write, and execute permission for all users except the file's owner:

```
chmod og-rwx filenameCopy
```

The same thing can be also accomplished by using the following form:

```
chmod og= filenameCopy
```

- ◇ Give read, write and execute permission to the file's owner, read permissions to the file's group and no permissions to all other users:

```
chmod u=rwx,g=r,o= filenameCopy
```

- ◇ Add the file's owner permissions to the permissions that the members of the file's group have:

```
chmod g+u filenameCopy
```

- ◇ Add a sticky bit to a given directory:

```
chmod o+t dirnameCopy
```

### Numeric Method

The syntax of the chmod command when using numeric method has the following format:

```
chmod [OPTIONS] NUMBER FILE...
```

Copy

When using the numeric mode, you can set the permissions for all three user classes (owner, group, and all others) at the same time.

The NUMBER can be a 3 or 4-digits number.

When 3 digits number is used, the first digit represents the permissions of the file's owner, the second one the file's group, and the last one all other users.

Each write, read, and execute permissions have the following number value:

- ◇ r (read) = 4
- ◇ w (write) = 2
- ◇ x (execute) = 1
- ◇ no permissions = 0

The permissions number of a specific user class is represented by the sum of the values of the permissions for that group.

### 3.5.3 Changing Permissions

#### Setting Permissions With chmod

You can change the permissions with chmod command accordingly to your need. Below are some examples to change the permissions for different groups.

## Notes

To add permissions to a group.

Syntax:

1. `chmod <groupName>+<permissionName> <fileName>`

Example:

1. `chmod u+x file`

```

sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ ls -l file
-rw-rw-r-- 1 sssit php 59 Jun 30 10:01 file
sssit@JavaTpoint:~$ chmod u+x file
sssit@JavaTpoint:~$ ls -l file
-rwxrw-r-- 1 sssit php 59 Jun 30 10:01 file
sssit@JavaTpoint:~$

```

Source: <https://www.javatpoint.com/>

Look at the above snapshot, permission to execute is added to the user owner group.

To remove permissions from a group

Syntax:

1. `chmod <groupName>-<permissionName> <fileName>`

Example:

1. `chmod g-x file`
2. `chmod u-w file`

```

sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ ls -l file
-rwxrw-r-- 1 sssit php 59 Jun 30 10:01 file
sssit@JavaTpoint:~$ chmod g-x file
sssit@JavaTpoint:~$ ls -l file
-rwxrw-r-- 1 sssit php 59 Jun 30 10:01 file
sssit@JavaTpoint:~$ chmod u-w file
sssit@JavaTpoint:~$ ls -l file
-r-xrw-r-- 1 sssit php 59 Jun 30 10:01 file
sssit@JavaTpoint:~$

```

Source: <https://www.javatpoint.com/>

Look at the above snapshot, permission to execute is removed from the group and permission to write is removed from the user owner.

To add permission to all the groups together

Syntax:

1. `chmod a+<permissionName> <fileName>`

Example:

1. `chmod a+w file`



```

ssstt@JavaTpoint: ~
ssstt@JavaTpoint:~$ ls -l file
-r----- 1 sssit php 59 Jun 30 10:01 file
ssstt@JavaTpoint:~$ chmod a+w file
ssstt@JavaTpoint:~$ ls -l file
-rw-rw-rw- 1 sssit php 59 Jun 30 10:01 file
ssstt@JavaTpoint:~$

```

Source: <https://www.javatpoint.com/>

Look at the above snapshot, we have given permission to write for all the groups.

Note: Similarly, you can also remove the permission for all the groups.

To add permission to all the groups without typing a

Syntax:

1. **chmod +<permissionName> <fileName>**

Example:

1. **chmod +w file**

```

ssstt@JavaTpoint: ~
ssstt@JavaTpoint:~$ ls -l file
-r----- 1 sssit php 59 Jun 30 10:01 file
ssstt@JavaTpoint:~$ chmod +w file
ssstt@JavaTpoint:~$ ls -l file
-rw-rw-rw- 1 sssit php 59 Jun 30 10:01 file
ssstt@JavaTpoint:~$

```

Source: <https://www.javatpoint.com/>

Take a look at the screenshot above; this example is identical to the previous one; the only difference is that we have not typed an in this one. To set explicit permission

Syntax:

1. **chmod <groupName>=<permissions> <fileName>**

Example:

1. **chmod o=rw file**

```

ssstt@JavaTpoint: ~
ssstt@JavaTpoint:~$ ls -l file
-rw-rw-rw- 1 sssit php 59 Jun 30 10:01 file
ssstt@JavaTpoint:~$ chmod o=rw file
ssstt@JavaTpoint:~$ ls -l file
-rw-rw-rw- 1 sssit php 59 Jun 30 10:01 file
ssstt@JavaTpoint:~$

```

Source: <https://www.javatpoint.com/>

Look at the above snapshot, we have set explicit permission to read and write for others.

To set explicit permissions for different groups

Syntax:

1. **chmod <groupName>=<permissions> <fileName>**

Example:

1. **chmod u=rwx,g=rw,o=r file**



## Notes

```

sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ ls -l file
-rw-rw-rw- 1 sssit php 59 Jun 30 10:01 file
sssit@JavaTpoint:~$ chmod u=rwx,g=rw,o=r file
sssit@JavaTpoint:~$ ls -l file
-rwxrwx-r-- 1 sssit php 59 Jun 30 10:01 file
sssit@JavaTpoint:~$

```

Source: <https://www.javatpoint.com/>

Look at the screenshot above; we have specified permissions for all three groups.

### Configuring Octal Permissions

The groups can also be given octal permissions.

For example, setting r octal to 4 will result in a 4, setting w octal to 2, and setting x octal to 1.

Octal Table:

| binary | Octal | Permissions |
|--------|-------|-------------|
| 000    | 0     | ---         |
| 001    | 1     | --x         |
| 010    | 2     | -w-         |
| 011    | 3     | -wx         |
| 100    | 4     | r--         |
| 101    | 5     | r-x         |
| 110    | 6     | rw-         |
| 111    | 7     | Rwx         |

From this we can conclude that,

1. 777 = rwxrwxrwx
2. 765 = rwxrw-r-x
3. 654 = rw-r-xr--

and so on.

```

sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ chmod 777 file
sssit@JavaTpoint:~$ ls -l file
-rwxrwxrwx 1 sssit php 59 Jun 30 10:01 file
sssit@JavaTpoint:~$ chmod 274 file
sssit@JavaTpoint:~$ ls -l file
--w-rwxr-- 1 sssit php 59 Jun 30 10:01 file
sssit@JavaTpoint:~$ chmod 111 file
sssit@JavaTpoint:~$ ls -l file
---x---x--x 1 sssit php 59 Jun 30 10:01 file
sssit@JavaTpoint:~$

```

Source: <https://www.javatpoint.com/>

Look at the above snapshot, we have shown some random octal examples with the numbers 777, 274 and 111.

umask

A set of permissions is applied by default when creating a file or directory. The `umask` command displays the default permissions.

All Unix systems do not grant execution permission to freshly created files for security concerns.

It is entirely up to you whether or not to grant execution authority.

```

sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ umask
0002
sssit@JavaTpoint:~$ touch abc
sssit@JavaTpoint:~$ ls -l abc
-rw-rw-r-- 1 sssit sssit 0 Jul 7 14:26 abc
sssit@JavaTpoint:~$

```

Source: <https://www.javatpoint.com/>

### **mkdir -m**

The '`mkdir -m`' command can be used to set the mode.

Syntax:

1. `mkdir -m <mode> <fileName>`

Example:

1. `mkdir -m 777 new1`
2. `mkdir -m 000 new2`

```

sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ mkdir -m 777 new1
sssit@JavaTpoint:~$ ls -ld new1
drwxrwxrwx 2 sssit sssit 4096 Jul 7 14:36 new1
sssit@JavaTpoint:~$ mkdir -m 000 new2
sssit@JavaTpoint:~$ ls -ld new2
d----- 2 sssit sssit 4096 Jul 7 14:36 new2
sssit@JavaTpoint:~$

```

Source: <https://www.javatpoint.com/>

Look at the above snapshot, we have created two files `new1` and `new2` with mode 777 and 000 respectively.

### **cp -p**

The '`cp -p`' command preserves the permissions and time stamps from source files.

Syntax:

1. `cp -p <sourceFile> <destinationFile>`

Example:

1. `cp -p list dupli.txt`

```

sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ ls -l list dupli.txt
-rwxrwxrwx 1 sssit sssit 37 Jun 13 17:08 dupli.txt
-rw-rw-r-- 1 jtp sssit 37 Jul 7 14:56 list
sssit@JavaTpoint:~$ cp -p list dupli.txt
sssit@JavaTpoint:~$ ls -l list dupli.txt
-rw-rw-r-- 1 sssit sssit 37 Jul 7 14:56 dupli.txt
-rw-rw-r-- 1 jtp sssit 37 Jul 7 14:56 list
sssit@JavaTpoint:~$

```

## Notes

Source: <https://www.javatpoint.com/>

Look at the above snapshot, earlier permissions for files list and dupli.txt were different. But after passing the command “cp -p list dupli.txt”, both the files have same permissions.

### Check Your Understanding

#### Fill-Ups

1. In the Linux system, there are \_\_\_\_\_ sorts of permissions.
2. When you are the User owner, you have access to the \_\_\_\_\_.
3. The \_\_\_\_\_ command allows to change the permissions on a file.
4. The \_\_\_\_\_ command displays the default permissions.
5. The \_\_\_\_\_ command can be used to set the mode.

#### True/False

1. The three sorts of permissions are represented by nineteen characters.
2. Under the read permission you can edit or modify the file.
3. When you are the Group, you have access to the group permissions.
4. Different directories and files have different permissions.
5. All Unix systems do not grant execution permission to freshly created files for security concerns.

#### Summary

- In the Linux system, three sorts of permissions are defined for each of the three owners (user owner, group, and others).
- The three sorts of permissions are represented by nine characters.
- Such a permission is not available in Windows.
- When you are the User owner, you have access to the User owner privilege. Other permissions do not apply to you.
- When you are the Group, you have access to the group permissions. Other permissions do not apply to you.
- When you are the Other, you have the other permission. You are uninterested in user and group permissions.
- There are two ways to represent the MODE.
- Using symbolic modes (letters to indicate the categories and permission)
- Using numeric modes (An octal (base 8) number that represents the mode)
- Using the “numeric modes” way of setting these permissions is shorter than the symbolic method, but not as flexible because you can’t build on top of existing permissions which is possible when using “symbolic modes”.
- Permission could be changed with chmod command.

### Activity

1. Get divided into the groups of five and prepare a presentation on chmod command.

### Glossary

- Read (r)-The read permission allows you to open and read a file's content.
- Write (w)-The write permission allows you to change, delete, or rename files.
- u - The file owner.
- g - The users who are members of the group.
- o - All other users.
- a - All users, identical to ugo.
- - Removes the specified permissions.
- + Adds specified permissions.
- = Changes the current permissions to the specified permissions.

### Question & Exercises

1. Write in brief about the permissions under the Linux System.
2. What are the two ways to represent MODE?
3. Write a short note on Symbolic mode.
4. Elucidate chmod command.
5. How setting Permissions be changed with chmod command.

### Further Reading

1. The Art of UNIX Programming
2. The Unix Programming Environment — Kernighan, Pike

### Answers to Check Your Understanding

#### Fill-Ups

- |             |                         |
|-------------|-------------------------|
| 1. Three    | 2. User owner privilege |
| 3. Chmod    | 4. Umask                |
| 5. mkdir -m |                         |

#### True/False

- |          |          |
|----------|----------|
| 1. False | 2. False |
| 3. True  | 4. True  |
| 5. True  |          |

Notes

## Notes

### Unit-3.6: Unix Links

#### Objectives:

At the end of this unit, the reader will be able to:

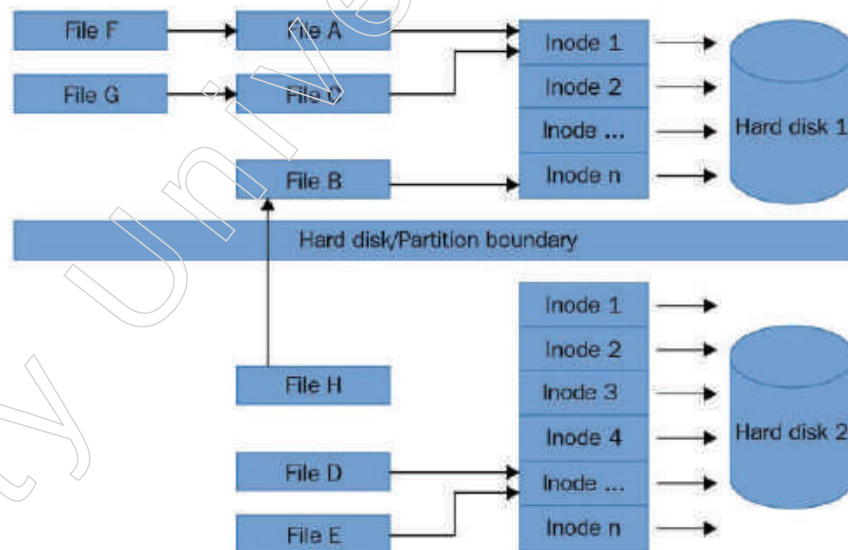
- Understand the concept of file links
- Comprehend the types of links
- Talk about the ln command

#### Introduction

In UNIX, a link is a file pointer. Links in UNIX, like pointers in other programming languages, are pointers pointing to a file or directory. Creating links is a type of shortcut for gaining access to a file. Links allow multiple file names to refer to the same file in different locations. There are two kinds of links: direct and indirect.

#### 3.6.1 File Links

As you already know, files are stored on the hard disk. In a Linux filesystem, the filename and the data are two separate concepts and are not stored together. A general structure is shown in the following diagram:



The filesystem manages the connection between a filename and the actual data using a table or data structure known as a file allocation table. An Inode is the actual entry point to the data of a given file on the hard disc in the Linux filesystem. To simplify, consider the Inode to be the actual data of a file. The filesystem management now ensures that, upon creation, every regular file has one link entry in its allocation table that connects the real filename to the Inode on the hard disc. This type of link is also known as a hard link. A hard link connects the original filename to the Inode relationship as well. The wonderful thing about the Linux filesystem is that you can make additional hard links to an existing Inode, which is similar to having many file names.

One disadvantage of a hard link is that it cannot be distinguished from the original filename or Inode. This can produce issues and side effects since if the original file's

content is modified, the hard link's content will be changed as well. Another constraint of hard links is that they can only be defined for Inodes that are on the same partition as the hard link. Furthermore, you cannot build hard links on directories. They can only be created on standard files. To overcome the constraints of hard connections, soft links, also known as symbolic links, can be used. These are links that you will almost certainly utilise in your day-to-day work as a Linux system administrator. Hard links have their own set of uses, such as creating file backups, but are rarely utilised by a Linux user.

A symbolic link is a link to the filename rather than the Inode. Symbolic links are also not restricted to being on the same partition as the original file. You can also make symbolic links in a directory. The biggest disadvantage is that if you remove or transfer the original file, you will have a broken symbolic link with no notice, which can have some negative side consequences. The main use cases and power of symbolic links in the Linux filesystem are referring configuration files or dynamic library versions. Because no actual data is moved, using Linux file links can save a lot of disc space, and they are also useful for quickly trying out things like different configuration files for services.

### 3.6.2 Types of Links

In UNIX, a link is a file pointer. Links in UNIX, like pointers in other programming languages, are pointers leading to a file or directory. Creating links is a type of shortcut for gaining access to a file. Links allow several file names to refer to the same file in different locations.

There are two types of links :

1. Soft link, also known as symbolic links,
2. Hard Links

When the source of the connection (what is being linked to) is moved or withdrawn, these links act differently. Symbolic links are not updated (they simply include a string containing the target's path name); hard links always point to the source, even if moved or destroyed.

Assume we have a file called a.txt. If we make a hard link to a file and subsequently remove it, we can still access it through the hard link. However, if we make a soft link to a file and then remove it, we cannot access the file via the soft link, and the soft link becomes dangling. Hard links, on the other hand, raise a location's reference count, whereas soft links serve as a shortcut (like in Windows)

#### 1. Hard Links

Because each hard linked file has the same Inode value as the original, they correspond to the same physical file location. Although hard links are more flexible and remain linked even if the original or linked files are relocated around the file system, they cannot cross multiple file systems.

- ◇ The ls -l command displays all links, with the link column displaying the number of links.
- ◇ The contents of links are actual file contents.



## Notes

- ◇ Removing any link reduces the link count but has no effect on other links.
- ◇ Even if we modify the filename of the original file, the hard links continue to function effectively.
- ◇ To avoid recursive cycles, we cannot build a hard link for a directory.
- ◇ If the original file is deleted, the link will still display the file's content.
- ◇ The size of any hard link file is the same as the original file, and if the content of any hard link is changed, the size of all hard link files is updated.
- ◇ The downside of hard links is that they cannot be made for files on multiple file systems or for custom files or directories.

The command to make a hard link is:

```
$ ln [initial filename] [Name of the link]
```

### 2. Soft Links

A soft link is analogous to the file shortcut capability found in Windows operating systems. Each soft linked file has its own Inode value, which points to the original file. Any changes to the data in either file are reflected in the other, much like hard links. Soft links can be linked between file systems, but if the source file is removed or transferred, the soft linked file will no longer function properly (called hanging link).

- ◇ The `ls -l` command displays all links that have the first column value `l?` and point to the original file.
- ◇ The path to the original file is contained in the Soft Link, not the contents.
- ◇ Removing a soft link has no effect, but removing the original file causes the link to become a “dangling” link that leads to a nonexistent file.
- ◇ A soft link can be used to point to a directory.
- ◇ A soft link's size is equal to the name of the file for which it is formed. E.g If the file's name is `file1`, the size of its soft link will be 5 bytes, which is the same as the size of the original file's name.
- ◇ If we alter the name of the original file, all of the soft links for that file become dangling, which means they are no longer useful.
- ◇ Link files across file systems: Only symlinks/soft links can be used to connect files across file systems.

To make a Soft link, use the following command:

```
$ ln -s [initial filename] [Name of the link]
```

### 3.6.3 The ln command

The `ln` command in Linux manages file links. The basic format is `ln [OPTION]`, followed by the filename on which you wish to make the link, and lastly the link name. Use the following code to create a hard link to a file called `fileX` in your home directory:



```

root@localhost ~]# ls -l
total 16
-rw-r--r--. 1 root root 1161 Jun 26 11:31 anaconda-ks.cfg
-rw-r--r--. 1 root root 153 Jun 27 05:45 edit
-rwxrwxrwx. 1 root root 8 Jun 27 02:09 file-to-delete
-rw-r--r--. 1 root root 8 Jun 27 05:46 fileX
drwx----- 3 olip olip 74 Jun 27 01:40 list
-rw-r--r--. 1 root root 34 Jun 27 04:31 my-source-file.source
drwxr-xr-x. 2 root root 75 Jun 27 05:31 scripts
drwxr-xr-x. 2 root root 66 Jun 27 05:24 stuff
drwxr-xr-x. 6 root root 4096 Jun 27 01:31 test_files
root@localhost ~]# ln fileX fileX_link
root@localhost ~]# ls -l
total 16
-rw-r--r--. 1 root root 1161 Jun 26 11:31 anaconda-ks.cfg
-rw-r--r--. 1 root root 153 Jun 27 05:45 edit
-rwxrwxrwx. 1 root root 8 Jun 27 02:09 file-to-delete
-rw-r--r--. 2 root root 8 Jun 27 05:46 fileX
-rw-r--r--. 2 root root 8 Jun 27 05:46 fileX_link
drwx----- 3 olip olip 74 Jun 27 01:40 list
-rw-r--r--. 1 root root 34 Jun 27 04:31 my-source-file.source
drwxr-xr-x. 2 root root 75 Jun 27 05:31 scripts
drwxr-xr-x. 2 root root 66 Jun 27 05:24 stuff
drwxr-xr-x. 6 root root 4096 Jun 27 01:31 test_files
root@localhost ~]#
root@localhost ~]# rm fileX_link2
rm: remove regular empty file 'fileX_link2'? y
root@localhost ~]# df -i
Filesystem Inodes IUsed IFree IUse% Mounted on
/dev/mapper/centos-root 6991872 58516 6933356 1% /
devtmpfs 124544 36 124508 1% /dev
tmpfs 127187 1 127186 1% /dev/shm
tmpfs 127187 412 126775 1% /run
tmpfs 127187 13 127174 1% /sys/fs/cgroup
/dev/sda1 512000 336 511664 1% /boot
tmpfs 127187 1 127186 1% /run/user/0
root@localhost ~]# mount_

```

As you can see, there is no way to tell the additional hard link from the original one. You can also add several links to the same file. The `rm` command is used to erase a hard link. There is a limited number of Inodes on each filesystem, or you can simply say files, which you may view using `df -i`. If you run the `mount` command, you will notice that the user's `tmp` filesystem is on a separate partition than the home directory, which is on the root disc, as shown in the screenshot below:

```

cgroup on /sys/fs/cgroup/cpu,cpuacct type cgroup (rw,nosuid,nodev,noexec,relatime,cpuacct,cpu)
cgroup on /sys/fs/cgroup/cpuset type cgroup (rw,nosuid,nodev,noexec,relatime,cpuset)
cgroup on /sys/fs/cgroup/freezer type cgroup (rw,nosuid,nodev,noexec,relatime,freezer)
cgroup on /sys/fs/cgroup/net_cls type cgroup (rw,nosuid,nodev,noexec,relatime,net_cls)
cgroup on /sys/fs/cgroup/blkio type cgroup (rw,nosuid,nodev,noexec,relatime,blkio)
cgroup on /sys/fs/cgroup/devices type cgroup (rw,nosuid,nodev,noexec,relatime,devices)
configfs on /sys/kernel/config type configfs (rw,relatime)
/dev/mapper/centos-root on / type xfs (rw,relatime,seclabel,attr2,inode64,noquota)
selinuxfs on /sys/fs/selinux type selinuxfs (rw,relatime)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=31,pgrp=1,timeout=300,minproto=5,maxproto=5,direct)
debugfs on /sys/kernel/debug type debugfs (rw,relatime)
mqueue on /dev/mqueue type mqueue (rw,relatime,seclabel)
hugetlbfs on /dev/hugepages type hugetlbfs (rw,relatime,seclabel)
/dev/sda1 on /boot type xfs (rw,relatime,seclabel,attr2,inode64,noquota)
tmpfs on /run/user/0 type tmpfs (rw,nosuid,nodev,relatime,seclabel,size=101688k,mode=700)
root@localhost ~]#

```

So, the next command, `ln ~/folderABC ~/folderABC_link`, will fail because it is not allowed to create hard links between partitions. A hard link cannot be created on a directory, and changing the origin of the file's content will also modify the hard link's file content. This has the potential to have negative consequences. Use the `ln -s` option to create a symbolic link:

## Notes

```
[root@localhost ~]# ln -s fileX fileX_sym_link
[root@localhost ~]# ls -l
total 16
-rw-----. 1 root root 1161 Jun 26 11:31 anaconda-ks.cfg
-rw-r--r--. 1 root root 153 Jun 27 05:45 edit
-rwxrwxrwx. 1 root root 0 Jun 27 02:09 file-to-delete
-rw-r--r--. 2 root root 0 Jun 27 05:46 fileX
-rw-r--r--. 2 root root 0 Jun 27 05:46 fileX_link
lrwxrwxrwx. 1 root root 5 Jun 27 05:56 fileX_sym_link -> fileX
drwx-----. 3 olip olip 74 Jun 27 01:40 list
-rw-r--r--. 1 root root 34 Jun 27 04:31 my-source-file.source
drwxr-xr-x. 2 root root 75 Jun 27 05:31 scripts
drwxr-xr-x. 2 root root 66 Jun 27 05:24 stuff
drwxr-xr-x. 6 root root 4096 Jun 27 01:31 test_files
[root@localhost ~]#
```

As you can see, it is simple to display a file as a symbolic link denoted by the arrow. You can use `ln -s /etc/passwd` to create a symbolic link to a file in another directory while keeping the original file's name. This creates a symbolic link in the current directory to the `/etc/passwd` file with the same name, `passwd`. The `rm` command is used to erase a symbolic link; the original file is not affected. A symbolic link can also be created on a directory. If you remove the original file to which the symbolic link is pointing (`fileX` in this case), the symbolic connection will be broken. This can be problematic, as indicated by the blue colour:

```
[root@localhost ~]# rm fileX
rm: remove regular empty file 'fileX'? y
[root@localhost ~]# ls -l
total 4
-rw-----. 1 0 root 1169 Jun 26 11:40 anaconda-ks.cfg
-rw-r--r--. 1 0 root 0 Jun 26 18:03 fileX_link
drwxr-xr-x. 2 0 root 6 Jun 26 18:30 folderABC
[root@localhost ~]#
```

### Check Your Understanding

#### Fill-Ups

- Files are stored on the \_\_\_\_\_.
- The filesystem manages the connection between a filename and the actual data using a table or data structure known as a \_\_\_\_\_.
- A \_\_\_\_\_ connects the original filename to the Inode relationship.
- A \_\_\_\_\_ is a link to the filename rather than the Inode.
- The \_\_\_\_\_ in Linux manages file links.

#### True/False

- In a Linux filesystem, the filename and the data are same concepts.
- A hard link can be distinguished from the original filename or Inode.
- Symbolic links are also not restricted to being on the same partition as the original file.
- In UNIX, a link is a file pointer.
- Soft link also known as symbolic links.

## Summary

- Files are stored on the hard disk. In a Linux filesystem, the filename and the data are two separate concepts and are not stored together.
- The filesystem manages the connection between a filename and the actual data using a table or data structure known as a title allocation table.
- An Inode is the actual entry point to the data of a given file on the hard disc in the Linux filesystem.
- To simplify, consider the Inode to be the actual data of a file. The filesystem management now ensures that, upon creation, every regular file has one link entry in its allocation table that connects the real filename to the Inode on the hard disc.
- This type of link is also known as a hard link. A hard link connects the original filename to the Inode relationship as well.
- The wonderful thing about the Linux filesystem is that you can make additional hard links to an existing Inode, which is similar to having many file names.
- A symbolic link is a link to the filename rather than the Inode.
- Symbolic links are also not restricted to being on the same partition as the original file.
- In UNIX, a link is a file pointer. Links in UNIX, like pointers in other programming languages, are pointers leading to a file or directory.
- Creating links is a type of shortcut for gaining access to a file. Links allow several file names to refer to the same file in different locations.
- The `ln` command in Linux manages file links. The basic format is `ln [OPTION]`, followed by the filename on which you wish to make the link, and lastly the link name.

## Activity

1. Get divided into the groups of five and prepare a presentation on direct and indirect link in UNIX.

## Glossary

- **Inode**- The actual entry point to the data of a given file on the hard disc in the Linux.
- **Symbolic link**- A link to the filename rather than the Inode.
- **Soft link**- Analogous to the file shortcut capability found in Windows operating systems.

## Question & Exercises

1. Distinguish between soft and hard link.
2. Write a short note on `ln` Command.
3. What are the disadvantages of hard link?
4. What does file link means?

## Further Reading

1. The Linux Programming Interface: A Linux and UNIX System Programming
2. How Linux Works: What Every Superuser Should Know

## Notes

3. The Linux Command Line: A Complete Introduction
4. Advanced Programming in the UNIX Environment
5. Advanced UNIX Programming — Rochkind
6. UNIX and Linux System Administration Handbook, 4th Edition
7. Unix Network Programming, Volume 1: The Sockets Networking API
8. UNIX Network Programming, Volume 2: Interprocess Communications
9. The Art of UNIX Programming
10. The Unix Programming Environment — Kernighan, Pike

### Answers to Check Your Understanding

#### Fill-Ups

1. Hard Disk
2. Title Allocation Table
3. Hard Link
4. symbolic link
5. In command

#### True/False

1. False
2. False
3. True
4. True
5. True

## Module-IV: VI Editor and Shell Programming

Notes

### Structure:

#### 4.1 Vi editor

4.1.1 Command mode, insert mode and last line mode;

4.1.2 “Change word, Change line, Delete current line, Delete n lines, Delete

4.1.3 “command to delete character, insert line; deleting text, command for moving the cursor;

#### 4.2 Shell Programming

4.2.1 Shell Script Introduction

4.2.2 Shell scripts and execution methods

4.2.3 Setting positional parameters (set command), Shift ,

4.2.4 shell variables, environment variables

4.2.5 “Assignment Statements, Arithmetic, Logical and relational operators, read, echo”

4.2.6 Test Command through IF Statement: Numerical Test, File Test and String Test;

4.2.7 Test Command: Numerical Test, File Test and String Test;

4.2.8 for Loop

4.2.9 while and until loop

4.2.10 case and switch

## Notes

### Unit-4.1: Vi Editor

#### Objectives:

At the end of this unit, you will be able to understand:

- Command Mode, Insert Mode and Last Line Mode
- Change Word, Change Line, Delete Current Line, Delete n Lines, Delete Remainder
- Command to Delete Character, insert Line, Deleting text, command for moving the Cursor

#### Introduction

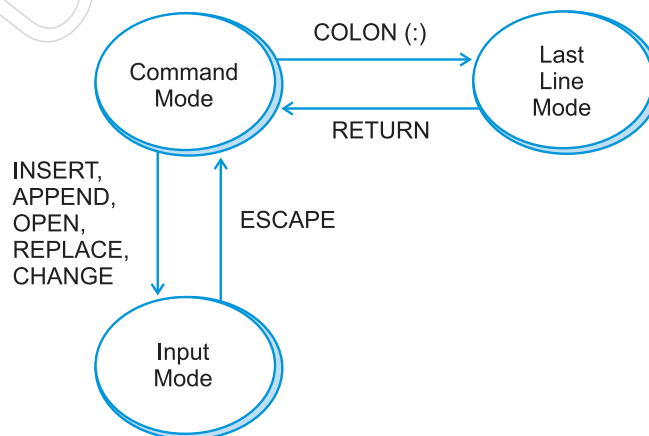
The standard UNIX system editor was ed (available on most Linux systems), a line-oriented editor that made it impossible to perceive the context of your editing until vi (Visual mode) was developed. Then there was ex, which is a superset of ed. The most apparent advantage of ex over ed is the display-editing feature, which lets you to work with a full screen of text rather than just a single line. Giving the vi (Visual mode) command while using ex will bring up the display-editing feature. People utilised this display-editing feature so much that ex's developers made it possible to start the editor with the display-editing feature already running, rather than having to start ex and then type vi. We can edit an existing file or create a new one using the vi editor. This editor can also be used to just read a text file.

Syntax:

```
vi filename
```

#### 4.1.1 Command Mode, Insert Mode and Last Line Mode

**Modes of Operation in vi editor:** vi has three modes of operation:



**Figure 8-1 Modes of operations in vi Editor**

- ◇ **Command Mode:** Vi is in Command Mode when it first starts up. Vi considers any characters we input as commands in this mode, and hence does not display them in the window. This mode enables us to go through a file and delete, copy, or paste text.



To enter into Command Mode from any other mode, it requires pressing the [Esc] key. If we press [Esc] when we are already in Command Mode, then vi will beep or flash the screen.

- ◇ **Insert mode:** This mode allows you to add text to the document. In this mode, everything you type is interpreted as input and then saved to a file. The vi opens in command mode by default. You must be in insert mode to enter text. Simply type I to enter insert mode. Insert mode can be exited by using the Esc key, which will return you to command mode.

[illegible]

Vim will display line numbers next to the text you're editing if you use the following command:

```
:set number RETURN
```

**Last Line mode:** Vim switches to Last Line mode after the colon (:) in the preceding command. Vim maintains the cursor on the bottom line of the screen while in this mode. When you press RETURN after finishing the command, vim returns the cursor to its original position in the text. Give the following command: To disable line numbering, type `nonumber` RETURN.

**vim is case sensitive:** Keep in mind that vim is case sensitive while issuing commands. In other words, depending on whether you type an uppercase or lowercase character, vim interprets the identical letter as two separate commands. The CAPS LOCK (SHIFTLOCK) key should be used with extreme care. Vim interprets your commands as capital letters if you configure this key to enter uppercase text while in Input mode and then leave to Command mode. When this happens, it can be perplexing because vim does not appear to be executing the instructions you are typing.

#### 4.1.2 Change Word, Change Line, Delete Current Line, Delete n Lines, Delete

- Change word:** Place the cursor at the start of the word to be replaced to replace it. Type cw and then the new word. Press Esc to exit.

Place the cursor on the word, to the right of the chunk to be saved, to modify a part of it. Press Esc after typing cw and the correction.



## Notes

### Change a Line:

Place the cursor anywhere on the line and type `cc` to replace it. The line vanishes, leaving you with a blank line to write your new text on (which can be of any length). To exit, press `Esc`.

### Deleting current Line:

Place the cursor anywhere on the current line and type `dd` to erase it. The line is removed, as well as the space it occupied

- ◇ The following are detailed methods for deleting a line:  
To return to normal mode, Press the **Esc** key
- ◇ Put the cursor on the line you want to delete.
- ◇ Type **dd** and press Enter to delete the line.

### Deleting n Lines:

Prefix the `dd` command with the number of lines to be erased to delete many lines at once. To eliminate five lines, for example, perform the following:

- ◇ Hit the `Esc` key to return to normal mode.
- ◇ Put the cursor on the first line you want to remove.
- ◇ Type `5dd` and press Enter to delete the next five lines.

### Delete remainder of lines:

The syntax for deleting a range of lines is illustrated below:

```
:[start],[end]d
```

For instance, to delete lines starting from 3 to 5 follow the steps listed below:

- ◇ Hit the `Esc` key to return to normal mode.
- ◇ Type `:3,5d` and press Enter to delete the lines starting from 3.

You can also declare the range with the following characters:

- ◇ `.` (dot) - The current line.
- ◇ `$` - The last line.
- ◇ `%` - All lines.

Below are a few examples:

- ◇ `:$d` - From the current line to the end of the file.
- ◇ `:,1d` - From the current line to the beginning of the file.
- ◇ `10,$d` - From the 10th line to the end of the file

### Copying and moving:

During an editing session, the vim editor has a General-Purpose buffer and 26 Named buffers that can hold text. If you want to relocate or copy a portion of text from one position in the Work buffer to another, you can use these buffers. Text is removed from one spot in the Work buffer and placed in another using a combination of the

Delete and Put commands. The Yank and Put commands copy text from one point in the Work buffer to another without modifying it.

**Notes**

### 4.1.3 Command to Delete Character, Insert Line, Deleting Text, Command for Moving the Cursor

#### Delete Character Command:

To erase a single character, place the cursor over the character you want to remove and press the x key. When a letter is deleted from the midst of a word, the following letters close up, leaving no gap. The x command also deletes the space the character occupied.

Use the vim delete character command - the lowercase letter x — to delete the character at the current cursor position when you're in "command mode" in the vim editor (simply click the [Esc] key to get there):

x

The character at your current cursor position is deleted using this command. You can erase ten characters by pressing the letter x ten times, or you can use this command instead:

10x

#### Insert Line:

The lowercase letter "o" allows you to start a new line just below the one you're on. Vi inserts a new blank line below your current line and puts you in insert mode at that location when you execute this command. The uppercase letter "O" allows you to start a new line just above the one you're on. Vi adds a new blank line above your current line and puts you in insert mode there when you use this command.

#### Deleting Text:

Use the vi delete word command when you're in command mode in the vi editor and want to erase the current word :

dw

If you wish to delete the next five words, use the dw command with the number 5, just like the delete character command:

5dw

#### Moving the Cursor:

To delete, insert, and correct text, you must be able to move the pointer on the screen. The RETURN key, the SPACE bar, and the ARROW keys move the cursor in vim's Command mode. If you prefer to keep your hand closer to the middle of the keyboard, if your terminal lacks ARROW keys, or if the emulator you're using doesn't support them, you can move the cursor left, down, up, and right using the h, j, k, and l (lowercase "l") keys, respectively.

h left, one space

## Notes

- ◇ j down, one line
- ◇ k up, one line
- ◇ l right, one space

### Check your Understanding

#### Fill Ups

1. The standard UNIX system editor was ed (available on most Linux systems), a line-oriented editor that made it impossible to perceive the context of your editing until vi (\_\_\_\_\_) was developed.
2. Vi is in \_\_\_\_\_ Mode when it first starts up.
3. You must be in \_\_\_\_\_ mode to enter text.
4. Vim switches to \_\_\_\_\_ mode after the colon (:) in the preceding command.
5. Place the cursor anywhere on the line and type cc to replace it.

#### True or False

1. The most apparent advantage of ex over ed is the display-editing feature, which lets you to work with a full screen of text rather than just a single line.
2. We can edit an existing file or create a new one using the ed editor.
3. To enter into Command Mode from any other mode, it requires pressing the [Esc] key.
4. Command mode can be exited by using the Esc key, which will return you to insert mode.
5. Keep in mind that vim is not case sensitive while issuing commands.

#### Summary

- vi (Visual mode) was developed, the standard UNIX system editor was ed (available on most Linux systems), a line-oriented editor that made it difficult to see the context of your editing.
- There are three modes of operation in vi command, insert and last line mode.
- The x command also deletes the space the character occupied—when a letter is removed from the middle of a word, the remaining letters will close up, leaving no gap.
- The lowercase letter “o” lets you open a new line just below your current line. The uppercase letter “O” lets you open a new line just above your current line.
- vim is in Command mode, the RETURN key, the SPACE bar, and the ARROW keys move the cursor

#### Questions and Exercises

1. What is a command? Explain the different types of command.
2. Differentiate between delete and insert character.
3. Explain moving the cursor.
4. Explain the process used to delete and insert text.

## Glossary

- **Vim:** It is an advanced and highly configurable text editor built to enable efficient text editing
- **Command mode:** It is the mode to be in when giving commands which will move the cursor, delete text, copy and paste, save the file etc.
- **X:** That command deletes the character at your current cursor positioned
- **dw:** deletes the current word where the cursor is place.

## Further Readings

1. Blum, Richard; Bresnahan, Christine. Mastering Linux System Administration. Sybex; 1st edition (July 27, 2021)
2. Gift, Noah; Jones, Jeremy. Python for Unix and Linux System Administration. O'Reilly Media; 1st edition (September 1, 2008)
3. Hein, Trent; Mackin, Dan; Nemeth, Evi; Snyder, Garth; Whaley, Ben. UNIX and Linux System Administration Handbook. Addison-Wesley Professional; 5th edition (August 8, 2017)
4. Kerrisk, Michael. The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press; 1st edition (October 28, 2010)
5. Negus, Christopher. Linux Bible. Wiley; 10th edition (June 10, 2020)

## Check your Understanding: Answers

### Fill Ups

1. Visual Mode
2. Command
3. Insert
4. Last Line
5. cc

### True or False

1. True
2. False
3. True
4. False
5. False

## Notes

## Notes

### Unit-4.2: Shell Programming

#### Objectives:

**At the end of this unit, you will be able to understand:**

- Introduction to Shell Script
- Shell Scripts and Execution Methods
- Setting Positional Parameters(set command), Shift
- Shell Variables, Environment Variables
- Assignment Statements, Arithmetic, Logical and Relational Operators, Read, Echo
- Test Command through IF Statement: Numerical Test, File Test, String Test
- Test Command: Numerical Test, File Test and String Test

#### Introduction

The shell is a program that interacts with the user through a terminal or takes the input from a file and executes a sequence of commands that are passed to the Operating System. A shell program is an application that allows interacting with the computer. In a shell the user can run programs and also redirect the input to come from a file and output to come from a file. Shells also provide programming constructions such as if, for, while, functions, variables etc. Additionally, shell programs offer features such as line editing, history, file completion, wildcards, environment variable expansion, and programming constructions.

#### Shell as an interpreter:

The shell—the command interpreter—serves as an interface between the user and the operating system in a textual environment. When you type a command on the screen, the shell analyses it and runs the desired programme. For Linux, a variety of shells are available. The four most popular shells are the following

- ◇ The Bourne Again Shell (bash), an enhanced version of the original Bourne Shell (an original UNIX shell).
- ◇ The Debian Almquist Shell (dash;), a smaller version of bash with fewer features. Many startup shell scripts call dash in place of bash to speed the boot process.
- ◇ The TC Shell (tcsh;), an enhanced version of the C Shell, developed as part of BSD UNIX.
- ◇ The Z Shell (zsh), which incorporates features from a number of shells, including the Korn Shell.

Multiuser systems might have numerous different shells in operation at any same moment because different users may prefer various shells. The ability to create a personalised interface for each user is one of the advantages of the Linux operating system, as evidenced by the option of shells.

## 4.2.1 Shell Script Introduction

### Notes

#### Shell Scripts

A shell script is just a set of commands that are listed in the sequence in which they should be executed. Comments, preceded by the # symbol, describe the steps of a decent shell script.

There are conditional checks, such as value A is greater than value B, loops for processing large amounts of data, files for reading and storing data, variables for reading and storing data, and the script may include functions.

In the following parts, we'll write a lot of scripts. It would be a basic text file containing all of our commands as well as a few more necessary constructs that instruct the shell environment what to do and when to perform it.

Scripts and functions in the shell are both interpreted. This indicates that they have not been compiled.

#### Example Script

Suppose we generate a **test.sh** script. Note all the scripts would have the **.sh** extension. Before you add anything else to your script, you need to alert the system that a shell script is being started. This is done using the **shebang** construct. For example –

◇ `#!/bin/sh`

This tells the system that the commands that follow are to be executed by the Bourne shell. It's called a shebang because the # symbol is called a hash, and the ! symbol is called a bang.

To create a script containing these commands, you put the shebang line first and then add the commands –

◇ `#!/bin/bash`

◇ `pwd`

◇ `ls`

A **Shell** offers you with an interface to the UNIX system. It gathers input from you and executes programs based on that input. When a programme completes its execution, the output of that programme is displayed.

Shell is a command-line interface that allows us to run commands, programmes, and shell scripts. Shells come in a variety of flavours, much as operating systems come in a variety of flavours. Each shell flavour has its own collection of well-known commands and functions.

#### Shell Prompt

The prompt, \$, known as the **command prompt**, is created by the shell. While the prompt is displayed, you can type a command.

Shell reads your input after you press **Enter**. It determines the command you want executed by looking at the first word of your input. A word is an unbroken set of characters. Spaces and tabs separate words.

## Notes

Following is a simple example of the **date** command, which displays the current date and time –

- ◇ `$date`
- ◇ Thu Jun 25 08:30:19 MST 2009

You can customize your command prompt using the environment variable `PS1` explained in the Environment tutorial.

### Shell Types

In Unix, there are two major types of shells –

- ◇ **Bourne shell** – When using a Bourne-type shell, the `$` character is the default prompt.
- ◇ **C shell** – When using a C-type shell, the `%` character is the default prompt.

The Bourne Shell has the following subcategories –

- ◇ Bourne shell (`sh`)
- ◇ Korn shell (`ksh`)
- ◇ Bourne Again shell (`bash`)
- ◇ POSIX shell (`sh`)

The different C-type shells follow –

- ◇ C shell (`csh`)
- ◇ TENEX/TOPS C shell (`tcsh`)

Stephen R. Bourne, who worked at AT&T Bell Labs in New Jersey at the time, created the first Unix shell in the mid-1970s.

Bourne shell is known as “the shell” since it was the first shell to debut on Unix computers.

On most Unix versions, Bourne shell is installed as `/bin/sh`. As a result, it is the shell of choice for building scripts that can run on several Unix versions.

### Shell Comments

**Source:** <https://www.javatpoint.com/>

**Source:** <https://www.javatpoint.com/>

- ◇ You can put your comments in your script as follows-
- ◇ `#!/bin/bash`
- ◇ `# Author : Zara Ali`
- ◇ `# Copyright (c) Tutorialspoint.com`
- ◇ `# Script follows here:`
- ◇ `pwd`
- ◇ `ls`
- ◇ Save the above content and make the script executable-
- ◇ `$chmod +x test.sh`
- ◇ The shell script is now ready to be executed -



- ◇ `./test.sh`
- ◇ Upon execution, you will receive the following result-
- ◇ `/home/amrood`
- ◇ `index.htm unix-basic_utilities.htm unix-directories.htm`
- ◇ `test.sh unix-communication.htm unix-environment.htm`
- ◇ Note - To execute a program available in the current directory, use `./program_name|`

**Source:** <https://www.javatpoint.com/>

### Extended Shell Scripts

Shell scripts have several required constructs that tell the shell environment what to do and when to do it. Of course, most scripts are more complex than the above one.

The shell is, after all, a real programming language, complete with variables, control structures, and so forth. No matter how complicated a script gets, it is still just a list of commands executed sequentially.

The following script uses the `read` command which takes the input from the keyboard and assigns it as the value of the variable `PERSON` and finally prints it on `STDOUT`.

```
#!/bin/sh

Author : Zara Ali
Copyright (c) Tutorialspoint.com
Script follows here:

echo "What is your name?"
read PERSON
echo "Hello, $PERSON"
```

Here is a sample run of the script -

- `./test.sh`
- What is your name?
- Zara Ali
- Hello, Zara Ali
- `$`

**Source:** <https://www.javatpoint.com/>

### 4.2.2 Shell Scripts and Execution Methods

#### Shell scripts:

Besides performing its function of interpreting commands from a keyboard and sending those commands to the operating system, the shell is a high-level programming language. Shell commands can be arranged in a file for later execution (Linux calls these files shell scripts; Windows calls them batch files). This flexibility

## Notes

allows users to perform complex operations with relative ease, often by issuing short commands, or to build with surprisingly little effort elaborate programs that perform highly complex operations.

### Filename Generation

When you type commands to be processed by the shell, you can construct patterns using characters that have special meanings to the shell. These characters are called wildcard characters. The patterns, which are called ambiguous file references, are a kind of shorthand: Rather than typing in complete filenames, you can type patterns; the shell expands these patterns into matching filenames. An ambiguous file reference can save you the effort of typing in a long filename or a long series of similar file-names. For example, the shell might expand the pattern `mak*` to `make-3.80.tar.gz`. Patterns can also be useful when you know only part of a filename or cannot remember the exact spelling of a filename.

### File name completion

The shell performs command, filename, pathname, and variable completion in conjunction with the ReadLine library: When you type a prefix and hit TAB, the shell displays a list of items that start with that prefix, or completes the item if the prefix denotes a unique item.

### History

The shell keeps track of all the commands you've entered. Use the cursor keys to browse up and down the list or type history to see a list of past commands if you need to repeat a command.

### Device-Independent Input And Output:

Redirection Devices (such as a printer or a terminal) and disk files appear as files to Linux programs. When you give a command to the Linux operating system, you can instruct it to send the output to any one of several devices or files. This diversion is called output redirection.

### Device independence:

In a similar manner, a program's input, which normally comes from a keyboard, can be redirected so that it comes from a disk file instead. Input and output are device independent; that is, they can be redirected to or from any appropriate device.

As an example, the `cat` utility normally displays the contents of a file on the screen. When you run a `cat` command, you can easily cause its output to go to a disk file instead of the screen.

### Shell Functions:

One of the most important features of the shell is that users can use it as a programming language. Because the shell is an interpreter, it does not compile programs written for it but rather interprets programs each time they are loaded from the disk. Loading and interpreting programs can be time-consuming.

Many shells, including the Bourne Again Shell, support shell functions that the shell holds in memory so it does not have to read them from the disk each time you execute them. The shell also keeps functions in an internal format so it does not have to spend as much time interpreting them.

### Shell Scripts:

Shell programming is akin to a carpentry project. You'll need the correct tools to make something out of wood. The tools you employ in UNIX are known as utilities or commands. There are basic commands such as ls and cd, as well as more powerful tools such as awk, sed, and the shell.

Using the incorrect tool or approach while making a project is one of the most common mistakes in woodworking. It takes practise to know which tool to employ. The simple tools are simple to pick up. Many of these you're undoubtedly already familiar with. The power tools take a little longer to understand, but once you do, you'll be able to solve any difficulty. The shell, UNIX's most powerful tool, is the main focus.

As shell can also take commands as input from file we can write these commands in a file and can execute them in shell to avoid this repetitive work. These files are called Shell Scripts or Shell Programs. Shell scripts are similar to the batch file in MS-DOS. Each shell script is saved with .sh file extension eg. myscript.sh

A shell script have syntax just like any other programming language. If you have any prior experience with any programming language like Python, C/C++ etc. it would be very easy to get started with it.

A shell script comprises following elements –

- ◇ Shell Keywords – if, else, break etc.
- ◇ Shell commands – cd, ls, echo, pwd, touch etc.
- ◇ Functions
- ◇ Control flow – if..then..else, case and shell loops etc.

There are many reasons to write shell scripts –

- ◇ To circumvent repetitive work and automation
- ◇ System admins use shell scripting for routine backups
- ◇ System monitoring
- ◇ Adding new functionality to the shell etc.

### Advantages of shell scripts:

- ◇ The command and syntax are exactly the same as those directly entered in command line, so programmer do not need to switch to entirely different syntax
- ◇ Writing shell scripts are much quicker
- ◇ Quick start
- ◇ Interactive debugging etc.

### Disadvantages of shell scripts:

- ◇ Prone to costly errors, a single mistake can change the command which might be harmful

## Notes

- ◇ Slow execution speed
- ◇ Design flaws within the language syntax or implementation
- ◇ Not well suited for large and complex task
- ◇ Provide minimal data structure unlike other scripting languages. Etc

### How to Write Shell Script in Linux/Unix:

Shell Scripts are written using text editors. On your Linux system, open a text editor program, open a new file to begin typing a shell script or shell programming, then give the shell permission to execute your shell script and put your script at the location from where the shell can find it.

Let us understand the steps in creating a Shell Script:

- ◇ Create a file using a vi editor(or any other editor). Name script file with extension .sh
- ◇ Start the script with `#!/bin/sh`
- ◇ Write some code.
- ◇ Save the script file as filename.sh
- ◇ For executing the script type `bash filename.sh`

"#!" is an operator called shebang which directs the script to the interpreter location. So, if we use `#!/bin/sh` the script gets directed to the bourne-shell.

### Execution Methods:

A Linux system begins execution by starting the init daemon, a single process called spontaneous process, with PID number 1. This process holds the same position in the process structure as the root directory does in the file structure: It is the ancestor of all processes the system and users work with. When a command-line system is in multiuser mode, init runs `getty` or `mingetty` processes, which display login: prompts on terminals and virtual consoles. When a user responds to the prompt and presses RETURN, `getty` or `mingetty` passes control to a utility named `login`, which checks the user- name and password combination. After the user logs in, the `login` process becomes the user's shell process.

The shell forks a new process when you type the name of a programme on the command line, generating a replica of the shell process (a subshell). The new process tries to run the programme (execute it). `exec()`, like `fork()`, is a system call. If the programme is a binary executable, such as a compiled C programme, `exec()` succeeds, and the system places the executable programme over the newly constructed subshell. `exec()` fails if the command is a shell script. When `exec` fails, it is presumed that the programme is a shell script, and the subshell executes the script's commands. Unlike a login shell, which demands input from the command line, the subshell gets its information from a file, specifically the shell script.

If it finds an executable file with the name specified on the command line, the shell starts a new process. A process is the execution of a command by Linux. The shell makes each command-line argument, including options and the name of the command, available to the called program. While the command is executing, the shell waits for the process to finish. At this point the shell is in an inactive state named sleep. When the

program finishes execution, it passes its exit status to the shell. The shell then returns to an active state (wakes up), issues a prompt, and waits for another command.

Because the shell does not process command-line arguments but merely passes them to the called program, the shell has no way of knowing whether a particular option or other argument is valid for a given program. Any error or usage messages about options or arguments come from the program itself. Some utilities ignore bad options.

### 4.2.3 Setting Positional Parameters (set command), Shift

The set command in the shell serves two purposes: it may be used to set numerous shell settings as well as reassign positional arguments like \$1, \$2, and so on.

#### Trace -x Option

This option activates the shell's trace mode. What the command does to the current shell

```
sh -x ctype a
```

Did for the execution of the ctype program," Decisions." From the point that the

```
set -x
```

After the shell has completed filename, variable, and command substitution and I/O redirection, the shell will print all following executed commands to standard error. Plus signs come before the traced commands.

```
$ x=*
```

```
$ set -x Set command trace option
```

```
$ echo $x
```

```
+ echo add greetings lu rem rolo add greetings lu rem rolo
```

```
$ cmd=wc
```

```
+ cmd=wc
```

```
$ ls | $cmd -l
```

```
+ ls
```

```
+ wc -l 5|
```

```
$
```

Trace mode can be turned off at any moment by running set with the +x option:

Notes

## Notes

```
$ set +x
+ set +x
$ ls | wc -l
5 Back to normal
$
```

It's worth noting that subshells don't get the trace option. However, you may trace the execution of a subshell by running it with the `-x` option and the name of the application to be executed, as in

```
sh -x rolo
```

You can also use the `set -x` command within the file. In fact, you can use as many `set -x` and `Set +x` statements as you want to switch trace mode on and off in your application.

### Shift Operators:

The shift built-in promotes each positional parameter. The first argument (which was represented by `$1`) is discarded. The second argument (which was represented by `$2`) becomes the first argument (now `$1`), the third argument becomes the second, and so on. Because no "unshift" command exists, you cannot bring back arguments that have been discarded. An optional argument to shift specifies the number of positions to shift (and the number of arguments to discard); the default is 1.

The following `demo_shift` script is called with three arguments. Double quotation marks around the arguments to echo preserve the spacing of the output but allow the shell to expand variables. The program displays the arguments and shifts them repeatedly until no arguments are left to shift.

```
$ cat demo_shift
echo "arg1= $1 arg2= $2 arg3= $3"
shift
echo "arg1= $1 arg2= $2 arg3= $3"
shift
echo "arg1= $1 arg2= $2 arg3= $3"
shift
echo "arg1= $1 arg2= $2 arg3= $3"
shift
```

```
$./demo_shift alice helen zach
arg1= alice arg2= helen arg3= zach arg1= helen arg2= zach
arg3= arg1= zach arg2= arg3=
arg1= arg2= arg3=
```

Repeatedly using shift is a convenient way to loop over all command-line arguments in shell scripts that expect an arbitrary number of arguments.

## Notes

### 4.2.4 Shell Variables, Environment Variables

There are several characteristics of your environment that you may want to customize but that cannot be expressed as an on/off choice. Characteristics of this type are specified in shell variables. Shell variables can specify everything from your prompt string to how often the shell checks for new mail.

Like an alias, a shell variable is a name that has a value associated with it. The Korn shell keeps track of several built-in shell variables; shell programmers can add their own. By convention, built-in variables have names in all capital letters. The syntax for defining variables is somewhat similar to the syntax for aliases:

```
varname = value
```

There must be no space on either side of the equal sign, and if the value is more than one word, it must be surrounded by quotes. To use the value of a variable in a command, precede its name by a dollar sign ( \$ ).

You can delete a variable with the command `unset varname`. Normally this isn't useful, since all variables that don't exist are assumed to be null, i.e., equal to the empty string `""`. But if you use the option `nounset`, which causes the shell to indicate an error when it encounters an undefined variable, then you may be interested in `unset`.

The easiest way to check a variable's value is to use the `print` built-in command. All `print` does is print its arguments, but not until the shell has evaluated them. This includes—among other things that will be discussed later—taking the values of variables and expanding filename wildcards. So, if the variable `fred` has the value `bob`, typing:

The Korn shell supports the old command `echo`, which does much the same thing, for backward compatibility reasons. However, we strongly recommend `print` because its options are the same on all UNIX systems, whereas `echo`'s options differ between BSD-derived and System V-derived UNIX versions.

```
$
print "$fred"
```

will cause the shell to simply print `bob`. If the variable is undefined, the shell will print a blank line. A more verbose way to do this is:

```
$
print "The value of \\
varname
is
\\"
varname
\\"
```



## Notes

The first dollar sign and the inner double quotes are backslash-escaped so that they appear literally in the output, which for the above example would be: The value of \$fred is "bob".

### Environment variables:

Environment variables contain information about your login session, stored for the system shell to use when executing commands. They exist whether you're using Linux, Mac, or Windows. Many of these variables are set by default during installation or user creation.

While environment variables apply to all modern systems, this article specifically addresses environment variables in the Bash shell on Linux, BSD, Mac, and Cygwin.

### Understanding environment variables:

Environment variables are no different, technically, than variables. They can be set, recalled, and cleared with exactly the same syntax used for variables. If you're not used to using variables in Bash, read my variables in Bash article before continuing.

You don't often use environment variables directly. They're referenced by individual applications and daemons as needed. For instance, your home directory is set as an environment variable when you log in.

### Scope of an environment variable:

Scope of any variable is the region from which it can be accessed or over which it is defined. An environment variable in Linux can have global or local scope.

#### Global

A globally scoped ENV that is defined in a terminal can be accessed from anywhere in that particular environment which exists in the terminal. That means it can be used in all kind of scripts, programs or processes running in the environment bound by that terminal.

#### Local

A locally scoped ENV that is defined in a terminal cannot be accessed by any program or process running in the terminal. It can only be accessed by the terminal( in which it was defined) itself.

### How to access ENVs?

SYNTAX:

\$NAME

NOTE: Both local and global environment variables are accessed in the same way.

### To display any ENV:

SYNTAX:

```
$ echo $NAME
```

### To display all the Linux ENVs:

SYNTAX:

```
$ printenv //displays all the global ENVs
or
$ set //display all the ENVs(global as well as local)
or
$ env //display all the global ENVs
```

### How to Set and Unset Environment Variables?

There are multiple ways to manipulate the environment variables in a Bash shell.

#### The declare built-in

- ◇ An environment variable can be added to the exported environment with the declare -x <KEY>=<VALUE> syntax
- ◇ An environment variable can be removed from the exported environment with the declare +x <KEY> syntax
- ◇ The existing environment variables can be listed using declare [-p]. It will produce an output with environment variables using the declare -x syntax which allows for easy export of an environment to a file and can be easily sourced later on.

#### The export built-in

- ◇ An environment variable can be added to the exported environment with the export <KEY>=<VALUE> syntax
- ◇ An environment variable can be removed from the exported environment with the export -n <KEY> syntax
- ◇ The existing environment variables can be listed using export [-p]. In Bash, it will produce an output with environment variables using the declare -x syntax as described above.

#### The unset built-in

- ◇ This built-in is solely used to remove (unset) a shell variable or function with the syntax unset <KEY>. If the variable is an environment variable, it will also be removed from the list of exported variables.

#### The printenv command or built-in

- ◇ Exclusively print out the names and values of the environment variables, with one key/value pair per line.

#### The set built-in

- ◇ The set built-in will list all existing shell variables (local and environment variables) when no argument is provided.
- ◇ The set -k option will export all shell local variables to the environment. This is there for a legacy reason and should be avoided. Exporting only the

Notes

## Notes

necessary environment variables individually with declare or export should be preferred.

### The env command:

- ◇ The env command can be used to alter the environment variable before executing a command or print the environment. The changes to the environment are not persisted in the current shell environment.
- ◇ An environment variable can be added to the exported environment with the env [<KEY>=<VALUE> ...] command syntax
- ◇ An environment variable can be removed from the exported environment with the env [-u KEY] command syntax
- ◇ The syntax can combine adding or removing variables, for example, env -u TZ -u HOME MY\_ENV=prod HOME=/opt/app myApp

### List Of Special And Environment Variables in Bas:

Some of the Bash Shell special and environment variables are described in separate sections of this site.

- ◇ You can learn about the LINEO, BASH\_LINENO, BASH\_ARGV0, BASH\_COMMAND, and BASH\_XTRACEFD variables in my post on How To Debug a Bash Shell Scripticon mdi-link-variant
- ◇ The TZ, EPOCHREALTIME, and EPOCHSECONDS variables are covered in my post on How To Format Date and Time in Linux, macOS, and Bash?icon mdi-link-variant
- ◇ The FUNCNAME and FUNCNEST variables are described in my Bash Functionsicon mdi-link-variant Guide
- ◇ The BASH\_REMATCH variable is described in my Bash If Statementicon mdi-link-variant Guide
- ◇ The MAPFILE variable is described in the Bash Arraysicon mdi-link-variant Guide
- ◇ The PROMPT\_COMMAND, PROMPT\_DIRTRIM, PS0, PS1, PS2, PS3, and PS4 are extensively detailed in the Bash Prompticon mdi-link-variant Guide
- ◇ The BASH\_ALIASES variable is covered in my post on How to Use Bash Aliasicon mdi-link-variant
- ◇ The SUDO\_ASKPASS variable is covered in my post on how to fix the error sudo: no tty present and no askpass program specifiedicon mdi-link-variant
- ◇ Finally, the INPUTRC, READLINE\_LINE, and READLINE\_POINT variables can be found in my post on The Most Useful Bash Shortcuts That You Will Want To Useicon mdi-link-variant

The other Bash special and environment variables can be found below.

### The User Home Directory Variable: HOME:

The \$HOME environment variable contains the current user's home directory. The cd builtin command uses the \$HOME variable as default when used with no parameters. The value of this variable is also used by the tilde expansion.

```
[me@linux /usr]$ cd
[me@linux ~]$ pwd
/home/jdoe
[me@linux ~]$ echo $HOME ~
/home/jdoe /home/jdoe
```

### Filename Expansion Variable: GLOBIGNORE:

The \$GLOBIGNORE variable is a colon-separated list of patterns defining the set of filenames to be ignored by filename expansion. If a filename matched by a filename expansion pattern also matches one of the patterns in \$GLOBIGNORE, it is removed from the list of matches. This should be used carefully as it may impact other assumptions in your scripts like when counting files in a directory.

The Host Environment Variables: HOSTNAME, HOSTTYPE, OSTYPE, and MACHTYPE

The \$HOSTNAME contains the name of the current host and \$HOSTTYPE refers to the machine type on which Bash is running. The \$MACHTYPE is very similar to \$HOSTTYPE but with a full description using standard GNU cpu-company-system format. The \$OSTYPE variable exposes a string describing the operating system.

```
[me@linux ~]$ echo hostname=$HOSTNAME type=$HOSTTYPE
os=$OSTYPE mach=$MACHTYPE
hostname=linux type=x86_64 os=linux-gnu mach=x86_64-pc-linux-gnu
```

### The Internal Field Separator Variable: IFS:

IFS stands for Internal Field Separator (or Input Field Separator). The \$IFS variable is often used with bash loops or with the read or printf builtin commands. It is set to a list of delimiting characters used for field separation in the shell. The default value is set to the space, tab, and newline characters, Example: IFS=\$' \t\n'. When the \$IFS variable is unset, it acts as if it was set back to its default value.

```
Using default IFS=$' \t\n'
[me@linux ~]$ read -r a b c <<< ' A B C'
[me@linux ~]$ echo "$a $b $c"
A B C

Using a custom IFS value
[me@linux ~]$ IFS=: read -r user pwhash uid gid gecost home shell \
<<< 'root:*:0:0:System Administrator:/var/root:/bin/sh'
[me@linux ~]$ echo "$user $pwhash $uid $gid $gecost $home $shell"
root * 0 0 System Administrator /var/root /bin/sh
```

## Notes

### The Mail Variables: MAIL, MAILPATH, and MAILCHECK:

The \$MAIL, \$MAILPATH, and \$MAILCHECK are environment variables related to the local inbox system and to notify users of new emails.

The \$MAIL environment variable can be optionally set to a filename or directory name used by Bash to notify the user of new mail in the stated file or Maildir-format directory. Bash uses the \$MAIL variable only when the \$MAILPATH variable is not set.

The \$MAILPATH environment variable is a colon-separated list of filenames used by the Bash shell to periodically check for new mail. Additionally, by using the question mark ? as a field separator, the variable allows to customize the notification message displayed to the user and the \$\_ variable can be used inside the custom message to be expanded to the name of the current mail file. Example: MAILPATH=<filename>?<message>;<filename>?<message>.

```
[me@linux ~]$ MAILPATH="/var/mail/root?Root Got a mail in $_! ;/var/mail/
jdoe?You've got a mail! "
```

The \$MAILCHECK variable set how often, in seconds, the shell should check for mail in the files specified in the \$MAILPATH or \$MAIL variables. The default is 60 seconds. The shell disables mail checking when this variable is unset or is not set to a number greater than zero.

### The getopts Variables: OPTARG, OPTIND, and OPTERR

The \$OPTARG and \$OPTIND are special variables local to the caller of the getopts builtin. The \$OPTARG is used to retrieve the value of the last option argument processed by getopts and \$OPTIND is used to retrieve the index of the last option argument processed.

When the \$OPTERR variable is set to the value 1 (default), Bash displays error messages generated by the getopts builtin command.

### The Executable Command Variables: PATH and EXECIGNORE:

\$PATH is probably the most known and commonly used of the environment variables. It's a colon-separated list of directories in which the shell looks for commands. This often includes the current directory and some variations of /bin and /usr/bin. The dot . or a zero-length (null) directory name designates the current directory. A null directory name might be displayed as two adjacent colons :: or as an initial or trailing colon.

```
[me@linux ~]$ echo $PATH
:/home/jdoe:/usr/local/sbin:/usr/local/bin:/usr/local/bin:/usr/bin:/bin:/usr/
sbin:/sbin
```

The \$EXECIGNORE variable is a colon-separated list of shell patternsicon mdi-link-variant defining the list of filenames to be ignored by command search using \$PATH. Files whose full pathnames match one of these patterns are not considered executable files for the purposes of completion and command execution via \$PATH lookup.

### The Shell Options Variables: BASHOPTS and SHELLOPTS:

The \$BASHOPTS variable is a read-only colon-separated list of shell options enabled with the shopt builtin command. The list of options in this variable will be the same as the one reported as on by the shopt command.

The \$SHELLOPTS variable is a read-only colon-separated list of shell options enabled with the set builtin command and -o option. The list of options in this variable will be the same as the one reported as on by the set -o command.

### The Current Shell Path Variables: BASH and SHELL:

The \$BASH and \$SHELL variables store the full pathname used to execute the current instance of Bash.

```
[me@mac ~]$ echo $BASH $SHELL
/usr/local/bin/bash /usr/local/bin/bash
```

### The Shell Process IDs variables: BASHPID and PPID:

The \$PPID readonly variable contains the process ID of the current shell's parent process while the \$BASHPID provide the current Bash process ID. \$BASHPID may differs from the \$\$ since subshells do not require Bash to be re-initialized. \$BASHPID will lose its special properties when being unset.

```
[me@mac ~]$ echo PPID=$PPID BASHPID=$BASHPID $$=$$
PPID=23693 BASHPID=23764 $$=23764
[me@mac ~]$ (echo PPID=$PPID BASHPID=$BASHPID $$=$$)
PPID=23693 BASHPID=39074 $$=23764
The Bash Hash Commands Variable: BASH_CMDS
```

The \$BASH\_CMDS is an associative array on mdi-link-variant variable that contains the list of hash commands as created by the hash builtin. \$BASH\_CMDS will lose its special properties when being unset.

### The Shell Compatibility Variables: POSIXLY\_CORRECT and BASH\_COMPAT:

The \$POSIXLY\_CORRECT environment variable can be set to force bash to run in POSIX mode as if bash --posix or set -o posix were used. When using set -o posix the \$POSIXLY\_CORRECT variable is automatically set to y.

The \$BASH\_COMPAT is used to enforce compatibility between Bash versions. You can set the value to a decimal number or an integer value representing the Bash version to be compatible with. Acceptable values are the compat<vers> options listed by the shopt builtin and the current Bash version. When the \$BASH\_COMPAT variable is unset or set to an empty string, the default value becomes the current Bash version.

### The Shell Init File Variables: ENV and BASH\_ENV:

The \$BASH\_ENV environment variable is used by Bash to define an init file to read before executing a script. This is often used for debugging purposes to define shell options or debugging traps.

## Notes

### 4.2.5 Assignment Statements, Arithmetic, Logical and Relational Operators, Read, Echo

#### Assignment Statements

T Variable assignments can be placed at the start of a command line in the Bourne Again Shell. This type of assignment puts variables in the command shell's environment, which means they're only accessible from the programme (and its children) that the command runs. It isn't available via the command's executing shell. The value of TEMPDIR is displayed by the my\_script shell script. The following command runs my\_script with TEMPDIR set to /home/sam/temp. The echo builtin shows that the interactive shell has no value for TEMPDIR after running my\_script. If TEMPDIR had been set in the interactive shell, running my\_script in this manner would have had no effect on its value.

```
$ cat my_script
echo $TEMPDIR
$ TEMPDIR=/home/sam/temp ./my_script
/home/sam/temp
$ echo $TEMPDIR
$
```

When you run a command with a variable as an argument, the shell changes the variable's name with its value and provides that value to the programme you're running. If the value of the variable contains a special character, such as

\* or ?, the shell might expand that variable.

The first line in the following sequence of commands assigns the string max\* to the variable memo. All shells interpret special characters as special when you reference a variable that contains an unquoted special character. In the following example, the shell expands the value of the memo variable because it is not quoted:

```
$ memo=max*
$ ls max.report max.summary
$ echo $memo
max.report max.summary
```

Above, the shell expands the \$memo variable to max\*, expands max\* to max.report and max.summary, and passes these two values to echo. In the next example, the Bourne Again Shell does not expand the string because bash does not perform path- name expansion (page 148) when it assigns a value to a variable.

```
$ echo "$memo"
max*
```

All shells process a command line in a specific order. Within this order bash (but not tcsh) expands variables before it interprets commands. In the preceding echo



command line, the double quotation marks quote the asterisk (\*) in the expanded value of \$memo and prevent bash from performing pathname expansion on the expanded memo variable before passing its value to the echo command.

### Arithmetic Operator:

Arithmetic expansion and arithmetic evaluation in bash use the same syntax, precedence, and associativity of expressions as the C language. These operators are used to perform normal arithmetics/mathematical operations. There are 7 arithmetic operators:

- ◇ Addition (+): Binary operation used to add two operands.
- ◇ Subtraction (-): Binary operation used to subtract two operands.
- ◇ Multiplication (\*): Binary operation used to multiply two operands.
- ◇ Division (/): Binary operation used to divide two operands.
- ◇ Modulus (%): Binary operation used to find remainder of two operands.
- ◇ Increment Operator (++): Unary operator used to increase the value of operand by one.
- ◇ Decrement Operator (--): Unary operator used to decrease the value of a operand by one.

### Example:

```
#!/bin/bash
#reading data from the user
read -p 'Enter a : ' a
read
 p 'Enter b : ' b

add
= $((a + b))
echo Addition of a and b are $add

sub
= $((a - b))
echo Subtraction of a and b are $sub

mul
= $((a * b))
echo Multiplication of a and b are $mul

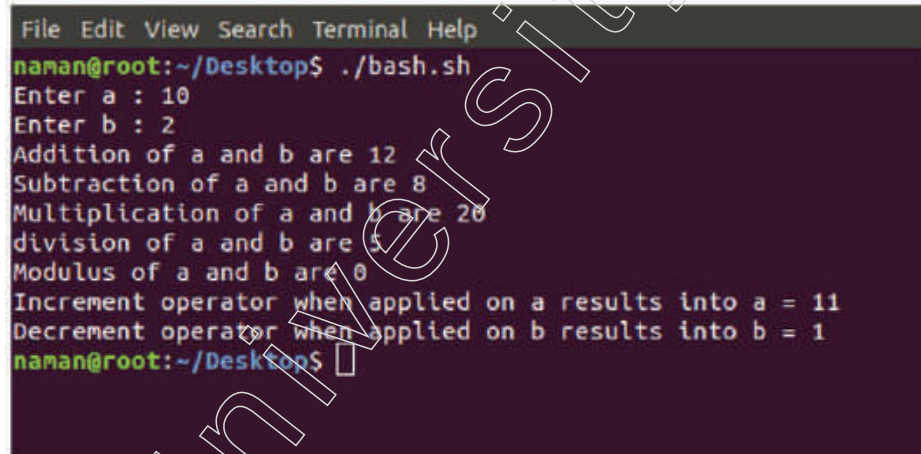
div
= $((a / b))
echo division of a and b are $div

mod
```

## Notes

```
= $((a % b))
echo Modulus of a
and b are $mod
((++a))
echo Increment
operator when applied on "a" results into a = $a
(--b)
echo Decrement
operator when applied on "b" results into b = $b
```

### Output:



```
File Edit View Search Terminal Help
naman@root:~/Desktop$./bash.sh
Enter a : 10
Enter b : 2
Addition of a and b are 12
Subtraction of a and b are 8
Multiplication of a and b are 20
division of a and b are 5
Modulus of a and b are 0
Increment operator when applied on a results into a = 11
Decrement operator when applied on b results into b = 1
naman@root:~/Desktop$
```

### Logical Operators:

They are also known as boolean operators. These are used to perform logical operations. They are of 3 types:

- ◇ **Logical AND (&&):** This is a binary operator, which returns true if both the operands are true otherwise returns false.
- ◇ **Logical OR (||):** This is a binary operator, which returns true if either of the operand is true or both the operands are true and return false if none of them is false.
- ◇ **Not Equal to (!):** This is a unary operator which returns true if the operand is false and returns false if the operand is true.

```
#!/bin/bash
#reading data from the user
read -p 'Enter a : ' a
read -p 'Enter b : ' b
if(($a == "true" & $b == "true"))
then
 echo Both are true.
else
 echo Both are not true.
fi
if(($a == "true" || $b == "true"))
then
 echo Atleast one of them is true.
else
 echo None of them is true.
fi
if((! $a == "true"))
then
 echo "a" was initially false.
else
 echo "a" was initially true.
Fi
```

Output:

```
File Edit View Search Terminal Help
naman@root:~/Desktop$./bash.sh
Enter a : true
Enter b : false
Both are true.
Atleast one of them is true.
a was intially true.
naman@root:~/Desktop$
```

### Relational Operators:

Relational operators are those operators which define the relation between two operands. They give either true or false depending upon the relation. They are of 6 types:

- ◇ **'==' Operator:** Double equal to operator compares the two operands. Its returns true is they are equal otherwise returns false.
- ◇ **'!=' Operator:** Not Equal to operator return true if the two operands are not equal otherwise it returns false.
- ◇ **'<' Operator:** Less than operator returns true if first operand is less than second operand otherwise returns false.
- ◇ **'<=' Operator:** Less than or equal to operator returns true if first operand is less than or equal to second operand otherwise returns false

## Notes

- ◇ **'>' Operator:** Greater than operator return true if the first operand is greater than the second operand otherwise return false.
- ◇ **'>=' Operator:** Greater than or equal to operator returns true if first operand is greater than or equal to second operand otherwise returns false

### Example:

```
#!/bin/bash
#reading data from the user
read -p 'Enter a : ' a
read -p 'Enter b : ' b

if(($a==$b))
then
 echo a is equal to b.
else
 echo a is not equal to b.
fi

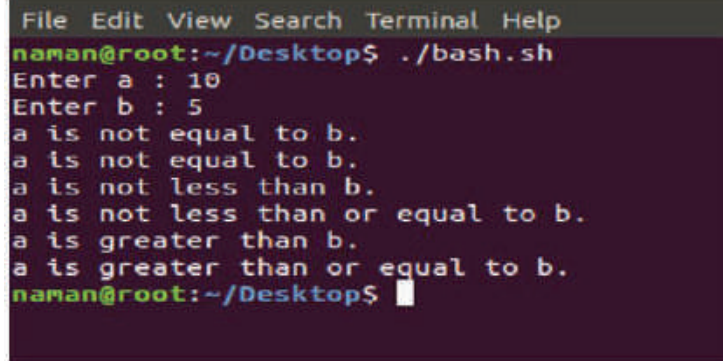
if(($a!= $b))
then
 echo a is not equal to b.
else
 echo a is equal to b.
fi

if(($a<$b))
then
 echo a is less than b.
else
 echo a is not less than b.
fi

if(($a<=$b))
then
 echo a is less than or equal to b.
else
 echo a is not less than or equal to b.
fi

if(($a>$b))
then
 echo a is greater than b.
else
 echo a is not greater than b.
fi

if(($a>=$b))
then
 echo a is greater than or equal to b.
else
 echo a is not greater than or equal to b.
fi
```

**Output:**


```
File Edit View Search Terminal Help
naman@root:~/Desktop$./bash.sh
Enter a : 10
Enter b : 5
a is not equal to b.
a is not equal to b.
a is not less than b.
a is not less than or equal to b.
a is greater than b.
a is greater than or equal to b.
naman@root:~/Desktop$
```

**Read Command:**

A line of input is read into a variable using the read command. Each argument must be a variable name that does not begin with the letter “\$.” The built-in command reads a line of input and uses the “IFS” inter field separator to break it down into individual words. The “IFS” is set to a space by default. From left to right, each word in the line is kept in a variable. The first word goes into the first variable, the second into the second, and so on. If there are fewer variables than words, the last variable is assigned to all remaining words. If there are more variables defined than words, the extra variables are set to null. If no variable names are specified on the read line, the default variable is used REPLY.

**Syntax:**

```
read [options] VAR1 VAR2 VARN
```

**example**

```
#!/bin/bash
#
read first middle last
echo "Hello $first $middle $last"
```

**Output from above read example**

```
$./read1.sh
land of linux
Hello land of linux
```

Other parameters that can be used with the read built in in command are included in the table below:

**Notes**

## Notes

| OPTION     | DESCRIPTION                                                                                    |
|------------|------------------------------------------------------------------------------------------------|
| -a ANAME   | Words are assigned sequentially to the array variable ANAME                                    |
| -d DELIM   | The first character of DELIM is used to terminate the input line                               |
| -e         | readline is used to get line                                                                   |
| -n NCHARS  | read returns after reading NCHARS                                                              |
| -p PROMPT  | Display PROMPT without a trailing newline. Prompt is only displayed if coming from a terminal. |
| -r         | Backslash does not act as an escape character                                                  |
| -s         | Silent Mode. Characters are not echoed coming from a Terminal                                  |
| -t TIMEOUT | read will timeout after TIMEOUT seconds. Only from a Terminal                                  |
| -u FD      | read input from File Descriptor FD                                                             |

### Echo Command:

The echo command is a built-in Linux feature that prints out arguments as the standard output. echo is generally used to display text strings or command results as messages. The echo command in Linux is used to display a string provided by the user.

Syntax :

```
echo [option] [string]
```

Displaying a text/string :

Syntax :

```
echo [string]
```

For example, use the following command to print Hello, World! as the output:

```
echo Hello, World!
```

```
test@test:~$ echo Hello, World!
Hello, World!
test@test:~$
```

The echo command uses the following options:

- ◇ -n: Displays the output while omitting the newline after it.
- ◇ -E: The default option, disables the interpretation of escape characters.

- ◇ -e: Enables the interpretation of the following escape characters:
- ◇ \\: Displays a backslash character (\).
- ◇ \a: Plays a sound alert when displaying the output.
- ◇ \b: Creates a backspace character, equivalent to pressing Backspace.
- ◇ \c: Omits any output following the escape character.
- ◇ \e: The escape character, equivalent to pressing Esc.
- ◇ \f: The form feed character, causes the printer to automatically advance to the start of the next page.
- ◇ \n: Adds a new line to the output.
- ◇ \r: Performs a carriage return.
- ◇ \t: Creates horizontal tab spaces.
- ◇ \v: Creates vertical tab spaces.
- ◇ \NNN: Byte with the octal value of NNN.
- ◇ \xHH: Byte with the hexadecimal value of H.

#### 4.2.6 Test Command through IF Statement: Numerical Test, File Test and String Test

Depending on the success or failure of a command, you may need to define multiple courses of action in a shell script. You can describe such criteria using the if construct.

The if command's most compact syntax is::

**if TEST-COMMANDS; then CONSEQUENT-COMMANDS; fi**

If the return status of the TEST-COMMAND list is zero, the CONSEQUENT-COMMANDS list is run. The exit status of the last command performed, or zero if no condition was true, determines the return status.

TEST-COMMAND is usually used for numerical or string comparison tests, but it can also be used for any command that returns a status of zero when it succeeds and a different status when it fails. When examining the state of a file, unary expressions are frequently employed. The file descriptor "N" is verified if the FILE argument to one of the primaries is of the form /dev/fd/N. Tests can also be run on stdin, stdout, and stderr, as well as their appropriate file descriptors.

#### Expressions used with if

The table below summarises the "primaries" that make up the TEST-COMMAND command or list of commands. To show the test of a conditional statement, these primaries are enclosed in square brackets.

#### Primary expressions

| Primary   | Meaning                                              |
|-----------|------------------------------------------------------|
| [-a FILE] | True if FILE exists.                                 |
| [-b FILE] | True if FILE exists and is a block-special file.     |
| [-c FILE] | True if FILE exists and is a character-special file. |



## Notes

|                         |                                                                                                                                                                                                                                                                     |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [-d FILE]               | True if FILE exists and is a directory.                                                                                                                                                                                                                             |
| [-e FILE]               | True if FILE exists.                                                                                                                                                                                                                                                |
| [-f FILE]               | True if FILE exists and is a regular file.                                                                                                                                                                                                                          |
| [-g FILE]               | True if FILE exists and its SGID bit is set.                                                                                                                                                                                                                        |
| [-h FILE]               | True if FILE exists and is a symbolic link.                                                                                                                                                                                                                         |
| [-k FILE]               | True if FILE exists and its sticky bit is set.                                                                                                                                                                                                                      |
| [-p FILE]               | True if FILE exists and is a named pipe (FIFO).                                                                                                                                                                                                                     |
| [-r FILE]               | True if FILE exists and is readable.                                                                                                                                                                                                                                |
| [-s FILE]               | True if FILE exists and has a size greater than zero.                                                                                                                                                                                                               |
| [-t FD]                 | True if file descriptor FD is open and refers to a terminal.                                                                                                                                                                                                        |
| [-u FILE]               | True if FILE exists and its SUID (set user ID) bit is set.                                                                                                                                                                                                          |
| [-w FILE]               | True if FILE exists and is writable.                                                                                                                                                                                                                                |
| [-x FILE]               | True if FILE exists and is executable.                                                                                                                                                                                                                              |
| [-O FILE]               | True if FILE exists and is owned by the effective user ID.                                                                                                                                                                                                          |
| [-G FILE]               | True if FILE exists and is owned by the effective group ID.                                                                                                                                                                                                         |
| [-L FILE]               | True if FILE exists and is a symbolic link.                                                                                                                                                                                                                         |
| [-N FILE]               | True if FILE exists and has been modified since it was last read.                                                                                                                                                                                                   |
| [-S FILE]               | True if FILE exists and is a socket.                                                                                                                                                                                                                                |
| [FILE1 -nt FILE2]       | True if FILE1 has been changed more recently than FILE2, or if FILE1 exists and FILE2 does not.                                                                                                                                                                     |
| [FILE1 -ot FILE2]       | True if FILE1 is older than FILE2 exists and FILE1 does not.                                                                                                                                                                                                        |
| [FILE1 -ef FILE2]       | True if FILE1 and FILE2 refer to the same device and in ode numbers.                                                                                                                                                                                                |
| [-o OPTIONNAME]         | True if shell option "OPTIONNAME" is enabled.                                                                                                                                                                                                                       |
| [-z STRING]             | True of the length if "STRING" is zero.                                                                                                                                                                                                                             |
| [-n STRING] or [STRING] | True if the length of "STRING" is non-zero.                                                                                                                                                                                                                         |
| [ STRING1==STRING2]     | True if the strings are equal. "=" may be used instead of "==" for strict POSIX compliance.                                                                                                                                                                         |
| [ STRING1 != STRING2]   | True if the strings are not equal.                                                                                                                                                                                                                                  |
| [STRING1 < STRING2]     | True if "STRING1" sorts before "STRING2" lexicographically in the current locale.                                                                                                                                                                                   |
| [STRING1 > STRING2]     | True if "STRING1" sorts after "STRING2" lexicographically in the current locale.                                                                                                                                                                                    |
| [ARG1 OP ARG2]          | "OP" is one of -eq, -ne, -lt, -le, -gt or -ge. These arithmetic binary operators return true if "ARG1" is equal to, not equal to, less than, less than or equal to, greater than, or greater than or equal to "ARG2", respectively. "ARG1" and "ARG2" are integers. |

The following operators, listed in decreasing order of precedence, can be used to combine expressions:

## Combining expressions

| Operation          | Effect                                                                                              |
|--------------------|-----------------------------------------------------------------------------------------------------|
| [ ! EXPR ]         | True if <b>EXPR</b> is false.                                                                       |
| [ ( EXPR ) ]       | Returns the value of <b>EXPR</b> . This may be used to override the normal precedence of operators. |
| [ EXPR1 -a EXPR2 ] | True if both <b>EXPR1</b> and <b>EXPR2</b> are true.                                                |
| [ EXPR1 -o EXPR2 ] | True if either <b>EXPR1</b> or <b>EXPR2</b> is true.                                                |

Conditional expressions are evaluated using a set of criteria dependent on the number of arguments by the `[`(or `test`) built-in. The Bash documentation provide more information on this issue. The opening square bracket should be closed after the criteria have been listed, much like the `if` is closed with `fi`.

## Commands following the then statement

With the exception of the ending `fi`, the **CONSEQUENT-COMMANDS** list that follows the `then` statement can be any legal UNIX command, executable programme, executable shell script, or shell statement. It's vital to understand that the shell considers the `then` and `fi` commands to be independent statements. As a result, they are separated by a semi-colon when supplied on the command line.

The distinct sections of the `if` statement are usually well-separated in a script. A number of simple examples are shown below.

### Checking files

The first example checks for the existence of a file:

```

anny ~-> cat msgcheck.sh
#!/bin/bash

echo "This scripts checks the existence of the messages file."
echo "Checking..."
if [-f /var/log/messages]
then
 echo "/var/log/messages exists."
fi
echo
echo "...done."

anny ~-> ./msgcheck.sh
This scripts checks the existence of the messages file.
Checking...
/var/log/messages exists.
...done.
```

## Notes

## Notes

### Checking shell options

To add in your Bash configuration files:

```
These lines will print a message if the noclobber option is set:
if [-o noclobber]
then
 echo "Your files are protected against accidental overwriting using
redirection."
fi
```

### The environment

The example above will work when you enter it on the command line:

```
anny ~> if [-o noclobber] ; then echo ; echo "your files are protected
against overwriting." ; echo ; fi

your files are protected against overwriting.

anny ~>
```

When you employ testing of conditions that are dependent on the environment in a script, however, you may receive different results since the script will launch a new shell in which the expected variables and options may not be set automatically.

### Simple applications of if

#### Testing exit status

The exit status of the previously performed command is stored in the `?` variable (the recently completed foreground process).

A simple test is demonstrated in the following example:

```
anny ~> if [$? -eq 0]
More input> then echo 'That was a good job!'
More input> fi
That was a good job!

anny ~>
```

TEST-COMMANDS can be any UNIX command that produces an exit status, as seen in the following example, and it always returns a zero exit status:

```
anny ~> if ! grep $USER /etc/passwd
More input> then echo "your user account is not managed locally"; fi
your user account is not managed locally

anny > echo $?
0

anny >
```

The following is a way to get the same outcome:

```
anny > grep $USER /etc/passwd

anny > if [$? -ne 0]; then echo "not a local account" ; fi
not a local account

anny >
```

### Numeric comparisons

The following examples make use of numerical comparisons:

```
anny > num=`wc -l work.txt`

anny > echo $num
201

anny > if ["$num" -gt "150"]
More input> then echo ; echo "you've worked hard enough for today."
More input> echo ; fi

you've worked hard enough for today.

anny >
```

This script is run every Sunday by cron. If the week number is even, it serves as a reminder to empty the trash cans:

```
#!/bin/bash

Calculate the week number using the date command:

WEEKOFFSET=$((date +%V) % 2)

Test if we have a remainder. If not, this is an even week so send a
message.
Else, do nothing.

if [$WEEKOFFSET -eq "0"]; then
 echo "Sunday evening, put out the garbage cans." | mail -s "Garbage
cans out" your@your_domain.org
fi
```

### String comparisons

Example: A comparison of strings for the purpose of testing the user ID:

```
if ["$(whoami)" != 'root']; then
 echo "You have no permission to run $0 as non-root user."
 exit 1;
fi
```

This type of construct can be made shorter with Bash. The following is the condensed version of the above test:

## Notes

```
["$(whoami)" != 'root'] && (echo you are using a non-privileged account; exit 1)
```

In the same way as the “&&” expression provides what to do if the test is true, the “||” expression specifies what to do if the test is false.

In comparisons, regular expressions can also be employed:

```
anny > gender="female"
anny > if [["$gender" == f*]]
More input> then echo "Pleasure to meet you, Madame."; fi
Pleasure to meet you, Madame.
anny >
```

The test built-in command, which is comparable to using square brackets for comparison, is preferred by most programmers:

```
test "$(whoami)" != 'root' && (echo you are using a non-privileged account; exit 1)
```

### No exit?

The exit will not send variables to the parent if it is performed from a subshell. If you don't want Bash to fork a subshell, use { and } instead of ( and ).

### 4.2.7 Test Command: Numerical Test, File Test and String Test

The condition is a series of parameters and values that the test command evaluates. When used in an if-then statement, the test command looks like this:

```
if test condition
then
commands
fi
```

The bash shell provides an alternative way of declaring the test command in an if-then statement:

```
if [condition]
then
commands
fi
```

The square brackets define the condition that's used in the test command. Be careful; you must have a space after the first bracket, and a space before the last bracket or you'll get an error message.

There are three classes of conditions the test command can evaluate:

- ◇ Numeric comparisons
- ◇ String comparisons
- ◇ File comparisons

## Numeric comparisons

The most common method for using the test command is to perform a comparison of two numeric values. Table below shows the list of condition parameters used for testing two values.

The numeric test conditions can be used to evaluate both numbers and variables. Here's an example of doing that:

```
$ cat test5
#!/bin/bash
using numeric test comparisons
val1=10
val2=11
if [$val1 -gt 5]
then
echo The test value $val1 is greater than 5
fi
if [$val1 -eq $val2]
then
echo The values are equal
else
echo The values are different
fi
```

**The test Numeric Comparisons**

| Comparison    | Description                                      |
|---------------|--------------------------------------------------|
| $n1 - eq\ n2$ | Check if $n1$ is equal to $n2$ .                 |
| $n1 - ge\ n2$ | Check if $n1$ is greater than or equal to $n2$ . |
| $n1 - gt\ n2$ | Check if $n1$ is greater than $n2$ .             |
| $n1 - le\ n2$ | Check if $n1$ is less than or equal to $n2$ .    |
| $n1 - lt\ n2$ | Check if $n1$ is less than $n2$ .                |
| $n1 - ne\ n2$ | Check if $n1$ is not equal to $n2$ .             |

The first test condition:

```
if [$val1 -gt 5]
```

tests if the value of the variable val1 is greater than 5. The second test condition:

```
if [$val1 -eq $val2]
```

tests if the value of the variable val1 is equal to the value of the variable val2. Run the script and watch the results:

```
$./test5
The test value 10 is greater than 5
The values are different
$
```

Notes

## Notes

Both of the numeric test conditions evaluated as expected. There is a limitation to the test numeric conditions though. Try this script:

```
$ cat test6
#!/bin/bash
testing floating point numbers
val1=echo scale=4; 10 / 3 | bc
echo "The test value is $val1"
if [$val1 -gt 3]
then
echo "The result is larger than 3"
fi
$./test6
The test value is 3.3333
./test6: line 5: [: 3.3333: integer expression expected
$
```

This illustration uses the bash calculator to produce a floating-point value, stored in the val1 variable. Next, it uses the test command to evaluate the value. Something obviously went wrong here.

In the previous section, you learned how to trick the bash shell into handling floating-point values; there's still a problem in this script. The test command wasn't able to handle the floating-point value that was stored in the val1 variable.

Remember, the only numbers the bash shell can handle are integers. When we utilize the bash calculator, we just fool the shell into storing a floating-point value in a variable as a string value. This works perfectly fine if all you need to do is display the result, using an echo statement, but this doesn't work in numeric-oriented functions, such as our numeric test condition. The bottom line is that you're not able to use integer values in the test command.

### String comparisons

The test command also allows you to perform comparisons on string values. Performing comparisons on strings can get tricky, as you'll see. Table below shows the comparison functions you can use to evaluate two string values.

The following sections describe the different string comparisons available.

#### String equality

With strings, the equal and not equal requirements are very self-explanatory. It's rather simple to determine whether two string values are the same or not :

```
$ cat test7
#!/bin/bash
9
testing string equality
testuser=rich
if [$USER = $testuser]
then
```



```
echo "Welcome $testuser"
fi
$./test7
Welcome rich
$
```

Similarly, using the not equals string comparison:

### The test Command String Comparisons

| Comparison                  | Description                                                          |
|-----------------------------|----------------------------------------------------------------------|
| <code>str1 = str2</code>    | Check if <code>str1</code> is the same as string <code>str2</code> . |
| <code>str1 != str 2</code>  | Check if <code>str1</code> is not the same as <code>str2</code> .    |
| <code>str1 &lt; str2</code> | Check if <code>str1</code> is less than <code>str2</code> .          |
| <code>str1 &gt; str2</code> | Check if <code>str1</code> is greater than <code>str2</code> .       |
| <code>-n str1</code>        | Check if <code>str1</code> has a length greater than zero.           |
| <code>-z str1</code>        | Check if <code>str1</code> has a length of zero.                     |

```
$ cat test8
#!/bin/bash
testing string equality
testuser=baduser
if [$USER != $testuser]
then
echo "This isn't $testuser"
else
echo "Welcome $testuser"
fi
$./test8
This isn't baduser
$
```

When comparing strings for equality, the test comparison considers all punctuation and capitalization.

### String order

Trying to figure out if one string is more or less than another is when things get tough. When using the greater-than or less-than features of the test command, shell programmers frequently run into two issues:

- ◇ The greater-than and less-than symbols must be escaped, else the shell will use the string values as filenames and utilise them as redirection symbols.
- ◇ The order of greater-than and less-than is not the same as with the sort command.

The first item can result in a huge problem that often goes undetected when programming your scripts. Here's a typical example of what sometimes happens to novice shell script programmers:

Notes

**Notes**

```
$ cat badtest
#!/bin/bash
mis-using string comparisons
val1=baseball
val2=hockey
if [$val1 > $val2]
then
echo "$val1 is greater than $val2"
else
echo "$val1 is less than $val2"
fi
$./badtest
baseball is greater than hockey
$ ls -l hockey
-rw-r--r-- 1 rich rich 0 Sep 30 19:08 hockey
$
```

By just using the greater-than symbol itself in the script, no errors are generated, but the results are wrong. The script interpreted the greater-than symbol as an output redirection, so it created a file called hockey. Since the redirection completed successfully, the test command returns a zero exit status code, which the if statement evaluates as though things completed successfully!

To fix this problem, you need to properly escape the greater-than symbol:

```
$ cat test9
#!/bin/bash
mis-using string comparisons
val1=baseball
val2=hockey
if [$val1 > $val2]
9
then
echo "$val1 is greater than $val2"
else
echo "$val1 is less than $val2"
fi
$./test9
baseball is less than hockey
$
```

Now that answer is more along the lines of what you would expect from the string comparison. The second issue is a little more subtle, and you may not even run across it unless you're working with upper-case and lower-case letters. The way the sort command handles upper-case letters is opposite of the way the test command considers them. Let's test this feature in a script:

**Notes**

```
$ cat test9b
#!/bin/bash
testing string sort order
val1=Testing
val2=testing
if [$val1 > $val2]
then
echo "$val1 is greater than $val2"
else
echo "$val1 is less than $val2"
fi
$./test9b
Testing is less than testing
$ sort testfile
testing
Testing
$
```

Capitalized letters appear less than lower-case letters in the test command. However, when you put the same strings in a file and use the sort command, the lower-case letters appear first. This is due to the ordering technique each command uses. The test command uses standard ASCII ordering, using each character's ASCII numeric value to determine the sort order. The sort command uses the sorting order defined for the system locale language settings. For the English language, the locale settings specify that lower-case letters appear before upper-case letters in sort order.

**String size**

The -n and -z comparisons are handy when trying to evaluate if a variable contains data or not:

```
$ cat test10
#!/bin/bash
testing string length
val1=testing
val2=""
if [-n $val1]
then
echo "The string '$val1' is not empty"
else
echo "The string '$val1' is empty"
fi
if [-z $val2]
then
echo "The string '$val2' is empty"
else
echo "The string '$val2' is not empty"
fi
if [-z $val3]
```

## Notes

```

then
echo "The string '$val3' is empty"
else
echo "The string '$val3' is not empty"
fi
$./test10
The string 'testing' is not empty
The string "" is empty
The string "" is empty
$

```

This example creates two string variables. The `val1` variable contains a string, and the `val2` variable is created as an empty string. The following comparisons are made:

```
if [-n $val1]
```

determines if the `val1` variable is non-zero in length, which it is, so the then section is processed:

```
if [-z $var2]
```

determines if the `val2` variable is zero in length, which it is, so the then section is processed:

```
if [-z $val3]
```

determines if the `val3` variable is zero in length. This variable was never defined in the shell script, so it indicates that the string length is still zero, even though it wasn't defined.

### The test Command File Comparisons

| Comparison                   | Description                                                                         |
|------------------------------|-------------------------------------------------------------------------------------|
| <code>-d file</code>         | Check if <i>file</i> exists and is a directory.                                     |
| <code>-e file</code>         | Checks if <i>file</i> exists.                                                       |
| <code>-f file</code>         | Checks if <i>file</i> exists and is a file.                                         |
| <code>-r file</code>         | Checks if <i>file</i> exists and is readable.                                       |
| <code>-s file</code>         | Checks if <i>file</i> exists and is not empty.                                      |
| <code>-w file</code>         | Checks if <i>file</i> exists and is writable.                                       |
| <code>-x file</code>         | Checks if <i>file</i> exists and is executable.                                     |
| <code>-O file</code>         | Checks if <i>file</i> exists and is owned by the current user.                      |
| <code>-G file</code>         | Checks if <i>file</i> exists and the default group is the same as the current user. |
| <code>file1 -nt file2</code> | Checks if <i>file1</i> is newer than <i>file2</i> .                                 |
| <code>file1 -ot file2</code> | Checks if <i>file1</i> is older than <i>file2</i> .                                 |

### File comparisons

The final category of test comparisons is likely the most powerful and often utilised in shell programming. Test the status of files and directories on the Linux filesystem with the test command. These similarities are listed in the table below.

These conditions allow you to verify files on your filesystem from within shell scripts, and they're commonly used in file-accessing scripts. Let's take a closer look at each of these because they're so common.

### Checking directories

The `-d` test checks if a specified filename exists as a directory on the system. This is usually a good thing to do if you're trying to write a file to a directory, or before you try to change to a directory location:

```
$ cat test11
#!/bin/bash
look before you leap
if [-d $HOME]
then
echo "Your HOME directory exists"
cd $HOME
ls -a
else
echo "There's a problem with your HOME directory"
fi
$./test11
Your HOME directory exists
. Drivers .ICEauthority .recently
.. .emacs.d joomla .redhat
4rich .evolution .lesshst rpm.list
backup.zip .fontconfig .metacity .spamassassin
.bash history .gconf mnt store
.bash logout .gconfd .mozilla store.sql
.bash profile .gftp Music store.zip
.bashrc .gimp-2.2 myprob Templates
.bittorrent .gnome myproblem test
.ccache .gnome2 myprog test1
.config .gnome2 private .mysql history .thumbnails
Desktop .gphedit .nautilus .Trash
.dmrc .gstreamer-0.10 .openoffice.org2.0 Videos
Documents .gtk-bookmarks Pictures .viminfo
Download .gtkrc-1.2-gnome2 Public
$
```

The sample code uses the `-d` test condition to see if the HOME directory exists for the user. If it does, it proceeds to use the `cd` command to change to the HOME directory and performs a directory listing.

### Checking if an object exists

The `-e` comparison lets you see if a file or directory object is there before you attempt to use it in your script:

## Notes

```
$ cat test12
#!/bin/bash # checking if a directory exists
if [-e $HOME]
then
echo "OK on the directory, now let's check the file"
checking if a file exists
if [-e $HOME/testing]
then
the file exists, append data to it
echo "Appending date to existing file"
date <$HOME/testing
else
the file doesn't exist, create a new file
9
echo Creating new file
date $HOME/testing
fi
else
echo Sorry, you don't have a HOME directory
fi
$./test12
OK on the directory, now let's check the file
Creating new file
$./test12
OK on the directory, now let's check the file
Appending date to existing file
$
```

The `-e` comparison is used in the first check to see if the user has a HOME directory. If this is the case, the following `-e` comparison determines if the testing file exists in the HOME directory. If the file doesn't exist, the shell script creates a new file with the date command's output using the single greater-than redirect symbol. The shell script utilises the double greater-than sign the second time you execute it, so it just appends the date output to the existing file.

### Checking for a file

The `-e` comparison works for both files and directories. To be sure that the object specified is a file, you must use the `-f` comparison:

```
$ cat test13
#!/bin/bash
check if a file
if [-e $HOME]
then
echo "The object exists, is it a file?"
if [-f $HOME]
then
echo "Yes, it's a file!"
else
```

```
echo "No, it's not a file!"
if [-f $HOME/.bash history]
then
echo "But this is a file!"
fi
fi
else
echo "Sorry, the object doesn't exist"
fi
$./test13
The object exists, is it a file?
No, it's not a file!
But this is a file!
$
```

This little script does a whole lot of checking! First, it uses the `-e` comparison to test if `$HOME` exists. If it does, it uses `-f` to test if it's a file. If it isn't a file (which of course it isn't), we use the `-f` comparison to test if it's a file, which it is.

### Can you read it?

Before trying to read data from a file, it's usually a good idea to test if you can read from the file first. This is done using the `-r` comparison:

```
$ cat test14
#!/bin/bash
testing if you can read a file
pwfile=/etc/shadow
first, test if the file exists, and is a file
if [-f $pwfile]
then
now test if you can read it
if [-r $pwfile]
then
tail $pwfile
else
echo Sorry, I'm unable to read the $pwfile file
fi
else
echo Sorry, the file $file doesn't exist
fi
$./test14
Sorry, I'm unable to read the /etc/shadow file
$
```

The `/etc/shadow` file contains the encrypted passwords for system users, so it's not readable by normal users on the system. The `-r` comparison determined that I didn't have read access to the file, so the test command failed, and the bash shell executed the `else` section of the `if-then` statement.



## Notes

### Checking for empty files

You should use `-s` comparison to check if a file is empty, especially if you're trying to remove a file. Be careful, because when the `-s` comparison succeeds, it indicates that a file has data in it:

```
$ cat test15
#!/bin/bash
testing if a file is empty
file=t15test
touch $file
if [-s $file]
then
 9
 echo The $file file exists and has data in it
else
 echo The $file exists and is empty
fi
date $file
if [-s $file]
then
 echo The $file file has data in it
else
 echo The $file is still empty
fi
$./test15
The t15test exists and is empty
The t15test file has data in it
$
```

The `touch` command creates the file, but doesn't put any data in it. After we use the `date` command and redirect the output to the file, the `-s` comparison indicates that there's data in the file.

### Checking if you can write to a file

The `-w` comparison determines if you have permission to write to a file:

```
$ cat test16
#!/bin/bash
checking if a file is writeable
logfile=$HOME/t16test
touch $logfile
chmod u-w $logfile
now=date +%Y%m%d-%H%Mdate +%Y%m%d-%H%M
if [-w $logfile]
then
 echo The program ran at: nowlogfile
 echo The first attempt succeeded
else
```

```
echo "The first attempt failed"
fi
chmod u+w $logfile
if [-w $logfile]
then
echo "The program ran at: $now" > $logfile
echo "The second attempt succeeded"
else
echo "The second attempt failed"
fi
$./test16
The first attempt failed
The second attempt succeeded
$ cat $HOME/t16test
The program ran at: 20070930-1602
$
```

This is a pretty busy shell script! First, it defines a log file in your HOME directory, stores the filename of it in the variable logfile, creates the file, and then removes the write permission for the user, using the chmod command. Next, it creates the variable now and stores a timestamp, using the date command. After all of that, it checks if you have write permission to the new log file (which you just took away). Since you don't have write permission, you should see the failed message appear.

After that, the script uses the chmod command again to grant the write permission back for the user, and tries to write to the file again. This time the write is successful.

### Checking if you can run a file

The -x comparison is a handy way to determine if you have execute permission for a specific file. While this may not be needed for most commands, if you run a lot of scripts from your shell scripts, it could come in handy:

```
$ cat test17
#!/bin/bash
testing file execution
if [-x test16]
then
echo "You can run the script:"
./test16
else
echo "Sorry, you are unable to execute the script"
fi
$./test17
You can run the script:
The first attempt failed
The second attempt succeeded
$ chmod u-x test16
$./test17
Sorry, you are unable to execute the script
$
```

## Notes

This example shell script uses the `-x` comparison to test if you have permission to execute the `test16` script. If so, it runs the script (notice that even in a shell script, you must have the proper path to execute a script that's not located in your `PATH`). After successfully running the `test16` script the first time, change the permissions on it, and try again. This time the `-x` comparison fails, as you don't have execute permission for the `test16` script.

### Checking ownership

The `-O` comparison lets you to easily test if you're the owner of a file:

```
$ cat test18
#!/bin/bash
check file ownership
if [-O /etc/passwd]
then
echo "You're the owner of the /etc/passwd file"
else
echo "Sorry, you're not the owner of the /etc/passwd file"
fi
$./test18
Sorry, you're not the owner of the /etc/passwd file
$ su
Password:
./test18
You're the owner of the /etc/passwd file
#
```

the script uses the `-O` comparison to test if the user running the script is the owner of the `/etc/passwd` file. The first time the script is run under a normal user account, so the test fails. The second time, I used the `su` command to become the root user, and the test succeeded. The `-G` comparison checks the default group of a file, and it succeeds if it matches the group of the default group for the user. This can be somewhat confusing, because the `-G` comparison only checks the default groups, and not all the groups the user belongs to. See the example of this below:

```
$ cat test19
#!/bin/bash
check file group test
if [-G $HOME/testing]
then
echo "You're in the same group as the file"
else
echo "The file is not owned by your group"
fi
$ ls -l $HOME/testing
-rw-rw-r-- 1 rich rich 58 2007-09-30 15:51 /home/rich/testing
$./test19
You're in the same group as the file
```

## Notes

```
$ chgrp sharing $HOME/testing
$./test19
The file is not owned by your group
$
```

The first time the script is run, the \$HOME/testing file is in the rich group, and the -G comparison succeeds. Next, I changed the group to the sharing group, which I'm also a member of. However, the -G comparison failed, since it only compares the default groups, not any additional groups I belong to.

### Checking file date

The last set of comparisons deal with comparing the creation times of two files. This comes in handy when writing scripts to install software. Sometimes you don't want to install a file that is older than a file already installed on the system.

The -nt comparison determines if a file is newer than another file. If a file is newer, it'll have a more recent file creation time. The -ot comparison determines if a file is older than another file. If the file is older, it'll have an older file creation time:

```
$ cat test20
#!/bin/bash
testing file dates
if [./test19 -nt ./test18]
then
echo "The test19 file is newer than test18"
else
echo "The test18 file is newer than test19"
fi
if [./test17 -ot ./test19]
then
echo "The test17 file is older than the test19 file"
fi
$./test20
The test19 file is newer than test18
The test17 file is older than the test19 file
$ ls -l test17 test18 test19
-rwxrw-r-- 1 rich rich 167 2007-09-30 16:31 test17
-rwxrw-r-- 1 rich rich 185 2007-09-30 17:46 test18
-rwxrw-r-- 1 rich rich 167 2007-09-30 17:50 test19
$
```

The filepaths used in the comparisons are relative to the directory from where you run the script. This can cause problems if the files you're checking can be moved around. Another problem is that neither of these comparisons check if the file exists first. Try this test:

## Notes

```
$ cat test21
#!/bin/bash
testing file dates
if [./badfile1 -nt ./badfile2]
then
echo "The badfile1 file is newer than badfile2"
else
echo "The badfile2 file is newer than badfile1"
fi
$./test21
The badfile2 file is newer than badfile1
$
```

This little example demonstrates that if the files don't exist, the -nt comparison just returns a failed condition. It's imperative that you ensure the files exist before trying to use them in the -nt or -ot comparison.

### 4.2.8 for Loop

#### The for loop:

The distinction between if and case, as stated in the Control Flow section, is that the first is based on command exit statuses, whereas the second is directly tied to data available in the shell. While and until use command exit statuses and for uses data directly available in the shell, this type of pairing also occurs for repetition statements.

The for-statement loops over a set of values that is fixed and finite. Its general form is

```
for name in w1 w2 ...
do command-list
done
```

This statement executes the command list for each value named after the 'in'. The variable name in the command list provides access to the "current" value wi. A semicolon or a newline must be used to separate the value list from the 'do'. A semicolon or a newline must be used to separate the command list from the 'done'. As an example:

```
for myval in Abel Bertha Charlie Delta Easy Fox Gumbo Henry India
do
echo $myval Company
done
```

**Output:**

```
Abel Company
Bertha Company
Charlie Company
Delta Company
Easy Company
Fox Company
Gumbo Company
Henry Company
India Company
```

**Notes**

### 4.2.9 While and Until Loop

**The while loop:**

In Bourne shell, the while-statement is the simplest and most fundamental kind of repeating statement. It's also the most common. It takes the following general form:

```
while command-list1
do command-list2
done
```

The while-statement is interpreted as follows:

1. Execute the commands in command list 1.
2. If the exit status of the last command is non-zero, the statement terminates.
3. Otherwise execute the commands in command list 2 and go back to step 1.
4. If the statement does not contain a syntax error and it ever terminates, it terminates with exit status zero.

The while-statement, like the if-statement, can be controlled by a whole command list, with only the final command in the list controlling the statement. But, as with the if-statement, you'll typically want to limit yourself to just one command, which you'll usually execute using the 'test' programme.

**Example:**

```
counter=0
while [$counter -lt 10]
do
 echo $counter
 counter=`expr $counter + 1`
done
```

## Notes

### Output:

```
0
1
2
3
4
5
6
7
8
9
```

### The until loop:

The until-statement is also a repetition statement, but it is sort of the semantic opposite of the while-statement. The general form of the until-statement is

```
until command-list1
do command-list2
done
```

The interpretation of this statement is almost the same as that of the while-statement. Only the commands in command list 2 are executed if the last command in command list 1 returns a non-zero status. To put it another way, command list 2 is run as long as the loop's condition is not met.

While **while-statements** are typically used to create an effect ("repeat until done"), until-statements are more commonly used to check for the presence of a condition or to wait until one is met. Assume that a process is running that will write 10000 lines to a particular file. The till-statement that follows waits until the file has increased to 10,000 lines:

### Example:

```
until [$lines -eq 10000]
do
 lines=`wc -l dates | awk '{print $1}'`
 sleep 5
done
```

## 4.2.10 case and switch

### Case Command:

case command in Linux is the best alternative when we had to use multiple if/elif on a single variable. It is used to execute the commands based on the pattern matching.



Syntax:

```
case {value} in
 pattern/value) Statements ;;
 pattern/value) Statements ;;
 ...
 *) Default case statements ;;
esac
```

Here,

- ◇ ;; are used as break statement. If they are not used, all following cases will be automatically executed.
- ◇ \*) is used to define the default case when no pattern/value has match with iteration variable.

**Example:**

```
Exam rank=captain

case $rank in
 colonel) echo Hannibal Smith;;
 captain) echo Howling Mad Murdock;;
 lieutenant) echo Templeton Peck;;
 sergeant) echo B.A. Baracus;;
 *) echo OOPS;;
esacple:
```

**Output:**

Howling Mad Murdock

Switch Command

Depending on whether a provided value equals one of multiple wildcarded values, the switch statement executes one of several blocks of commands. To select which block should be executed, the case statement is combined with the switch statement.

One or more parameters are assigned to each case command. The first case command that has a parameter that matches the switch command's string will be evaluated. Wildcards may be used in case parameters. To avoid ordinary wildcard expansion utilising filenames, these must be escaped or quoted.

It's worth noting that fish does not fall between the cracks in case statements. Though the syntax resembles C switch statements, it behaves more like regular shell case statements.

Also, even if the case statement's body isn't taken, command substitutions will be examined. This may appear confusing at first, but it is unavoidable since it is impossible

## Notes

to tell if a case command will evaluate to true until all types of parameter expansion for the case command have been completed.

### Check your Understanding

#### Fill ups

1. Multiuser systems might have numerous different \_\_\_\_\_ in operation at any same moment because different users may prefer various shells.
2. The \_\_\_\_\_ command in the shell serves two purposes: it may be used to set numerous shell settings as well as reassign positional arguments like \$1, \$2, and so on.
3. \_\_\_\_\_ variables contain information about your login session, stored for the system shell to use when executing commands.
4. An \_\_\_\_\_ file reference can save you the effort of typing in a long filename or a long series of similar file-names.
5. \_\_\_\_\_ shell is known as “the shell” since it was the first shell to debut on Unix computers.

#### True or False

1. In a shell the user can run programs and also redirect the input to come from a file and output to come from a file.
2. A shell script is just a set of commands that are listed in the sequence in which they should be executed.
3. Relational operators are those operators which define the relation between two operands.
4. Input and output are device independent; that is, they can be redirected to or from any appropriate device.
5. The most common method for using the test command is to perform a comparison of two numeric values.

#### Summary

- The shell is a program that interacts with the user through a terminal or takes the input from a file and executes a sequence of commands that are passed to the Operating System.
- A shell script is just a set of commands that are listed in the sequence in which they should be executed. There are conditional checks, such as value A is greater than value B, loops for processing large amounts of data, files for reading and storing data, variables for reading and storing data, and the script may include functions.
- One of the most important features of the shell is that users can use it as a programming language. Because the shell is an interpreter, it does not compile programs written for it but rather interprets programs each time they are loaded from the disk. Loading and interpreting programs can be time-consuming.
- There are several characteristics of your environment that you may want to customize but that cannot be expressed as an on/off choice. Characteristics of this

type are specified in shell variables. Shell variables can specify everything from your prompt string to how often the shell checks for new mail.

- Environment variables contain information about your login session, stored for the system shell to use when executing commands. They exist whether you're using Linux, Mac, or Windows. Many of these variables are set by default during installation or user creation.
- Arithmetic expansion and arithmetic evaluation in bash use the same syntax, precedence, and associativity of expressions as the C language. These operators are used to perform normal arithmetics/mathematical operations.
- Relational operators are those operators which define the relation between two operands. They give either true or false depending upon the relation.
- The echo command is a built-in Linux feature that prints out arguments as the standard output. echo is generally used to display text strings or command results as messages. The echo command in Linux is used to display a string provided by the user.

### Activity

1. Create a presentation on the different Shells of Linux and their functions.

### Questions and Exercises

1. What are the four most popular shells of Linux?
2. List the subcategories of Bourne Shell.
3. What are the advantages and disadvantages of Shell Script?
4. What is the syntax used to define variables?
5. What are the seven arithmetic operators?

### Glossary

- **Bourne shell** – When using a Bourne-type shell, the \$ character is the default prompt.
- **C shell** – When using a C-type shell, the % character is the default prompt.
- A **globally scoped** ENV that is defined in a terminal can be accessed from anywhere in that particular environment which exists in the terminal.
- A **locally scoped** ENV that is defined in a terminal cannot be accessed by any program or process running in the terminal.
- **Logical AND (&&):** This is a binary operator, which returns true if both the operands are true otherwise returns false.
- **Logical OR (||):** This is a binary operator, which returns true if either of the operand is true or both the operands are true and return false if none of them is false.
- **Not Equal to (!):** This is a unary operator which returns true if the operand is false and returns false if the operand is true.

## Notes

### Further Readings

1. Blum, Richard; Bresnahan, Christine. Mastering Linux System Administration. Sybex; 1st edition (July 27, 2021)
2. Gift, Noah; Jones, Jeremy. Python for Unix and Linux System Administration. O'Reilly Media; 1st edition (September 1, 2008)
3. Hein, Trent; Mackin, Dan; Nemeth, Evi; Snyder, Garth; Whaley, Ben. UNIX and Linux System Administration Handbook. ↑ Addison-Wesley Professional; 5th edition (August 8, 2017)
4. Kerrisk, Michael. The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press; 1st edition (October 28, 2010)
5. Negus, Christopher. Linux Bible. Wiley; 10th edition (June 10, 2020)

### Check your Understanding: Answers

#### Fill Ups

1. Shells
2. Set
3. Environment
4. Ambiguous
5. Bourne

#### True or False

1. True
2. True
3. True
4. True
5. True

## Module-V: sed AND awk

Notes

### Structure:

- 5.1 sed
  - 5.1.1 Introduction and features sed
  - 5.1.2 Execution process of sed
  - 5.1.3 Syntax of sed and Addressing:
  - 5.1.4 “sed commands: Inserting / Deleting/Printing Text/Lines”
  - 5.1.5 sed commands:Substitution
  - 5.1.6 Multiple edits, reading from file and writing to file
  - 5.1.7 quit command, Modify a File with sed
- 5.2 awk
  - 5.2.1 Syntax for awk
  - 5.2.2 working of awk,
  - 5.2.3 Input from files, Input from commands
  - 5.2.4 Formatting output-print and printf function
  - 5.2.5 “concept of record and field in awk .”
- 5.3 System Administration
  - 5.3.1 Adding and Removing Users, Changing password
  - 5.3.2 Disk Management, Starting and shutting down the system
  - 5.3.3 File System Mounting and Unmounting
  - 5.3.4 Monitoring System Usage,
  - 5.3.5 Ensuring System Security

## Notes

### Unit-5.1: sed

#### Objectives:

At the end of this unit, you will be able to understand:

- Introduction to sed
- Executions Process of sed
- Syntax of sed and Addressing
- Sed Commands: Substitution
- Multiple edits, reading from file and writing to file
- Quit command, Modify a File with sed

#### 5.1.1 Introduction and Features of sed

There are a few programs that are the real workhorse in the UNIX toolbox. These programs are simple to use for simple applications, yet have a rich set of commands for performing complex actions. Don't let the complex potential of a program keep you from making use of the simpler aspects. I'll start with the simple concepts and introduce the advanced topics later on.

When I first wrote this (in 1994), most versions of sed did not allow you to place comments inside the script. Lines starting with the '#' characters are comments. Newer versions of sed may support comments at the end of the line as well.

Two utilities found on every Unix installation—awk and sed—merit special attention in this context. awk is a lightweight scripting language that lets you write succinct programs to operate line-by-line on the contents of text files. It is particularly useful for handling text files that have columns of data separated by white spaces or tabs. sed on the other hand, is particularly useful for automating “find-and-replace” operations on text files. Each of them is optimized to handle large files without storing a lot of information in memory, so they can be useful for quick operations on large bioinformatic data sets. Neither is a fully-featured programming language that you would want to write large, complex programs in (that said, I did once implement a complete program for full-sibling inference from multiallelic markers in awk); however they do share many of the useful text-manipulation capabilities of such languages, such as Perl and Python. Additionally, awk and sed are deployed in a consistent fashion across most Unix operating systems, and they don't require much time to learn to use effectively for common text-processing tasks. As a consequence awk and sed are a useful addition to the bioinformatician's toolbox.

Both awk and sed rely heavily on regular expressions to describe patterns in text upon which some operation should be performed. You can think of regular expressions as providing a succinct language for performing very advanced “find” and “find-and-replace” operations in a text file.

**Sed:** sed is a stream editor. On an input stream, a stream editor is used to execute basic text alterations (a file or input from a pipeline). While sed is similar to an editor that allows scripted edits (such as ed), it is more efficient because it simply makes one pass over the input(s)..

## 5.1.2 Execution Process of sed

Generally, sed is used like this:

```
sed SCRIPT INPUTFILE...
```

For example, to replace all occurrences of 'hello' to 'world' in the file input.txt:

```
sed 's/hello/world/' input.txt > output.txt
```

sed filters the contents of the standard input if you don't specify INPUTFILE or if INPUTFILE is -. The following commands are interchangeable:

```
sed 's/hello/world/' input.txt > output.txt
sed 's/hello/world/' < input.txt > output.txt
cat input.txt | sed 's/hello/world/' - > output.txt
```

The output of sed is written to standard output. Instead of outputting to standard output, use -i to modify files in-place. For writing output to other files, see the W and s/w commands. The command below edits file.txt without producing any output:

```
sed -i 's/hello/world/' file.txt
```

sed prints all processed input by default (except input that has been modified/deleted by commands like d). To suppress output, use -n, and to print specific lines, use the p command. Only line 45 of the input file is printed with the following command:

```
sed -n '45p' file.txt
```

Multiple input files are treated as a single continuous stream by sed. The initial line of the first file (one.txt) and the last line of the last file (last.txt) are printed in the following example (three.txt). To reverse this behaviour, use -s.

```
sed -n '1p ; $p' one.txt two.txt three.txt
```

sed utilises the first non-option parameter as the script and the subsequent non-option parameters as input files when there are no -e or -f options. All non-option parameters are treated as input files when the -e or -f options are used to specify a script. The options -e and -f can be mixed and matched, and they can be used many times (in which case the final effective script will be concatenation of all the individual scripts).

The following examples are equivalent:

```
sed 's/hello/world/' input.txt > output.txt
sed -e 's/hello/world/' input.txt > output.txt
sed --expression='s/hello/world/' input.txt > output.txt

echo 's/hello/world/' > myscript.sed
sed -f myscript.sed input.txt > output.txt
sed --file=myscript.sed input.txt > output.txt
```

Notes



## Notes

### Command-Line Options

The full format for invoking sed is:

```
sed OPTIONS... [SCRIPT] [INPUTFILE...]
```

sed may be invoked with the following command-line options:

#### **--version**

Exit after printing the current version of sed and a copyright notice.

#### **--help**

Then exit with a usage message that quickly summarises these command-line options as well as the bug-reporting address.

#### **-n**

#### **--quiet**

#### **--silent**

sed prints the pattern space at the end of each cycle through the script by default. These parameters stop automatic printing, and sed only prints when the p command is used to tell it to.

#### **--debug**

Print the input sed program in canonical form, and annotate program execution.

```
$ echo 1 | sed '%1%s21232'
3

$ echo 1 | sed --debug '%1%s21232'
SED PROGRAM:
/1/ s/1/3/
INPUT: 'STDIN' line 1
PATTERN: 1
COMMAND: /1/ s/1/3/
PATTERN: 3
END-OF-CYCLE:
3
```

#### **-e script**

#### **--expression=script**

Add the scripted commands to the list of commands to run while the input is being processed.

#### **-f script-file**

#### **--file=script-file**

Add the commands in the script-file file to the list of commands to run while the input is being processed.

**-i[SUFFIX]**

**--in-place[=SUFFIX]**

This option specifies that files will be modified in their current location. GNU sed accomplishes this by creating a temporary file and writing to it instead of the standard output.

-s is implied by this option.

When the temporary file reaches the end of the file, it is renamed to the original name of the output file. If provided, the extension is used to change the name of the original file before renaming the temporary file, creating a backup copy2).

If the extension does not contain a \* character, it is appended as a suffix to the end of the current filename; if the extension does contain one or more \* characters, each asterisk is substituted with the current filename. This allows you to use a prefix instead of (or in addition to) a suffix on the backup file, or even place backup copies of the original files in a different directory (provided the directory already exists).

If no extension is supplied, the original file is overwritten without making a backup.

Because -i takes an optional argument, it should not be followed by other short options:

**sed -Ei '...' FILE**

Same as -E -i with no backup suffix - FILE will be edited in-place without creating a backup.

**sed -iE '...' FILE**

This is equivalent to --in-place=E, creating FILEE as backup of FILE

When using -n with -i, be aware that the former disables automatic line printing and the latter updates the file in situ without a backup. The output file will be empty if used carelessly (and without an explicit p command):

```
WRONG USAGE: 'FILE' will be truncated.
sed -ni 's/foo/bar/' FILE
```

**-l N**

**--line-length=N**

Specify the default line-wrap length for the l command. A length of 0 (zero) means to never wrap long lines. If not specified, it is taken to be 70.

**--posix**

Several extensions to POSIX sed are included in GNU sed. This option disables all of the extensions that this manual documents, including new commands, to make developing portable scripts easier. Most of the extensions accept sed programmes that deviate from the POSIX syntax, but some of them (such as the N command's behaviour detailed in Reporting Bugs) truly break the standard. Set the POSIXLY CORRECT variable to a non-empty value if you simply want to disable the latter type of extension.

## Notes

### **-b**

#### **--binary**

This option is accessible on all platforms, but it works only if the operating system distinguishes between text and binary files. Text files are constituted of lines separated by a carriage return and a line feed character when such a distinction is made—as is the case for MS-DOS, Windows, and Cygwin—and sed does not see the closing CR. When this option is used, sed will open input files in binary mode, obviating the need for additional processing and treating lines as terminating at a line feed.

#### **--follow-symlinks**

This option is only accessible on platforms that enable symbolic links, and it only has an effect if the `-i` option is used. In this situation, sed will follow the link and edit the link's ultimate destination if the file supplied on the command line is a symbolic link. The default behaviour is to break the symbolic link, preventing any changes to the link destination.

### **-E**

### **-r**

#### **--regexp-extended**

Instead of using basic regular expressions, use extended regular expressions. Extended regexps are those that egrep accepts; they are usually less ambiguous because they have fewer backslashes. This was originally a GNU addition, but it has subsequently been incorporated to the POSIX standard (<http://austingroupbugs.net/view.php?id=528>), so use it for portability. For years, GNU sed has recognised `-E` as an undocumented option, and \*BSD sed's have as well, however scripts written with `-E` may not port to other older systems.

### **-s**

#### **--separate**

sed treats the files supplied on the command line as a single continuous long stream by default. The user can treat them as independent files using the GNU sed extension: Line numbers are relative to the start of each file, `$` refers to the last line of each file, and files invoked from `R` commands are rewound at the beginning of each file.

#### **--sandbox**

`E/w/r` instructions are rejected in sandbox mode, and programmes containing them will be aborted without being run. Sandbox mode assures that sed exclusively works with the input files specified on the command line, and that it does not launch any external programmes.

### **-u**

#### **--unbuffered**

As little as possible, buffer both input and output. (This is especially useful if the input is something like `'tail -f'` and you want to see the modified output right away.)

**-z**

**--null-data**

**--zero-terminated**

Instead of a newline, treat the input as a series of lines, each ending with a zero byte (the ASCII 'NUL' character). To process arbitrary file names, use this option with programmes like 'sort -z' and 'find -print0.'

If no -e, -f, —expression, or —file options are specified on the command line, the script to be executed is assumed to be the first non-option argument.

After processing the above, any command-line options that remain are understood as the names of input files to be processed. The standard input stream is identified by the file name '-'. If no file names are supplied, the standard input will be used.

### Exit status

A value of zero signifies success, while a value of nonzero indicates failure. The exit status error values returned by GNU sed are as follows:

0

Successful completion.

1

Invalid command, syntax, regular expression, or a GNU sed extension command combined with —posix.

2

It was not possible to open one or more of the input files supplied on the command line (e.g. if a file is not found, or read permission is denied). Other files were processed in the meantime.

4

GNU sed aborted promptly if it encountered an I/O problem or a major processor error during runtime.

The commands q and Q can also be used to quit sed with a custom exit code value (this is a GNU sed addition):

```
$ echo | sed 'Q42'; echo $?
42
```

### 5.1.3 Syntax of sed and Addressing

#### Syntax for sed , regular expressions:

sed is a "stream editor". Though it is capable of all sorts of things, to be honest, I use it almost exclusively to do simple find-and-replace operations. The syntax for that is:

```
sed 's/regex/replacement/g;' file
```

## Notes

where regex is a regular expression and replacement is the string you wish to replace any segments of file that match regex. The s means “substitute” and is like a command to sed, and the g means “globally”, without which only the first match of regex on each line would be replaced.

Multiple instances of the “s” command can be given, like:

```
sed '
s/regex1/replacement1/g;
s/regex2/replacement2/g;
s/regex3/replacement3/g;
' file
```

The separate commands are done in order, line by line.

Now it's your turn!

Take the output of:

```
WINTERS=$(awk -F"," '$NF == "Winter" {print $1}' data/wgs-chinook-
samples.csv)
for i in $WINTERS; do
 echo "FASTQS are: $i.R1.fq.gz $i.R2.fq.gz"
done
```

and pipe it into sed to changes the R1 and R2 into r1 and r2, and to remove the .gz from the end of each file name. Remember to backslash-escape that period!

### Writing Regular Expressions:

For all practical purposes, you can rely on a program to produce the correct result. However, that doesn't mean the program always works correctly as far as you are concerned. Most of the time, you can bet that if a program does not produce the output that you expected, the real problem (putting aside input or syntax errors) is how you described what you wanted. In other words, the place to look to correct the problem is the expression where you described the result you wanted. Either the expression is incomplete or it is improperly formulated. For instance, if a program evaluates this expression:

```
PAY = WEEKLY_SALARY * 52
```

and knows the values of these variables, it will calculate the correct result. But someone might object that the formula did not account for salespeople, who also receive a commission. To describe this instance, the expression would need to be reformulated as:

```
PAY = WEEKLY_SALARY * 52 + COMMISSION
```

You could say that whoever wrote the first expression did not fully understand the scope of the problem and thus did not describe it well. It is important to know just how detailed a description must be. If you ask someone to bring you a book, and there are

multiple books in view, you need to describe more specifically the book that you want (or be content with an indeterminate selection process).

The same is true with regular expressions. A program such as `grep` is simple and easy to use. Understanding the elements of regular expressions is not so hard, either. Regular expressions enable you to write basic or sophisticated patterns descriptions. However, the complexity of the application: the range of occurrences or circumstances in which a pattern emerges, makes developing regular expressions challenging (and intriguing). This complexity is built into language, just as you can't always decipher a statement by looking up each word in a dictionary.

The process of writing a regular expression involves three steps:

- ◇ Understanding what you're looking for and how it might appear in the text.
- ◇ Create a pattern to define what you're looking for to match.
- ◇ Checking the pattern to check whether it matches anything.

This approach is nearly identical to the one used by a programmer to create a programme.

- ◇ Step 1 may be thought as as the specification, which should reflect an understanding of both the problem and how to address it.
- ◇ Step 2 is similar to the program's real coding, and
- ◇ The third step entails running the programme and comparing it against the specification.
- ◇ Steps 2 and 3 constitute a loop that is continued until the software functions properly.

It is important to test your description of what you want to match to confirm that it performs as planned. It frequently reveals a few surprises. Examining the results of a test and comparing the output to the input can help you grasp regular expressions a lot better. Consider examining the outcomes of a pattern matching-operation in the following manner:

### Hits

The lines that I wanted to match.

### Misses

The lines that I didn't want to match.

### Omissions

The lines that I didn't match but wanted to match.

### False alarms

The lines that I matched but didn't want to match.

Trying to perfect your pattern description is a two-pronged approach: you reduce the number of possible matches to eliminate false alarms, and you expand the number of available matches to capture omissions.

## Notes

When you have to define patterns using fixed strings, the difficulty becomes even more apparent. The number of possible matches increases with each character removed from the fixed-string pattern. For example, while looking for the string “what,” you decide that you’d also like to match “What.” The only fixed-string pattern that will match “What” and “what” is “hat,” the longest string common to both. However, it is self-evident that a search for “hat” will provide undesirable results. The number of possible matches decreases with each character added to a fixed-string pattern. The string “them” will typically provide fewer matches than “the.”

When metacharacters are used in patterns, the range of matches can be extended or narrowed more easily. Metacharacters, when combined with literals or other metacharacters, can be used to broaden the range of possible matches while still excluding the ones you don’t want.

### Standard Regular Expressions:

#### Start of line (^):

In regular expressions terminology, the caret(^) symbol matches the start of a line. The example below prints all the lines that start with the pattern “The”.

```
[jerry]$ sed -n '/^The/ p' books.txt
```

On executing the above code, you get the following result:

```
The Two Towers, J. R. R. Tolkien
The Alchemist, Paulo Coelho
The Fellowship of the Ring, J. R. R. Tolkien
The Pilgrimage, Paulo Coelho
```

#### End of Line (\$):

End of line is represented by the dollar(\$) symbol. The following example prints the lines that end with “Coelho”.

```
[jerry]$ sed -n '/Coelho$/ p' books.txt
```

On executing the above code, you get the following result:

```
The Alchemist, Paulo Coelho
The Pilgrimage, Paulo Coelho
```

#### Single Character (.):

Except for the end of line character, the Dot(.) matches any single character. The example below prints all three-letter words that conclude in the letter “t.”

```
[jerry]$ echo -e "cat\nbat\nrat\nmat\nbatting\nrats\nmats" | sed -n '/^..t$/p'
```

On executing the above code, you get the following result:



```
cat
bat
rat
mat
```

**Match Character Set ([ ]):**

In regular expression terminology, a character set is represented by square brackets ([ ]). It is used to match only one out of several characters. Below is an example that matches the patterns “Call” and “Tall” but not “Ball”.

```
[jerry]$ echo -e "Call\nTall\nBall" | sed -n '[CT]all/ p'
```

On executing the above code, you get the following result:

```
Call
Tall
Exclusive Set ([^])
```

In exclusive set, the caret negates the set of characters in the square brackets. The following example prints only “Ball”.

```
[jerry]$ echo -e "Call\nTall\nBall" | sed -n '[^CT]all/ p'
```

On writing the above code, you get the subsequent result:

```
Ball
```

**Character Range ([-]):**

When a character range is provided, the regular expression matches any character within the range specified in square brackets. The following example matches “Call” and “Tall” but not “Ball”.

```
[jerry]$ echo -e "Call\nTall\nBall" | sed -n '[C-Z]all/ p'
```

On executing the above code, you get the following result:

```
Call
Tall
```

Now let us modify the range to “A-P” and observe the result.

```
[jerry]$ echo -e "Call\nTall\nBall" | sed -n '[A-P]all/ p'
```

On executing the above code, you get the following result:

```
Call
Ball
```

**Zero or One Occurrence (?):**

In SED, the question mark (?) matches zero or one occurrence of the preceding

## Notes

character. The following example matches “Behaviour” as well as “Behavior”. Here, we made “u” as an optional character by using “\?”.

```
[jerry]$ echo -e "Behaviour\nBehavior" | sed -n '/Behaviour\?r/ p'
```

On executing the above code, you get the following result:

```
Behaviour
Behavior
```

### One or More Occurrence (\+):

In SED, the plus symbol(\+) matches one or more occurrences of the preceding character. The following example matches one or more occurrences of “2”.

```
[jerry]$ echo -e "111\n22\n123\n234\n456\n222" | sed -n '/2\+/ p'
```

On executing the above code, you get the following result:

```
22
123
234
222
```

### Zero or More Occurrence (\*):

Asterisks (\*) matches the zero or more occurrence of the preceding character. The following example matches “ca”, “cat”, “catt”, and so on.

```
[jerry]$ echo -e "ca\ncat" | sed -n '/cat*/ p'
```

On executing the above code, you get the following result:

```
ca
cat
```

### Exactly N Occurrences {n}:

{n} matches exactly “n” occurrences of the preceding character. The following example prints only three digit numbers. But before that, you need to create the following file which contains only numbers.

```
[jerry]$ cat numbers.txt
```

On executing the above code, you get the following result:

```
1
10
100
1000
10000
```

```
100000
1000000
10000000
100000000
1000000000
```

Let us write the SED expression.

```
[jerry]$ sed -n '/^[0-9]\{3\}$/ p' numbers.txt
```

On executing the above code, you get the following result:

```
100
```

Note that the pair of curly braces is escaped by the “\” character.

### At least n Occurrences {n,}:

{n,} matches at least “n” occurrences of the preceding character. The following example prints all the numbers greater than or equal to five digits.

```
[jerry]$ sed -n '/^[0-9]\{5,\}$/ p' numbers.txt
```

On executing the above code, you get the following result:

```
10000
100000
1000000
10000000
100000000
1000000000
```

### M to N Occurrence {m, n}:

{m, n} matches at least “m” and at most “n” occurrences of the preceding character. The example below prints all the numbers having at least five digits but not more than eight digits.

```
[jerry]$ sed -n '/^[0-9]\{5,8\}$/ p' numbers.txt
```

When executing the code shown above, you get the resulting:

```
10000
100000
1000000
10000000
```

### Pipe (|):

In SED, the pipe character behaves like logical OR operation. It matches items from either side of the pipe. The following example either matches “str1” or “str3”.

## Notes

```
[jerry]$ echo -e "str1\nstr2\nstr3\nstr4" | sed -n '/str\([1|3|)\| p'
```

On executing the above code, you get the following result:

```
str1
str3
```

Note that the pair of the parenthesis and pipe (|) is escaped by the “\” character.

### Escaping Characters:

There are certain special characters. For example, newline is represented by “\n”, carriage return is represented by “\r”, and so on. To use these characters into regular ASCII context, we have to escape them using the backward slash(\) character. This illustrates escaping of special characters.

#### Escaping “\”:

The following example matches the pattern “\”.

```
[jerry]$ echo 'str1\str2' | sed -n '/\\ p'
```

On executing the above code, you get the following result:

```
str1\str2
```

#### Escaping “\n”:

The following example matches the new line character.

```
[jerry]$ echo 'str1\nstr2' | sed -n '/\\n/ p'
```

On executing the above code, you get the following result:

```
str1\nstr2
```

#### Escaping “\r”:

The following example matches the carriage return.

```
[jerry]$ echo 'str1\rstr2' | sed -n '/\\r/ p'
```

On executing the above code, you get the following result:

```
str1\rstr2
```

#### Escaping “\dnnn”:

This matches a character whose decimal ASCII value is “nnn”. The following example matches only the character “a”.

```
[jerry]$ echo -e "a\nb\nc" | sed -n '/\d97/ p'
```

On executing the above code, you get the following result:

```
a
```

### Escaping “\onnn”:

This matches a character whose octal ASCII value is “nnn”. The following example matches only the character “b”.

```
[jerry]$ echo -e "a\nb\nc" | sed -n '/\o142/ p'
```

On executing the above code, you get the following result:

```
b
```

This matches a character whose hexadecimal ASCII value is “nnn”. The following example matches only the character “c”.

```
[jerry]$ echo -e "a\nb\nc" | sed -n '/\x63/ p'
```

On executing the above code, you get the following result:

```
c
```

### 5.1.4 sed Commands: Inserting / Deleting/Printing Text/Lines

#### Insert Command:

The insert command works similarly as append does. The only difference is that it inserts a line before a specific position. Represented below is the syntax of the insert command:

```
[address]i\
Insert text
```

Let us understand the insert command with some examples. The following command inserts a new entry before the fourth line.

```
[jerry]$ sed '4 i 7) Adultry, Paulo Coelho, 324' books.txt
```

On writing the above code, you get the subsequent result:

```
1) A Storm of Swords, George R. R. Martin, 1216
2) The Two Towers, J. R. R. Tolkien, 352
3) The Alchemist, Paulo Coelho, 197
7) Adultry, Paulo Coelho, 324
4) The Fellowship of the Ring, J. R. R. Tolkien, 432
5) The Pilgrimage, Paulo Coelho, 288
6) A Game of Thrones, George R. R. Martin, 864
```

In the above example, 4 is the location number, i implies the insert command, and the remaining part is the text to be inserted.

To insert text at the start of a file, provide the line address as 1. The following command illustrates this:

```
[jerry]$ sed '1 i 7) Adultry, Paulo Coelho, 324' books.txt
```

## Notes

On writing the above code, you get the subsequent result:

```
7) Adultry, Paulo Coelho, 324
1) A Storm of Swords, George R. R. Martin, 1216
2) The Two Towers, J. R. R. Tolkien, 352
3) The Alchemist, Paulo Coelho, 197
4) The Fellowship of the Ring, J. R. R. Tolkien, 432
5) The Pilgrimage, Paulo Coelho, 288
6) A Game of Thrones, George R. R. Martin, 864
```

Additionally, we can insert multiple lines. The following command inserts two lines before the last line.

```
[jerry]$ sed '$ i 7) Adultry, Paulo Coelho, 324'
```

On writing the above code, you get the subsequent result:

```
8) Eleven Minutes, Paulo Coelho, 304' books.txt
1) A Storm of Swords, George R. R. Martin, 1216
2) The Two Towers, J. R. R. Tolkien, 352
3) The Alchemist, Paulo Coelho, 197
4) The Fellowship of the Ring, J. R. R. Tolkien, 432
5) The Pilgrimage, Paulo Coelho, 288
7) Adultry, Paulo Coelho, 324
8) Eleven Minutes, Paulo Coelho, 304
6) A Game of Thrones, George R. R. Martin, 864
```

Note that the entries to be inserted are entered on separate lines and delimited by the backslash(\) character.

### Delete Command:

SED provides various commands to manipulate text. Let us first explore about the delete command. Here is how you execute a delete command:

```
[address1[,address2]]d
```

address1 and address2 are the starting and the ending addresses respectively, which can be either line numbers or pattern strings. Both of these addresses are optional parameters.

As the name suggests, the delete command is used to perform delete operation and since the SED operates on line, we can say that this command is used to delete lines. Note that the delete command removes lines only from the pattern buffer; the line is not sent to the output stream and the original file remains unchanged. The following example illustrates the point.

```
[jerry]$ sed 'd' books.txt
```

But where is the output? If no line address is provided, then the SED operates on every line by default. Hence, it deletes all the lines from the pattern buffer. That is why the command does not print anything on the standard output.

Let us instruct the SED to operate only on certain lines. The following example removes the 4th line only.

```
[jerry]$ sed '4d' books.txt
```

On executing the above code, you get the following result:

```
1) A Storm of Swords, George R. R. Martin, 1216
2) The Two Towers, J. R. R. Tolkien, 352
3) The Alchemist, Paulo Coelho, 197
5) The Pilgrimage, Paulo Coelho, 288
6) A Game of Thrones, George R. R. Martin, 864
```

Additionally, SED also accepts address range using comma(.). We can instruct the SED to remove N1 to N2 lines. For instance, the following example deletes all the lines from 2 through 4.

```
[jerry]$ sed '2, 4 d' books.txt
```

On executing the above code, you get the following result:

```
1) A Storm of Swords, George R. R. Martin, 1216
5) The Pilgrimage, Paulo Coelho, 288
6) A Game of Thrones, George R. R. Martin, 864
```

SED's address range is not only limited to numbers. We can also specify patterns as an address. The following example removes all the books of the author Paulo Coelho.

```
[jerry]$ sed '/Paulo Coelho/d' books.txt
```

On executing the above code, you get the following result:

```
1) A Storm of Swords, George R. R. Martin, 1216
2) The Two Towers, J. R. R. Tolkien, 352
4) The Fellowship of the Ring, J. R. R. Tolkien, 432
6) A Game of Thrones, George R. R. Martin, 864
```

We can also specify an address range using textual pattern. The following example removes all lines between the patterns Storm and Fellowship.

```
[jerry]$ sed '/Storm/,/Fellowship/d' books.txt
```

```
5) The Pilgrimage, Paulo Coelho, 288
6) A Game of Thrones, George R. R. Martin, 864
```

In addition to this, we can also use dollar(\$), plus(+), and tilde(~) operators with SED.



## Notes

### Printing Command:

Linux Sed command allows you to print only specific lines based on the line number or pattern matches. “p” is a command for printing the data from the pattern buffer.

To suppress automatic printing of pattern space use -n command with sed. sed -n option will not print anything, unless an explicit request to print is found.

```
[address1[,address2]]P
```

Example:

```
[jerry]$ sed -n 'P' books.txt
```

On executing the above code, you get the following result:

```
A Storm of Swords
George R. R. Martin
The Two Towers
J. R. R. Tolkien
The Alchemist
Paulo Coelho
The Fellowship of the Ring
J. R. R. Tolkien
The Pilgrimage
Paulo Coelho
A Game of Thrones
George R. R. Martin
```

### 5.1.5 sed Commands: Substitution

Substitution We have already demonstrated many uses of the substitute command. Let's look carefully at its syntax

```
:[address]s/pattern/replacement/flags
```

where the flags that modify the substitution are:

- ◇ n A number (1 to 512) indicating that a replacement should only be made for the n<sup>th</sup> occurrence of the pattern.
- ◇ g Make changes globally on all occurrences in the pattern space. Normally only the first occurrence is replaced.
- ◇ p Print the contents of the pattern space.
- ◇ w file Write the contents of the pattern space to file.

The substitute command is applied to the lines matching the address. If no address is specified, it is applied to all lines that match the pattern, a regular expression. If a regular expression is supplied as an address, and no pattern is specified, the substitute

command matches what is matched by the address. This can be useful when the substitute command is one of multiple commands applied at the same address.

Unlike addresses, which require a slash (/) as a delimiter, the regular expression can be delimited by any character except a newline. Thus, if the pattern contained slashes, you could choose another character, such as an exclamation mark, as the delimiter.

```
s!/usr/mail!/usr2/mail!
```

It's worth noting that the delimiter appears three times and is required following the replacement. If the delimiter appears in the regular expression or the replacement text, use a backslash (\) to escape it, regardless of which delimiter you choose.

Once upon a time, computers stored text in fixed-length records. A line ended after so many characters (typically 80), and then the next line started. There was no explicit character in the data to mark the end of one line and the beginning of the next; every line had the same (fixed) number of characters. Modern systems are more flexible; they use a special character (referred to as newline) to mark the end of the line. This allows lines to be of arbitrary length.

Since newline is just another character when stored internally, a regular expression can use "\n" to match an embedded newline. This occurs, as you will see in the next chapter, in the special case when another line is appended to the current line in the pattern space.

The replacement is a string of characters that will replace what is matched by the regular expression. In the replacement section, only the following characters have special meaning:

- ◇ & Replaced by the string matched by the regular expression.
- ◇ \n Matches the n th substring (n is a single digit) previously specified in the pattern using "\(" and "\)".
- ◇ \ Used to escape the ampersand (&), the backslash (\), and the substitution command's delimiter when they are used literally in the replacement section. In addition, it can be used to escape the newline and create a multiline replacement string.

Thus, besides metacharacters in regular expressions, sed also has metacharacters in the replacement. See the next section, "Replacement Metacharacters," for examples of using them.

Flags can be used in combination where it makes sense. For instance, gp makes the substitution globally on the line and prints the line. The global flag is by far the most commonly used. Without it, the replacement is made only for the first occurrence on the line. The print flag and the write flag both provide the same functionality as the print and write commands (which are discussed later in this chapter) with one important difference. These actions are contingent upon a successful substitution occurring. In other words, if the replacement is made, the line is printed or written to file. Because the default action is to pass through all lines, regardless of whether any action is taken, the print and write flags are typically used when the default output is suppressed (the -n option). In addition, if a script contains multiple substitute commands that match the same line, multiple copies of that line will be printed or written to file.

## Notes

The numeric flag can be utilised in the unusual case where a regular expression repeats repeatedly on a line and only one of those occurrences must be replaced by position. A line containing tbl input, for example, could contain many tabs. Let's imagine each line has three tabs, and you want to replace the second tab with ">." It might be done with the following substitution command:

```
s/•/>/2 "•"
```

represents a real tab character that would otherwise go unnoticed on the screen. If the input is a single-line file like the one below:

```
Column1•Column2•Column3•Column4
```

the output produced by running the script on this file will be:

```
Column1•Column2>Column3•Column4
```

The substitute command would only replace the first tab if the numeric flag was not used. (As a result, the numeric flag "1" can be considered the default.)

### 5.1.6 Multiple Edits, Reading from File and Writing to File

#### Multiple Edits:

The first type of sed script demonstrates making a series of edits in a file. We use a script that converts a file created by a word processor into a file typesetting system.

One of the authors once did a writing project for a computer company, here referred to as BigOne Computer. The document had to include a product bulletin for "Horsefeathers Software." The company promised that the product bulletin was online and that they would send it. Unfortunately, when the file arrived, it contained the formatted output for a line printer, the only way they could provide it. A portion of that file (saved for testing in a file named horsefeathers) follows.

```
HORSEFEATHERS SOFTWARE PRODUCT BULLETIN
DESCRIPTION
```

```
+ _____
```

```
BigOne Computer offers three software packages from the suite of
Horsefeathers software products -- Horsefeathers Business BASIC,
BASIC Librarian, and LIDO. These software products can fill your
requirements for powerful, sophisticated, general-purpose business
software providing you with a base for software customization or
development.
```

```
Horsefeathers BASIC is BASIC optimized for use on the BigOne
machine with UNIX or MS-DOS operating systems. BASIC Librarian is
a full screen program editor, which also provides the ab
```

Note that the text has been justified with spaces added between words. There are also spaces added to create a left margin.

When we first start using sed to solve a problem, we find that making a mental list of all the things we want to do helps us focus. When we first start coding, we create a

script with a single command that accomplishes a single goal. We verify that it works, then add another command, and so on until we've completed all of the obvious tasks. ("All of this is self-evident," because the list isn't always comprehensive, and the implement-and-test cycle frequently adds new items to the list.)

Working in this manner may appear to be a time-consuming procedure, yet there are some scripts where it's OK to write the entire script in one sitting and then test it. The one-step-at-a-time method, on the other hand, is highly suggested for beginners because it isolates each command and allows you to quickly identify what is and is not functioning. When you try to run multiple instructions at once, you may find that you wind up recreating the advised process in reverse, that is, eliminating commands one by one until the problem is found.

Here is a list of the obvious edits that need to be made to the Horsefeathers Software bulletin:

1. Replace all blank lines with a paragraph macro (.LP).
2. Remove all leading spaces from each line.
3. Remove the printer underscore line, the one that begins with a "+".
4. Remove multiple blank spaces that were added between words.

The first edit requires that we match blank lines. However, in looking at the input file, it wasn't obvious whether the blank lines had leading spaces or not. As it turns out, they do not, so blank lines can be matched using the pattern "\$". (If there were spaces on the line, the pattern could be written "^ \*\$".) Thus, the first edit is fairly straightforward to accomplish:

```
s/^$/.LP/
```

It replaces each blank line with ".LP". Note that you do not escape the literal period in the replacement section of the substitute command. We can put this command in a file named sedscr and test the command as follows:

```
$ sed -f sedscr horsefeathers
HORSEFEATHERS SOFTWARE PRODUCT BULLETIN
.LP
DESCRIPTION + _____
.LP
BigOne Computer offers three software packages from the suite of
Horsefeathers software products -- Horsefeathers Business BASIC,
BASIC Librarian, and LIDO. These software products can fill your
requirements for powerful, sophisticated, general-purpose business
software providing you with a base for software customization or
development.
.LP
Horsefeathers BASIC is BASIC optimized for use on the BigOne
machine with UNIX or MS-DOS operating systems. BASIC Librarian is
a full screen program editor, which also provides the ability
```

## Notes

It is pretty obvious which lines have changed. (It is frequently helpful to cut out a portion of a file to use for testing. It works best if the portion is small enough to fit on the screen yet is large enough to include different examples of what you want to change. After all edits have been applied successfully to the test file, a second level of testing occurs when you apply them to the complete, original file.)

The next edit that we make is to remove the line that begins with a “+” and contains a line-printer underscore. We can simply delete this line using the delete command, `d`. In writing a pattern to match this line, we have a number of choices. Each of the following would match that line:

```
/^+/
/^+ /
/^+ */
/^+ * _ */
```

As you can see, each successive regular expression matches a greater number of characters. Only through testing can you determine how complex the expression needs to be to match a specific line and not others. The longer the pattern that you define in a regular expression, the more comfort you have in knowing that it won't produce unwanted matches. For this script, we'll choose the third expression:

```
/^+ */d
```

This command will delete any line that begins with a plus sign and is followed by at least one space. The pattern specifies two spaces, but the second is modified by “\*”, which means that the second space might or might not be there.

This command was added to the sed script and tested but since it only affects one line, we'll omit showing the results and move on. The next edit needs to remove the spaces that pad the beginning of a line. The pattern for matching that sequence is very similar to the address for the previous command.

```
s/^ */
```

This command removes any sequence of spaces found at the beginning of a line. The replacement portion of the substitute command is empty, meaning that the matched string is removed.

We can add this command to the script and test it.

```
$ sed -f sedscr horsefeathers
HORSEFEATHERS SOFTWARE PRODUCT BULLETIN
.LP
DESCRIPTION
.LP
BigOne Computer offers three software packages from the suite of
Horsefeathers software products -- Horsefeathers Business BASIC,
BASIC Librarian, and LIDO. These software products can fill your
```

requirements for powerful, sophisticated, general-purpose business software providing you with a base for software customization or development.

.LP

Horsefeathers BASIC is BASIC optimized for use on the BigOne machine with UNIX or MS-DOS operating systems. BASIC Librarian is a full screen program editor, which also provides the ability

The next edit attempts to deal with the extra spaces added to justify each line. We can write a substitute command to match any string of consecutive spaces and replace it with a single space.

```
s/ */ /g
```

We add the global flag at the end of the command so that all occurrences, not just the first, are replaced. Note that, like previous regular expressions, we are not specifying how many spaces are there, just that one or more be found. There might be two, three, or four consecutive spaces. No matter how many, we want to reduce them to one.\*

Let's test the new script:

```
$ sed -f sedscr horsefeathers
```

```
HORSEFEATHERS SOFTWARE PRODUCT BULLETIN
```

```
.LP
```

```
DESCRIPTION
```

```
.LP
```

```
BigOne Computer offers three software packages from the suite of
Horsefeathers software products -- Horsefeathers Business BASIC,
BASIC Librarian, and LIDO. These software products can fill your
requirements for powerful, sophisticated, general-purpose business
software providing you with a base for software customization or
development
```

```
.LP
```

```
Horsefeathers BASIC is BASIC optimized for use on the BigOne
machine with UNIX or MS-DOS operating systems. BASIC Librarian is
a full screen program editor, which also provides the ability.
```

It works as advertised, reducing two or more spaces to one. On closer inspection, though, you might notice that the script removes a sequence of two spaces following a period, a place where they might belong.

We could perfect our substitute command such that it does not make the replacement for spaces following a period. The problem is that there are cases when three spaces follow a period and we'd like to reduce that to two. The best way seems to be to write a separate command that deals with the special case of a period followed by spaces.

```
s/\. */ ./g
```

## Notes

This command replaces a period followed by any number of spaces with a period followed by two spaces. It should be noted that the previous command reduces multiple spaces to one, so that only one space will be found following a period. \* Nonetheless, this pattern works regardless of how many spaces follow the period, as long as there is at least one. (It would not, for instance, affect a filename of the form test.ext if it appeared in the document.)

This command is placed at the end of the script and tested:

```
$ sed -f sedscr horsefeathers
HORSEFEATHERS SOFTWARE PRODUCT BULLETIN
.LP
DESCRIPTION
.LP
BigOne Computer offers three software packages from the suite of
Horsefeathers software products -- Horsefeathers Business BASIC,
BASIC Librarian, and LIDO. These software products can fill your
requirements for powerful, sophisticated, general-purpose business
software providing you with a base for software customization or
development.
.LP
Horsefeathers BASIC is BASIC optimized for use on the BigOne
machine with UNIX or MS-DOS operating systems. BASIC Librarian is
a full screen program editor, which also provides the ability
```

It works. Here's the completed script:

```
s/^$.LP/
/^+ */d
s/^ */
s/ */ /g
s/^. */. /g
```

As we said earlier, the next stage would be to test the script on the complete file (hf.product.bulletin), using testsed, and examine the results thoroughly. When we are satisfied with the results, we can use runsed to make the changes permanent:

```
$ runsed hf.product.bulletin
done
```

By executing runsed, we have overwritten the original file. Before leaving this script, it is instructive to point out that although the script was written to process a specific file, each of the commands in the script is one that you might expect to use again, even if you don't use the entire script again. In other words, you may well write other scripts that delete blank lines or check for two spaces following a period. Recognizing how commands can be reused in other situations reduces the time it takes to develop and test new scripts. It's like a singer learning a song and adding it to his or her repertoire.



## Reading and Writing Files :

The read (r) and write (w) commands allow you to work directly with files. Both take a single argument, the name of a file. The syntax follows:

```
[line-address]r file
[address]w file
```

The read command reads the contents of file into the pattern space after the addressed line. It cannot operate on a range of lines. The write command writes the contents of the pattern space to the file.

You must have a single space between the command and the filename. (Everything after that space and up to the newline is taken to be the filename. Thus, leading or embedded spaces will become part of the filename.) The read command will not complain if the file does not exist. The write command will create a file if it does not exist; if the file already exists, the write command will overwrite it each time the script is invoked. If there are multiple instructions writing to the same file in one script, then each write command appends to the file. Also, be aware that you can only open up to 10 files per script.

The read command can be useful for inserting the contents of one file at a particular place in another file. For instance, let's say that there is a set of files and each file should close with the same one- or two-paragraph statement. A sed script would allow you to maintain the closing separately while inserting it as needed, for instance, when sending the file to the printer.

```
sed '$r closing' $* | pr | lp
```

The \$ is an addressing symbol specifying the last line of the file. The contents of the file named closing are placed after the contents of pattern space and output with it. This example does not specify a pathname, assuming the file to be in the same directory as the command. A more general-purpose command should use the full pathname.

You may want to test out a few quirks of the read command. Let's look at the following command:

```
/^<company list>/r company. List
```

That is, when sed matches a line beginning with the string "", it is going to append the contents of the file company. List to the end of the matched line. No subsequent command will affect the lines read from the file. For instance, you can't make any changes to the list of companies that you've read into the file. However, commands that address the original line will work. The previous command could be followed by a second command:

```
/^<company list>/d
```

to delete the original line. So that if the input file was as follows:

For service, contact any of the following companies:

```
<company list>
Thank you.
```

Notes

## Notes

running the two-line script would produce:

For service, contact any of the following companies:

```
Allied
Mayflower
United
Thank you.
```

Suppressing the automatic output, using the `-n` option or `#n` script syntax, prevents the original line in the pattern space from being output, but the result of a `read` command still goes to standard output.

Now let's look at examples of the `write` command. One use is to extract information from one file and place it in its own file. For instance, imagine that we had a file listing the names of salespeople alphabetically. For each person, the listing designates which of four regions the person is assigned to. Here's a sample:

Adams, Henrietta	Northeast
Banks, Freda	South
Dennis, Jim	Midwest
Garvey, Bill	Northeast
Jeffries, Jane	West
Madison, Sylvia	Midwest
Sommes, Tom	South

Writing a script for a seven-line file, of course, is ridiculous. Yet such a script can potentially handle as many names as you can put together, and is reusable.

If all we wanted was to extract the names for a particular region, we could easily use `grep` to do it. The advantage with `sed` is that we can break up the file into four separate files in a single step. The following four-line script does it:

```
/Northeast$/w region. Northeast
/South$/w region. South
/Midwest$/w region. Midwest
/West$/w region. West
```

All of the names of salespeople that are assigned to the Northeast region will be placed in a file named `region. Northeast`.

The `write` command writes out the contents of the pattern space when the command is invoked, not when end of the script is reached. In the previous example, we might want to remove the name of the region before writing it to file. For each case, we could handle it as we show for the Northeast region:

```
/Northeast${
s///
w region. Northeast
}
```

The substitute command matches the same pattern as the address and removes it. There are many different uses for the write command; for example, you could use it in a script to generate several customized versions of the same source file.

### 5.1.7 Quit Command, Modify a File with sed

#### Quit Command:

Quit command instructs the SED to quit the current execution flow. It is represented by the q command. Given below is the syntax of the quit command:

```
[address]q
[address]q [value]
```

Note that the quit command does not accept range of addresses, it only supports a single address. By default, SED follows read, execute, and repeat workflow; but when the quit command is encountered, it simply stops the current execution.

Let us print the first 3 lines from the file.

```
[jerry]$ sed '3 q' books.txt
```

On executing the above code, you get the following result:

```
1) A Storm of Swords, George R. R. Martin, 1216
2) The Two Towers, J. R. R. Tolkien, 352
3) The Alchemist, Paulo Coelho, 197
```

In addition to line number, we can also use textual patterns. The following command quits when pattern match succeeds.

```
[jerry]$ sed '/The Alchemist/ q' books.txt
```

On executing the above code, you get the following result:

```
1) A Storm of Swords, George R. R. Martin, 1216
2) The Two Towers, J. R. R. Tolkien, 352
3) The Alchemist, Paulo Coelho, 197
```

In addition to this, SED can also accept a value which can be used as the exit status. The following command shows its exit status as 100.

```
[jerry]$ sed '/The Alchemist/ q 100' books.txt
```

On writing the above code, you get the subsequent result:

## Notes

- 1) A Storm of Swords, George R. R. Martin, 1216
- 2) The Two Towers, J. R. R. Tolkien, 352
- 3) The Alchemist, Paulo Coelho, 197

Now let us verify the exit status.

```
[jerry]$ echo $?
```

On executing the above code, you get the following result:

```
100
```

### Modify a File with sed:

The most common use of sed is in making a set of search and replacement edits across a set of files. Many times these scripts aren't very unusual or interesting, just a list of substitute commands that change one word or phrase to another. Of course, such scripts don't need to be interesting as long as they are useful and save doing the work manually.

The example we look at in this section is a conversion script, designed to modify various "machine-specific" terms in a UNIX documentation set. One person went through the documentation set and made a list of things that needed to be changed. Another person worked from the list to create the following list of substitutions.

- ◇ s/ON switch/START switch/g
- ◇ s/ON button/START switch/g
- ◇ s/STANDBY switch/STOP switch/g
- ◇ s/STANDBY button/STOP switch/g
- ◇ s/STANDBY/STOP/g
- ◇ s/[cC]abinet [L]ight/control panel light/g
- ◇ s/core system diskettes/core system tape/g
- ◇ s/TERM=542[05] /TERM=PT200 /g
- ◇ s/Teletype 542[05]/BigOne PT200/g
- ◇ s/542[05] terminal/PT200 terminal/g
- ◇ s/Documentation Road Map/Documentation Directory/g
- ◇ s/Owner/Operator Guide/Installation and Operation Guide/g
- ◇ s/AT&T 3B20 [cC]omputer/BigOne XL Computer/g
- ◇ s/AT&T 3B2 [cC]omputer/BigOne XL Computer/g
- ◇ s/3B2 [cC]omputer/BigOne XL Computer/g
- ◇ s/3B2/BigOne XL Computer/g

The script is straightforward. The beauty is not in the script itself but in sed's ability to apply this script to the hundreds of files comprising the documentation set. Once this script is tested, it can be executed using `runsed` to process as many files as there are at once.

Such a script can be a tremendous time-saver, but it can also be an opportunity to make big-time mistakes. What sometimes happens is that a person writes the script, tests it on one or two out of the hundreds of files and concludes from that test that

the script works fine. While it may not be practical to test each file, it is important that the test files you do choose be both representative and exceptional. Remember that text is extremely variable and you cannot typically trust that what is true for a particular occurrence is true for all occurrences.

Using `grep` to examine large amounts of input can be very helpful. For instance, if you wanted to determine how “core system diskettes” appears in the documents, you could `grep` for it everywhere and pore over the listing. To be thorough, you should also `grep` for “core,” “core system,” “system diskettes,” and “diskettes” to look for occurrences split over multiple lines. Examining the input is the best way to know what your script must do.

In some ways, writing a script is like devising a hypothesis, given a certain set of facts. You try to prove the validity of the hypothesis by increasing the amount of data that you test it against. If you are going to be running a script on multiple files, use `testsed` to run the script on several dozen files after you’ve tested it on a smaller sample. Then compare the temporary files to the originals to see if your assumptions were correct. The script might be off slightly and you can revise it. The more time you spend testing, which is actually rather interesting work, the less chance you will spend your time unraveling problems caused by a botched script.

### Check your understanding

#### Fill Ups

1. \_\_\_\_\_ is a lightweight scripting language that lets you write succinct programs to operate line-by-line on the contents of text files.
2. \_\_\_\_\_ on the other hand, is particularly useful for automating “find-and-replace” operations on text files.
3. In SED, the \_\_\_\_\_ character behaves like logical OR operation.
4. The \_\_\_\_\_ flag is by far the most commonly used.
5. The \_\_\_\_\_ flag can be utilised in the unusual case where a regular expression repeats repeatedly on a line and only one of those occurrences must be replaced by position.

#### True or False

1. Both `awk` and `sed` rely heavily on regular expressions to describe patterns in text upon which some operation should be performed.
2. As the name suggests, the delete command is used to perform delete operation and since the SED operates on line, we can say that this command is used to delete lines.
3. The print flag and the write flag both provide the same functionality as the print and write commands (which are discussed later in this chapter) with one important difference.
4. The read command reads the contents of file into the pattern space after the addressed line.
5. Note that the quit command does not accept range of addresses, it only supports a single address.

## Notes

### Summary

- Two utilities found on every Unix installation—awk and sed—merit special attention in this context. awk is a lightweight scripting language that lets you write succinct programs to operate line-by-line on the contents of text files. sed on the other hand, is particularly useful for automating “find-and-replace” operations on text files. Each of them is optimized to handle large files without storing a lot of information in memory, so they can be useful for quick operations on large bioinformatic data sets. Neither is a fully-featured programming language that you would want to write large, complex programs in (that said, I did once implement a complete program for full-sibling inference from multiallelic markers in awk); however they do share many of the useful text-manipulation capabilities of such languages, such as Perl and Python.
- For all practical purposes, you can rely on a program to produce the correct result. However, that doesn't mean the program always works correctly as far as you are concerned. Most of the time, you can bet that if a program does not produce the output that you expected, the real problem (putting aside input or syntax errors) is how you described what you wanted. In other words, the place to look to correct the problem is the expression where you described the result you wanted.
- As the name suggests, the delete command is used to perform delete operation and since the SED operates on line, we can say that this command is used to delete lines. Note that the delete command removes lines only from the pattern buffer; the line is not sent to the output stream and the original file remains unchanged. The following example illustrates the point.
- Linux Sed command allows you to print only specific lines based on the line number or pattern matches. “p” is a command for printing the data from the pattern buffer. To suppress automatic printing of pattern space use -n command with sed. sed -n option will not print anything, unless an explicit request to print is found.
- The substitute command is applied to the lines matching the address. If no address is specified, it is applied to all lines that match the pattern, a regular expression. If a regular expression is supplied as an address, and no pattern is specified, the substitute command matches what is matched by the address. This can be useful when the substitute command is one of multiple commands applied at the same address.
- Once upon a time, computers stored text in fixed-length records. A line ended after so many characters (typically 80), and then the next line started. There was no explicit character in the data to mark the end of one line and the beginning of the next; every line had the same (fixed) number of characters. Modern systems are more flexible; they use a special character (referred to as newline) to mark the end of the line. This allows lines to be of arbitrary length.
- The read command reads the contents of file into the pattern space after the addressed line. It cannot operate on a range of lines. The write command writes the contents of the pattern space to the file.
- The most common use of sed is in making a set of search and replacement edits across a set of files. Many times these scripts aren't very unusual or interesting, just a list of substitute commands that change one word or phrase to another. Of

course, such scripts don't need to be interesting as long as they are useful and save doing the work manually.

### Activity

1. Write your own bulletin on any topic and exchange it with a partner in your class. Then go through the bulletin and see if any edits need to be made. If you feel no edits need to be made, write why you feel so and provide examples.

### Questions and Exercises

1. List the three steps involved in writing a regular expression.
2. What does the insert command do?
3. Explain the printing command.
4. What happens when you write / in your code?
5. What is the read command used for?
6. Explain what the quit command does.

### Glossary

- **awk** is a lightweight scripting language that lets you write succinct programs to operate line-by-by line on the contents of text files.
- **sed** on the other hand, is particularly useful for automating “find-and-replace” operations on text files.
- **Hits** The lines that I wanted to match.
- **Misses** The lines that I didn't want to match.
- **Omissions** The lines that I didn't match but wanted to match.
- **False alarms** The lines that I matched but didn't want to match.
- **n** A number (1 to 512) indicating that a replacement should only be made for the n<sup>th</sup> occurrence of the pattern.
- **g** Make changes globally on all occurrences in the pattern space. Normally only the first occurrence is replaced.
- **p** Print the contents of the pattern space.
- **w file** Write the contents of the pattern space to file.

### Further Readings

1. Blum, Richard; Bresnahan, Christine. Mastering Linux System Administration. Sybex; 1st edition (July 27, 2021)
2. Gift, Noah; Jones, Jeremy. Python for Unix and Linux System Administration. O'Reilly Media; 1st edition (September 1, 2008)
3. Hein, Trent; Mackin, Dan; Nemeth, Evi; Snyder, Garth; Whaley, Ben. UNIX and Linux System Administration Handbook. Addison-Wesley Professional; 5th edition (August 8, 2017)



## Notes

4. Kerrisk, Michael. The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press; 1st edition (October 28, 2010)
5. Negus, Christopher. Linux Bible. Wiley; 10th edition (June 10, 2020)

### Check your Understanding: Answers

#### Fill Ups

1. awk
2. sed
3. pipe
4. global
5. numeric

#### True or False

1. True
2. True
3. True
4. True
5. True

## Unit-5.2: awk

### Notes

### Objectives:

At the end of this unit, you will be able to understand:

- Syntax for awk
- Working of awk
- Input from files, Input from commands
- Formatting output-print and print function
- Concept of record and field in awk

### Introduction

Awk is a scripting language used for manipulating data and generating reports. The awk command programming language requires no compiling and allows the user to use variables, numeric functions, string functions, and logical operators.

Awk is a utility that enables a programmer to write tiny but effective programs in the form of statements that define text patterns that are to be searched for in each line of a document and the action that is to be taken when a match is found within a line. Awk is mostly used for pattern scanning and processing. It searches one or more files to see if they contain lines that matches with the specified patterns and then perform the associated actions.

Awk is abbreviated from the names of the developers – Aho, Weinberger, and Kernighan.

### 5.2.1 Syntax for awk

#### AWK Command Line

We can use single quotes at the command line to provide an AWK command, as shown –

```
awk [options] file ...
```

Example

Consider a text file marks.txt with the following content –

1) Amit	Physics	80
2) Rahul	Maths	90
3) Shyam	Biology	87
4) Kedar	English	85
5) Hari	History	89

Let us display the complete content of the file using AWK as follows –

Example

## Notes

```
[jerry]$ awk '{print}' marks.txt
```

On executing this code, you get the following result –

Output

1) Amit	Physics	80
2) Rahul	Maths	90
3) Shyam	Biology	87
4) Kedar	English	85
5) Hari	History	89

### AWK Program File

We can provide AWK commands in a script file as shown –

```
awk [options] -f file
```

First, create a text file command.awk containing the AWK command as shown below –

```
{print}
```

Now we can tell AWK to read commands from the text file and execute the command. In this case, we get the same outcome as in the previous example.

Example

```
[jerry]$ awk -f command.awk marks.txt
```

On writing this code, you get the subsequent result –

Output

1) Amit	Physics	80
2) Rahul	Maths	90
3) Shyam	Biology	87
4) Kedar	English	85
5) Hari	History	89

### AWK Standard Options

The command line can be used to provide the following standard options to AWK.

The -v option

This option gives a variable a value. It enables assignment prior to the execution of the programme. The -v option is demonstrated in the following example.

Example

```
[jerry]$ awk -v name=Jerry 'BEGIN{printf "Name = %s\n", name}'
```

On writing this code, you get the subsequent result –

Output

```
Name = Jerry
```

Notes

### The --dump-variables[=file] option

It saves a file containing a sorted list of global variables and their final values. awkvars.out is the default file.

Example

```
[jerry]$ awk --dump-variables "
[jerry]$ cat awkvars.out
```

On writing the above code, you get the subsequent result –

Output

```
ARGC: 1
ARGIND: 0
ARGV: array, 1 elements
BINMODE: 0
CONVFMT: "%.6g"
ERRNO: ""
FIELDWIDTHS: ""
FILENAME: ""
FNR: 0
FPAT: "[^[:space:]]+"
FS: " "
IGNORECASE: 0
LINT: 0
NF: 0
NR: 0
OFMT: "%.6g"
OFS: " "
ORS: "\n"
RLENGTH: 0
RS: "\n"
RSTART: 0
RT: ""
SUBSEP: "\034"
TEXTDOMAIN: "messages"
```

### The --help option

This option prints the help message on standard output.

Example

```
[jerry]$ awk --help
```

On writing this code, you get the subsequent result –

## Notes

### Output

Usage: awk [POSIX or GNU style options] -f progfile [--] file ...

Usage: awk [POSIX or GNU style options] [--] 'program' file ...

POSIX options: GNU long options: (standard)

-f progfile	--file=progfile
-F fs	--field-separator=fs
-v var=val	--assign=var=val

Short options: GNU long options: (extensions)

-b	--characters-as-bytes
-c	--traditional
-C	--copyright
-d[file]	--dump-variables[=file]
-e 'program-text'	--source='program-text'
-E file	--exec=file
-g	--gen-pot
-h	--help
-L [fatal]	--lint[=fatal]
-n	--non-decimal-data
-N	--use-ic-numeric
-O	--optimize
-p[file]	--profile[=file]
-P	--posix
-r	--re-interval
-S	--sandbox
-t	--lint-old
-V	--version

### The --lint[=fatal] option

This option allows you to check non-portable or suspect structures. When the fatal option is used, warning messages are treated as errors. This is demonstrated in the following example –

#### Example

```
[jerry]$ awk --lint " /bin/l
```

On writing this code, you get the subsequent result –

Output

```
awk: cmd. line:1: warning: empty program text on command line
awk: cmd. line:1: warning: source file does not end in newline
awk: warning: no program text at all!
```

### The --posix option

This option disables all common and awk-specific extensions, resulting in strict POSIX compatibility.

The --profile[=file] option

This option saves a file with a pretty-printed version of the application. awkprof.out is the default file. This is demonstrated in the basic example below–

Example

```
[jerry]$ awk --profile 'BEGIN{printf"---|Header|--\n"} {print}
END{printf"---|Footer|---\n"}' marks.txt > /dev/null
[jerry]$ cat awkprof.out
```

On writing this code, you get the subsequent result –

Output

```
awk profile, created Sun Oct 26 19:50:48 2014

BEGIN block(s)
BEGIN {
 printf "---|Header|--\n"
}
Rule(s) {
 print $0
}
END block(s)
END {
 printf "---|Footer|---\n"
}
```

### The --traditional option

This option disables all awk-specific extensions.

The --version option

This option displays the version information of the AWK program.

Example

Notes

## Notes

```
[jerry]$ awk --version
```

When this code is executed, it produces the following result –

Output

```
. GNU Awk 4.0.1
Copyright (C) 1989, 1991-2012 Free Software Foundation
```

### 5.2.2 Working of awk

The below are the steps involved in working of awk:

#### Read

A line from the input stream (file, pipe, or stdin) is read and stored in memory by AWK..

#### Execute

On the input, all AWK commands are executed in order. AWK executes commands on each line by default. By offering patterns, we can limit this.

#### Repeat

This step is repeated until the file is finished.

#### Program Structure

Let us now understand the program structure of AWK.

#### BEGIN block

The syntax of the BEGIN block is as follows –

Syntax

```
BEGIN {awk-commands}
```

At the start of the programme, the BEGIN block is executed. It is only used once. This is a nice location to start setting up variables. BEGIN is an AWK keyword, hence it must be capitalised. Please keep in mind that this section is optional.

#### Body Block

The syntax of the body block is as follows –

Syntax

```
/pattern/ {awk-commands}
```

Every input line is subjected to AWK commands in the body block. AWK executes commands on each line by default. By offering patterns, we can limit this. It's worth noting that the Body block has no keywords.



## END Block

The syntax of the END block is as follows –

Syntax

```
END {awk-commands}
```

At the end of the programme, the END block is executed. Because END is an AWK keyword, it must be capitalised. Please keep in mind that this block is completely optional.

Let's make a file called marks.txt that contains the serial number, the student's name, the topic name, and the amount of marks earned.

1) Amit	Physics	80
2) Rahul	Maths	90
3) Shyam	Biology	87
4) Kedar	English	85
5) Hari	History	89

Now we will display the file contents with header by using AWK script.

Example

```
[jerry]$ awk 'BEGIN{printf "Sr No\tName\tSub\tMarks\n"} {print}' marks.txt
```

When this code is written, it produces the subsequent result –

Output

Sr No	Name	Sub	Marks
1)	Amit	Physics	80
2)	Rahul	Maths	90
3)	Shyam	Biology	87
4)	Kedar	English	85
5)	Hari	History	89

AWK prints the header from the BEGIN block at the start. The body block then reads a line from a file and uses AWK's print command to print the contents to the standard output stream. This procedure is repeated till the file reaches its conclusion.

## 5.2.3 Input from Files, Input from Commands

**Input from files:**

Awk reads all input from the standard input (by default, the keyboard, but it's commonly a pipe from another command) or from files whose names you give on the awk command line in a typical awk programme. If you specify input files, awk reads each one in turn, processing all of the data before moving on to the next. FILENAME is a predefined variable that contains the name of the current input file (see Predefined Variables).

Notes

## Notes

The data is read in units called records and processed by your program's rules one record at a time. Each record is one line by default. Each record is immediately broken down into fields. This makes it easier for programmes to work on different sections of a record.

You may need to utilise the `getline` command on rare circumstances. The `getline` command is useful because it can take explicit input from any number of files and it does not require that the files used with it be named on the `awk` command line (see Explicit Input with `getline`).

### How Input Is Split into Records

Your program's input is divided into records and fields by `awk`. It keeps track of how many records have been read from the current input file so far. This value is saved in the `FNR` predefined variable, which is reset to zero each time a new file is opened. Another preset variable, `NR`, keeps track of how many input records have been read from all datafiles thus far. It starts at zero but is never reset to that value.

### Record Splitting with Standard `awk`

The record separator is a character that separates records. The newline character is used as the record separator by default. This is why records are single-line by default. Simply assign a different character to the predefined variable `RS` to utilise a different record separator.

The value of `RS` can be altered in the `awk` programme using the assignment operator, '=', just like any other variable (see Assignment Expressions). The new record-separator character must be contained in quotation marks to show that it is a string constant. The best moment to do this is usually at the start of the execution, before any input is handled, so that the first record is read with the correct separator. To do this, use the special `BEGIN` pattern (see The `BEGIN` and `END` Special Patterns). For example:

```
awk 'BEGIN { RS = "u" }
 { print $0 }' mail-list
```

Before reading any input, alter the value of `RS` to 'u'. The new value is a string with the letter "u" as the first character; hence, records are separated by the letter "u." The input file is then read, and the `awk` program's second rule (the action with no pattern) outputs each record. This `awk` programme duplicates the input with each 'u' changed to a newline because each `print` statement adds a newline at the end of its output. The results of running the software on a mailing list are shown below:

```
$ awk 'BEGIN { RS = "u" }
> { print $0 }' mail-list
Amelia 555-5553 amelia.zodiac
sq
e@gmail.com F
Anthony 555-3412 anthony.assert
ro@hotmail.com A
Becky 555-7685 becky.algebrar
```

## Notes

```

m@gmail.com A
Bill 555-1675 bill.drowning@hotmail.com A
Broderick 555-0542 broderick.aliq
otiens@yahoo.com R
Camilla 555-2912 camilla.inf
sar
m@skynet.be R
Fabi
s 555-1234 fabi
s.
ndevicesim
s@
cb.ed
 F
J
lie 555-6699 j
lie.perscr
tabor@skeeve.com F
Martin 555-6480 martin.codicib
s@hotmail.com A
Sam
el 555-3430 sam
el.lanceolis@sh
.ed
 A
Jean-Pa
l 555-2127 jeanpa
l.campanor
m@ny
.ed
 R

```

It's worth noting that the 'Bill' entry isn't separated. The line looks like this in the original datafile (see Datafiles for Examples):

```

Bill 555-1675 bill.drowning@hotmail.com A

```

It contains no 'u', so there is no reason to split the record, unlike the others, which each have one or more occurrences of the 'u'. In fact, this record is treated as part of the previous record; the newline separating them in the output is the original newline in the datafile, not the one added by awk when it printed the record!

## Notes

Another way to change the record separator is on the command line, using the variable-assignment feature (see Other Command-Line Arguments):

```
awk '{ print $0 }' RS="u" mail-list
```

This sets RS to 'u' before processing mail-list.

Using an alphabetic character for the record separator, such as 'u,' is likely to generate unexpected results. In the vast majority of circumstances, using an odd character like '/' will result in correct behaviour, but there are no assurances. The moral of the story is to be aware of your data.

When using regular characters as the record separator, there is one unusual case that occurs when awk is being fully POSIX-compliant (see Command-Line Options). Then, the following (extreme) pipeline prints a surprising '1':

```
$ echo | awk --posix 'BEGIN { RS = "a" }; { print NF }'
```

1

There is only one field, which is a newline. The number of fields in the current record is the value of the built-in variable NF. (awk, like most other versions of awk, sees the newline as whitespace and prints '0' as a result in the typical situation.)

Even if the last character in the file is not the character in RS, reaching the end of an input file stops the current input record (d.c.)

As the value of RS, the empty string "" (a string with no characters) has a unique significance. It simply means that one or more blank lines are used to divide records. For more information, see Multiple-Line Records.

If you alter the value of RS in the middle of an awk run, the new value is utilised to delimit succeeding records, but the current record and previously processed records are unaffected.

awk assigns the variable RT to the text in the input that matched RS after determining the end of the record.

### Record Splitting with awk

The value of RS is not limited to a one-character string when using awk. Any regular expression will suffice. In general, each record ends with the next string that matches the regular expression, and the following record begins with the end of that string. When RS contains only a newline, this basic rule applies: a record terminates at the beginning of the next matching string (the next newline in the input), and the next record begins shortly after the end of this string (at the first character of the following line). The newline is not part of either record because it matches RS.

RT contains the same single character as RS when it is a single character. RT, on the other hand, includes the actual input text that matched the regular expression when RS is a regular expression.

If there is no text matching RS at the conclusion of the input file, awk sets RT to the null string.

Both of these characteristics are demonstrated in the following example. It assigns RS to a regular expression that matches a newline or a sequence of one or more uppercase letters with optional leading and/or trailing whitespace:

```
$ echo record 1 AAAA record 2 BBBB record 3 |
> awk 'BEGIN { RS = "\n(*[:upper:]]+ *)" }
> { print "Record =", $0,"and RT = [" RT "]" }'
Record = record 1 and RT = [AAAA]
Record = record 2 and RT = [BBBB]
Record = record 3 and RT = [
]
```

The square brackets divide RT's contents, allowing you to observe the leading and trailing whitespace. A newline is the final value of RT. For a more usable example of RS as a regexp with RT, see A Simple Stream Editor.

If you set RS to a regular expression with optional trailing text, such as "RS = "abc(XYZ)?"", it's possible that awk will match the leading part of the regular expression but not the trailing part due to implementation constraints, especially if the input text that could match the trailing part is quite long. Awk tries to avoid this problem, but there's no guarantee that it won't happen again.

### Input from commands:

There are various ways to give an awk programme input. The most frequent setup is to save input data in a file called data and then type it in

```
awk 'program' data
```

If no filenames are specified, awk reads its standard input; consequently, a second popular approach is to have another programme pipe its output into awk. The software egrep, for example, chooses input lines that contain a defined regular expression, but it does it considerably faster than awk. As a result, we could type the command

```
egrep 'Asia' countries : awk 'program'
```

egrep looks for lines that contain Asia and sends them to the awk programme for further processing.

### Input Separators

The built-in variable FS has the value 11 11 by default, which is a single blank. When FS is set to this value, input fields are separated by blanks and/or tabs, with leading blanks and tabs deleted, resulting in the same initial field on each subsequent line:

```
field1
field1
field1 field2
```

Leading blanks and tabs are not discarded when FS is set to any other value.

## Notes

The field separator can be altered by setting the built-in variable FS to a string. It's assumed that the string is a regular expression if it's longer than one character. The field separators on the current input line are the leftmost longest nonnull and nonoverlapping substrings matched by that regular expression. As an example,

```
BEGIN { FS = "[\t]*:[\t]+ 11 }
```

creates field separators from any string that contains a comma followed by blanks and tabs, as well as any string of blanks and tabs that does not contain a comma.

The field separator is set to a single character other than blank when FS is set to a single character other than blank. This practise makes it simple to utilise metacharacters in regular expressions as field separators:

```
FS = "."
```

makes `.` a field separator. But note that something indirect like

```
FS = "[]"
```

is required to set the field separator to a single blank.

FS can also be set on the command line with the `-F` argument. The command line

```
awk -F '[\t]*:[\t]+' 'program'
```

sets the field separator to the same strings as the BEGIN action shown above.

### Multiline Records

Because newlines are used to separate records by default, the terms "line" and "record" are often interchanged. However, you can modify the default record separator in a restricted way by changing the value of the built-in record-separator variable RS. If RS is set to null, as it is in

```
BEGIN { RS = "" }
```

The records are then separated by one or more blank lines, allowing each record to take up multiple lines. The default behaviour is restored by setting RS to newline with the assignment `RS = "\n"`. No matter what value FS has, newline is always one of the field separators in multiline records.

A common way to process multiline records is to use

```
BEGIN { RS = ""; FS = "\n" }
```

to make each line a separate field by setting the record separator to one or more blank lines and the field separator to a newline alone. The maximum length of a record is usually around 3000 characters.

### The getline Function

The getline function can be used to read data from the current input, a file, or a pipe. Getline obtains the next input record and conducts the standard field-splitting procedures on it on its own. It sets NF, NR, and FNR, and returns 1 if a record was found, 0 if an end-of-file error occurred, and -1 if an error occurred (such as failure to open a file).

The expression `getline x` increments NR and FNR while reading the next record into the variable x. There is no splitting; NF is not set.

The expression

```
getline <"file"
```

reads from file instead of the current input. It has no effect on NR or FNR, but field splitting is performed and NF is set.

The expression

```
getline x <"file" 11
```

gets the next record from file into x; no splitting is done, and NF, NR, and FNR are untouched. The value of each expression is the value returned by `getline`. As an example, this program copies its input to its output, except that each line like

```
#include "filename"
```

is replaced by the contents of the file filename.

```
include - replace #include "f" by contents of file f
```

```
/"#include/ {
```

```
gsub(/"1, "", $2) }
```

```
while (getline x 0)
```

```
print x
```

```
}
```

```
next
```

```
{ print }
```

You can also pipe the output of another command straight into `getline`. Consider the following example:

```
while ("who" : getline)
```

```
n++
```

executes the Unix programme `who` and feeds its output to `getline` (once only). The result of `who` is a list of logged-in users. Each iteration of the while loop reads one more line from this list and increments the variable n, resulting in n containing a count of the number of users when the while loop ends. In the same way, the expression

```
"date" : getline d
```

The `date` command output is piped into the variable d, setting d to the current date. On non-Unix systems, input pipes may not be available. In every scenario where `getline` is used, you should be aware that if the file cannot be accessed, an error will be returned. Despite the fact that writing is enticing

```
while (getline <"file") ... # Dangerous
```



## Notes

that's an infinite loop if file doesn't exist, because with a nonexistent file `getline` returns -1, a nonzero value that represents true. The preferred way is

```
while (getline <"file"> 0) ... #Safe
```

Here the loop will be executed only when `getline` returns 1.

### Command-Line Variable Assignments

As we have seen, an `awk` command line can have several forms:

```
awk 'program' f1 £2 ...
awk -f progfile f1 £2 ...
awk -Fsep 'program' f1 f2..
awk -Fsep -£ progfile f1 f2 ...
```

`f1`, `£2`, and so on are command-line parameters that generally represent filenames in these command lines. However, if a filename has the type `var-text`, it is interpreted as a text assignment to `var`, which is performed when the argument would normally be accessed as a file. Variables can be altered before and after a file is read with this type of assignment.

### Command-Line Arguments

The `awk` programme has a built-in array called `ARGV` that contains the command-line arguments. The built-in variable `ARGC` has a value one higher than the number of arguments. Using a command line

```
awk -f progfile a v=1 b
```

`ARGC` has the value 4, `ARGV [ 0 ]` contains `awk`, `ARGV [ 1 ]` contains `a`, `ARGV [ 2 ]` contains `v= 1`, and `ARGV [ 3 ]` contains `b`. `ARGC` is one more than the number of arguments because `awk`, the name of the command, is counted as argument zero, as it is in C programs. If the `awk` program appears on the command line, however, the program is not treated as an argument, nor is `-f` filename or any `-F` option. For example, with the command line

```
awk -F\t 'S3 > 100' countries
```

`ARGC` is 2 and `ARGV[ 1 ]` is `countries`.

The following program echoes its command-line arguments:

```
echo - print command-line arguments
BEGIN {
 for (i = 1; i < ARGC; i++)
 printf "%s ", ARGV[i]
 printf "\n"
}
```

Notice that everything happens in the BEGIN action: because there are no other pattern-action statements, the arguments are never treated as filenames, and no input is read. Another program using command-line arguments is seq, which generates sequences of integers:

```
seq - print sequences of integers
input: arguments q, p q, or p q r; q >= p; r > 0
output: integers 1 to q, p to q, or p to q in steps of r
BEGIN {
 if (ARGC == 2)
 for (i = 1; i <= ARGV[1]; i++)
 print i
 else if (ARGC == 3)
 for (i = ARGV[1]; i <= ARGV[2]; i++)
 print i
 else if (ARGC == 4)
 for (i = ARGV[1]; i <= ARGV[2]; i += ARGV[3])
 print i
}
```

The commands

```
awk -f seq 10
awk -f seq 1 10
awk -f seq 1 10 1
```

all generate the integers one through ten.

The arguments in ARGV may be modified or added to; ARGC may be altered. As each input file ends, awk treats the next nonnull element of ARGV (up through the current value of ARGC-1) as the name of the next input file. Thus, setting an element of ARGV to null means that it will not be treated as an input file. The name 11 - 11 may be used for the standard input.

## 5.2.4 Formatting output-print and printf function

Many of the scripts we've built so far have done a good job of data processing, but the output hasn't been structured appropriately. That's because the fundamental pr int statement only allows you to do so much. Because one of awk's most popular duties is to generate reports, we need to be able to style our reports in a logical manner. The filesum programme does a good job with the arithmetic problems, but the report is disorganised. The print f statement, taken from the C programming language, is an alternative to the pr int statement in Awk. The printf statement, like the pr int statement, can output a single string.

```
awk 'BEGIN { printf ("Hello, world\n") }'
```

The primary difference you'll notice right away is that, unlike print, printf does not supply a newline immediately. It must be specified as "n" explicitly.

## Notes

There are two sections to the full syntax of the `printf` statement:

```
printf (format-expression [, arguments])
```

The parentheses are not required. The first portion is an expression that explains the format requirements; this is normally provided as a string constant enclosed in quotes. The second element is an argument list that corresponds to the format standards, such as a list of variable names. A percent sign (%) precedes a format specification, and the specifier is one of the characters listed in the table below. `s` for strings and `d` for decimal numbers are the two most used format specifiers. \*

**Table: Format Specifiers Used in `printf`**

Character	Description
c	ASCII character
d	Decimal integer
i	Decimal integer. (Added in POSIX)
e	Floating-point format ([-]d.precision[+-]dd)
E	Floating-point format ([-]d.precisionE[+-]dd)
f	Floating-point format ([-]ddd.precision)
g	e or f conversion, whichever is shortest, with trailing zeros removed
G	E or f conversion, whichever is shortest, with trailing zeros removed
o	Unsigned octal value
s	String
u	Unsigned decimal value
x	Unsigned hexadecimal number. Uses a-f for 10 to 15
X	Unsigned hexadecimal number. Uses A-F for 10 to 15
%	Literal %

This example uses the `printf` statement to produce the output for rule 2 in the `filesun` program. It outputs a string and a decimal value found in two different fields:

```
printf("%d\t%s\n", $5, $9)
```

The value of `$5` is to be output, followed by a tab (`\t`) and `$9` and then a newline (`\n`). \* For each format specification, you must supply a corresponding argument.

The width and alignment of output fields can be specified using the `printf` statement. Following “percent” and before the format specifier, a format expression can include three optional modifiers:

```
%-width.precision format-specifier
```

The output field’s width is a numeric value. By default, when you specify a field width, the contents of the field are right-justified. To get left-justification, you must specify “-.” As a result, “percent -20s” produces a left-justified string in a 20-character field. If the string is less than 20 characters, whitespace will be used to fill the field. A

"|" is printed in the following examples to reflect the field's real width. The first example aligns the text to the right:

```
printf("|%10s\n", "hello")
```

It produces:

```
| hello|
```

The next example left-justifies the text:

```
printf("|%-10s\n", "hello")
```

It produces: |hello |

For decimal or floating-point data, the precision modifier determines how many digits show to the right of the decimal point. It determines the maximum amount of characters from a string that will be printed for string values.

It's worth noting that the default precision for numeric numbers output is "percent.6g."

Values in the printf or sprintf argument list can be used to dynamically specify the width and precision. This is accomplished by using asterisks instead of literal values.

```
printf("%*. *g\n", 5, 3, myvar);
```

In this example, the width is 5, the precision is 3, and the value to print will come from myvar.

Setting the system variable OFMT changes the default precision used by the printf statement when printing numbers. If you're using awk to produce reports with dollar values, for example, you might want to alter OFMT to "percent.2f."

The problem with filesum of having fields and headings properly aligned can be solved by using the entire syntax of the format statement. Because the fields had a better chance of aligning themselves if they were output in that sequence, we output the file size before the filename. The ability to fix the width of output fields is provided by printf; as a result, each field begins in the same column. Let's rearrange the filesum report's output fields. We need a minimum field width so that the second field starts in the same place as the first. Between the percent and the conversion specification, you define the field width. "percent -15s" specifies a minimum field width of 15 characters and left-justification of the value. Without the hyphen, "percent 10d" is right-justified, which is what we want for a decimal figure.

```
printf("%-15s\t%10d\n", $9, $5) # print filename and size
```

The data will be aligned in columns and the numbers will be right-justified as a result of this. Examine the END action's use of the printf statement:

```
printf("Total: %d bytes (%d files)\n", sum, filenum)
```

In the BEGIN rule, the column header is also adjusted correctly. filesum now generates the following result when using the printf command:

## Notes

```
$ filesum g*
FILE BYTES
g 23 awk 2237
awk.mail 1171
awk.test 74
awkro 264
gfilesum 610
grades 64
grades.awk 231
grepscript 6
Total: 4680 bytes (9 files)
```

### 5.2.5 Concept of Record and Field in awk

Awk assumes that its input is structured rather than a random collection of characters. Each input line is treated as a record, and each word separated by spaces or tabs is treated as a field. (The characters that separate the fields are known as delimiters.) Three fields are separated by a space or a tab in the following record in the file names.

John Robinson 666-555-1111

Two or more consecutive spaces and/or tabs count as a single delimiter.

#### Referencing and Separating Fields :

The field operator \$ in Awk allows you to refer to fields in actions. This operator is followed by a number or a variable that specifies a field's numerical position. The first field is denoted by "\$1," the second by "\$2," and so on. The value "\$0" denotes the complete input record. The surname name appears first, followed by the first name and then the phone number in the sample below.

```
$ awk '{ print $2, $1, $3 }' names Robinson John 666-555-1111
```

The first name is \$1, the last name is \$2, and the phone number is \$3. The commas that divide each argument in the print statement result in a space between the values being produced. (We'll talk about the output field separator (OFS) later, which the comma outputs and is set to a space by default.) A single input line in this example creates a record with three fields: a space between the first and last names, and a tab between the last name and the phone number. You could define the field separator explicitly to only recognise tabs if you wanted to grab the first and last name as a single field. Then just two fields in this record would be recognised by awk.

You can refer to a field with any expression that evaluates to an integer, not only numbers and variables.

```
$ echo a b c d | awk 'BEGIN { one = 1; two = 2 }
> { print $(one + two) }'
C
```

The -F option on the command line can be used to modify the field separator. The delimiter character comes next (either immediately, or separated by whitespace). The field separator is changed to a tab in the following example.

```
$ awk -F"\t" '{ print $2 }' names
666-555-1111 "
```

\t" is an escape sequence (discussed below) that represents an actual tab character. It should be surrounded by single or double quotes.

Commas delimit fields in the following two address records.

```
John Robinson,Koren Inc.,978 4th Ave.,Boston,MA 01760,696-0987
Phyllis Chapman,GVE Corp.,34 Sea Drive,Amesbury,MA 01881,879-0900
```

An awk program can print the name and address in block format.

```
blocklist.awk -- print name and address in block form.
input file -- name, company, street, city, state and zip, phone
{ print "" # output blank line
print $1 # name print $2 # company
print $3 # street
print $4, $5 # city, state zip
}
```

The empty string (") is specified in the first print declaration (remember, print by itself outputs the current line). The records in the report will be divided by blank lines as a result of this. Using the following command, we can run this script and specify that the field separator is a comma:

```
awk -F, -f blocklist.awk names
```

The following report is produced:

```
John Robinson
Koren Inc.
978 4th Ave.
Boston MA 01760
Phyllis Chapman
GVE Corp.
34 Sea Drive
Amesbury MA 01881
```

Specifying the field separator in the script itself is usually a better practise and more convenient. The field separator can be changed using the system variable FS. We must assign this variable in an action controlled by the BEGIN rule since it must be done before the first input line is read.

## Notes

```
BEGIN { FS = "," }
```

Now let's use it in a script to print out the names and phone numbers.

```
phonelist.awk -- print name and phone number.
input file -- name, company, street, city, state and zip, phone
BEGIN { FS = "," } # comma-delimited fields
{ print $1 " , " $6 }
```

Notice how we use blank lines in the script to make it easier to read. Between the two output fields, the `print` statement inserts a comma followed by a space. This script can be run directly from the command prompt:

```
$ awk -f phonelist.awk names
John Robinson, 696-0987
Phyllis Chapman, 879-0900
```

This offers you a basic notion of how to use `awk` to manipulate data with a recognisable structure. This script is set up to publish all lines of data, but we could change it to only print specified names or addresses by implementing a pattern-matching rule. So, if we had a vast list of names, we could narrow it down to simply the names of people who lived in a specific state. We might be able to write:

```
/MA/ { print $1 " , " $6 }
```

Where `MA` corresponds to Massachusetts' postal state abbreviation. We might be able to identify a firm name or another field where the letters "MA" appeared. We can look for a match in a specified field. The tilde ( `~` ) operator is used to compare a regular expression to a field.

```
$5 ~ /MA/ { print $1 " , " $6 }
```

You can reverse the meaning of the rule by using bang-tilde ( `!~` ).

```
$5 !~ /MA/ { print $1 " , " $6 }
```

This rule would match all records whose fifth field did not contain the letter "MA." A pattern-matching rule that exclusively matches long-distance phone numbers would be more difficult. The regular expression below looks for an area code.

```
$6 ~ /1?(-|)?(?([0-9]+)\?(-|)?[0-9]+-[0-9]+)/
```

This rule matches any of the following forms:

```
707-724-0000
(707) 724-0000
(707)724-0000
```

```
1-707-724-0000
1 707-724-0000
1(707)724-0000
```



By breaking down the regular expression into its constituent pieces, it may be read. "1?" denotes a single occurrence of "1." "(-|)?" looks for a hyphen, a space, or nothing at all in the next position. The backslash prevents "(" from being interpreted as the grouping metacharacter. "(?" looks for zero or one left parenthesis. "[0-9]+" searches for one or more digits; note that we chose the easy way out by specifying one or more digits rather than three. We're looking for an optional right parenthesis in the next spot, as well as either a space or a hyphen, or nothing at all. Then we look for one or more digits "[0-9]+," a hyphen, and another one or more digits "[0-9]+."

### Field Splitting:

You can have awk separate fields in three different ways. The first option is to use whitespace to separate fields. Set FS to a single space to accomplish this. In this example, the record is stripped of leading and following whitespace (spaces and/or tabs), and fields are separated by runs of spaces and/or tabs. Because the default value of FS is a single space, awk splits each record into fields in this manner:

The second option is to create some additional single-character fields. A ":" is commonly used as a field separator in awk programmes that parse the UNIX /etc/passwd file. When FS is a single character, each instance of that character denotes the start of a new field. If there are two instances in a row, the field between them has the value of the empty string.

Finally, if the field separator contains more than one character, it will be read as a regular expression. That is, the "leftmost longest non-null and nonoverlapping" substring\* that meets the regular expression will be used as the field separator. (The term "null string" refers to what we've been calling the "empty string" in technical terms.) The distinction between specifying and specifying can be seen:

FS = "\t"

which causes each tab to be interpreted as a field separator, and:

FS = "\t+"

which specifies that one or more consecutive tabs separate a field. Using the first specification, the following line would have three fields:

abc\t\tdef

whereas the second specification would only recognize two fields. Using a regular expression allows you to specify several characters to be used as delimiters:

FS = "[':\t]"

Any of the three characters in brackets will be interpreted as the field separator.

### Check your Understanding

#### Fill Ups

1. Awk is a \_\_\_\_\_ language used for manipulating data and generating reports.
2. \_\_\_\_\_ executes commands on each line by default.

## Notes

3. FILENAME is a predefined \_\_\_\_\_ that contains the name of the current input file.
4. The data is read in units called \_\_\_\_\_ and processed by your program's rules one record at a time.
5. The \_\_\_\_\_ command is useful because it can take explicit input from any number of files and it does not require that the files used with it be named on the awk command line.

### True or False

1. Awk assumes that its input is structured rather than a random collection of characters.
2. The field separator is set to a single character other than blank when FS is set to a single character other than blank.
3. At the start of the programme, the BEGIN block is executed.
4. Every input line is subjected to AWK commands in the body block.
5. Because END is an AWK keyword, it must be capitalised.

### Summary

- Awk is a utility that enables a programmer to write tiny but effective programs in the form of statements that define text patterns that are to be searched for in each line of a document and the action that is to be taken when a match is found within a line. Awk is mostly used for pattern scanning and processing. It searches one or more files to see if they contain lines that matches with the specified patterns and then perform the associated actions.
- Awk reads all input from the standard input (by default, the keyboard, but it's commonly a pipe from another command) or from files whose names you give on the awk command line in a typical awk programme. If you specify input files, awk reads each one in turn, processing all of the data before moving on to the next. FILENAME is a predefined variable that contains the name of the current input file (see Predefined Variables).
- The getline command is useful because it can take explicit input from any number of files and it does not require that the files used with it be named on the awk command line (see Explicit Input with getline).
- The record separator is a character that separates records. The newline character is used as the record separator by default. This is why records are single-line by default. Simply assign a different character to the predefined variable RS to utilise a different record separator.
- Awk assumes that its input is structured rather than a random collection of characters. Each input line is treated as a record, and each word separated by spaces or tabs is treated as a field. (The characters that separate the fields are known as delimiters.) Three fields are separated by a space or a tab in the following record in the file names

### Activity

1. Create a presentation on awk and its functions.

### Questions and Exercises

1. What are the main purposes of awk?
2. What are the steps involved in working of awk?
3. How is input split in awk?
4. Describe what ARGV is.
5. Explain the printf command.

### Glossary

- **Awk** is a scripting language used for manipulating data and generating reports.
- **FILENAME** is a predefined variable that contains the name of the current input file (see Predefined Variables).
- **ARGV** is a built-in array in the awk programme that contains the command-line arguments

### Further Readings

1. Blum, Richard; Bresnahan, Christine. Mastering Linux System Administration. Sybex; 1st edition (July 27, 2021)
2. Gift, Noah; Jones, Jeremy. Python for Unix and Linux System Administration. O'Reilly Media; 1st edition (September 1, 2008)
3. Hein, Trent; Mackin, Dan; Nemeth, Evi; Snyder, Garth; Whaley, Ben. UNIX and Linux System Administration Handbook. Addison-Wesley Professional; 5th edition (August 8, 2017)
4. Kerrisk, Michael. The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press; 1st edition (October 28, 2010)
5. Negus, Christopher. Linux Bible. Wiley; 10th edition (June 10, 2020)

### Check your Understanding: Answers

#### Fill Ups

1. Scripting
2. awk
3. variable
4. records
5. getline

#### True or False

- |         |         |
|---------|---------|
| 1. True | 2. True |
| 3. True | 4. True |
| 5. True |         |

### Notes

## Notes

### Unit-5.3: System Administration

#### Objectives:

**At the end of this unit, you will be able to understand:**

- Adding and Removing Users Changing Password
- Disk Management, Starting and Shutting Down the System
- File System Mounting and Unmounting
- Monitoring System Usage
- Ensuring System Security

#### Introduction

In terms of computing technology, Linux is a huge asset. Linux is used to power the majority of web servers, mobile phones, personal computers, supercomputers, and cloud servers. A Linux systems administrator's duty is to use Linux tools and command-line interface tools to administer the operations of a computer system, such as maintaining, enhancing, creating user accounts/reports, and taking backups. Because of its great stability, security, and open-source environment, Linux is used to power the majority of computing devices.

#### 5.3.1 Adding and Removing Users, Changing password

Adding and removing users on a Linux system is one of the most important system administration tasks to familiarize yourself with. When you create a new system, you are often only given access to the root account by default.

While running as the root user gives you complete control over a system and its users, it is also dangerous and can be destructive. For common system administration tasks, it is a better idea to add an unprivileged user and carry out those tasks without root privileges. You can also create additional unprivileged accounts for any other users you may have on your system. Each user on a system should have their own separate account.

For tasks that require administrator privileges, there is a tool installed on Ubuntu systems called `sudo`. Briefly, `sudo` allows you to run a command as another user, including users with administrative privileges. In this guide we will cover how to create user accounts, assign `sudo` privileges, and delete users.

#### Adding a User:

If you are signed in as the root user, you can generate a new user at any time by typing:

```
adduser newuser
```

If you are signed in as a non-root user who has been given `sudo` privileges, you are able to add a new user by typing:

```
sudo adduser newuser
```

Either way, you will be asked a series of questions. The procedure will be:

- ◇ Assign and confirm a password for the new user
- ◇ Enter any other information regarding the new user. This is entirely optional and can be skipped by hitting ENTER if you don't wish to utilize these fields.
- ◇ Finally, you'll be asked to verify that the information you provided was correct. Enter Y to continue.

Your new user is now ready for use. You can now log in using the password that you entered.

If you need your new user to have access to administrative functionality, continue on to the following section.

### Granting a User Sudo Privileges :

If your new user should have the ability to execute commands with root (administrative) privileges, you will need to give the new user access to sudo. Let's examine two approaches to this problem: adding the user to a pre-defined sudo user group, and specifying privileges on a per-user basis in sudo's configuration.

### Adding the New User to the Sudo Group:

By default, sudo on Ubuntu 18.04 systems is configured to extend full privileges to any user in the sudo group. You can see what groups your new user is in with the groups command:

```
groups newuser
```

Output

```
newuser : newuser
```

By default, a new user is only in their own group which adduser creates along with the user profile. A user and its own group share the same name. In order to add the user in a new group, use the following:

```
usermod command:
usermod -aG sudo newuser
```

The -aG option here tells usermod to add the user in the listed groups.

### Specifying Explicit User Privileges in /etc/sudoers:

As an alternative to putting your user in the sudo group, you can use the visudo command, which opens a configuration file called /etc/sudoers in the system's default editor, and explicitly specify privileges on a per-user basis.

Using visudo is the only recommended way to create alterations to /etc/sudoers, because it locks the file against multiple simultaneous edits and performs a sanity check on its contents before overwriting the file. This assists to prevent a situation where you misconfigure sudo and are prevented from fixing the problem because you have lost sudo privileges.

If you are currently signed in as root, type:

## Notes

```
Visudo
```

If you are signed in as a non-root user with sudo privileges, type:

```
sudo visudo
```

Traditionally, visudo opened /etc/sudoers in the vi editor, which can be confusing for inexperienced users. By default on new Ubuntu installations, visudo will instead use nano, which provides a more convenient and accessible text editing experience. Use the arrow keys to move the cursor, and search for the line that looks like this:

```
/etc/sudoers
root ALL=(ALL:ALL)
```

ALL Below this line, add the following highlighted line. Be sure to change newuser to the name of the user profile that you would like to grant sudo privileges:

```
/etc/sudoers
root ALL=(ALL:ALL) ALL
newuser ALL=(ALL:ALL) ALL
```

Add a new line like this for each user that should be given full sudo privileges. When you are finished, you can save and close the file by hitting CTRL+X, followed by Y, and then ENTER to confirm. **## Testing Your User's Sudo Privileges**

Now, your new user is able to execute commands with administrative privileges. When signed in as the new user, you can execute commands as your regular user by typing commands as normal:

```
some_command
```

One can perform the same command with administrative privileges by typing sudo ahead of the command:

```
sudo some_command
```

You will be prompted to enter the password of the regular user account you are signed in as.

### Deleting a User:

In the event that you no longer need a user, it is beneficial to delete the old account. You can delete the user itself, without deleting any of their files, by typing the following command as root:

```
deluser newuser
```

If you are signed in as another non-root user with sudo privileges, you could instead type:

```
sudo deluser newuser
```

If, instead, you want to delete the user's home directory when the user is deleted, you can issue the following command as root:

```
deluser --remove-home newuser
```

If you're running this as a non-root user with sudo privileges, you would instead type:

```
sudo deluser --remove-home newuser
```

If you had previously configured sudo privileges for the user you deleted, you might wish to remove the relevant line again by typing:

```
Visudo
```

Or use this if you are a non-root user with sudo privileges:

```
sudo visudo
root ALL=(ALL:ALL) ALL
newuser ALL=(ALL:ALL) ALL # DELETE THIS LINE
```

This will prevent a new user created with the same name from being accidentally given sudo privileges.

### Changing Password

How do I set or change Linux system password for any user account? How can I change user password on Linux operating system using the command-line options?

Both Linux and UNIX-like operating systems use the passwd command to change user password. The passwd is used to update a user's authentication token (password) stored in /etc/shadow file. The passwd change passwords for user and group accounts. A normal user may only change the password for his/her own account, the super user (or root) may change the password for any account. The overseer of a group may change the password for the group. passwd also changes account information, such as the full name of the user, user login shell, or password expiry date and interval.

#### Linux Set User Password

Type following passwd command to change your own password:

```
$ passwd
```

Sample Outputs:

```
Changing password for vivek
(current) UNIX password:
Enter new UNIX password:
Reenter new UNIX password:
passwd: password updated successfully
```

The user is first prompted for his/her old password if one is present. This password is then encrypted and compared against the stored password. There is just one chance for the user to type in the proper password. This step can be skipped by the super user in order to change forgotten passwords. The complexity of a new password is assessed. As a general guideline, passwords should consist of 10 to 20 characters including one or more from each of following sets:

1. Lower case alphabets



## Notes

2. Upper case alphabetics
3. Digits 0 thru 9
4. Punctuation marks/spacial characters

Linux change password for other user account

You need to login as the root user, type the following command to change password for user vivek:

```
passwd vivek
```

OR

```
$ sudo passwd vivek
```

Sample putput:

```
Enter new UNIX password:
Reenter new UNIX password:
passwd: password updated successfully
```

Where,

◇ vivek – is username or account name.

Passwords do not display to the screen when you enter them. For example:

```
[vivek@nixcraft-asus ~]$ passwd
Changing password for user vivek.
Current password:
New password:
Retype new password:
passwd: all authentication tokens updated successfully.
[vivek@nixcraft-asus ~]$
[vivek@nixcraft-asus ~]$
```

Linux changing user password using passwd

Linux Change Group Password

When the **-g** option is entered, the password for the named group is changed. In this example, change password for group sales:

```
passwd -g sales
```

The current group password is not prompted for. The **-r** option is used with the **-g** option to remove the current password from the named group. This allows group access to all members. The **-R** selection is used with the **-g** option to restrict the named group for all users.

Changing user passwords on Linux

As a Linux system administrator (sysadmin) you can change password for any users on your server. To change a password on behalf of a user:

1. First sign on or “su” or “sudo” to the “root” account on Linux, run: **sudo -i**
2. Then type, **passwd tom** to change a password for tom user

3. The system will issue a prompt for you to enter a password twice

To change or set a new root (superuser) password type:

```
$ sudo passwd
```

Forcing Linux user to change password at their next login

By default, Linux passwords never expire for users. We can, however, utilise GUI or CLI techniques to require users to update their password the next time they log in. The syntax is straightforward:

```
$ sudo passwd -e {username}
```

```
$ sudo passwd --expire {username}
```

Let us immediately expire an account's password:

```
$ sudo passwd -e Marlana
```

The system will confirm it:

```
passwd: password expiry information changed.
```

When a user attempts to log in using the ssh command, the following message appears on the screen:

```
$ ssh marlena@192.168.2.25
Last login: Fri Dec 4 04:12:48 2020 from 192.168.2.105
WARNING: Your password has expired.
You must change your password now and login again!
Changing password for marlena.
Current password:
New password:
Retype new password:
passwd: password updated successfully
Connection to 192.168.2.25 closed.
```

Locking and Unlocking user password of the named account

Note that the following `local` command does not disable the account. The user may still be able to login using another authentication token, such as an SSH key. To disable the account, administrators should use either **`usermod --expiredate 1 {username}`** or **`sudo passwd --expire {username}`** command. Also, users with a locked password are not allowed to change their password to get around the security policy set by sysadmin.

We can lock the password as follows:

```
$ sudo passwd -l {username}
```

This choice deactivates a password by changing it to a value which matches no possible encrypted value (it adds a `!` at the start of the password in the `/etc/shadow` file. Want to unlock the password, try:

```
$ sudo passwd -u {username}
```

## Notes

The above command option re-enables a password by changing the password back to its previous value. Basically, it changes the value before using the -l option.

### 5.3.2 Disk Management, Starting and Shutting down the System

#### Disk Management:

Disk management includes several important tasks such as adding or removing storage devices, creating and deleting partitions, mounting partitions on appropriate directories and making file system in partitions.

#### DISK PARTITIONING:

Partitioning and logical volume management are both methods for partitioning a disc (or, in the case of LVM, a pool of drives) into distinct portions of known size. Although all of our sample systems offer logical volume management, traditional partitioning is only available on Linux, Solaris, and occasionally HP-UX.

You can use a logical volume manager to handle individual partitions, but you can't partition a logical volume. Partitioning is the most fundamental kind of disc management.

Partitioning is required but largely redundant on Solaris; ZFS hides it so well that you might not even notice it's happening. This section offers some broad background material that may be valuable to Solaris administrators, however the Solaris path differs significantly from that of Linux, HP-UX, and AIX in terms of procedure. For further information, go to ZFS: All Your Storage Problems Solved on page 264. (Or don't: zpool create newpool newdevice will take care of the basics.)

Partitions and logical volumes simplify backups, prevent users from stealing each other's disc space, and limit the damage that runaway programmes might cause. Every system has a root "partition" that contains / and most of the configuration data for the local host. In theory, the root partition contains everything required to bring the system up to single-user mode. Various subdirectories (most often /var, /usr, /tmp, /share, and /home) can be partitioned or divided into their own volumes. At least one exchange area is included in most systems.

The optimum way to divide drives is a matter of opinion, as are the settings employed by many operating systems. Here are a few general guidelines to help you:

- ◇ It's a good idea to have a backup root partition that you can boot to if the primary root partition fails. The backup root should ideally be on a separate disc than the main root, so it can guard against both hardware failures and corruption. Even a backup root on the same drive, however, has some value.
- ◇ Check to see if you can boot from the backup root. The operation is frequently complicated. To get everything operating smoothly, you may need to give the kernel special boot-time arguments and make minor configuration changes within the alternate root itself.
- ◇ Because the root partition is frequently copied, it should be small enough that two copies don't take up too much disc space. This is why /usr is frequently a distinct drive; it contains the majority of the system's libraries and data.
- ◇ Putting /tmp on its own filesystem keeps transient files to a manageable size

and eliminates the need to back them up. For performance reasons, some systems store /tmp on a memory-based filesystem. Because memory-based filesystems are still backed up by swap space, they can be used in a variety of scenarios.

- ◇ Because log files are placed under /var, having a separate disc partition for /var is a smart idea. It's easy to fill the root and bring the machine to a halt by leaving /var as part of a small root partition
- ◇ Users' home directories should be stored on a separate partition or drive. User data has a decent chance of surviving even if the root partition is corrupted or erased. In contrast, even if a user's erroneous shell script fills up /home, the system can continue to function.
- ◇ Performance is improved by distributing swap space across multiple physical discs. Put the busy ones on other discs; this strategy also works for filesystems. Notes on this subject can be found on page 1129.
- ◇ You should also add swap space to your machine as you add memory. More information on virtual memory can be found on page 1124.
- ◇ If the entire partition can fit on one piece of media, backups of that partition may be simplified.
- ◇ Attempt to consolidate constantly changing data on a few frequently backed-up partitions.

### Traditional partitioning:

Partitions are implemented in systems that enable them by placing a "label" at the start of the disc to define the range of blocks contained in each partition. The label must frequently cohabit with additional startup information (such as a boot block), and it frequently carries extra information such as a name or unique ID that identifies the disc as a whole. The label is known as the MBR, or master boot record, in Windows.

The device driver in charge of representing the disc reads the label and calculates the actual location of each partition using the partition table. Each partition is usually represented by one or two device files (one block device and one character device; Linux has only block devices). A separate set of device files also reflects the entire disc.

Solaris refers to partitions as "slices," or more precisely, slices when implemented with a Solaris-style label and partitions when implemented with a Windows-style MBR. Slice 2 spans the entire disc, demonstrating the terrifying reality that a single disc block can be claimed by many slices. Perhaps "slices" was chosen over "partition" because partition implies a simple divide, but slices can overlap. Aside from that, the terms are interchangeable.

Despite the widespread availability of logical volume managers, classical partitioning is still required or beneficial in some scenarios.

- ◇ The boot disc on PC hardware must contain a partition table. MBR partitioning is required by most systems (see Windows-style partitioning below), however GPT partitioning is required by tanium systems. Unpartitioned data discs are possible.
- ◇ Using a Windows-style MBR allows Windows to understand the disc even if the contents of the individual partitions are not. You'll need to install a Windows MBR if you want to work with Windows (for example, by dual

## Notes

booting). Even if you have no plans to do so, it's worth considering Windows' pervasiveness and the chance that your disc may come into touch with it at some point.

Windows 10 is a well-behaved operating system that would never write at random to a disc it can't read. They will, however, strongly advise any administrator who logs in to take this course of action. A helpful "OK, mess up this disc!" button is included in the dialogue box. 10 Unless someone commits a mistake, nothing horrible will happen, yet safety is a structural and organisational process.

- ◇ Partitions have a specific position on the disc and ensure reference locality. Logical volumes are not the same as physical volumes (at least, not by default). This fact isn't really significant in most circumstances. Short seeks, on the other hand, are faster than long seeks, and the throughput of a disk's outer cylinders (those storing the lowest-numbered blocks) can be 30 percent or more faster than the throughput of its inner cylinders. You can use partitioning to gain an extra edge in situations where every ounce of performance counts. (You can always utilise logical volume management to reclaim some of the lost flexibility within partitions.)
- ◇ RAID systems make use of discs or partitions that are the same size. Although a RAID implementation may accept entities of various sizes, it will almost certainly only use the block ranges that all devices share. You can isolate surplus space in a separate partition rather than letting it go to waste.

### Windows-style partitioning:

The Windows MBR takes up a single 512-byte disc block, with boot code consuming the majority of it. There is only enough room to create four partitions. Because they are defined directly in the MBR, these are referred to as "primary" partitions. One of the primary partitions can be designated as a "extended" partition, which means it has its own auxiliary partition table. The expanded partition is a real partition that takes up a specific physical space on the disc. The subsidiary partition table is stored at the start of the data for that partition.

Secondary partitions are those that you construct within the extended partition. They are subsets of the extended partition in their own right.

When setting up Windows-partitioned discs, keep the following guidelines in mind. The first is a set of guidelines. The others are simply included because some BIOSes, boot blocks, or operating systems may require them.

- ◇ On a disc, there can only be one extended partition.
- ◇ The extended partition should be the last of the partitions defined in the MBR; no primary partitions should come after it.
- ◇ Some older operating systems aren't fond of being put on second partitions. Stick to primary partitions for OS installations to avoid problems.

One partition can be designated as "active" in Windows' partitioning mechanism. The boot loader searches for the active partition and attempts to load the operating system from there.

A one-byte type attribute is also assigned to each partition, which is designed to indicate the partition's contents. In most cases, the codes denote filesystem types or

operating systems. Although these codes are not given centrally, several common norms have developed over time. Andries E. Brouwer summarises them at [tinyurl.com/part-types](http://tinyurl.com/part-types).

The MS-DOS command that partitioned hard disks was called fdisk. Most operating systems that allow Windows-style partitions have taken this nomenclature for their own partitioning commands, but fdisks differ greatly. Windows has progressed: in newer versions, the command-line tool is known as diskpart. Windows also offers a partitioning GUI that can be accessed via the mmc Disk Management plug-in.

It makes no difference whether you use Windows or another operating system to partition a disc. The outcome is the same.

### **GPT: GUID partition tables**

The extensible firmware interface (EFI) effort by Intel intends to replace PC BIOSes' shaky protocols with a more current and effective architecture. Although full EFI firmware systems are still rare, EFI's partitioning method has acquired general acceptance among operating systems. The fact that MBR does not handle discs larger than 2TB is the major reason for its success. Since 2TB discs are now commonly available, this issue has taken on a new urgency.

The EFI partitioning system, referred to as a "GUID partition table" or GPT, eliminates MBR's glaring flaws. It simply describes one sort of division, but you can make as many as you like. Each partition has a type defined by a 16-byte ID code (the globally unique ID, or GUID) that does not need to be arbitrated centrally.

By dragging an MBR as the first block of the partition table, GPT maintains rudimentary compatibility with MBR-based systems. This "fake" MBR gives the impression that the disc is occupied by a single huge MBR partition (at least, up to the 2TB limit of MBR). The counterfeit MBR isn't particularly useful, but the goal is that it will deter naive computers from attempting to reformat the drive.

GPT drives are supported for data in Windows since Vista, but only PCs with EFI firmware may boot from them. GPT discs are supported by the OS and bootable on any system, thanks to Linux and its GRUB boot loader. Both EFI and GPT partitioning are used on Intel-based Mac OS computers. GPT partitioning is supported by Solaris, and ZFS defaults to it. Solaris boot discs, on the other hand, cannot employ GPT partitioning.

Despite the fact that GPT has been widely adopted by operating system kernels, its use by disc management utilities is still patchy. GPT is still considered a "bleeding edge" format. On discs that don't require it, there's no compelling reason to use it (that is, discs 2TB in size or smaller).

### **Linux partitioning:**

Linux systems have a variety of partitioning options. fdisk is a straightforward command-line partitioning utility. GNU's parted is a more advanced command-line utility that can move and resize partitions in addition to creating and removing them. It also recognises numerous label formats (including Solaris' native one). GNOME has a GUI version called gparted. Another option is to use cfdisk, which is an excellent terminal-based replacement for fdisk.



## Notes

Although parted and gparted may potentially expand a variety of filesystems and the partitions that comprise them, the project's home page defines this capability as "buggy and unreliable." Filesystem-specific utilities are likely to perform better when modifying filesystems, yet parted lacks a command to "resize the partition but not the filesystem.". If this isn't what you're looking for, go back to fdisk.

We prefer gparted to parted in general. Both are straightforward, but gparted allows you to define the partition size rather than the starting and ending block ranges. Most distributions' graphical installers are the best method for dividing the boot disc since they often advise a partitioning scheme that fits well with that distribution's layout.

### **Solaris partitioning:**

ZFS applies a GPT partition table to drives and labels them for you. The format command, on the other hand, can be used to manually partition discs. An fdisk command is also available on x86 systems. Both user interfaces are menu-driven and simple to use.

You can choose from a nice list of discs with format, but fdisk requires you to specify the disc on the command line with fdisk. Format provides a fdisk command that runs fdisk as a subprocess, so you can use it as a wrapper to help you choose the proper disc.

Windows MBR, GPT, and old-style Solaris partition tables, known as SMI, are all supported by Solaris. Depending on the hardware and whether you're running Solaris or OpenSolaris, you'll need to use MBR or SMI for the boot disc. For all manually partitioned discs under 2TB, it's probably preferable to stick to these alternatives for the time being.

### **HP-UX partitioning:**

Disk partitioning is only used by HP on Itanium (Integrity) boot discs, which require a GPT partition table and an EFI boot partition. Prints and builds partition tables with the idisk command. Instead than being an interactive partitioning tool, it reads a partitioning plan from a file or standard input and uses it to build the partition table.

The idisk partitioning standard is thankfully simple. The first line just provides a number that indicates how many divisions should be created. Each line after that has a partition type (EFI, HPUX, HPDUMP, or HPSP for swap), a space character, and a size specification like 128MB or 100%. When a percentage is used, it refers to the amount of space left on the drive after the previous partitions have been allotted.

### **RAID: REDUNDANT ARRAYS OF INEXPENSIVE DISKS:**

Even if you have backups, a disc failure on a server can be devastating. RAID, or "redundant arrays of cheap discs," is a data distribution or replication method that uses numerous discs to distribute or reproduce data. RAID not only prevents data loss, but it also reduces (often to zero) the downtime associated with hardware failures while potentially increasing speed.

RAID can be built using specialised hardware that exposes a group of hard drives as a single composite drive to the operating system. It can also be accomplished simply by having the operating system read or write several discs in accordance with RAID principles.



### Software vs. hardware:

**RAID** Because the discs are always the most significant bottleneck in a RAID implementation, there's no reason to believe that a hardware-based RAID implementation will be faster than a software- or OS-based RAID implementation. Due to a lack of software alternatives (no direct OS support for RAID) and hardware's ability to buffer writes in some form of nonvolatile memory, hardware RAID has dominated in the past.

Because it makes writes appear to complete instantly, the latter feature improves performance. It also guards against a potential corruption problem known as the "RAID 5 write hole," which we go over in depth on page 241. However, be aware that many "RAID cards" advertised for PCs include no nonvolatile memory at all; they are essentially glorified SATA ports with RAID software preinstalled. RAID implementations on PC motherboards are also included in this category. On these systems, you're far better off using the RAID facilities in Linux or OpenSolaris.

On a critical production server, we recently had a disc controller failure. Despite the fact that the data was replicated across numerous physical drives, the data on all discs was erased due to a malfunctioning hardware RAID controller. It took more than two months for the server to fully recover after a lengthy and unpleasant tape restore operation. The rebuilt server now manages its RAID environment using kernel software, removing the risk of another RAID controller failure.

### RAID levels:

RAID can perform two primary functions. First, it can boost speed by "striping" data across several drives, allowing numerous drives to supply or absorb a single data stream at the same time. Second, it can replicate data across numerous drives, reducing the danger of a single disc failing.

Mirroring, in which data blocks are replicated bit-for-bit on several distinct drives, and parity schemes, in which one or more drives contain an error-correcting checksum of the blocks on the remaining data drives, are the two primary types of replication. Mirroring is faster, but it takes up more space on the hard drive. Parity schemes use less storage space but provide less performance.

RAID is generally stated in terms of "levels," which define the technical features of an array's parallelism and redundancy. Because "higher" levels are not always "better," the word may be deceptive. The tiers are merely different setups; choose the one that best suits your needs.

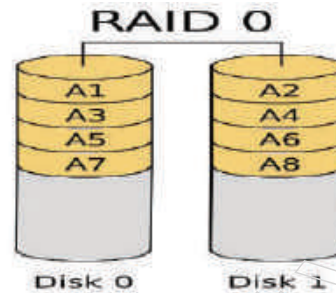
Numbers denote stripes, while letters a, b, and c denote data blocks inside a stripe in the diagrams below. Parity blocks are indicated by the letters p and q.

- ◇ Linear mode, commonly known as JBOD (short for "just a bunch of disks"), is not a true RAID level. Despite this, it appears that every RAID controller supports it. JBOD joins the block addresses of several drives to form a single, bigger virtual disc. It doesn't add any data redundancy or improve performance. Nowadays, a logical volume manager, rather than a RAID controller, is the ideal way to get JBOD capability.
- ◇ RAID level 0 is only used to boost performance. It combines two or more drives of the same size, but instead of stacking them end-to-end, it striped

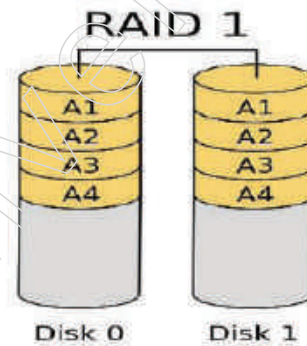
## Notes

data across the discs in the pool in a random order. As a result, sequential reads and writes are distributed across several discs, reducing write and access times.

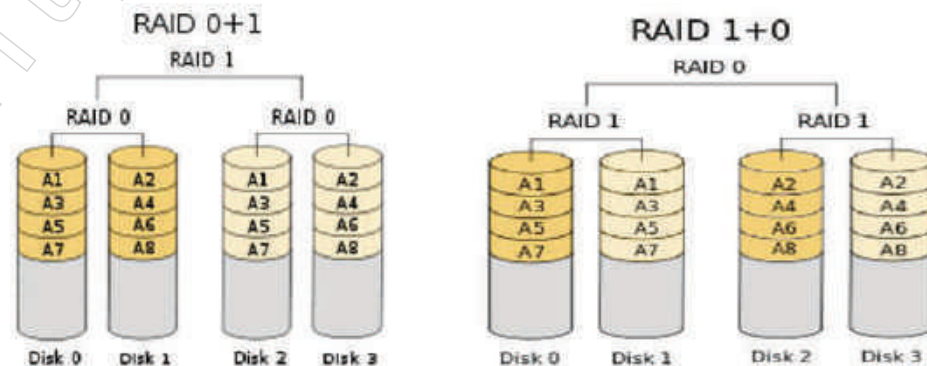
It's worth noting that RAID 0 has much lower dependability than individual discs. The annual failure rate of a two-drive array is nearly double that of a single drive, and so on.



- ◇ Mirroring is a common term for RAID level 1. Writes are simultaneously copied to two or more drives. Because of this configuration, writes are slightly slower than they would be on a single drive. Because reads may be farmed out among the numerous duplicate disc drives, it delivers read speeds comparable to RAID 0.

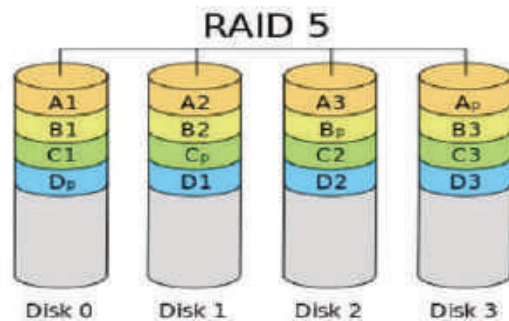


- ◇ RAID levels 1+0 and 0+1 are stripes of mirror sets or mirrors of stripe sets. Although they are logically concatenations of RAID 0 and RAID 1, several controllers and software implementations support them directly. The purpose of both modes is to achieve RAID 0 performance and RAID 1 redundancy at the same time.

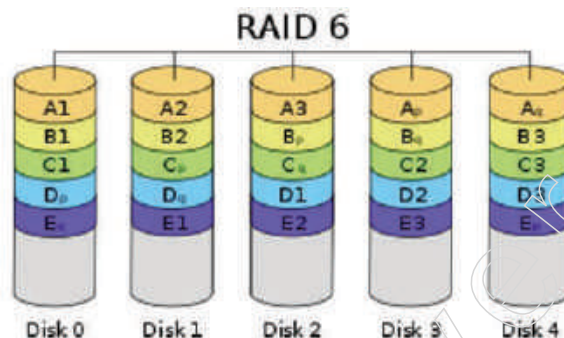


- ◇ RAID level 5 adds redundancy while enhancing read performance by striping both data and parity information. Furthermore, RAID 5 utilises disc space more efficiently than RAID 1. An array of N drives (at least three are

necessary) can store data on  $N-1$  of them. RAID 5 has a space efficiency of at least 67 percent, whereas mirroring has a space efficiency of no more than 50 percent.



- ◇ With two parity disks, RAID level 6 is comparable to RAID 5. Without losing data, a RAID 6 array can endure the full failure of two drives.



Levels 2, 3, and 4 of RAID are defined, however they are rarely used. Striping (RAID 0) and mirroring (RAID 1) are common features in logical volume managers.

Linux gives you the option of using a dedicated RAID system or the logical volume manager for simple striped and mirrored configurations. The LVM technique is likely to be more flexible, whilst the md approach is likely to be more rigidly predictable. You may still use LVM to manage the space on the RAID volume if you choose md. RAID 5 and RAID 6 software RAID must be implemented with md.

Solaris' ZFS system, which combines a RAID system, logical volume management, and filesystem, allows striping, mirroring, and configurations equivalent to RAID 5 and RAID 6. Mirroring and parity arrangements are done at the lowest level in the ZFS design, whereas striping is done per storage pool (one level up) and is automatic. This is a good method to arrange the features because it keeps the RAID configuration clear.

On HP-UX and AIX, logical volume management is the extent of OS-level RAID capabilities. (HP even requires you to buy the mirroring feature separately, despite the fact that it is included in some enterprise setups.) You'll need some more hardware if you want a parity-based system. However, AIX already has facilities for managing RAID hardware built in: see Disk Array under Devices in SMIT.

#### Disk failure recovery:

In most production systems, the Google disc failure study referenced on page 211 should be rather strong evidence of the requirement for some type of storage redundancy. With an annual failure rate of 8%, your company only needs 150 hard discs in service to predict one failure per month on average.

## Notes

When hardware failure occurs, JBOD and RAID 0 modes are useless; you must manually retrieve your data from backups. Other types of RAID go into a degraded mode, marking the problematic devices as faulty. From the standpoint of storage clients, the RAID arrays continue to work normally, albeit at a lower rate.

To restore array redundancy, bad discs must be replaced as soon as feasible with fresh ones. A single device failure can only be tolerated by a RAID 5 or two-disk RAID 1 array. The array is vulnerable to a second failure once that failure has happened.

The specifics of the procedure are usually straightforward. You replace the failed disc with a similar or larger one, then instruct the RAID implementation to replace the old with the new. The parity or mirror information is then rewritten to the fresh, blank disc over a long period of time. This is frequently an overnight operation. During this time, clients can still access the array, but performance is likely to be poor.

Most RAID implementations allow you to select one or more discs as “hot” spares to reduce downtime and the array’s vulnerability to a second failure. When a disc fails, it is automatically replaced with a spare, and the process of resynchronizing the array begins right away. Hot spares should be utilised as soon as possible if they are available.

The technique for starting and stopping the operating system, as well as a description of:

- ◇ Boot operation
- ◇ Different startup states and the corresponding boot preparation
- ◇ Run levels
- ◇ Resolving problems that occur during the boot operation

Shutting down the computer is a basic action that should be done on a regular basis. Normally, you can shut down the system quickly and with little impact on the consumers. You may need to quickly shut down the system on occasion, creating some user interruption. The system abruptly shuts down under certain conditions (out of your control), causing significant interruption to users.

### Understanding the Boot Operation:

When you boot the operating system, you initiate a set of tasks that the system must perform to operate successfully. The system is vulnerable during startup since it is loading the kernel into memory and initializing routines that it depends on for operation. Consequently, you should understand what is happening during the system boot, and be prepared to respond if problems occur.

Although certain boot operations are hardware dependent, some features typically apply to all systems.

For example:

### The system always boots either automatically or manually.

In an automatic boot, the system controls the entire operation. You rely on an automated boot when you boot into multiuser mode, power down the system using the reboot flag, or when the system panics and recovers. With an automatic boot, the system begins the initialization process and continues until completion or failure.

Manual intervention may be required if the automatic boot fails for some reason, for example, if the fsck command fails.

In a manual boot, the system controls the initial operation, turns control of the procedure over to you, then reinstates control to complete the operation. When you boot the system to single-user mode, you are relying on a manual boot. In an automatic or a manual boot, the operation either succeeds or fails:

- ◇ If the boot operation succeeds, the system is initialized. In single-user mode, the system displays the root prompt (#) on the console or on the terminal screen. In multiuser mode, the system displays the login prompt or a startup display. The prompt or startup display differs according to hardware capability and available startup software.
- ◇ If the boot operation fails, the system displays an error message followed by a console firmware prompt (>>>). In the worst case, the system hangs without displaying a console prompt.

**The system boots to either single-user or multiuser mode.**

- ◇ In a boot to single-user mode, the software loads the kernel and proceeds through the initialization tasks associated with process 0 (initialization) and process 1 (init). The init program creates a Bourne shell (sh), turns control over to you, and waits for you to exit from the shell with the exit command or Ctrl/d before continuing with its startup tasks.

Because init does not invoke the startup script prior to turning control over to you, the root file system is mounted read only, startup of the network and other daemons does not occur, file checking and correction are not enabled, and other operations necessary for full system use are not automatically available to you.

Usually you boot to single-user mode to perform specific administrative tasks that are best accomplished without the threat of parallel activity by other users. You perform these tasks manually before exiting from the Bourne shell. For example, you might check new hardware, mount and check aberrant file systems, change disk partitions, or set the system clock. When you finish your work, you return control to the system, and init continues with its startup tasks and boots to multiuser mode.

- ◇ In a boot to multiuser mode, the system loads the kernel and moves through various phases such as hardware and virtual memory initialization, resource allocation, scheduling, configuration, module loading, and so on. At the conclusion of the main initialization tasks (process 0), init (process 1) starts an additional set of tasks that includes reading the /etc/inittab file, acting on instructions found there, and executing the relevant run command scripts. These scripts contain entries that initiate activities such as mounting and checking file systems, removing temporary files, initializing the clock daemon, initializing the network daemon, setting up printer spooling directories and daemons, enabling error logging, and performing other tasks specified within the scripts or in related directories. At the conclusion of these activities, the system is enabled and accessible to users.

The Digital UNIX operating system allows you to boot an alternate kernel. For example, if you cannot boot your system, you could boot /genvmunix to troubleshoot



## Notes

the problem with your system. You could also boot an alternate kernel to test new drivers or to add options to the existing kernel.

### Preparing to Boot the Installed System:

As the system administrator, you set up or encounter various preboot or postshutdown states. This section describes and recommends procedures for preparing and initiating a reboot from a variety of system states. The states discussed here include the following:

- ◇ A powered-down system
- ◇ A powered-up, halted system
- ◇ A powered-up system in single-user mode
- ◇ A crashed system

### Note

If the system is running in single-user mode and you want to use the ed editor, you must change the protections of the root file system to read-write. At the prompt, enter the following command:

```
mount -u /
```

### Preparing to Boot a Powered-Down System:

A system is powered down when the hardware (processor, devices, and peripherals) is turned off. Administrators power down the hardware periodically for routine maintenance or to configure new devices.

If you are preparing to reboot your system from a powered-down state, follow these steps:

1. Confirm that the hardware and all peripheral devices are connected. Refer to the operator's guide for your hardware for information and instructions for interpreting diagnostic output.
2. Power up the hardware and peripheral devices. Remember to power up all devices that you powered down earlier. Refer to the operator's manual or the hardware user's guide for instructions on starting your hardware and peripherals.
3. Confirm that the hardware completed its restart and diagnostic operations. Most hardware provides a diagnostic check as a routine part of its startup operation. Refer to the operator's manual for your hardware for information about your hardware's restart and diagnostic operations.
4. Wait for the console prompt (>>>). If you have enabled your system to boot automatically upon powerup, press the halt button to display the console prompt. Refer to the hardware operator's guide for the location of the halt button on your system.
5. Decide which startup mode you want to initiate:
  - ◆ If you have tasks you need to accomplish and want the system to restrict access to all users but root, plan to boot to single-user mode.

- ◆ If you do not require single-user access and you want the system to initialize full functionality, plan to boot to one of the multiuser modes: multiuser without networking or multiuser with networking.
6. Enter the boot command that corresponds to the desired startup mode.

### Preparing to Boot a Powered-Up, Halted System:

When your machine is powered up and enabled but the processor is halted, the system is in console mode. For example, after you shut down the processor with the shutdown -h command or when you run the halt command, your system displays the console prompt (>>>).

When the system displays the console prompt, follow these steps to prepare to boot your system:

1. Decide which startup mode you want to initiate:
  - ◆ If you have tasks you need to accomplish and you want the system to restrict access to all users but root, plan to boot to single-user mode.
  - ◆ If you do not require single-user access and you want the system to initialize full functionality, plan to boot to one of the multiuser modes: multiuser without networking or multiuser with networking.
2. Enter the boot command that corresponds to the desired startup mode.

### Preparing to Transition from Single-User Mode:

When your machine is powered up and enabled, the processor is running, and access is limited to root, the system is in single-user mode.

When the system displays the single-user prompt (#), follow these steps to prepare to go to multiuser mode:

1. Decide if you should continue in single-user mode or if you should go to multiuser mode:
  - ◆ If you have additional tasks that you need to perform and you want the system to deny access to all users but root, plan to continue in single-user mode.
  - ◆ If you do not require single-user access, or if you have completed your tasks and you want the system to initialize full functionality, plan to go to one of the multiuser modes: multiuser without networking or multiuser with networking.
2. When you are ready to go to multiuser mode, press Ctrl/d. Refer to Section 3.3 for the commands and procedures required to boot your system.

### Preparing to Boot a Crashed System:

If your system crashes and is unable to recover automatically and reboot itself, follow these steps to prepare to boot the system:

1. Confirm that the hardware and all peripheral devices are connected.
2. Power up the hardware, if necessary. Always power up peripherals and devices before the processor.



## Notes

3. Monitor the hardware restart and diagnostic operations. Refer to the operator's guide for your hardware for information and instructions for interpreting diagnostic output.
  - ◆ In the unlikely event that the diagnostic test indicates hardware failure, contact your Digital field representative. Because hardware damage is a serious problem, do not continue or try to bypass the defective hardware.
  - ◆ If you have enabled your system to boot automatically, press the halt button to display the console prompt. Refer to the hardware operator's guide for the location of the halt button on your system.
4. Decide which startup mode you want to initiate:
  - ◆ If you need to deny access to all users but root, plan to work in single-user mode. After a crash, it is wise to work initially in single-user mode. You should check all file systems thoroughly for inconsistencies and perform other postcrash operations before enabling system access to other users.
  - ◆ If you need to allow access to you and to all other users with login permission, plan to boot to one of the multiuser modes: multiuser without networking or multiuser with networking.
5. Enter the required boot command.

### Booting the System:

The command that you use to boot the kernel depends on several factors:

- ◇ Processor type
- ◇ Run level
- ◇ Location of the kernel that you are booting (on the system disk or on a remote server)
- ◇ Whether you are booting all processors or a single processor (in a multiprocessor system)
- ◇ Whether any console environment variables are defined
- ◇ Whether you are booting the default kernel or an alternate kernel

### Defining the Console Environment Variables and Using the Boot Commands

To boot your system, you need to understand the use of certain console environment variables and their role in affecting the boot process. Below table lists each of the console environment variables and their associated actions. If you are booting a DEC 2000 processor, refer to the hardware manual that accompanied the processor for information on boot commands.

**Table: Console Environment Variables**

Variable	Action
boot_reset	When set to on, resets the hardware on boot
boot_osflags	A combination of flags used to control the boot loader and kernel
bootdef_dev	Identifies the boot device

boot_file	Identifies the kernel to boot (on DEC 4000 and DEC 7000 processors)
cpu_enable	Selectively enables particular processors from the console

To prepare the hardware:

1. Set the auto\_action variable to halt:

```
>>> set auto_action halt
```

The previous command will halt the system at the console prompt each time your system is turned on, when the system crashes, or when you press the halt button.

2. For DEC 3000 and DEC 7000 processors, set the boot\_reset variable to on to force the resetting of the hardware before booting:

```
>>> set boot_reset on
```

3. For DEC 3000 processors, set the time to wait to reset the SCSI device before booting:

```
>>> set scsi_reset 4
```

4. Use the following procedure to set the boot\_osflags variable and the boot device:

- a. Determine which options to the boot\_osflags variable you want. Below table lists the options.

**Table: Options to the boot\_osflags Variable**

Option	Action
a	Boot to multiuser mode. (By default, the kernel boots to single-user mode.)
k	Use the kdebug debugger to debug the kernel. (By default, kdebug is not used.)
d	Use full crash dumps. (By default, partial dumps are used.)
i	Prompt for the kernel and special arguments. (By default, no questions are asked.)

The options are concatenated into the boot\_osflags variable to achieve the desired effect. For example, to boot to multiuser mode and use full crash dumps, enter:

```
>>> set boot_osflags ad
```

If you want the defaults, clear the variable as shown in the following example:

```
>>> set boot_osflags ""
```

- b. Determine the unit numbers for your system's devices:

```
>>> show device
```

If you want to boot from the dual SCSI TURBOchannel option card (PMAZB or PMAZC), complete the following steps:

- ◆ Identify the slot number of the PMAZB or PMAZC option card:

## Notes

```
>>> show conf
```

The previous command displays the system configuration.

◇ Determine the unit number of your system's devices:

Use the conf command with the slot number to identify the unit numbers of the devices attached to that controller. For example, to look at the devices attached to the controller in slot 1, enter:

```
>>> t tc1 cnfg
```

A display appears identifying the unit number of each device attached to that controller. Identify the unit number of the device from which you want to boot.

### Set the default boot device:

By default, you must provide a boot device when you boot your system. If you always boot from the same device, use the following command syntax with the bootdef\_dev variable to set a default boot device:

```
set bootdef_dev device
```

For example, to boot the system off of disk dka0, enter:

```
>>> set bootdef_dev dka000
```

To boot the system from the first disk on the PMAZB or PMAZC option card in TURBOchannel slot 1, enter the following command. Note that the double quotes (") are necessary for the console to understand where it is booting from.

```
>>> set bootdef_dev "1/dka000"
```

You have the option of booting from an alternate kernel. If you want to do this, enter:

```
>>> set boot_osflags i
```

When booting, the system prompts you to enter a file name. For example:

```
Enter [kernel_name] [option_1 ... option_n]: genvmunix
```

The system will display informational messages.

On DEC 4000 and DEC 7000 processors, you can also boot an alternate kernel by setting the boot\_file variable to the name of the kernel you want to boot. For example, to boot a genvmunix kernel, enter:

```
>>> set boot_file genvmunix
```

On DEC 4000 and DEC 7000 processors, you must clear the boot\_file variable if you want to boot the default kernel, /vmunix. For example:

```
>>> set boot_file ""
```

In a multiprocessor configuration, you can use the set cpu\_enable command to selectively enable processors from the console. The mask is a bit field, where each bit represents a slot position. The easiest way to ensure all processors are enabled is to set the CPU mask to ff. After setting the mask, turn the power on the system off and then back on again.

The operating system also provides a mechanism for enabling or disabling processors at system boot time. See the description of the `cpu-enable-mask` attribute in the System Tuning and Performance Management guide for information.

After you have set the console variables, use the following command to boot the system:

```
>>> b
```

### Overriding the Boot Commands:

The previous section described how to set the boot commands. This section describes how to override those commands.

#### ◇ Overriding `bootdef_dev`

To override the `bootdef_dev` variable, supply the desired boot device as an argument to the boot command. For example, if your boot device is set to boot from disk `dka0` and you want to boot from disk `dkb0`, enter:

```
>>> b dkb0
```

#### ◇ Overriding `boot_osflags`

The `boot_osflags` variables are ignored if you specify the `-fl` option to the boot command, as follows:

```
>>> b -fl
```

To override the `boot_osflags` variables, pass the desired choices to the `-fl` option. For example, the following command boots to the interactive prompt so you can specify an alternate kernel, and then boots to multiuser mode:

```
>>> b -fl ai
```

#### ◇ Overriding `boot_file`

To boot a kernel other than that specified by `boot_file`, enter the boot command with the following syntax:

```
b -fi kernel
```

For example, to boot the `genvmunix` kernel, enter:

```
>>> b -fi genvmunix
```

### Identifying the System Run Levels:

A run level (mode) specifies the state of the system and defines which processes are allowed to run at that state. There are four basic run levels available, as follows:

0	Specifies the halt state
S or s	Specifies single-user mode
2	Specifies multiuser mode without network services
3	Specifies multiuser mode with network services
	Specifies the console mode

## Notes

The inittab file contains line entries that define the specific run levels and the run command scripts that are associated with the run level. When the init process starts, it reads the inittab file and executes the relevant run command scripts. The scripts, in turn, define which processes are to run (and which processes are to be killed if the system is changing from one level to another) at a specific run level.

### Changing the System Run Levels:

Before changing to a new run level, check the inittab file to confirm that the run level to which you intend to change supports the processes you need. Of particular importance is the getty process since it controls the terminal line access for the system console and other logins. Ensure that the getty entry in the inittab file allows system console access at all run levels.

Before changing to a new run level, use the wall or write command to warn users that you intend to change the run level. Since a change in run level could result in termination of the user's getty process (which disables their login capability) as well as termination of other processes that they are running, you should communicate the change to each logged in user. Check the getty entry for user terminals to confirm that the new run level is stated in the entry. If it is not, request that users log off so that their processes will not terminate in response to a kill signal from init.

When the system is initialized for the first time, it enters the default run level that is defined by the initdefault line entry in the inittab file. The system continues at that run level until init receives a signal to change run levels. The following sections describe these signals and provide instructions for changing run levels.

### Changing Run Levels from Single-User Mode:

Use the Bourne shell when working in single-user mode and press Ctrl/d to change run levels. The shell terminates in response to Ctrl/d and displays the following message if transitioning from single-user mode to multiuser mode during a boot operation:

INIT: New run level: 3

If this transition is made from single-user mode with the previous state having been multiuser mode, then a prompt is issued for input of the desired run level. The init process searches the inittab file for entries (at the new run level) with the boot or bootwait keywords, and then acts on these entries before it continues with the normal processing of the inittab file. The init process next scans the file for other entries with processes that are allowed to run at the new run level, and then acts on these entries.

### Changing Run Levels from Multiuser Mode:

You can use the init command to alter run levels when the system is running at one of the two multiuser run levels. Log in as root and use the following syntax to run the command:

init [0 | s | 2 | 3 | q]

The init command invokes the following run levels:

0	Specifies the halt state
s	Specifies the single-user run level
2	Specifies a multiuser run level with local processes and daemons
3	Specifies a multiuser run level with remote processes and daemons
q	Specifies that init should reexamine the inittab file

### Changing to a Different Multiuser Run Level:

To change from the current multiuser run level to a different multiuser run level, enter the init command with the argument that corresponds to the run level that you want to enter. For example, to change from run level 2 to run level 3, enter the following command:

```
init 3
```

In response to your entry, init reads the inittab file and follows the instructions that correspond to the change in run level.

### Changing to Single-User Mode:

The init command provides a way to change from the current multiuser mode to single-user mode by using the s run level argument. For example, to change from the current run level to single-user mode, enter:

```
init s
```

To change from a multiuser mode to single-user mode, giving users a 10-minute warning, enter:

```
/usr/sbin/shutdown +10 Bringing system down to single-user for testing
```

To return to multiuser mode from single-user mode, use Ctrl/d or enter exit at the prompt. This causes the init command as process 1 to prompt you for the run level. In response to the prompt, enter 2 to return to multiuser mode without networking daemons activated, or enter 3 to return to multiuser mode with networking daemons activated.

Alternatively, you can reboot the system by using one of the following commands:

```
/usr/sbin/shutdown -r now
/sbin/reboot
```

### Reexamining the inittab File:

To reexamine the inittab file, enter the init command with the q argument, as follows:

```
init q
```

In response, init reexamines the inittab file and starts new processes, if necessary. For example, if you recently added new terminal lines, init activates the getty process for these terminal lines in response to the init q command.

Refer to the getty(8) reference page for further information about the relationship between terminal lines and the init command.

## Notes

### Symmetric Multiprocessing:

Symmetric MultiProcessing (SMP) consists of two or more processors that execute the same copy of the operating system, address common memory, and can execute instructions simultaneously. In a multiprocessor system, multiple threads can run concurrently through simultaneous execution on multiple processors.

If your system is a multiprocessor system and it is running Digital UNIX, it is running in an SMP environment. The objective of the operating system in an SMP environment is to take advantage of the incremental computes available to the system as additional processors are added. To do this, the operating system must allow multiple threads of execution to operate concurrently across the available processors.

From a system administrator's point of view, this additional computing power requires little to no additional system management work. All the administrator should see is additional available computes. It may be that additional I/O capabilities will be required to more efficiently utilize these extra computes.

### Adding CPUs to an Existing System

At boot time, the system determines the number of CPUs available. Adding computing power to your multiprocessing systems is as simple as installing the processor board and rebooting the system. You do not have to reconfigure the kernel; you may have to modify any tuning that was done to limit the number of processors available, and you may need to install a Product Authorization Key (PAK). For more information on PAKs, see the Software License Management manual.

### Unattended Reboots on Multiprocessor Systems

If a processor in a multiprocessor system fails, the operating system notes which processor failed, then automatically reboots the system. Although the operating system continues, you must manually restart the failed processor. For instructions, see the Installation Guide.

### Setting and Resetting the System Clock:

The system has an internal clock that you set when you install the system. The clock maintains the time and date whether the power is on or off. Nevertheless, there are occasions when you might need to reset the time or date. For example, with battery-powered clocks, you might need to reset the time as a result of battery failure; or you may need to synchronize system time with standard time.

To set the date and time, log in as root and use the following syntax with the date command:

```
date [[cc]yy]mmddHHMM[.ss]
```

cc	Designates the first two numbers of the year (century) as a 2-digit integer.
yy	Designates the year as a 2-digit integer
MM	Designates the month as a 2-digit integer
dd	Designates the day as a 2-digit integer
HH	Designates the hour as a 2-digit integer, using a 24-hour clock
mm	Designates the minutes as a 2-digit integer



.	Serves as a delimiter
ss	Designates the seconds as a 2-digit integer (this field is optional)

## Notes

To set the date to 09:34:00 AM Jan 7, 2000 using the mmddHHMM[[cc]yy][.ss] Digital format:

```
date 010709342000
```

```
date 0107093400.00
```

```
date 010709342000.00
```

Note

If you are changing the year, the system disk must be updated with the new year information. In single-user mode, enter the mount -u / command after you enter a date containing a new year. This command writes the new year into the superblock on the system disk. Note also that the root file system will now be mounted read-write.

### Resolving Booting Problems:

Should your system not boot, the following list suggests some areas for further investigation:

#### ◇ Hardware failure

Check the hardware manual accompanying your system for hardware test procedures. If a hardware problem exists, follow the instructions in the guide for resolving the problem.

#### ◇ Software failure

Software can fail for the following reasons:

- ◆ An incorrect boot path was specified
- ◆ The kernel is corrupt

If you suspect that the kernel may be corrupt, try booting the generic kernel, / genvmunix. This will provide you with a fully functional system and you can begin debugging procedures using the kdbx or dbx utilities to analyze crash dumps. Refer to the kdbx(8) or dbx(1) reference pages for more information.

A disk or file system is corrupt

If a disk or file system is corrupt, run the fsck command on the file system. The fsck command checks and repairs UNIX File Systems (UFS). If fsck finds something wrong, it prompts you for an action to take. Use extreme care under these circumstances so that you do not inadvertently overwrite or remove any files.

AdvFS provides disk recovery during the mount procedure that corrects the disk structures. You do not need to run the fsck command or any other command. Consequently, recovery of AdvFS is very rapid.

### Shutting Down the System:

The following sections describe the shutdown procedures and the recovery strategies that you use in both controlled and unexpected shutdowns. The first part

## Notes

discusses procedures for handling controlled shutdowns. The second part discusses guidelines and recommendations for handling and recovering from unexpected shutdowns.

### Note

The SysMan `dxshutdown` command can also be used for some of these tasks.

There are several good reasons to stop the system in a controlled shutdown. For example:

- ◇ You need to upgrade your software or add new hardware to your configuration. You power down the system to set up the new additions, make the necessary adjustments to your configuration files, and build a new kernel.
- ◇ You have been monitoring the hardware error log and have noticed repeated warnings. You suspect that your hardware may soon fail so you shut down the system and examine the problem.
- ◇ You notice that system performance is degrading rapidly. You check the system statistics and conclude that some changes to the system would improve performance. You shut down and tune the system.
- ◇ You notice signs of possible file system corruption. You shut down the system and run the `fsck` program to fix problems or to confirm that none exist.

In each of these and similar situations a variety of options are available to you. Regardless of how you decide to resolve the situation, your first step is to initiate a controlled shutdown of the system. There are practical and reasonable ways to shut down your system from single-user mode or multiuser mode.

A system that has panicked or crashed presents you with a different set of circumstances than a system that has shut down in an orderly fashion. However, this chapter discusses orderly shutdowns only.

### Stopping Systems While in Multiuser Mode:

To power down the system while running in multiuser mode, use the shutdown command. When you issue the shutdown command with the `-h` or `-r` flags, the program typically performs the following operations:

- ◇ Runs the `wall` program to notify all users of the impending shutdown
- ◇ Disables new logins
- ◇ Stops all accounting and error-logging processes
- ◇ Runs the `killall` program to stop all other processes
- ◇ Runs the `sync` program to synchronize the disk(s)
- ◇ Logs the shutdown in the log file
- ◇ Unmounts file systems
- ◇ Halts the system

The following sections describe typical shutdown operations and provide examples of what happens when you use the command flags.

### Shutting Down the System and Warning Other Users

To shut down the system from multiuser mode to single-user mode at a specific time and warn users of the shutdown:

- ◇ Log in as root and change to the root directory:

```
cd /
```

- ◇ Utilize the following syntax with the shutdown command:

```
shutdown time [warning-message]
```

For example, to shut down the system in 10 minutes with a warning to users that the system is going down for routine maintenance tasks, enter:

```
shutdown +10 Maintenance shutdown
```

The system begins to notify users of the impending shutdown. Next, it disables logins, stops accounting and error logging, stops all remaining processes, logs the shutdown in the log file, and sends the init program a signal that causes the system to transition to single-user mode.

When the system's shutdown completion message appears, the shutdown is complete. You can access the system through the console to perform the desired administrative tasks. When you are finished, reboot the system.

### Shutting Down and Halting the System:

To shut down the system from multiuser mode, warn all users, and halt all systems:

- ◇ Log in as root and change to the root directory:

```
cd /
```

- ◇ Use the following syntax with the shutdown command:

```
shutdown -h time [warning-message]
```

For example, to shut down and halt the system in 5 minutes with a warning to users that the system is going down for maintenance, enter:

```
shutdown -h +5 Maintenance shutdown in five minutes
```

The shutdown program begins to notify users of the impending shutdown, disables logins, and proceeds with the standard shutdown activities. At the specified shutdown time, the systems are halted.

### Shutting Down and Automatically Rebooting the System:

To shut down the system from multiuser mode, warn all users, and automatically reboot the system to multiuser mode:

- ◇ Log in as root and change to the root directory:

```
cd /
```

- ◇ Use the following syntax with the shutdown command:

```
shutdown -r time [warning-message]
```

For example, to shut down and automatically reboot the system in 15 minutes with a warning to users that the system is going down for a reboot, enter the following command

```
shutdown -r +15 Shutdown and reboot in 15 minutes
```

## Notes

In this case, the system begins to notify users of the impending shutdown, disables logins, and proceeds with the standard shutdown activities. When it completes these activities, shutdown automatically starts the reboot operation, which involves running fsck for a consistency check on all mounted file systems. If problems are not encountered, the system reboots to multiuser mode.

### Note

If fsck finds file system inconsistencies, it displays a warning message, recommending that you run fsck again from single-user mode before operating the system in multiuser mode.

### Shutting Down and Halting Systems Immediately

To shut down and halt the system immediately:

- ◇ Log in as root and change to the root directory. For example, enter the following command:

```
cd /
```

- ◇ Enter the shutdown command as follows:

```
shutdown -h now
```

In response to this command, the system shuts down immediately and halts the system.

### Note

Do not use this command when there are other users on the system. Users get no warning messages and their processes are immediately stopped.

### Stopping Systems While in Single-User Mode:

Although the shutdown command is your best choice for shutting down systems, you can also use the halt command. This command should be invoked only from single-user mode. If you are working in single-user mode, you can stop systems by entering the following commands:

```
sync
```

```
sync
```

```
halt
```

In response to the halt command, the program logs the shutdown in the log file, kills all running processes, executes the sync system call and waits for all information to be written to disk, then halts the systems.

The procedures for starting up and shutting down the operating system and includes a discussion of:

- ◇ Boot operation
- ◇ Different startup states and the corresponding boot preparation
- ◇ Run levels
- ◇ Resolving problems that occur during the boot operation

Shutting down the system is a routine task that you should perform periodically. Usually, you can shut down the system easily and with minimal disruption to system users. Occasionally, you must shut down the system rapidly, causing a moderate degree of disruption to users. Under some circumstances (that are out of your control), the system shuts itself down suddenly, causing substantial disruption to users.

### 5.3.3 File System Mounting and Unmounting

It's normally apparent from context which type of "filesystem" is being discussed, but for clarity in the following discussion, we use the term "file tree" to refer to the overall layout and reserve the word "filesystem" for the chunks attached to the tree.

Most filesystems are disk partitions or disk-based logical volumes, but as we mentioned earlier, they can be anything that obeys the proper API: network file servers, kernel components, memory-based disk emulators, etc. Linux and Solaris even have a nifty "loopback" filesystem that lets you mount individual files as if they were distinct devices. It's great for developing filesystem images without having to worry about repartitioning your disks. In most situations, filesystems are attached to the tree with the mount command.<sup>3</sup> mount maps a directory within the existing file tree, called the mount point, to the root of the newly attached filesystem. The previous contents of the mount point become inaccessible as long as another filesystem is mounted there. Mount points are usually empty directories, however.

For example,

```
$ sudo mount /dev/sda4 /users
```

installs the filesystem stored on the disk partition represented by /dev/sda4 under the path /users. You could then use ls /users to see that filesystem's contents.

A list of the filesystems that are customarily mounted on a particular system is kept in the /etc/fstab, /etc/vfstab (Solaris), or /etc/filesystems (AIX) file. The information enclosed in this file allows filesystems to be checked (with fsck) and mounted (with mount) automatically at boot time. It also serves as documentation for the format of the filesystems on disk and enables short commands such as mount /usr. See page 260 for a discussion of the fstab file and its brethren. You detach filesystems with the umount command. umount complains if you try to unmount a filesystem that is in use; the filesystem to be detached must not have open files or processes whose current directories are located there, and if the filesystem contains executable programs, they cannot be running.

Linux has a "lazy" unmount option (umount -l) that removes a filesystem from the naming hierarchy but does not truly unmount it until all existing file references have been closed. It's debatable whether this is a useful option. To begin with, there's no guarantee that existing references will ever close on their own. In addition, the "semi-unmounted" state can present inconsistent filesystem semantics to the programs that are using it; they can read and write through existing file handles but cannot open new files or perform other filesystem operations.

umount -f force-unmounts a busy filesystem and is supported on all our example systems. However, it's almost always a bad idea to use it on non-NFS mounts, and it may not work on certain types of filesystems (e.g., those

## Notes

that keep journals, such as ext3 or ext4). Instead of reaching for `umount -f` when a filesystem you're trying to unmount turns out to be busy, run the `fuser` command to find out which processes hold references to that filesystem. `fuser -c mountpoint` prints the PID of every process that's using a file or directory on that filesystem, plus a series of letter codes that show the nature of the activity.

For example,

```
$ fuser -c /
usr /usr: 157tm 315ctom 474tom 5049tom 84tm 496ctom 490tm
16938c 16902ctm 358ctom 484tm
```

The exact letter codes vary from system to system. Below table summarizes the meanings of the codes, but the details are usually unimportant; the PIDs are what you want.

Table 2

Codes	Meaning
f,o	The process has a file open for reading or writing.
c	The process's current directory is on the filesystem.
e,t	The process is currently executing a file.
r	The process's root directory (set with <code>chroot</code> ) is on the filesystem.
m,s	The process has mapped a file or shared library.

To investigate the offending processes, just run `ps` with the list of PIDs returned by `fuser`. For example,

```
$ ps -fp "157 315 5049"
UID PID PPID C STIME TTY TIME CMD
root 5049 490 0 Oct 14 ? 0:00 /usr/bin/X11/xdm
root 157 1 0 Jun 27 ? 5:26 /usr/sbin/named
lp 315 1 0 Jun 27 ? 0:00 /usr/lib/lpsched ...
```

Here, the quotation marks force the shell to pass the list of PIDs to `ps` as a single argument. On Linux systems, you can avoid the need to launder PIDs through `ps` by running `fuser` with the `-v` flag. This option produces a more readable display that includes the command name.

```
$ fuser -cv /usr
USER PID ACCESS COMMAND /usr
root 444m atd
root 499m sshd
root 520m lpd ...
```



The letter codes in the ACCESS column are the same ones used in fuser's nonverbose output.

A more elaborate alternative to fuser is the lsof utility by Vic Abell. lsof is a more multifaceted and sophisticated program than fuser, and its output is correspondingly verbose. lsof is available from [people.freebsd.org/~abe](http://people.freebsd.org/~abe) and works on all of our example systems.

Under Linux, scripts in search of specific information about processes' use of filesystems can read the files in /proc directly. However, lsof -F, which formats lsof's output for easy parsing, is an easier and more portable solution. Use additional command-line flags to request just the information you need.

### 5.3.4 Monitoring System Usage

Large installations require vigilant supervision. Don't expect users to report problems to you unless the issues are severe. Working around a problem is usually faster than taking the time to document and report it, so users often follow the path of least resistance. Regularly ensure that email and web services are working correctly, watch log files for early signs of trouble, make sure that local networks are properly connected, and keep an eye on the availability of system resources such as disk space. All of these chores are excellent opportunities for automation, and a variety of off-the-shelf monitoring systems can help sysadmins with this task.

#### PS: MONITOR PROCESSES:

The key tool for monitoring processes for the system administrator is ps. While the arguments and display of different versions of ps varies, they all convey fundamentally the same information. Differences in the development history of UNIX can be traced back to part of the large diversity among versions of ps. ps, on the other hand, is a command that vendors frequently tweak for additional purposes. Because it is so tightly linked to how the kernel handles processes, it tends to reflect all of a vendor's underlying kernel changes.

ps can display a process's PID, UID, priority, and control terminal. It also tells you how much memory a process is consuming, how much CPU time it's taking up, and how it's doing right now (running, stopped, sleeping, etc.). In a ps listing, zombies appear as or.

Over the last decade, ps implementations have become impossibly complicated. Several vendors have given up on defining meaningful displays in favour of making their pses entirely flexible. Almost any desired output may be created with a little modification work. For example, Linux's ps is a trisexual and hermaphroditic version that recognises several option sets and relies on an environment variable to determine which universe it is in.

Don't be intimidated by the complexity: it's primarily intended for developers, not system administrators. You only need to remember a few specific incantations because you'll be using ps regularly.

With ps aux on Linux and AIX, you can get a good overview of all the processes operating on the system. The a option displays all processes, the x option displays even processes without a control terminal, and the u option displays the "user oriented" output format.



## Notes

### Explanation of ps aux output

Field	Contents
USER	Username of the process's owner
PID	Process ID
%CPU	Percentage of the CPU this process is using
%MEM	Percentage of real memory this process is using
VSZ	Virtual size of the process
RSS	Resident set size (number of pages in memory)
TTY	Control terminal ID
STAT	Current process status:
	§ R = Runnable D= In uninterruptible sleep
	§ S = Sleeping (< 20 sec) T = Traced or stopped
	§ Z = Zombie
	Additional flags:
	§ W= Process is swapped out
	§ < = Process has higher than normal priority
	§ N= Process has lower than normal priority
	§ L = Some pages are locked in core
	§ s = Process is a session leader
TIME	CPU time the process has consumed
COMMAND	Command name and arguments

I selects the “long” output format. ps lax is also slightly faster to run than ps aux because it doesn't have to translate every UID to a username—efficiency can be important if the system is already bogged down. Under Solaris and HP-UX, ps -ef is a good place to start. The e option selects all processes, and the f option sets the output format. (ps -ef also works on AIX and Linux systems; note the dash.)

solaris\$ ps -ef							
UID	PID	PPID	C	STIME	TTY	TIME	COMD
Root	0	0	80	Dec 21	?	0:02	sched
root	1	0	2	Dec 21	?	4:32	/etc/init
root	2	0	8	Dec 21	?	0:00	pageout
root	171	1	80	Dec 21	?	0:02	/usr/lib/sendmail-bd
trent	8482	8444	35	14:34:10	pts/7	0:00	ps-ef
trent	8444	8442	203	14:32:50	pts/7	0:01	-csh

The columns in the ps -ef output are explained in below Table.

Field	Content	Field	Content
UID	Username of the owner	STIME	Time the process was started
PID	Process ID	TTY	Control terminal

PPID	PID of the parent process	TIME	CPU time consumed
C	CPU use/scheduling info	COMD	Command and arguments

Like `ps` in the Linux and AIX worlds, `ps -elf` shows additional gory details on Solaris and HP-UX systems:

% ps -elf												
F	S	UID	PID	PPID	C	P	NI	ADDR	SZ	WCHAN	TIME	COMD
19	T	root	0	0	80	0	SY	f00c2fd8	0	0:02	sched	
8	S	root	1	0	65	1	20	ff26a800	88	ff2632c8	4:32	init
8	S	root	142	1	41	1	20	ff2e8000	176	f00cb69	0:00	syslogd ...

The STIME and TTY columns have been omitted to fit this page; they are identical to those produced with `ps -ef`.

### 12.5.2 Dynamic Monitoring with TOP, PRSTAT, and TOPAS:

Since commands like `ps` offer only a one-time snapshot of your system, it is often difficult to grasp the big picture of what's really happening. `top` is a free utility that runs on many systems and provides a regularly updated summary of active processes and their use of resources. On AIX, an equivalent utility is `topas`, and on Solaris the analogous tool is `prstat`.

Table

Explanation of `ps -ef` output

Field	Contents
F	Process flags; possible values vary by system (rarely useful for sysadmins)
S	Process status:
	O= Currently running
	S = Sleeping (waiting for event)
	R = Eligible to run
	T = Stopped or being traced
	Z = Zombie
	D= Uninterruptible sleep (disk, usually)
C	Process CPU utilization/scheduling info
P	Scheduling priority (internal to the kernel, different from nice value)
NI	Nice value or SY for system processes
ADDR	Memory address of the process
SZ	Size (in pages) of the process in main memory
WCHAN	Address of the object the process is waiting for

For example:

```
ubuntu$ top
```

```
top - 16:37:08 up 1:42, 2 users, load average: 0.01, 0.02, 0.06
```

## Notes

Tasks: 76 total, 1 running, 74 sleeping, 1 stopped, 0 zombie

Cpu(s): 1.1% us, 6.3% sy, 0.6% ni, 88.6% id, 2.1% wa, 0.1% hi, 1.3% si

Mem: 256044k total, 254980k used, 1064k free, 15944k buffers

Swap: 524280k total, 0k used, 524280k free, 153192k cached

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3175	root	15	0	35436	12m	4896	S	4.0	5.2	01:41.9	X
3421	root	25	10	29916	15m	9808	S	2.0	6.2	01:10.5	rhn-applet-gui
1	root	16	0	3356	560	480	S	0.0	0.2	00:00.9	init
2	root	34	19	0	0	0	S	0.0	0	00:00.0	ksoftirqd/0
3	root	5	-10	0	0	0	S	0.0	0	00:00.7	events/0
4	root	5	-10	0	0	0	S	0.0	0	00:00.0	khelper
5	root	15	-10	0	0	0	S	0.0	0	00:00.0	kacpid
18	root	5	-10	0	0	0	S	0.0	0	00:00.0	kblockd/0
28	root	15	0	0	0	0	S	0.0	0	00:00.0	pdflush
29	root	15	0	0	0	0	S	0.0	0	00:00.3	pdflush
31	root	13	-10	0	0	0	S	0.0	0	00:00.0	aio/0
19	root	15	0	0	0	0	S	0.0	0	00:00.0	khubd
30	root	15	0	0	0	0	S	0.0	0	00:00.2	kswapd0
187	root	6	-10	0	0	0	S	0	0	00:00.0	kmirror0/0
196	root	15	0	0	0	0	S	0	0	00:01.3	kjournald ...

By default, the display updates every 10 seconds. The most CPU-consumptive processes appear at the top. top also accepts input from the keyboard and allows you to send signals and to renice processes, so you can observe how your actions affect the overall condition of the machine. Root can run top with the -q option to goose it up to the highest possible priority. This option can be very useful when you are trying to track down a process that has already brought the system to its knees.

### 5.3.5 Ensuring System Security

#### BIOS Security:

An Operating System is needed security from its booting up to shutting down. If an attacker has access to the BIOS, nothing will be remain safe. Whenever an operating system is boot up it is very important whether an unauthorized user has full physical access to computer. The BIOS is the lowest level of software that configures or manipulates of x86 based hardware. LILO and other Linux boot methods access the BIOS to determine how to boot up Linux machine.

We can set BIOS password to prevent from unauthorized physical access. To prevent changes to BIOS Settings BIOS password is needed .Some examples are, disallow booting from floppy drives and passwords to access some BIOS features. Many x86 BIOS's also allow to specify various other good security settings.

#### Boot Loader Security:

At this moment we prevents an illegal user to change the BIOS settings by setting BIOS password. But In Linux Operating System there is an option to change the root

password during booting. The two common boot loaders for Linux are LILO (Linux Loader) and GRUB (Grand Unified Boot loader).

## Notes

### Linux booting process

The boot process of Linux on a Intel i386 architecture has following steps:

- ◇ A Linux Loader is placed at the first sector read by the bios.
- ◇ The kernel is loaded.
- ◇ Init is started and executes various scripts.

### Securing boot process:

#### Reasons for securing boot loader

The following are the primary reasons for protecting a LINUX boot loader.

- a. Prevent Access to Single User Mode 15 If an attacker can boot into single user mode , he becomes the root user and a root user can change anything to his own way which is directly threat to a secure operating system.
- b. Prevent Access to the GRUB Console If an illegal user get access to GRUB editor, he can change its configuration or he can get information by using cat command.
- c. Prevent Access to Non-Secure Operating Systems- If the system is using dual booting, any body can select an operating system which is less secure than Linux which ignores access controls and file permissions[11].

### LILO security:

LILO is the Linux boot loader, it handles all the tasks of getting the kernel into memory and bootstrapping them machine into something that resembles a useful computing device. To secure LILO we have to set a password, and use the restricted keyword. We can also set security on a per image basis, or adding a password directive in to the global section of its configuration file. Add a passwords directive to /etc/lilo.conf file.

```
password= <my password>
```

If we want to allow booting a kernel without password verification, but do not want to allow users to add arguments without password add restricted keyword below the password line[11].

```
image=/boot/vmlinuz- <version>
```

```
password= <thisisapassword>
restricted
```

### Grub security

GRUB adds an extra level of security by supporting MD5 encryption for the password in the configuration file. To generate an encrypted password, run the command

```
$ /sbin/grub-md5-crypt
```

When prompted for password type a password and it will return an MD5 hash of the password. Next edit the /boot/grub/grub.conf and set the MD5 hash value of password[11].

```
password --md5<password-hash>
```

## Notes

### Users and Groups

User accounts are necessary for confirming a person's identity when utilising a computer system. The system can identify if a user is permitted to log into the system and, if so, which resources the user is permitted to access by confirming the user's accounts.

There are three types of users:

- i) Root
- ii) Normal users
- iii) System users

The superuser, normally named root, has complete control over the entire system. The root user can access all files on the system and the root user is generally the only user who can execute certain programs. The root has a user ID 0. Any account with a user ID of 0 is a root user, even if the username is not root. Normal users are users who can log in.

Normal users usually have a home directory and can create and manipulate files in their home directory and in other directory. Normal users typically have restricted access to files and directories on the machine and as a result they cannot perform many system-level functions.

System users don't login. They are accounts that are used for specific purposes and are not allowed by a specific person. For example user nobody and lp. The user nobody is the user who typically handle the http request and lp handles print request.

User accounts can be grouped together for a specific reason using groups, which are logical entities. Access to restricted resources can be preserved for those who need them but denied to others through careful group creation and privilege assignment. The owner and group owner are allocated to the user who creates the file. The owner, the group, and everyone else have distinct read, write, and execute permissions on the file. Only the root user has the ability to alter the owner of a file. If the owner of the file is a member of the group being added to the file, root or the owner of the file can modify the group to which it belongs.

For security purposes we should be aware at least the following:

- ◇ Login activity
- ◇ Authorization information
- ◇ Authentication information
- ◇ Commands users have run
- ◇ Restarts and shutdowns of the system
- ◇ Network transactions records

### Managing user accounts:

Linux provides a large number of tools including account permissions, passwords, account aging, adding and deleting of users, etc. Following commands are useful to manage user and group.

- ◇ chage - change user password expiry information

- ◇ groups - print the groups a user is in
- ◇ newusers - update and create new users in batch
- ◇ passwd - update a user's authentication tokens(s)
- ◇ nologin - prevent non-root users from log into the system
- ◇ su - run a shell with substitute user and group IDs
- ◇ useradd - Create a new user or update default new user information
- ◇ userdel - Delete a user account and related files 18
- ◇ usermod - Modify a user account
- ◇ chgrp - change the group ownership of files
- ◇ chown - change the user and group ownership of files
- ◇ gpasswd - administer the /etc/group file
- ◇ groupadd - Create a new group
- ◇ groupdel - Delete a group
- ◇ groupmod - Modify a group
- ◇ groups - print the groups a user is in
- ◇ grpck - verify integrity of group files
- ◇ pwconv - convert to and from shadow passwords
- ◇ pwunconv - convert to and from shadow passwords
- ◇ grpconv - convert to and from shadow passwords
- ◇ grpunconv - convert to and from shadow passwords
- ◇ vipw - edit the password or group files
- ◇ vigr - edit the password or group files

### Passwords:

Password is one of the most important security features and basic means of authentication. It is important to set secure, unguessable passwords. Password security is the most critical means to protect system from compromise. An effective well-chosen password is always desirable not be compromised the system.

Linux have several characteristics of password storing mechanism.

- i) In a file that is readable only by root. 19
- ii) In a one way hash format.

Following guideline should be maintain for strong password

- ◇ Use of shadow passwords.
- ◇ Do not use only words or numbers.
- ◇ Do not use recognizable words.
- ◇ Do not use hacker terminology.
- ◇ Do not use personal Information.
- ◇ Do not invert recognizable words.
- ◇ Make the password at least eight characters long.

## Notes

- ◇ Mix upper and lower case letters.
- ◇ Mix letters and numbers.
- ◇ Include non-alphanumeric characters.

### Guest accounts privilege:

Guest accounts on servers don't have to be set up with the same privilege as those of regular users. If these guests only need to run a few programs or access a collection of local files, then enable them to do just this and nothing more.

### File Systems:

Security File systems security is important to keep a system safe. By changing important file like server configuration, network configuration and system configuration a machine can be compromised. A file system is the methods and data structures that an operating system uses to keep track of files on a disk or partition that is, the way the files are organized on the disk. The super block, inode, data block, directory block, and indirection block are all important concepts in Linux file systems. The super block holds data on the entire file system, such as its size. All information about a file, save its name, is contained in an inode. The name, together with the inode number, is saved in the directory. A filename and the number of the inode that represents the file make up a directory entry. The numbers of numerous data blocks used to store data in the file are contained in the inode. However, the inode only has room for a few data block numbers, and if more are required, more space for pointers to the data blocks is dynamically created. These dynamically created blocks are called indirect blocks because finding the data block requires first finding its number in the indirect block.

Linux prefers a single hierarchical directory structure. Instead of having 'drives,' everything starts with the root directory, represented by /, and then grows into sub-directories. Linux, on the other hand, arranges directories in order of significance to the boot process, starting with the root directory. Another reason for the unified file system is that Linux caches a large number of disc requests in system memory while it executes correct commands. This will shut down the system in a proper manner, ensuring the integrity of files.

/bin Essential command binaries

/boot Static files of the boot loader

/dev Device files /etc Host-specific system configuration

/lib Essential shared libraries and kernel modules

/mnt Mount point for mounting a filesystem temporarily

/opt Add-on application software packages

/sbin Essential system binaries

/tmp Temporary files

/usr Secondary hierarchy

/var Variable data For each object in the file system,

Linux stores administrative information in a structure known as an inode. Instead, they have indices (numbers) indicating their positions in the array of inodes.



Each inode generally contains:

- ◇ The location of the item's contents on the disk, if any
- ◇ The item's type (e.g., file, directory, symbolic link) 21
- ◇ The item's size, in bytes, if applicable
- ◇ The time the file's inode was last modified
- ◇ The time the file's contents were last modified
- ◇ The time the file was last accessed (the atime) for read ( ), exec ( ), etc
- ◇ A reference count: the number of names the file has
- ◇ The file's owner (a UID)
- ◇ The file's group (a GID)
- ◇ The file's mode bits (also called file permissions or permissionbits)

Linux separates access control on files and directories according to three characteristics: owner, group, and other. There is always exactly one owner, any number of members of the group, and everyone else. Any user will be able to view contents of a file and edit by setting read ,write permission of a file.

Following table is the list file protection command with chmod [6].

**Table - Linux File protection command**

Command	Meaning
chmod 400 file	To protect a file against accidental overwriting
chmod 500 directory	To protect from accidentally removing, renaming or moving files from this directory
chmod 600 file	A private file only changeable by the user who entered this command
chmod 644 file	A publicly readable file that can only be changed by the issuing user
chmod 660 file	Users belonging to group can change these files, others don't have any access to it
at all. chmod 700	File Protects a file against any access from other users, while the issuing user still has full access
chmod 755	directory for files that should be readable and executable by others, but only changeable by the issuing user.
chmod 775	file Standard file sharing mode for a group.
chmod 777	file Everybody can do everything to this file.

### Check your Understanding

#### Fill Ups

1. Adding and removing users on a \_\_\_\_\_ system is one of the most important system administration tasks to familiarize yourself with
2. For tasks that require administrator privileges, there is a tool installed on Ubuntu systems called \_\_\_\_\_.

## Notes

3. The \_\_\_\_\_ is used to update a user's authentication token stored in /etc/shadow file.
4. \_\_\_\_\_ is the most fundamental kind of disc management.
5. \_\_\_\_\_ mode, commonly known as JBOD (short for "just a bunch of disks"), is not a true RAID level.

### True and False

1. If your new user should have the ability to execute commands with root (administrative) privileges, you will need to give the new user access to sudo.
2. Using visudo is the only recommended way to create alterations to /etc/sudoers, because it locks the file against multiple simultaneous edits and performs a sanity check on its contents before overwriting the file.
3. Both Linux and UNIX-like operating systems use the passwd command to change user password.
4. Partitioning and logical volume management are both methods for partitioning a disc (or, in the case of LVM, a pool of drives) into distinct portions of known size.
5. Traditionally, visudo opened /etc/sudoers in the vi editor, which can be confusing for inexperienced users.

### Summary

- Adding and removing users on a Linux system is one of the most important system administration tasks to familiarize yourself with. When you create a new system, you are often only given access to the root account by default.
- For tasks that require administrator privileges, there is a tool installed on Ubuntu systems called sudo. Briefly, sudo allows you to run a command as another user, including users with administrative privileges. In this guide we will cover how to create user accounts, assign sudo privileges, and delete users.
- As an alternative to putting your user in the sudo group, you can use the visudo command, which opens a configuration file called /etc/sudoers in the system's default editor, and explicitly specify privileges on a per-user basis.
- Both Linux and UNIX-like operating systems use the passwd command to change user password. The passwd is used to update a user's authentication token (password) stored in /etc/shadow file. The passwd change passwords for user and group accounts. A normal user may only change the password for his/her own account, the super user (or root) may change the password for any account. The overseer of a group may change the password for the group. passwd also changes account information, such as the full name of the user, user login shell, or password expiry date and interval.
- Disk management includes several important tasks such as adding or removing storage devices, creating and deleting partitions, mounting partitions on appropriate directories and making file system in partitions.
- Partitioning and logical volume management are both methods for partitioning a disc (or, in the case of LVM, a pool of drives) into distinct portions of known size.

Although all of our sample systems offer logical volume management, traditional partitioning is only available on Linux, Solaris, and occasionally HP-UX.

- Partitions are implemented in systems that enable them by placing a “label” at the start of the disc to define the range of blocks contained in each partition. The label must frequently cohabit with additional startup information (such as a boot block), and it frequently carries extra information such as a name or unique ID that identifies the disc as a whole. The label is known as the MBR, or master boot record, in Windows.
- RAID can perform two primary functions. First, it can boost speed by “striping” data across several drives, allowing numerous drives to supply or absorb a single data stream at the same time. Second, it can replicate data across numerous drives, reducing the danger of a single disc failing.
- When you boot the operating system, you initiate a set of tasks that the system must perform to operate successfully. The system is vulnerable during startup since it is loading the kernel into memory and initializing routines that it depends on for operation. Consequently, you should understand what is happening during the system boot, and be prepared to respond if problems occur.
- Most filesystems are disk partitions or disk-based logical volumes, but as we mentioned earlier, they can be anything that obeys the proper API: network file servers, kernel components, memory-based disk emulators, etc. Linux and Solaris even have a nifty “loopback” filesystem that lets you mount individual files as if they were distinct devices. It’s great for developing filesystem images without having to worry about repartitioning your disks.
- An Operating System is needed security from its booting up to shutting down. If an attacker has access to the BIOS, nothing will be remain safe. Whenever an operating system is boot up it is very important whether an unauthorized user has full physical access to computer. The BIOS is the lowest level of software that configures or manipulates of x86 based hardware. LILO and other Linux boot methods access the BIOS to determine how to boot up Linux machine.

### Activity

1. Create a presentation on the importance of partitioning. Include all the different forms of partitioning and their functions.

### Questions and Exercises

1. What is sudo?
2. How does one add a user?
3. What is visudo?
4. What are the steps to setting user password?
5. What is the need for partitioning?
6. What is RAID?
7. What are the reason for securing boot loader?

## Notes

### Glossary

- Disk management includes several important tasks such as adding or removing storage devices, creating and deleting partitions, mounting partitions on appropriate directories and making file system in partitions.
- Label is known as the MBR, or master boot record, in Windows.
- RAID, or “redundant arrays of cheap discs,” is a data distribution or replication method that uses numerous discs to distribute or reproduce data.
- Mirroring, in which data blocks are replicated bit-for-bit on several distinct drives, and parity schemes, in which one or more drives contain an error-correcting checksum of the blocks on the remaining data drives, are the two primary types of replication.
- LILO is the Linux boot loader, it handles all the tasks of getting the kernel into memory and bootstrapping them machine into something that resembles a useful computing device.

### Further Readings

1. Blum, Richard; Bresnahan, Christine. Mastering Linus System Administration. Sybex; 1st edition (July 27, 2021)
2. Gift, Noah; Jones, Jeremy. Python for Unix and Linux System Administration. O'Reilly Media; 1st edition (September 1, 2008)
3. Hein, Trent; Mackin, Dan; Nemeth, Evi; Snyder, Garth; Whaley, Ben. UNIX and Linus System Administration Handbook. ↑ Addison-Wesley Professional; 5th edition (August 8, 2017)
4. Kerrisk, Michael. The Linus Programming Interface: A Linus and UNIX System Programming Handbook. No Starch Press; 1st edition (October 28, 2010)
5. Negus, Christopher. Linux Bible. Wiley; 10th edition (June 10, 2020)

### Check your Understanding: Answers

#### Fill Ups

1. Linux
2. Sudo
3. Password
4. Partitioning
5. Linear

#### True and False

1. True
2. True
3. True
4. True
5. True